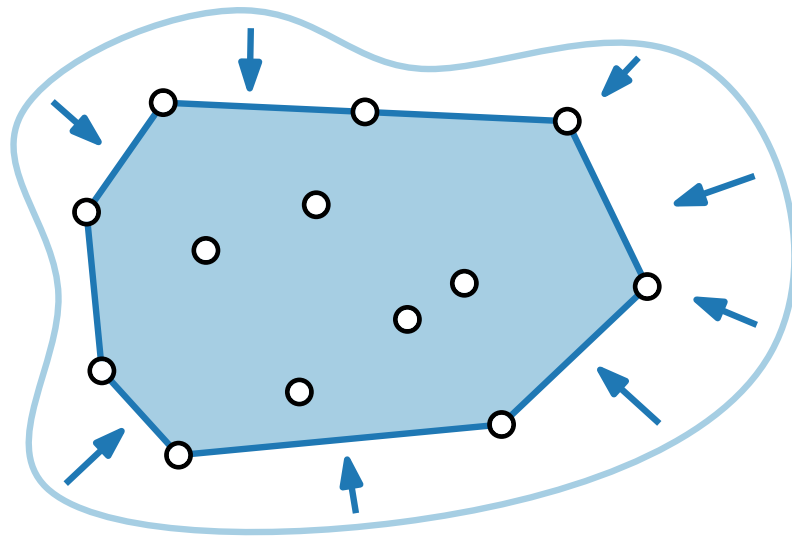


Fortgeschrittene Algorithmen

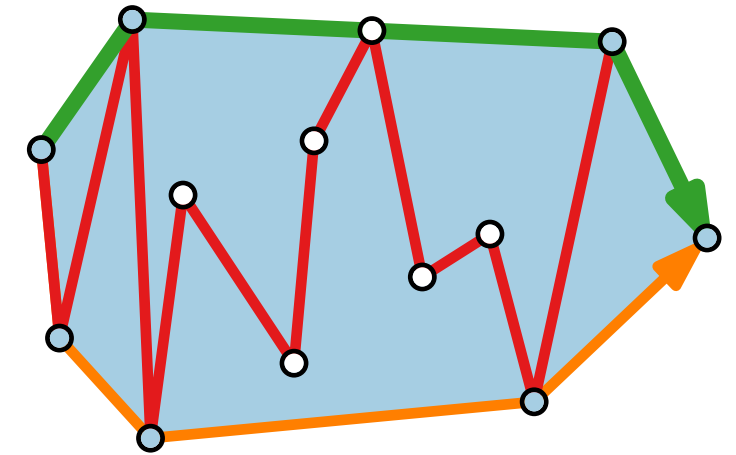
12. Vorlesung

Algorithmische Geometrie I:

Konvexe Hülle



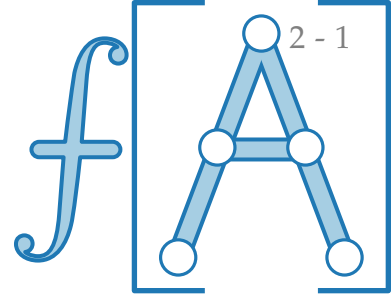
Teil I:
Problemstellung



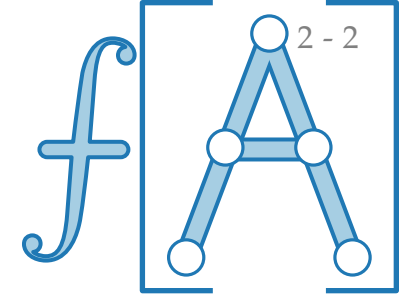
Mischen von Zutaten

Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %



Mischen von Zutaten



Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

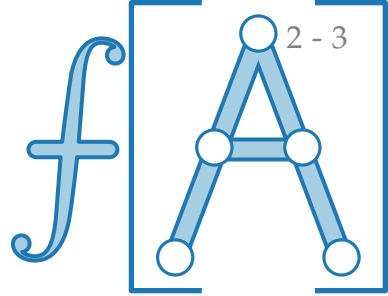
Ges.

Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3

Mischen von Zutaten



Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3

Mischen von Zutaten

Geg.

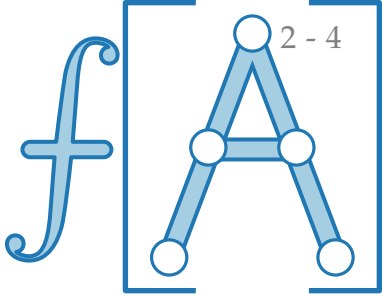
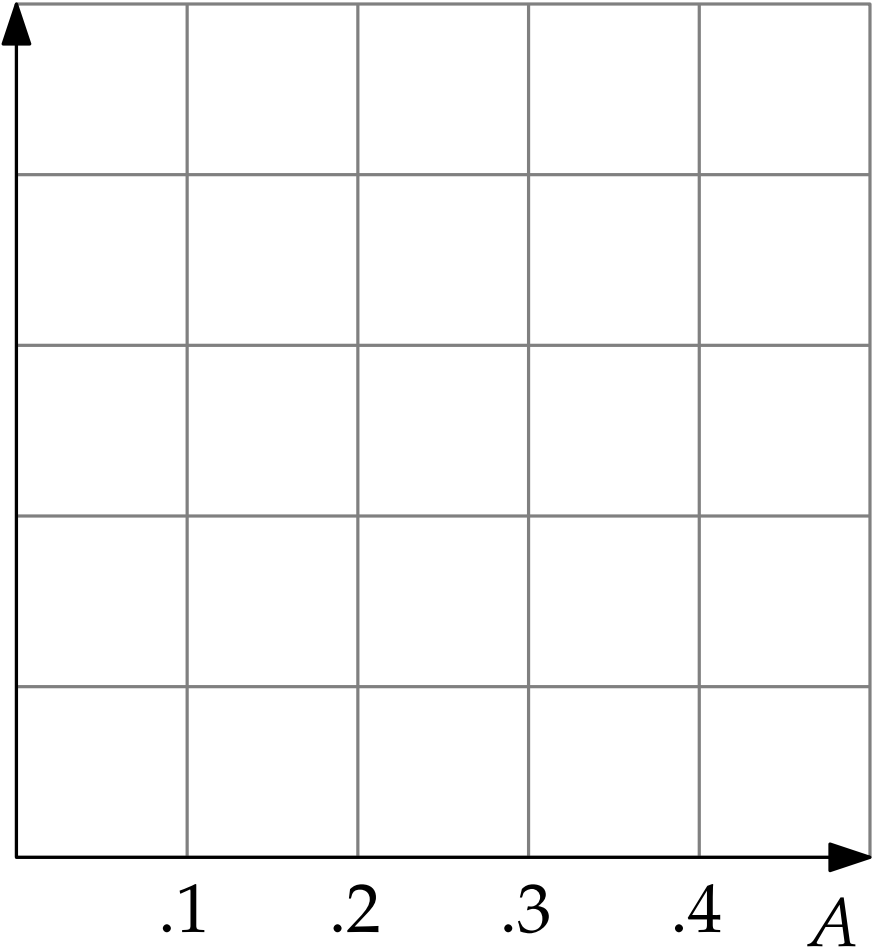
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

Geg.

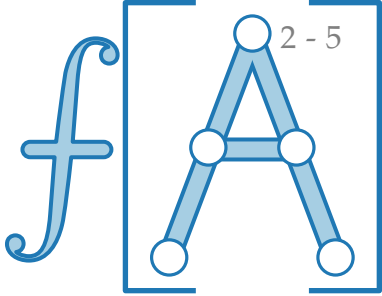
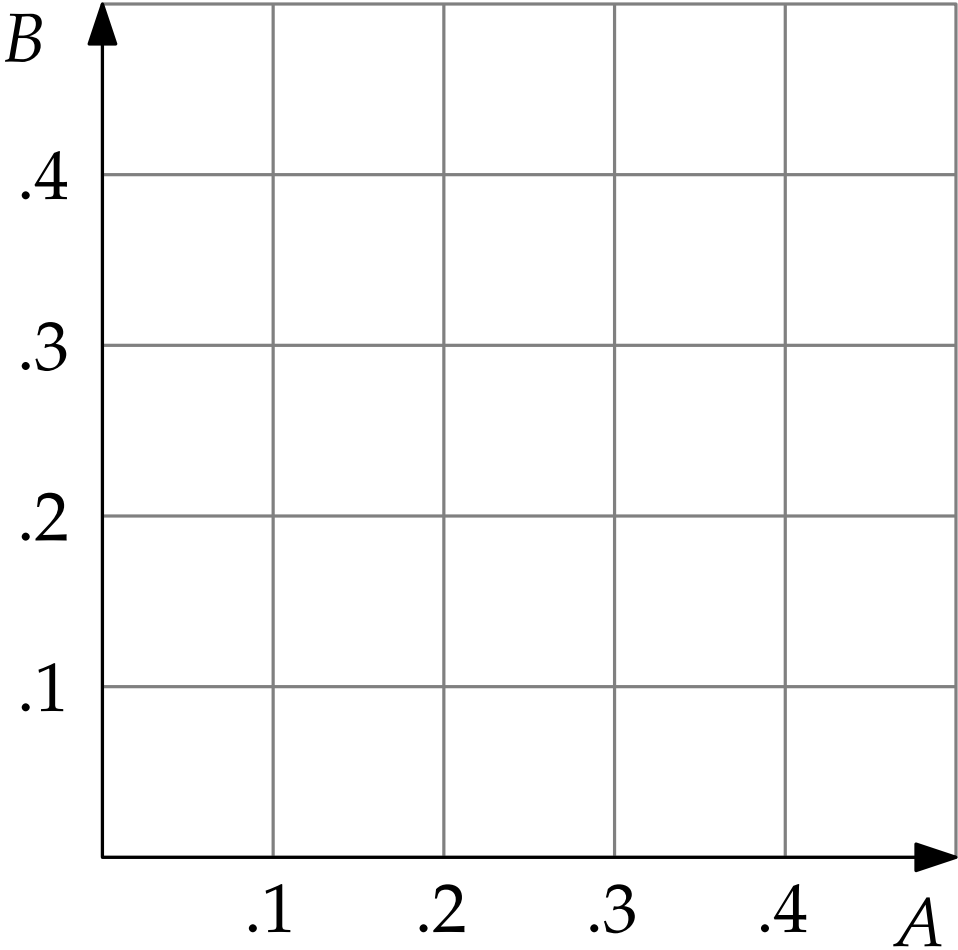
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

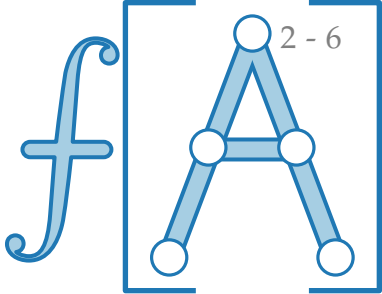
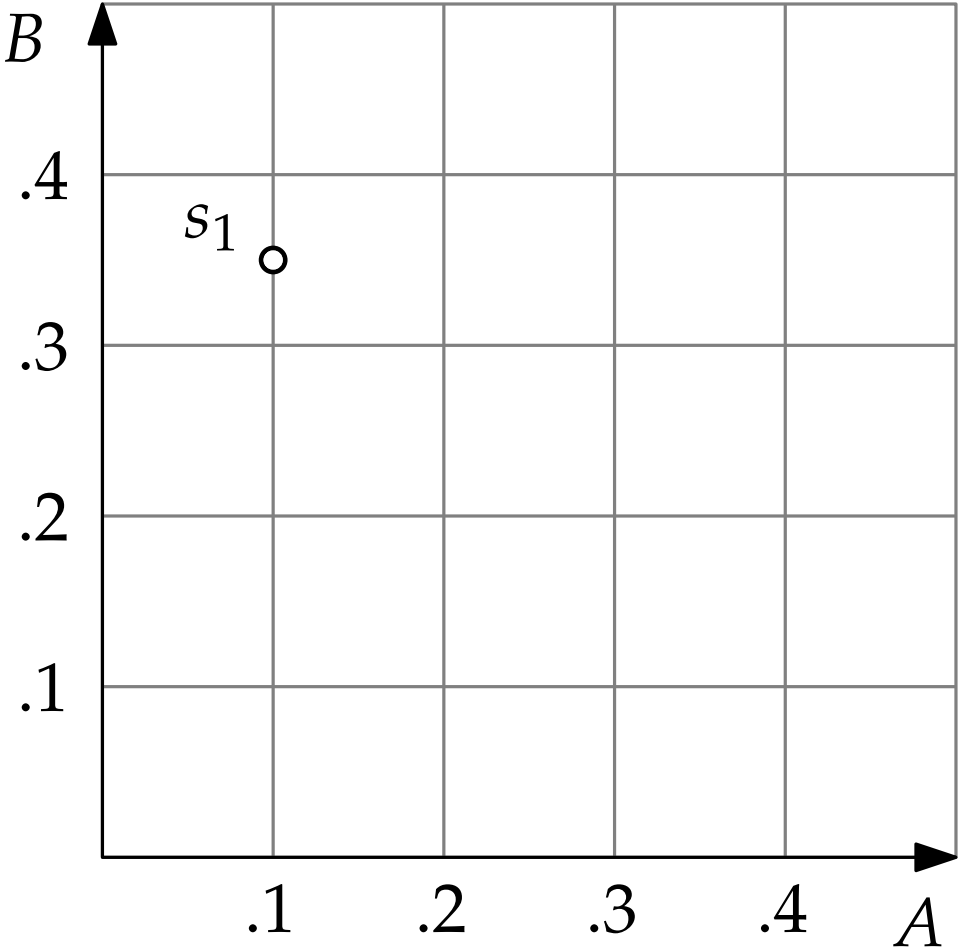
Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen		
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

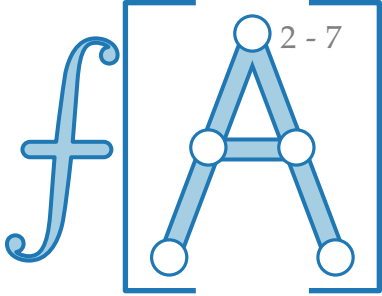
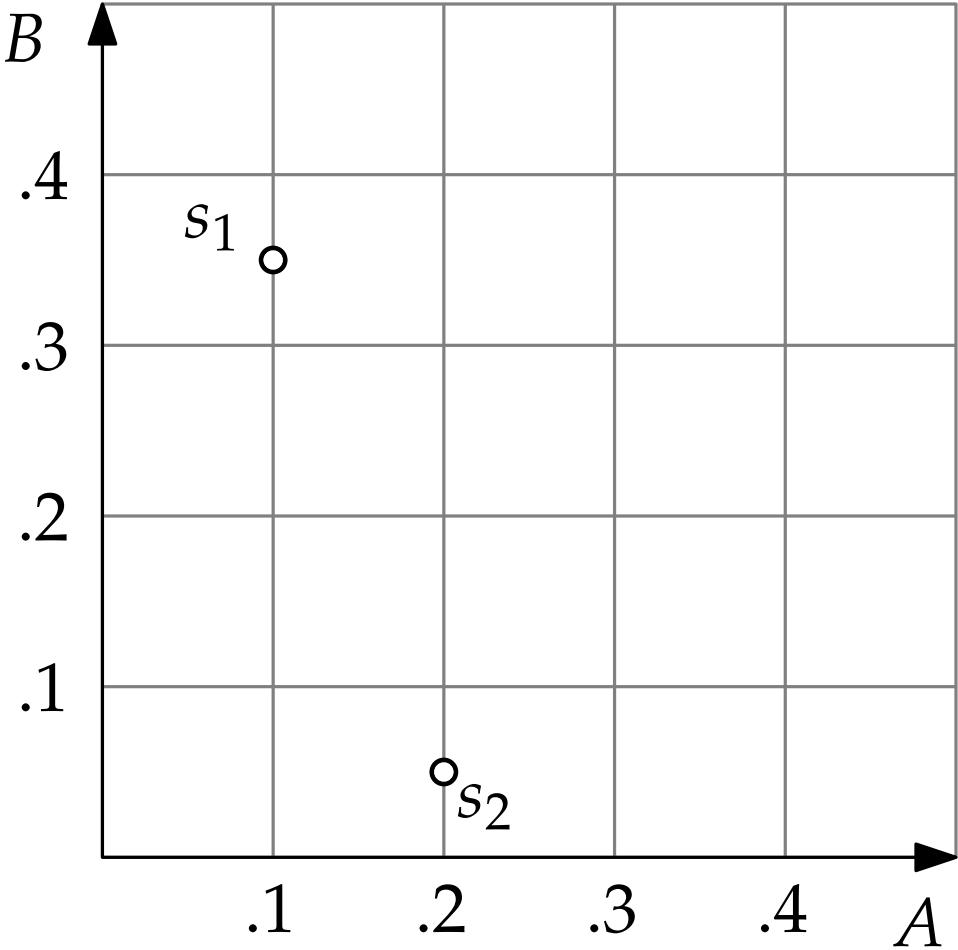
Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen		
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

Geg.

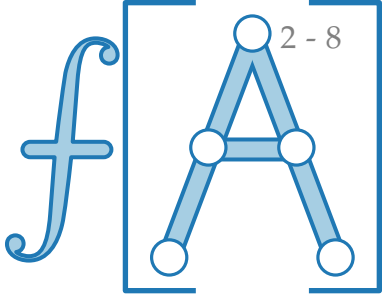
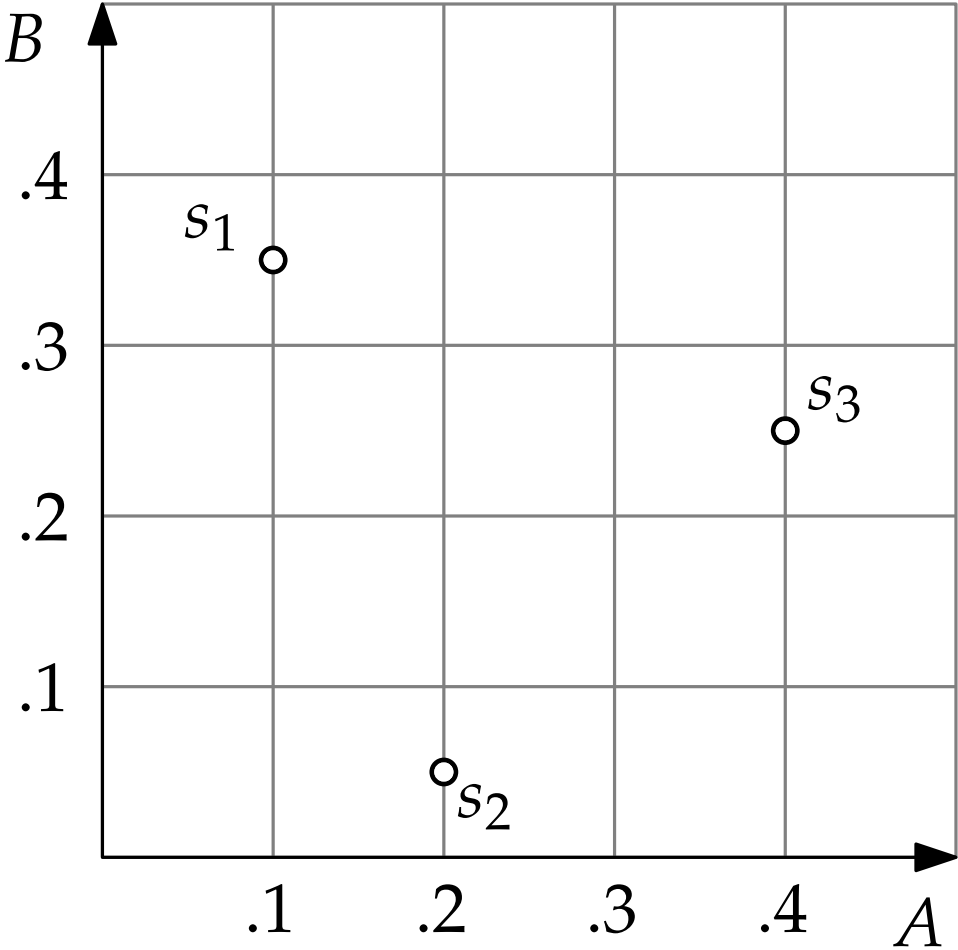
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

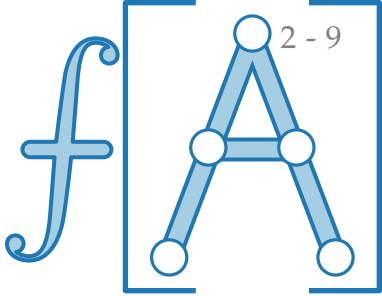
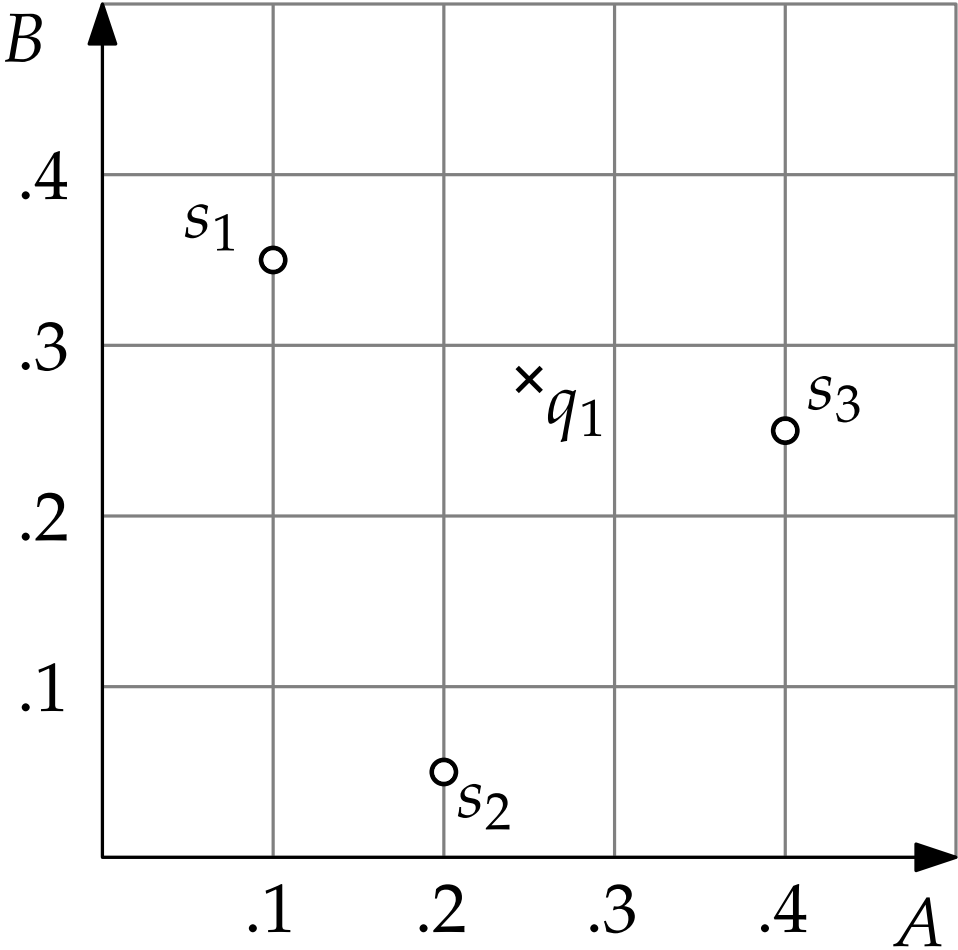
Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen		
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

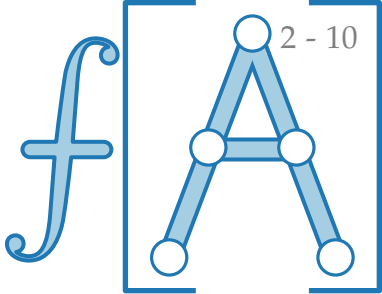
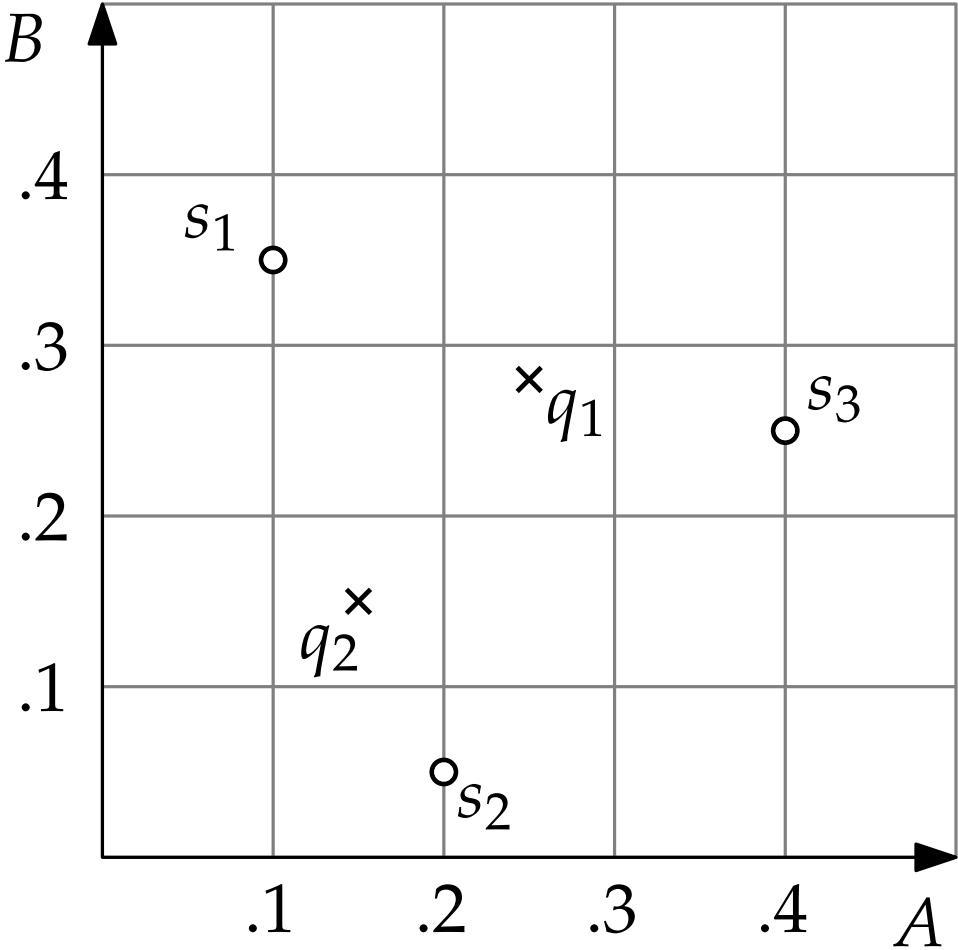
Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen		
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

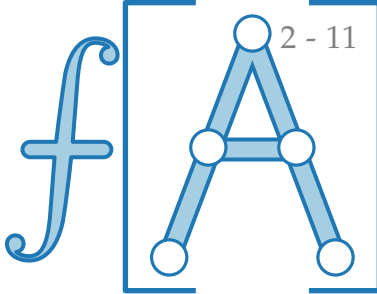
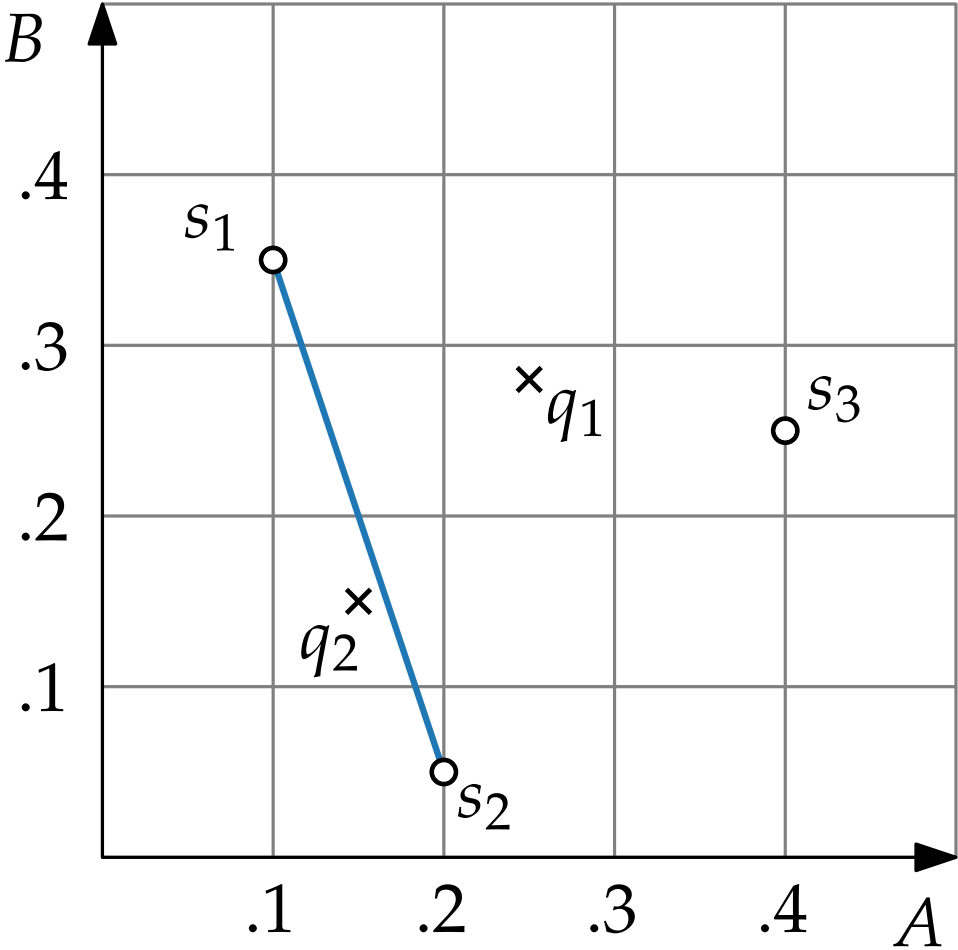
Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen		
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

Geg.

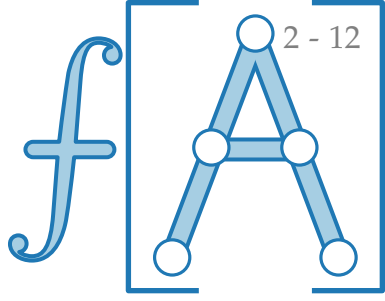
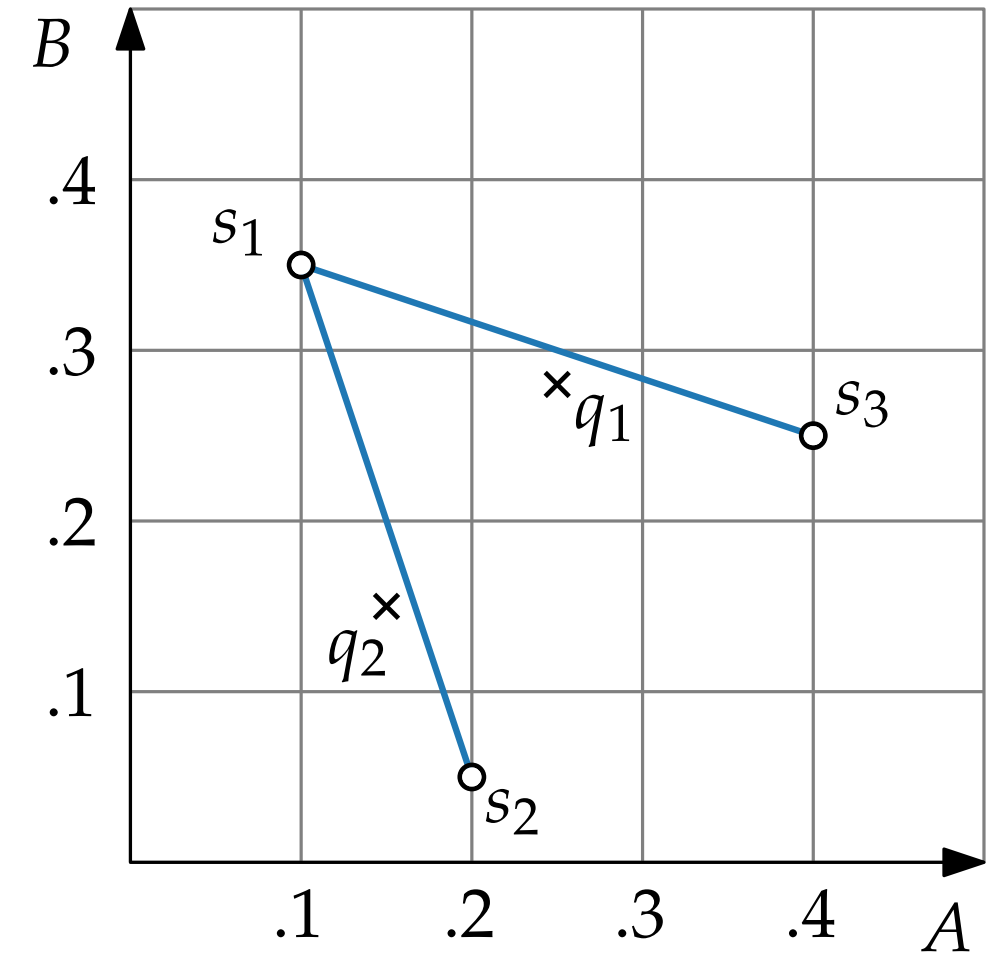
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

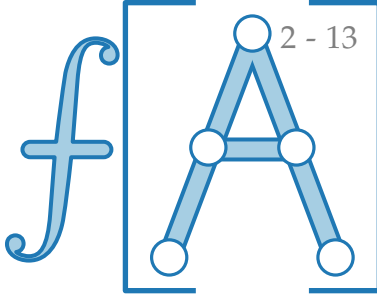
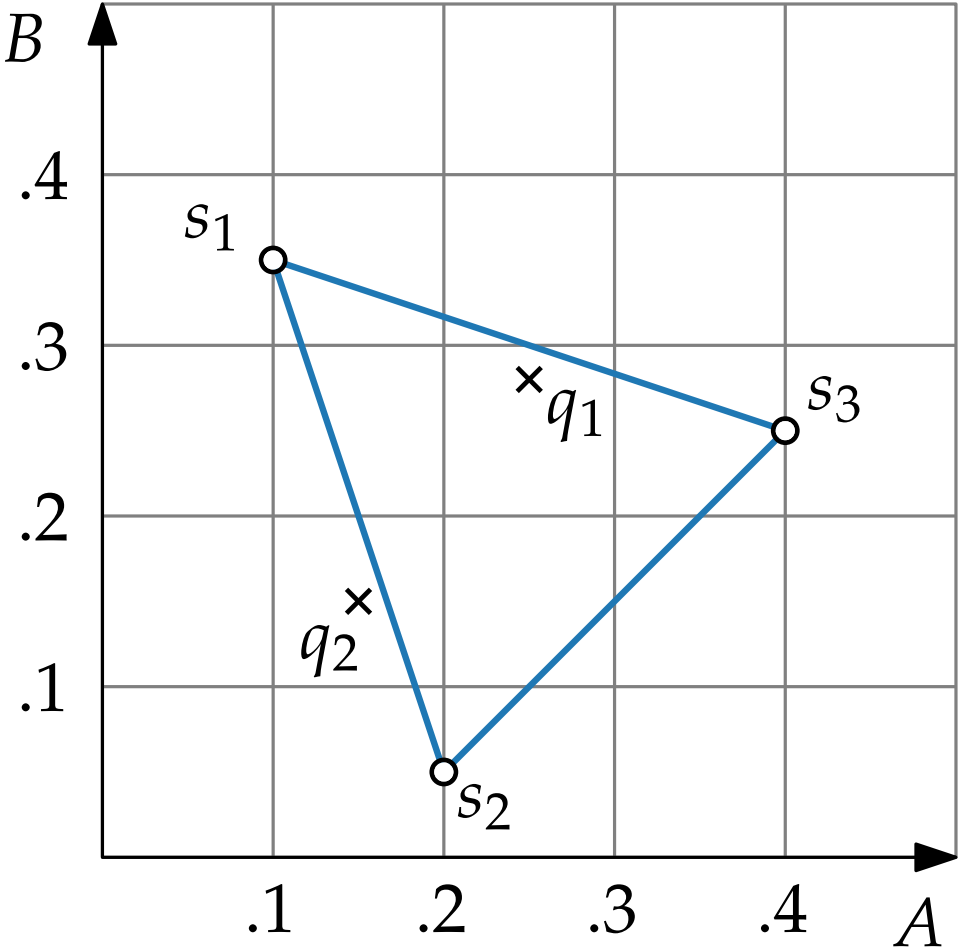
Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen		
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

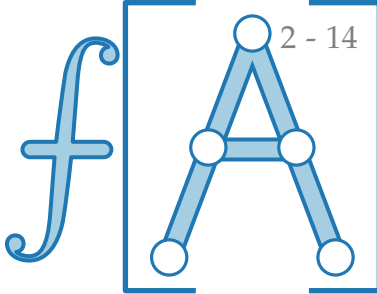
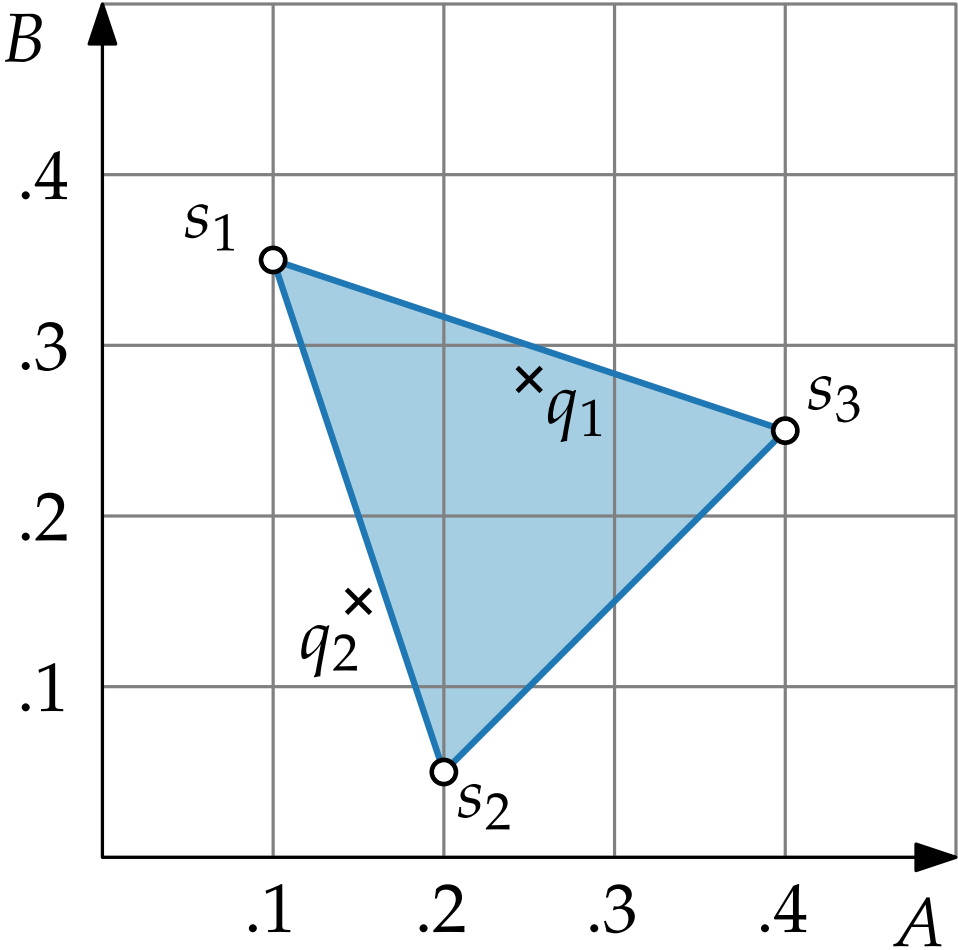
Geg.

Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen		
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Mischen von Zutaten

Geg.

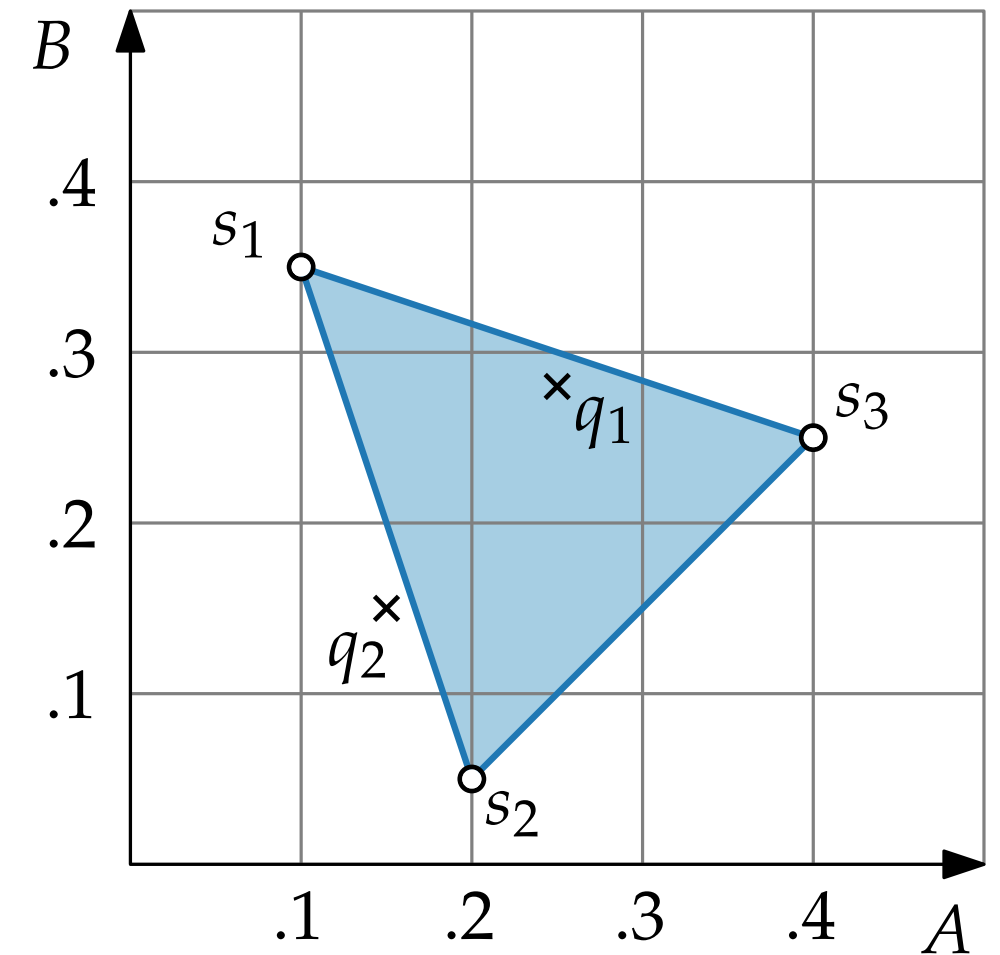
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

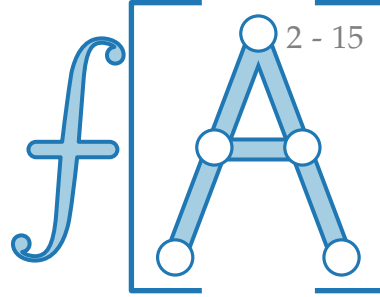
Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Beobachtung. Gegeben eine Menge $S \subset \mathbb{R}^2$ von Substanzen können wir eine Substanz $q \in \mathbb{R}^2$ aus denen in S mischen $\Leftrightarrow$



Mischen von Zutaten

Geg.

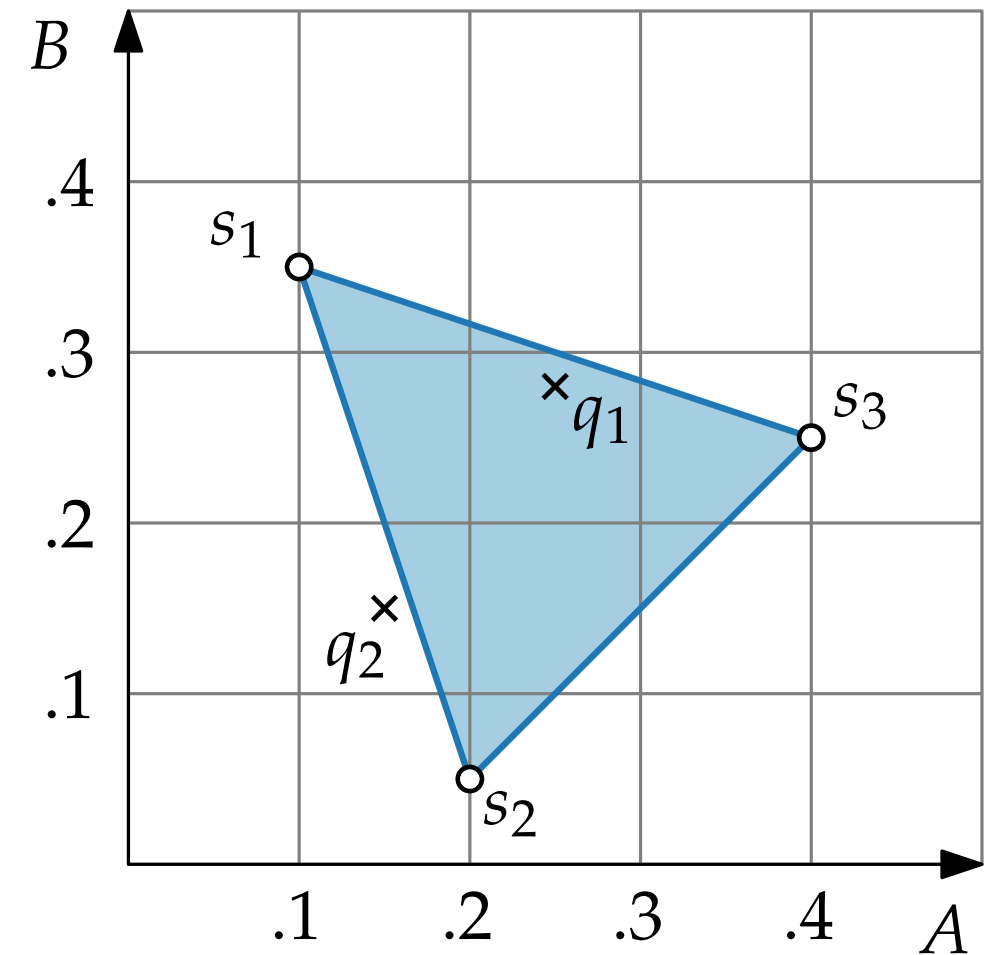
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

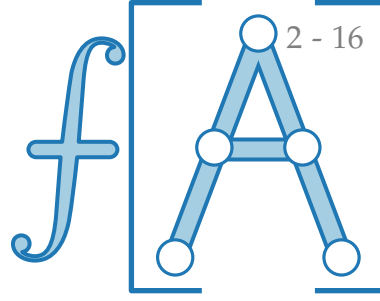
Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Beobachtung. Gegeben eine Menge $S \subset \mathbb{R}^2$ von Substanzen können wir eine Substanz $q \in \mathbb{R}^2$ aus denen in S mischen $\Leftrightarrow q \in \text{CH}(S)$.



Mischen von Zutaten

Geg.

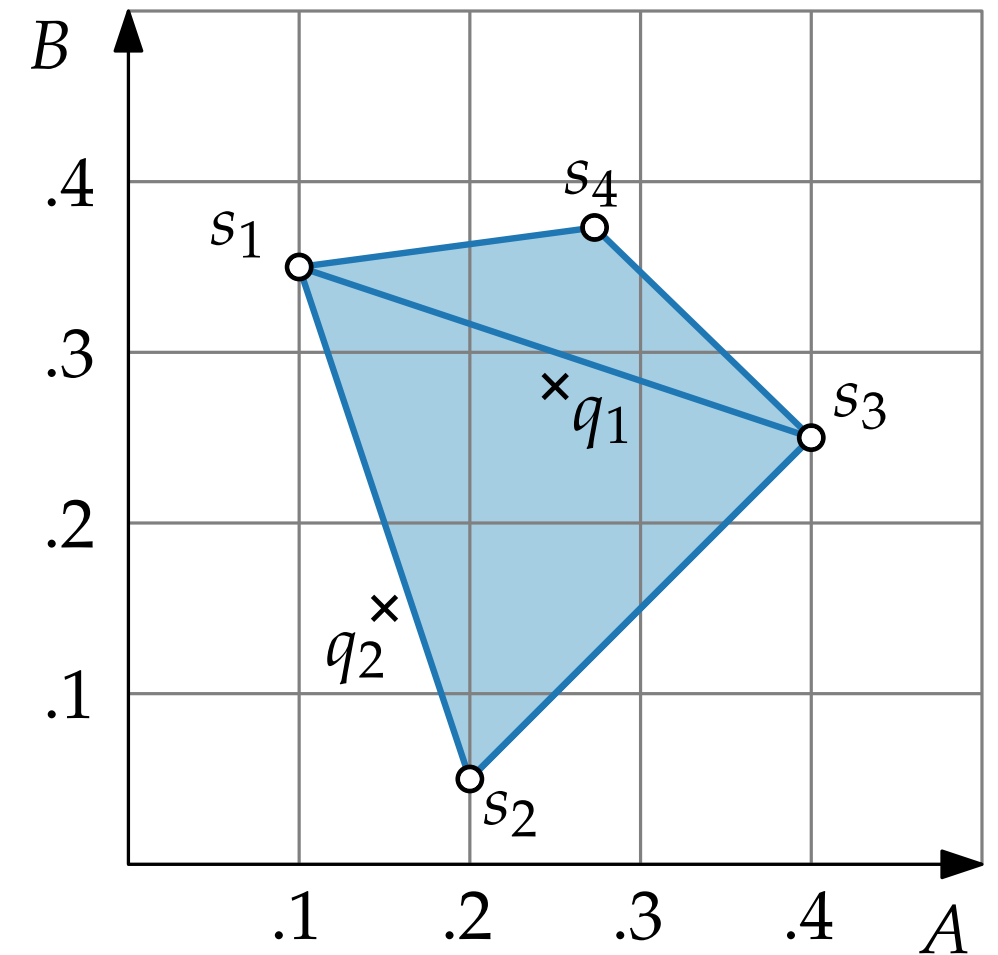
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

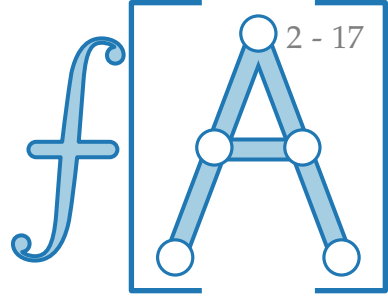
Mischungen

q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Beobachtung. Gegeben eine Menge $S \subset \mathbb{R}^2$ von Substanzen können wir eine Substanz $q \in \mathbb{R}^2$ aus denen in S mischen $\Leftrightarrow q \in \text{CH}(S)$.



Mischen von Zutaten

Geg.

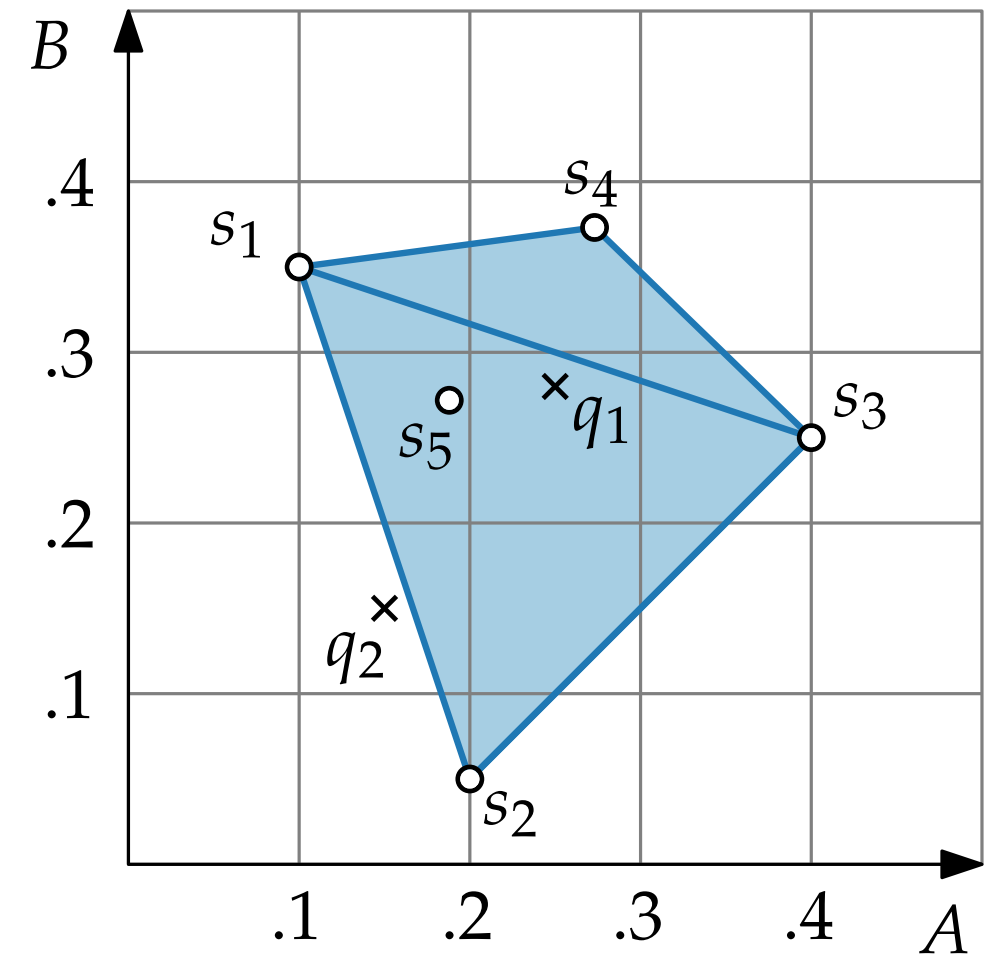
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen

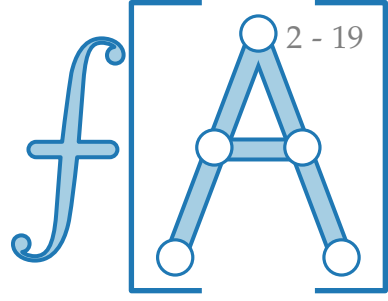
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



Beobachtung. Gegeben eine Menge $S \subset \mathbb{R}^2$ von Substanzen können wir eine Substanz $q \in \mathbb{R}^2$ aus denen in S mischen $\Leftrightarrow q \in \text{CH}(S)$.

Mischen von Zutaten



Geg.

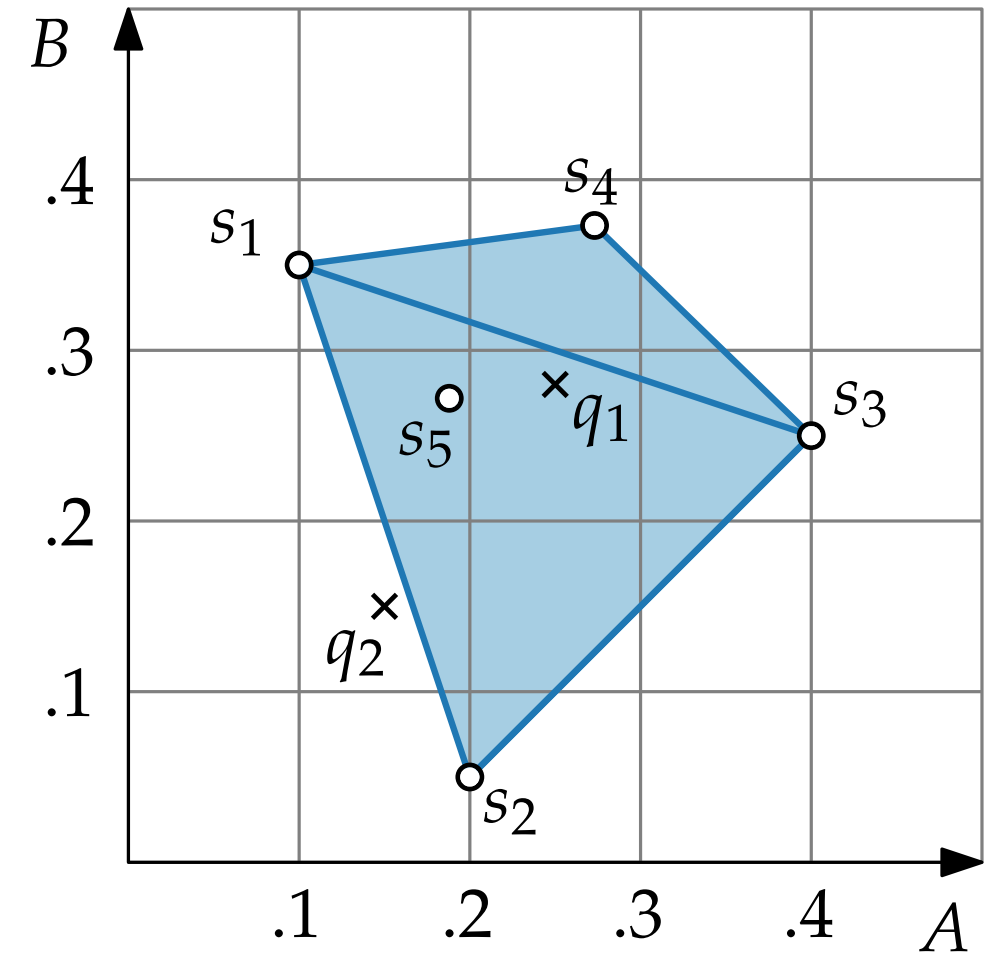
Substanz	Teil A	Teil B
s_1	10 %	35 %
s_2	20 %	5 %
s_3	40 %	25 %

Ges.

Mischungen

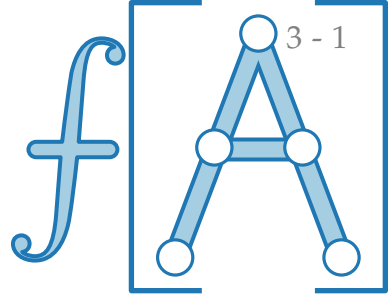
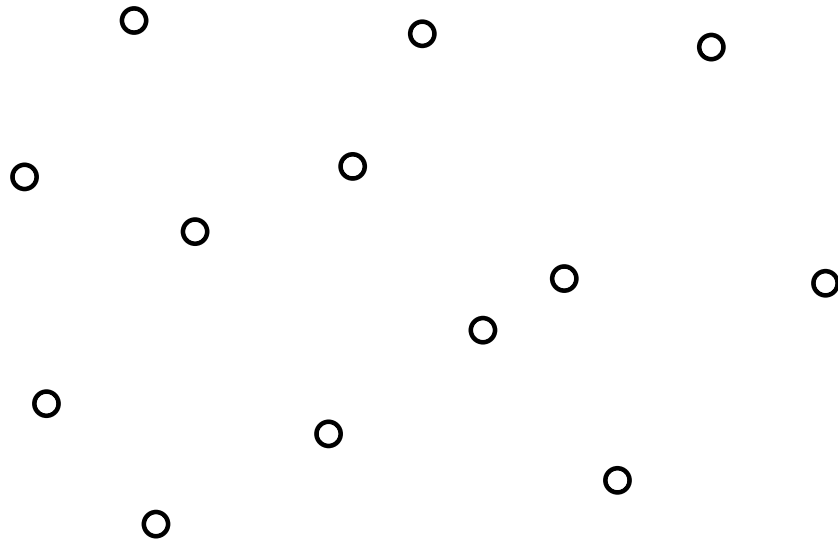
q_1	25 %	28 %
q_2	15 %	15 %

durch Substanzen s_1, s_2, s_3



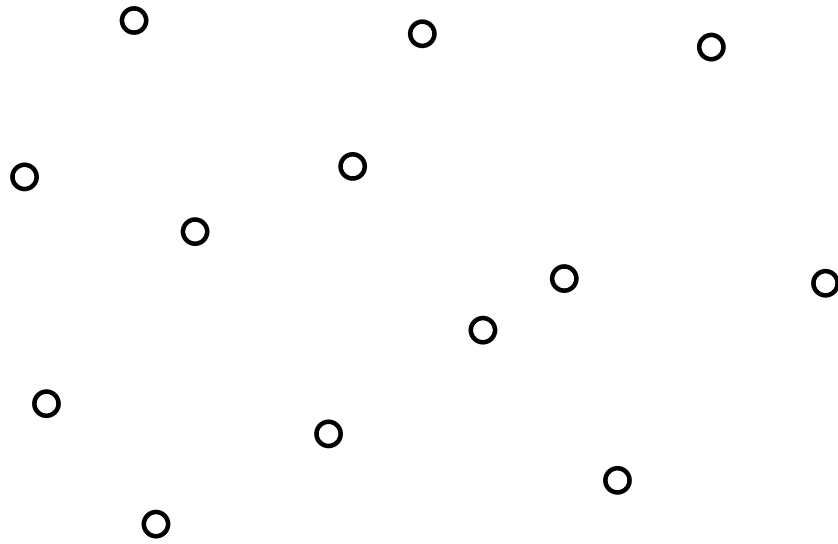
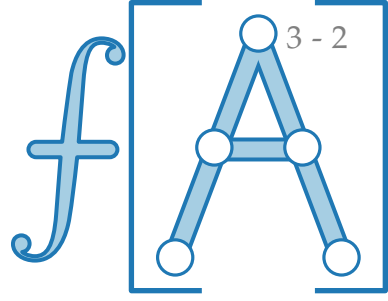
Beobachtung. Gegeben eine Menge $S \subset \mathbb{R}^d$ von Substanzen können wir eine Substanz $q \in \mathbb{R}^d$ aus denen in S mischen $\Leftrightarrow q \in \text{CH}(S)$.

Definition



Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $\text{CH}(S)$ definieren?

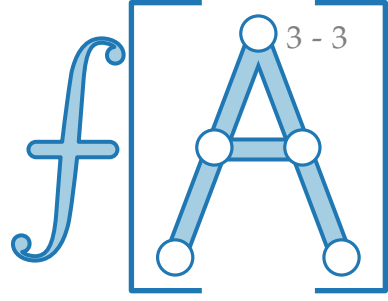
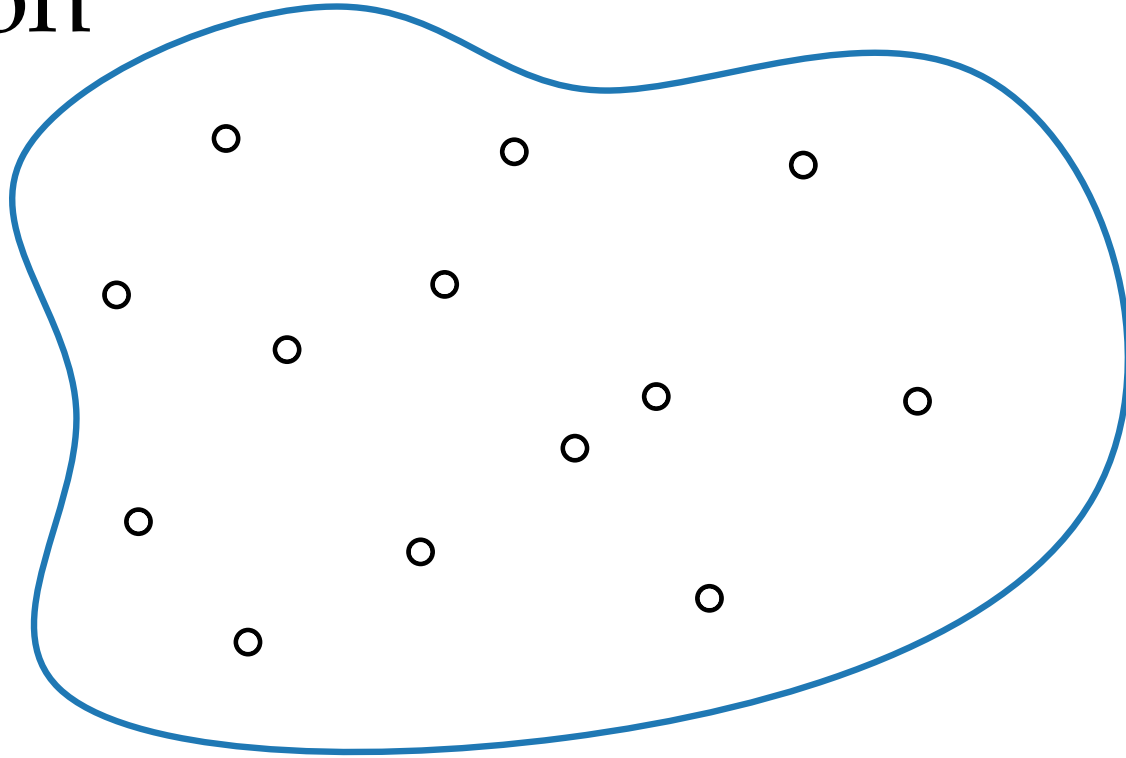
Definition



Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $\text{CH}(S)$ definieren?

Physikalischer Ansatz.

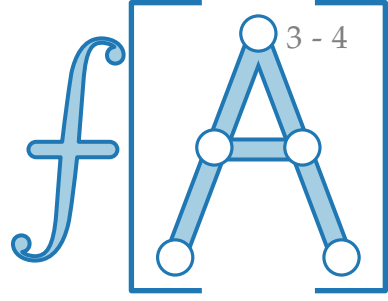
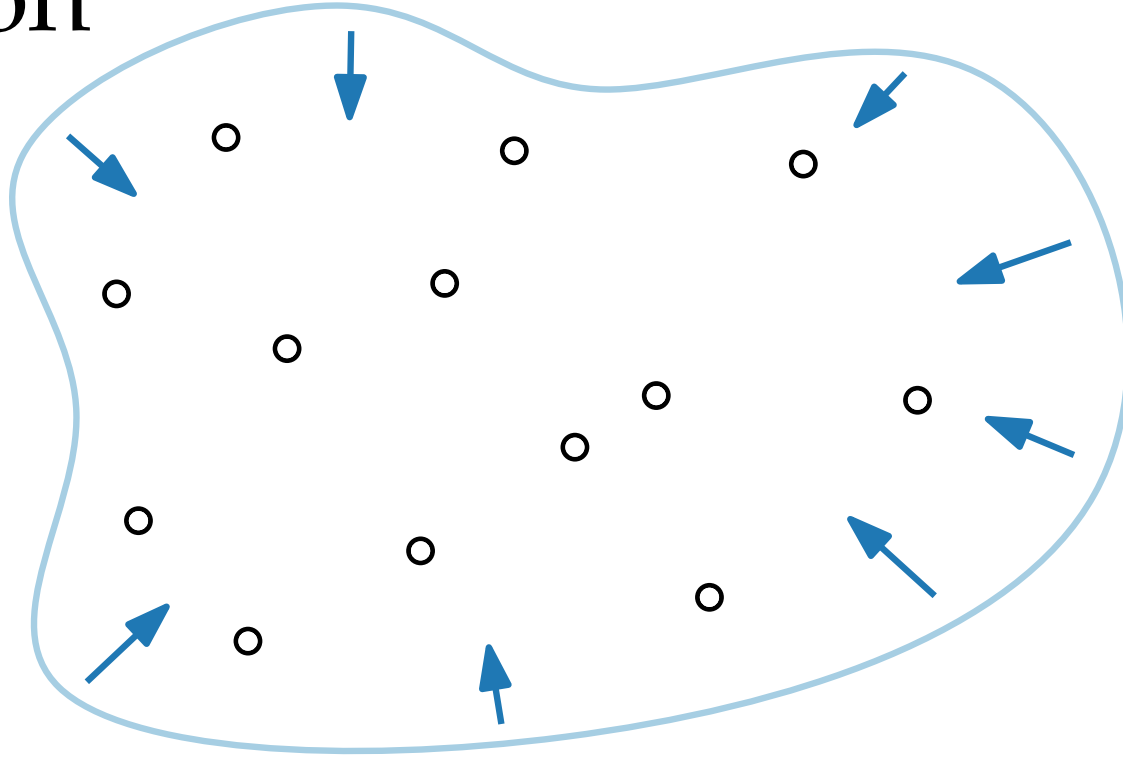
Definition



Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

Physikalischer Ansatz. ■ nehme (groß genug) elastisches **Seil**

Definition

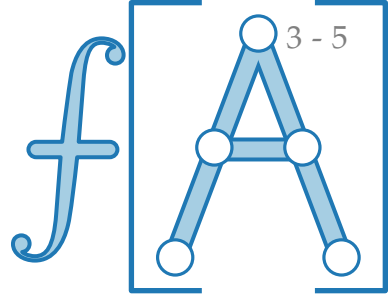
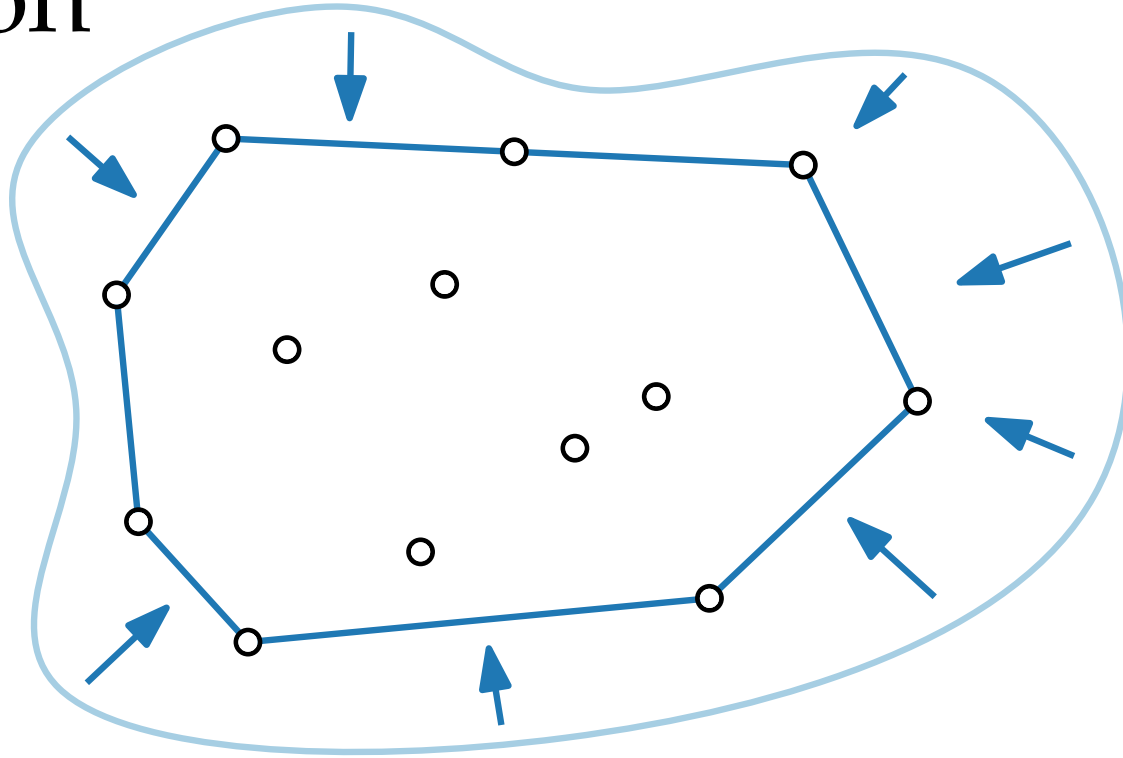


Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

Physikalischer Ansatz.

- nehme (groß genug) elastisches **Seil**
- strecke es und lasse es los

Definition

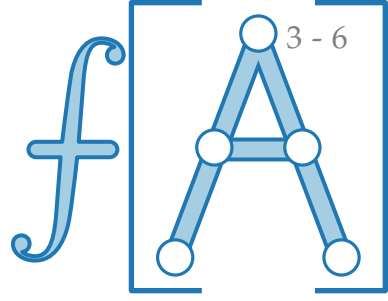
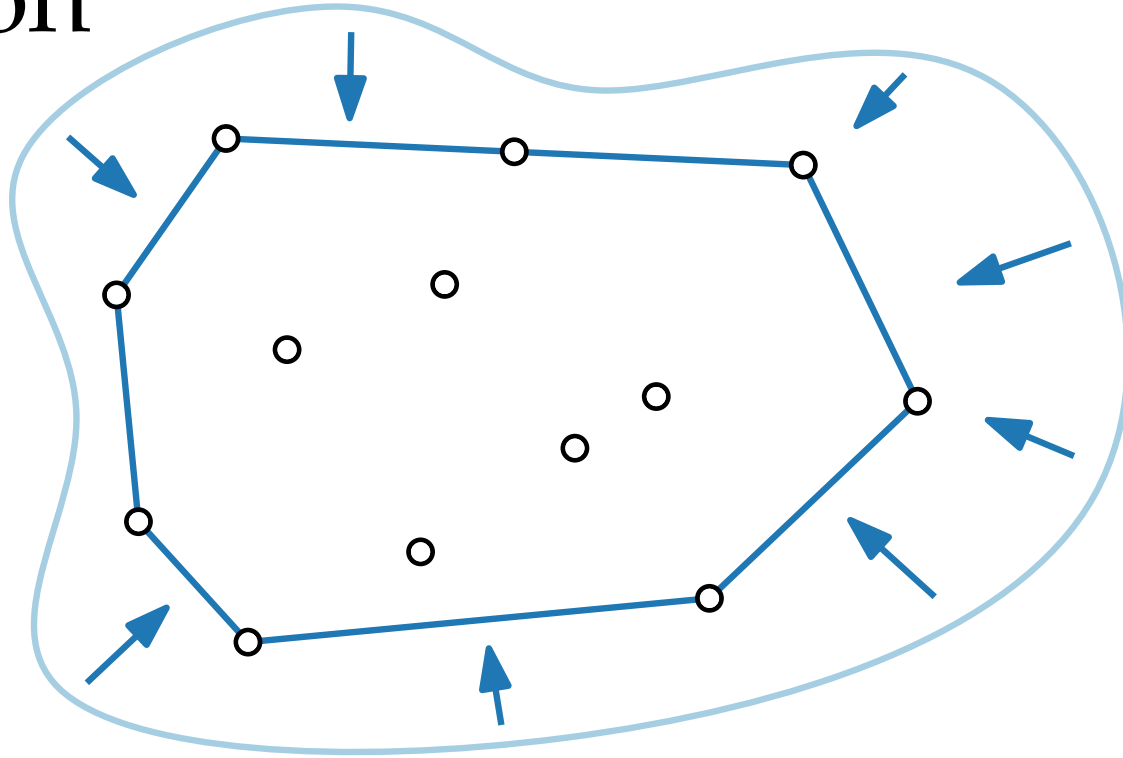


Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

Physikalischer Ansatz.

- nehme (groß genug) elastisches **Seil**
- strecke es und lasse es los

Definition

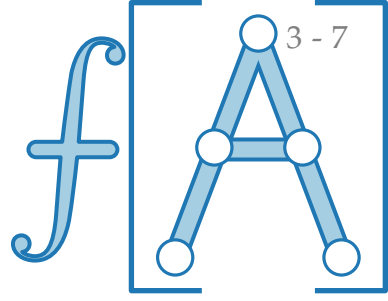
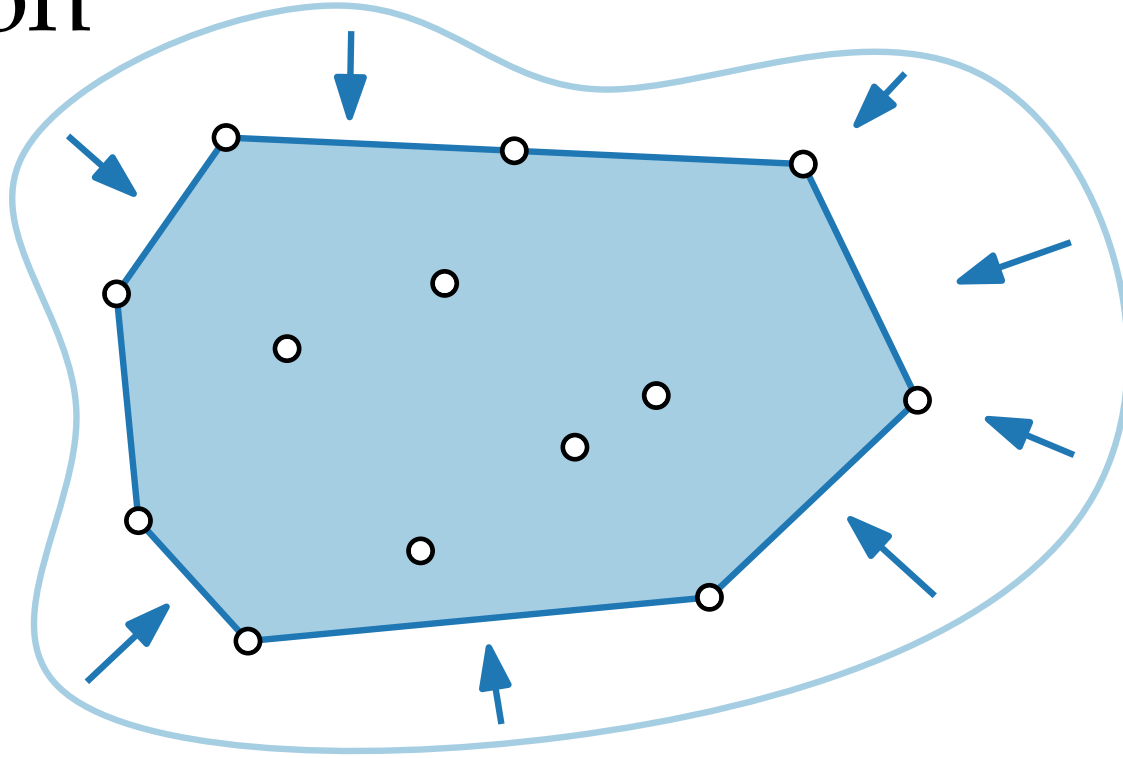


Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

Physikalischer Ansatz.

- nehme (groß genug) elastisches **Seil**
- strecke es und lasse es los
- nehme Fläche in (und auf) dem Seil

Definition

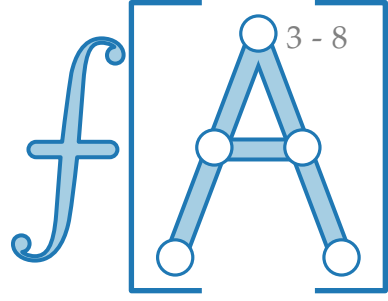
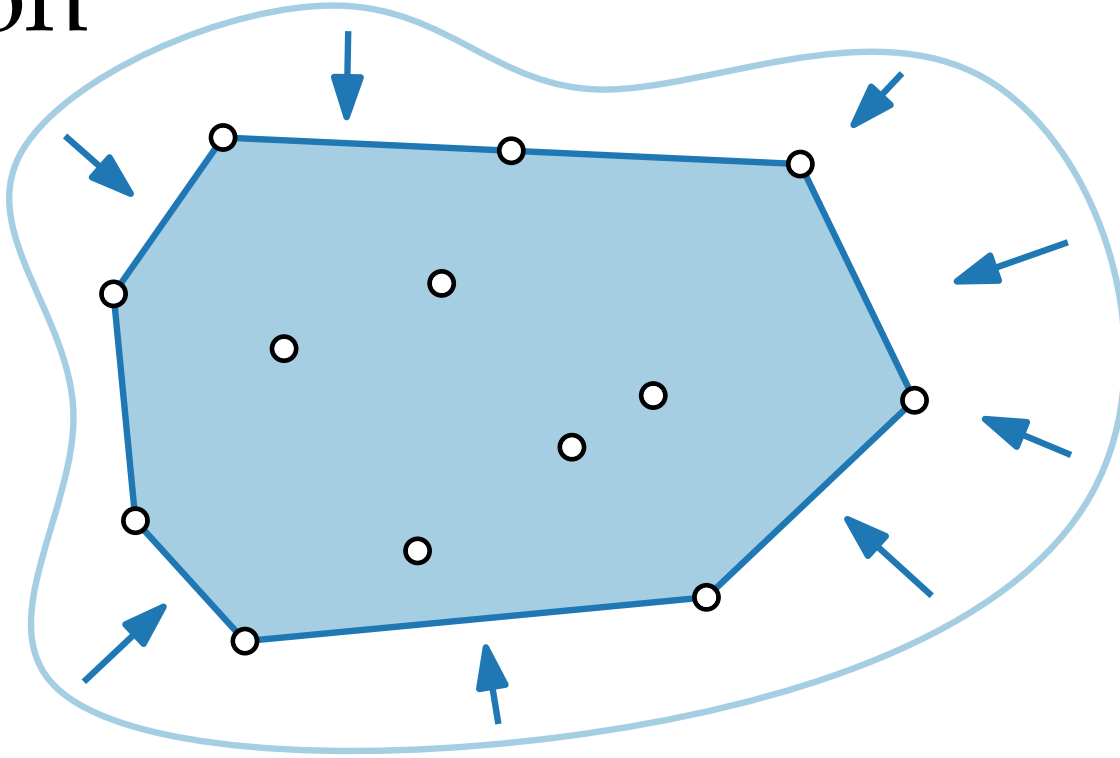


Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

Physikalischer Ansatz.

- nehme (groß genug) elastisches **Seil**
- strecke es und lasse es los
- nehme Fläche in (und auf) dem Seil

Definition



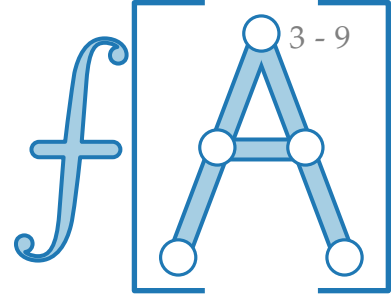
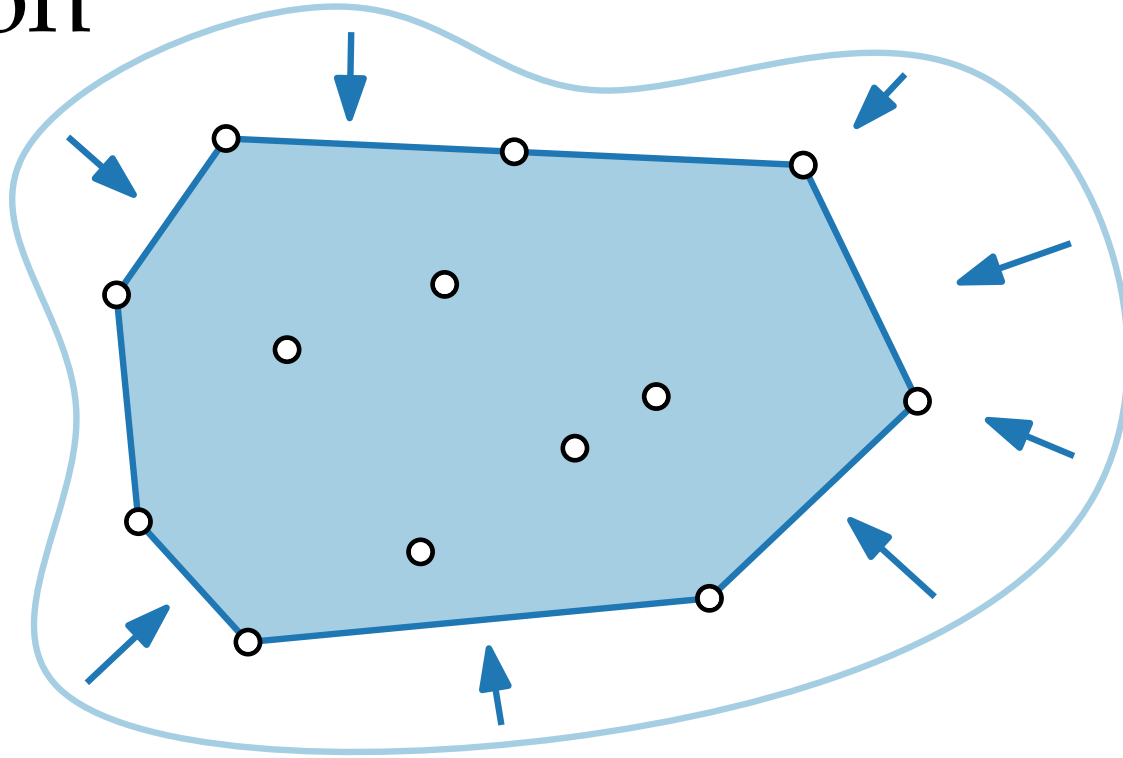
Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

Physikalischer Ansatz.

- nehme (groß genug) elastisches **Seil**
- strecke es und lasse es los
- nehme Fläche in (und auf) dem Seil

Mathematischer Ansatz.

Definition



Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

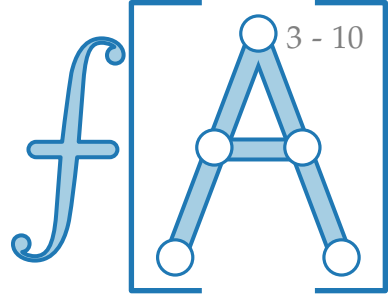
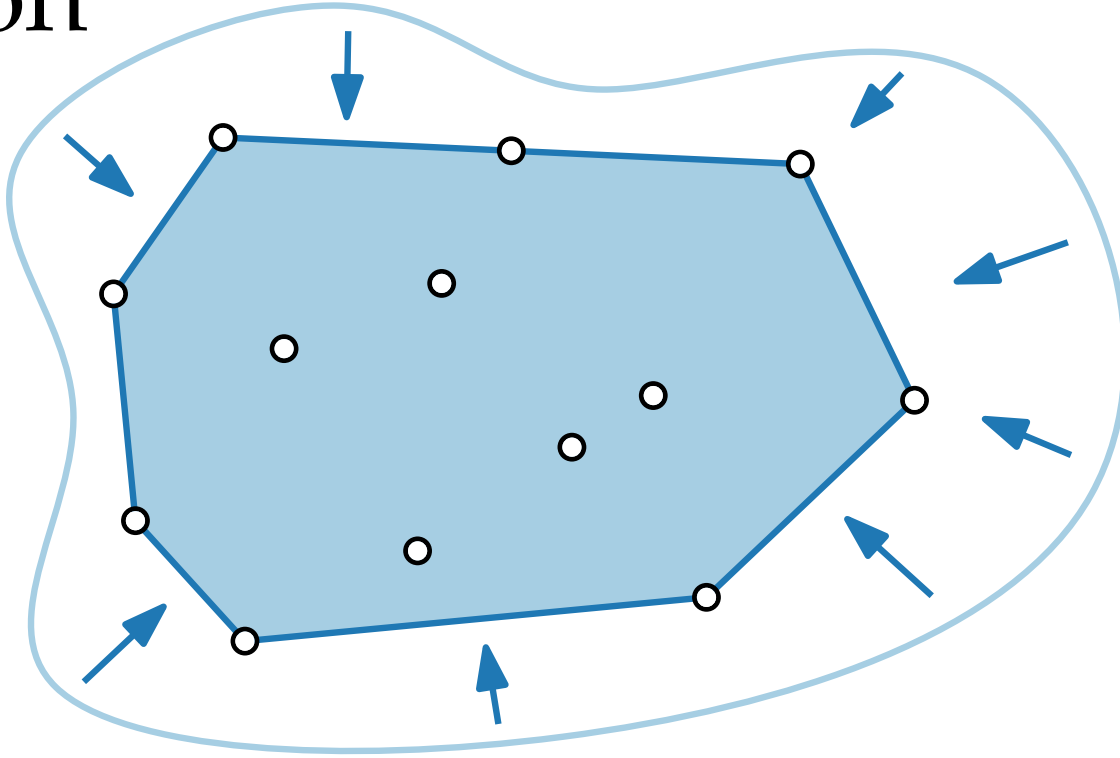
Physikalischer Ansatz.

- nehme (groß genug) elastisches **Seil**
- strecke es und lasse es los
- nehme Fläche in (und auf) dem Seil

Mathematischer Ansatz.

- definiere **konvex**

Definition



Gegeben $S \subset \mathbb{R}^2$, wie können wir die **konvexe Hülle** $CH(S)$ definieren?

Physikalischer Ansatz.

- nehme (groß genug) elastisches **Seil**
- strecke es und lasse es los
- nehme Fläche in (und auf) dem Seil

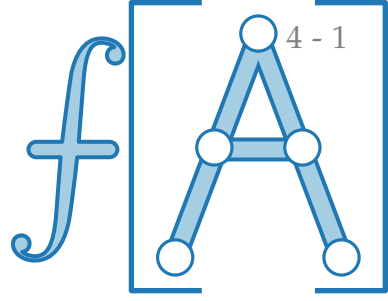
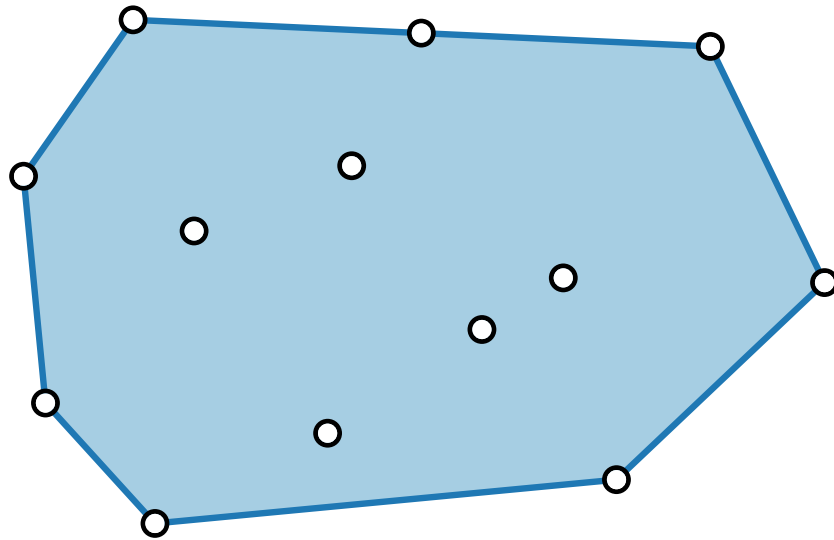
Mathematischer Ansatz.

- definiere **konvex**
- definiere $CH(S) = \bigcap_{C \supseteq S: C \text{ konvex}} C$

Berechnung

$$\text{CH}(S) \stackrel{\text{def}}{=} \bigcap_{C \supseteq S: C \text{ konvex}} C$$

Problem mit mathematischem Ansatz:

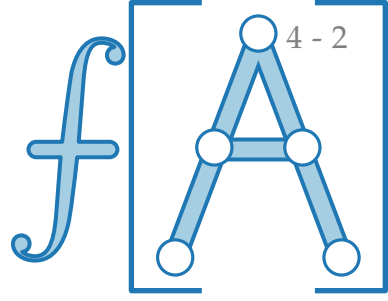
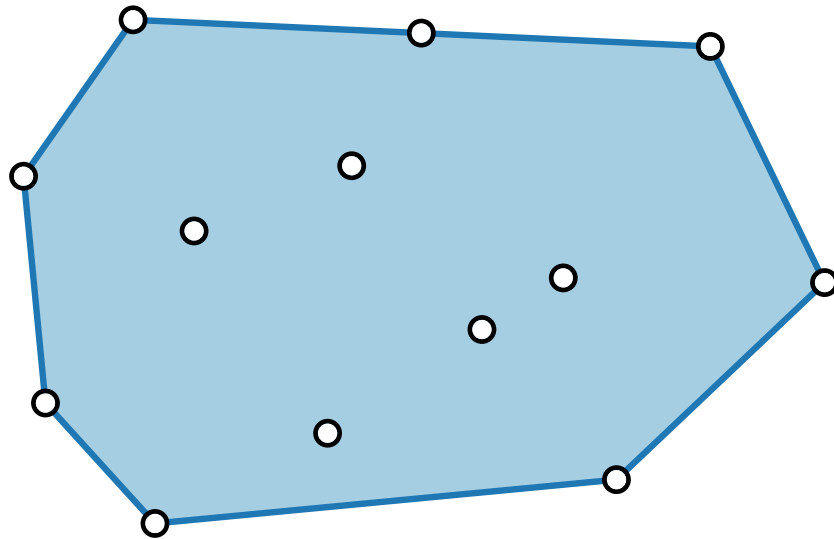


Berechnung

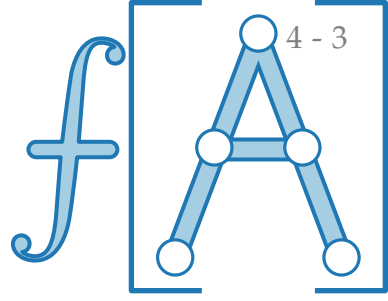
$$\text{CH}(S) \stackrel{\text{def}}{=} \bigcap_{C \supseteq S: C \text{ konvex}} C$$

Problem mit mathematischem Ansatz:

↖ Diese Menge ist **RIESIG!**

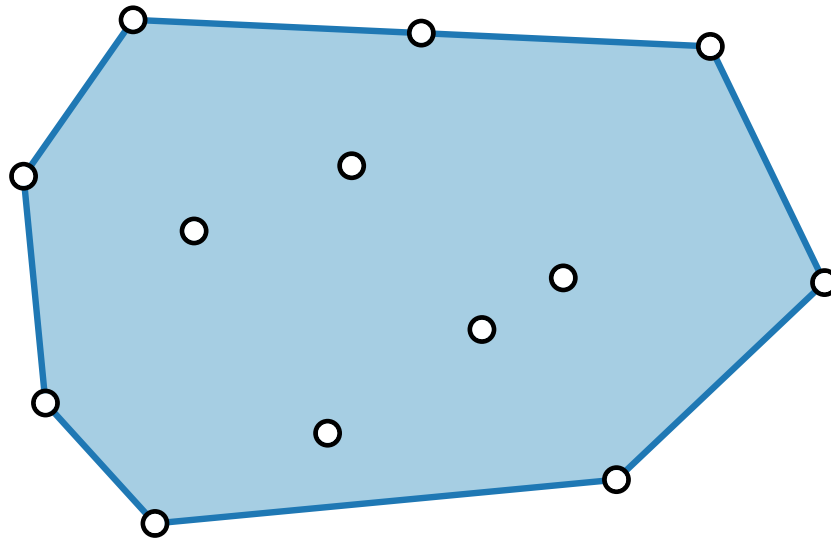


Berechnung



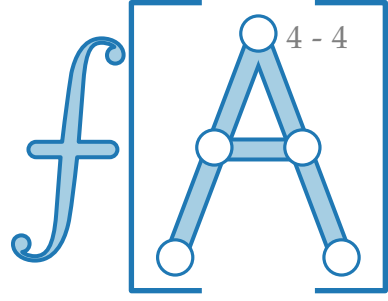
$$\text{CH}(S) \stackrel{\text{def}}{=} \bigcap_{C \supseteq S: C \text{ konvex}} C$$

Problem mit mathematischem Ansatz: — Diese Menge ist **RIESIG!**



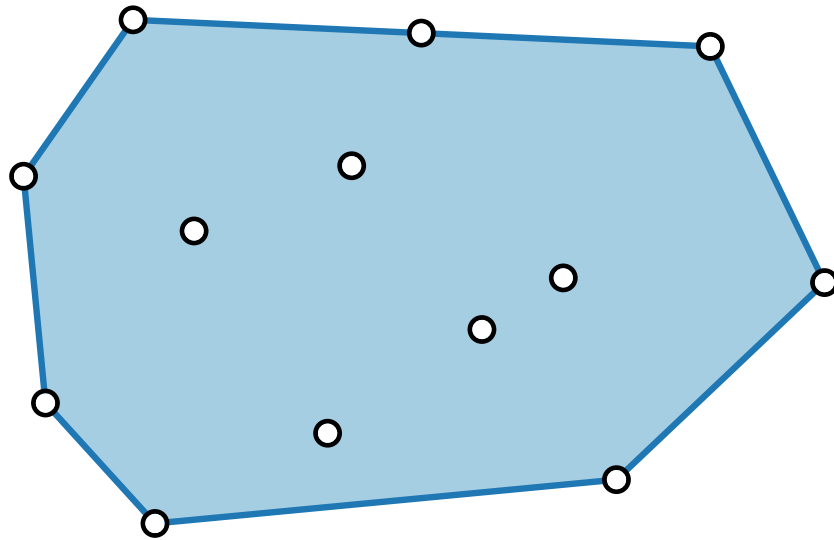
Kommen wir mit weniger Mengen aus?

Berechnung



$$\text{CH}(S) \stackrel{\text{def}}{=} \bigcap_{C \supseteq S: C \text{ konvex}} C$$

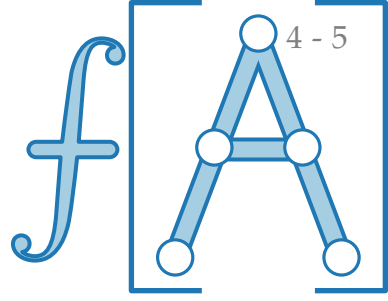
Problem mit mathematischem Ansatz: — Diese Menge ist **RIESIG!**



Kommen wir mit weniger Mengen aus?

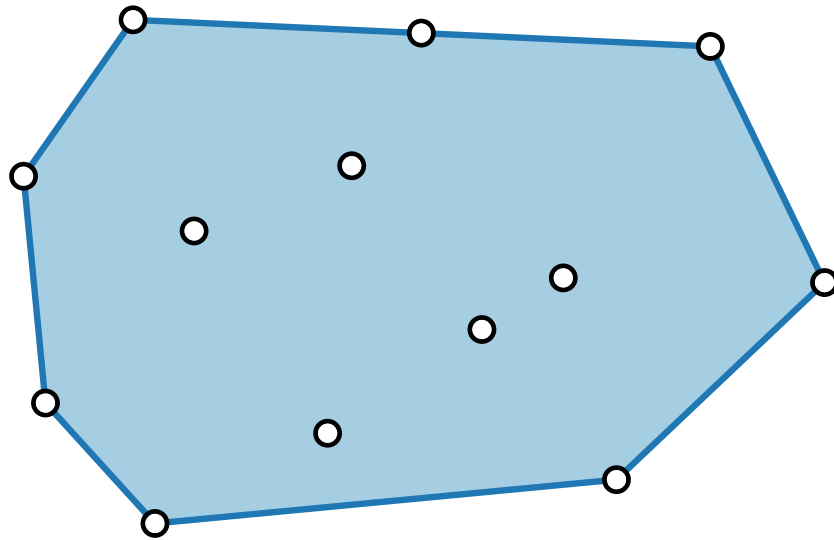
Behauptung. $\text{CH}(S) =$

Berechnung



$$\text{CH}(S) \stackrel{\text{def}}{=} \bigcap_{C \supseteq S: C \text{ konvex}} C$$

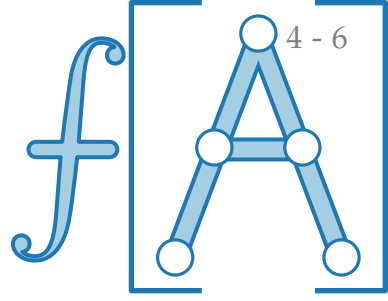
Problem mit mathematischem Ansatz: — Diese Menge ist **RIESIG!**



Kommen wir mit weniger Mengen aus?

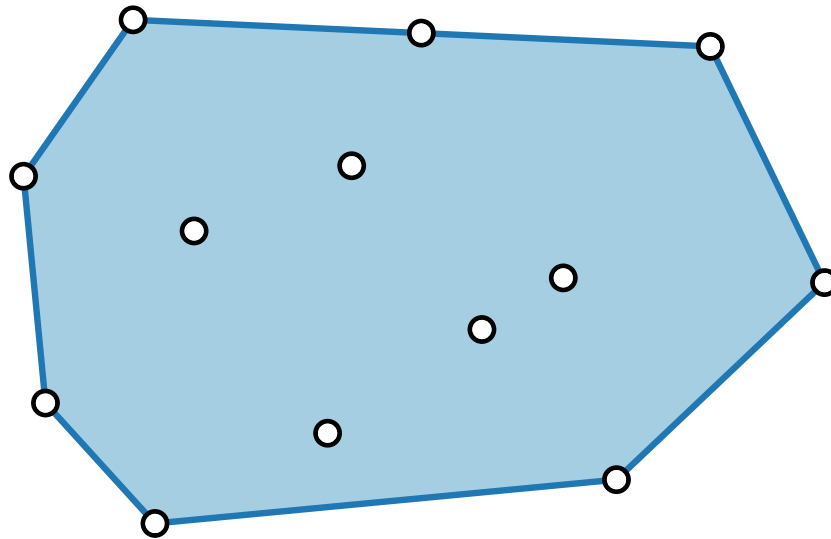
Behauptung. $\text{CH}(S) = \bigcap_{\substack{H \supseteq S: \\ H \text{ geschlossene Halbebene}}} H$

Berechnung



$$\text{CH}(S) \stackrel{\text{def}}{=} \bigcap_{C \supseteq S: C \text{ konvex}} C$$

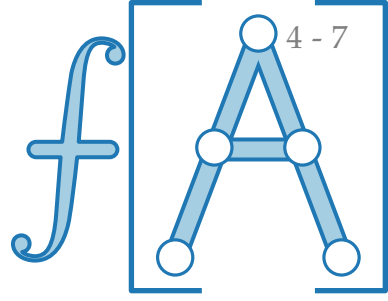
Problem mit mathematischem Ansatz: Diese Menge ist **RIESIG!**



Kommen wir mit weniger Mengen aus?

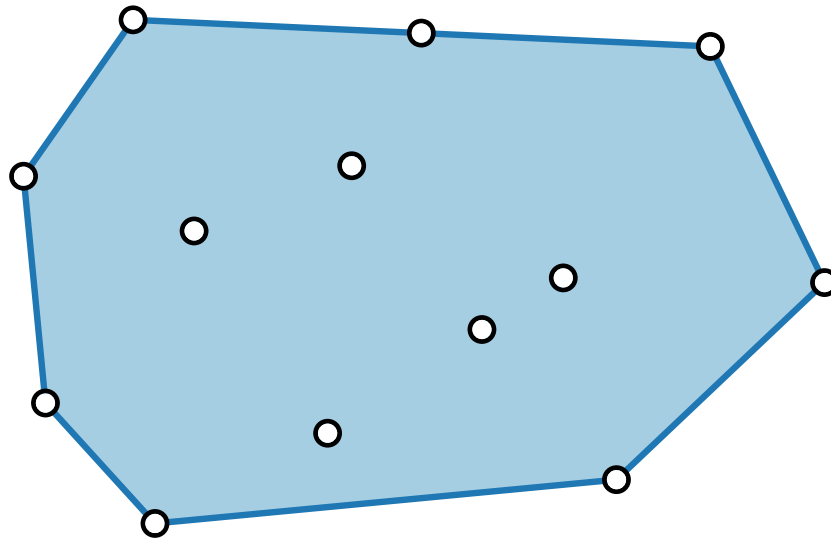
Behauptung. $\text{CH}(S) = \bigcap_{\substack{H \supseteq S: \\ H \text{ geschlossene Halbebene}}} H =$

Berechnung



$$\text{CH}(S) \stackrel{\text{def}}{=} \bigcap_{C \supseteq S: C \text{ konvex}} C$$

Problem mit mathematischem Ansatz: Diese Menge ist **RIESIG!**



Kommen wir mit weniger Mengen aus?

Behauptung. $\text{CH}(S) = \bigcap_{\substack{H \supseteq S: \\ H \text{ geschlossene Halbebene}}} H = \bigcap_{\substack{H \supseteq S: H \text{ geschl. Halbebene,} \\ |\partial H \cap S| \geq 2}} H$

Rand

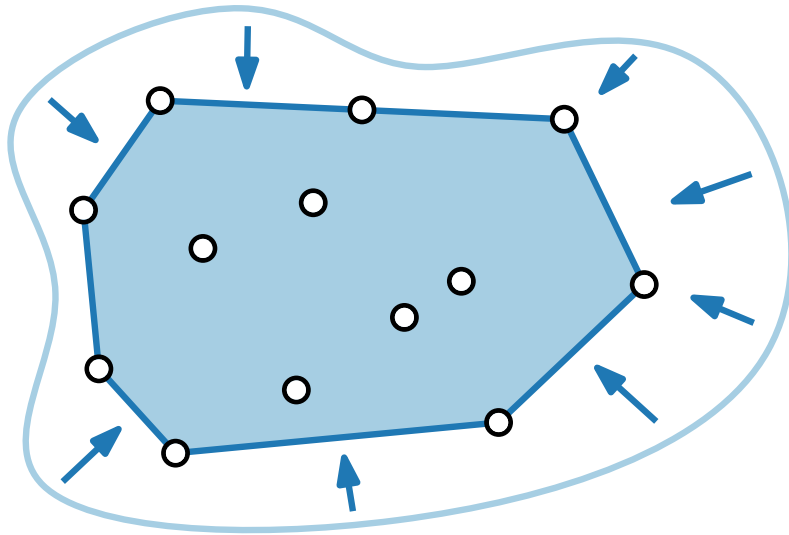
Fortgeschrittene Algorithmen

12. Vorlesung

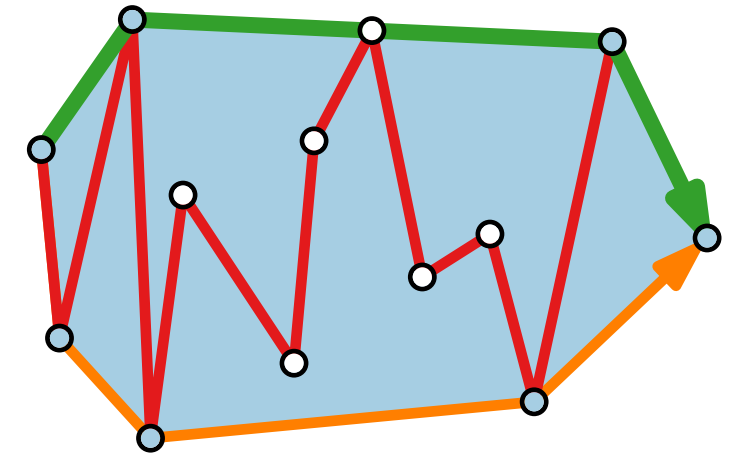
Algorithmische Geometrie I:

Konvexe Hülle

Teil II: Algorithmischer Ansatz



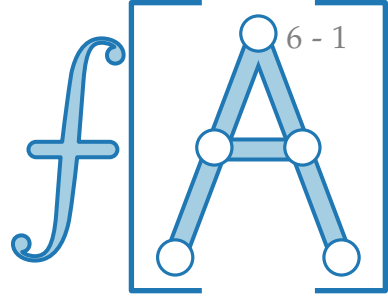
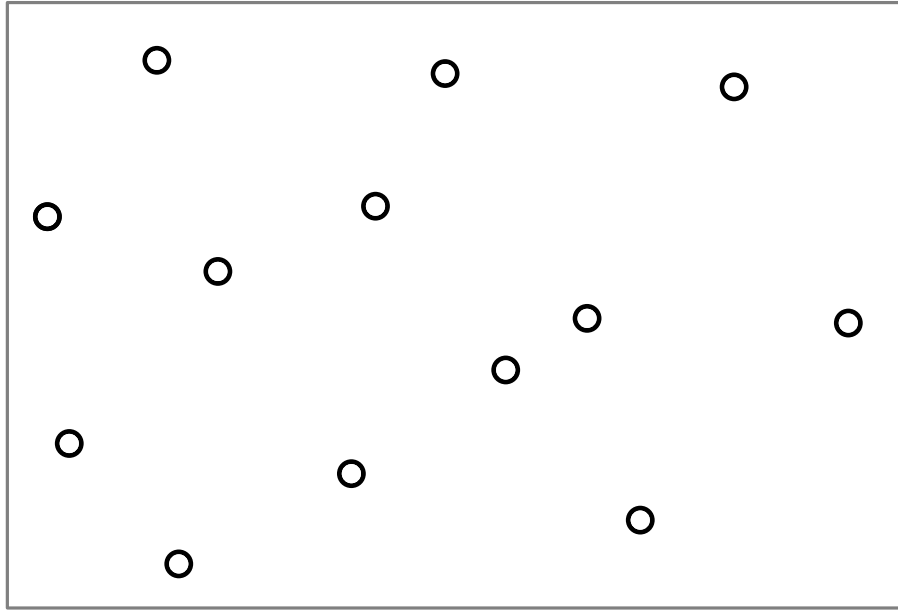
Philipp Kindermann



Wintersemester 2025/26

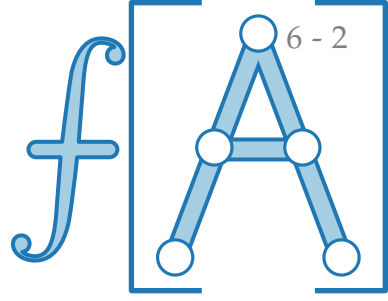
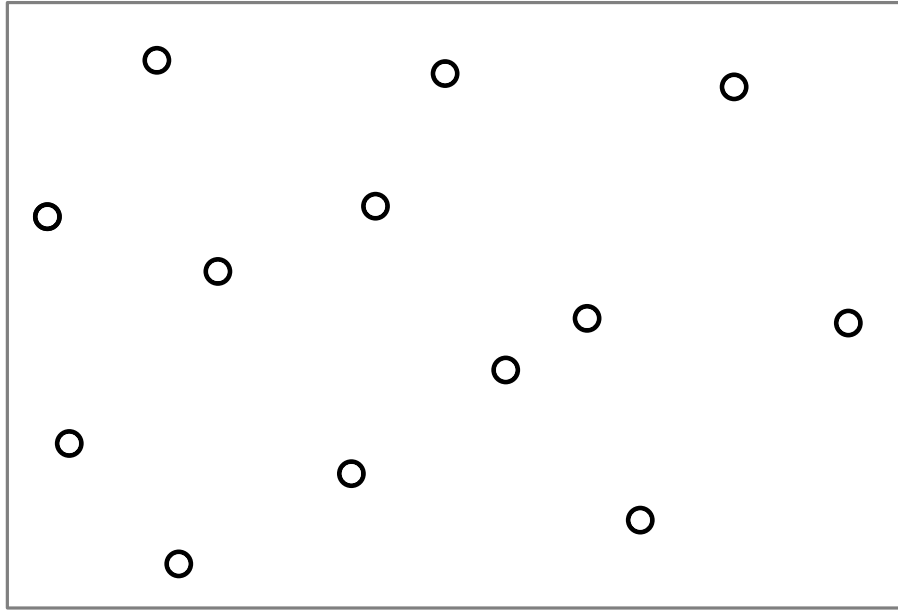
Algorithmischer Ansatz

Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



Algorithmischer Ansatz

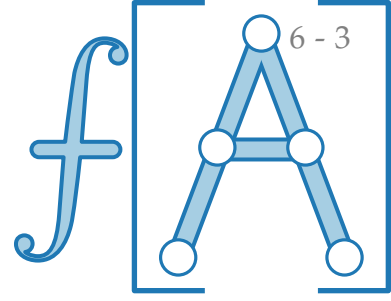
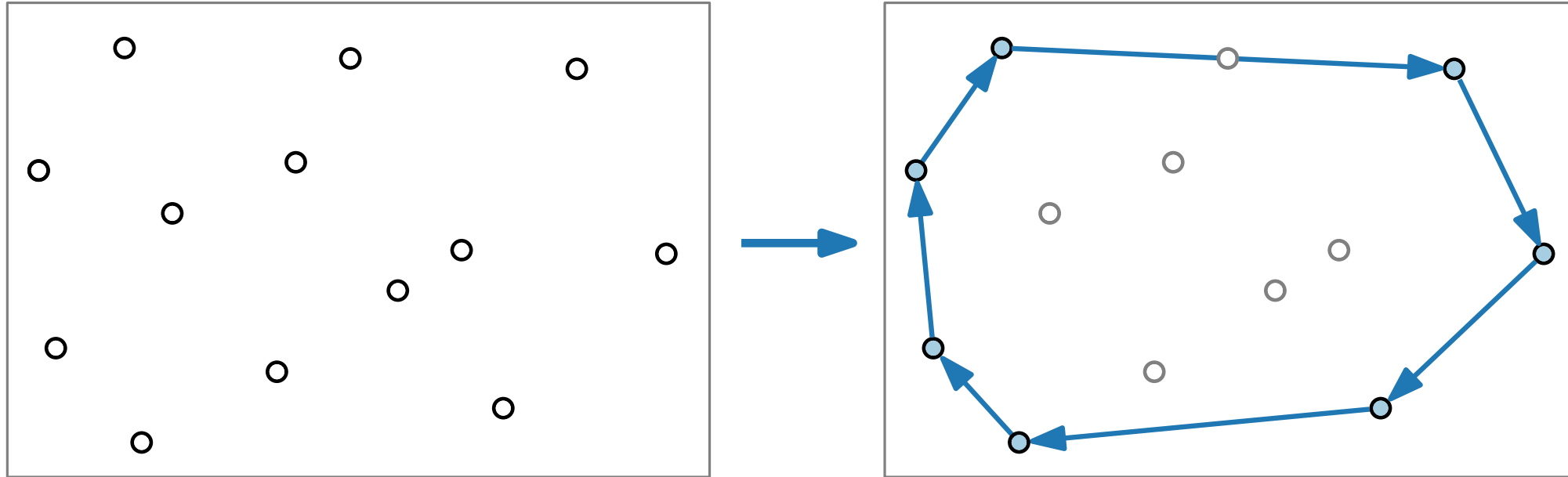
Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



Algorithmischer Ansatz

Geg.

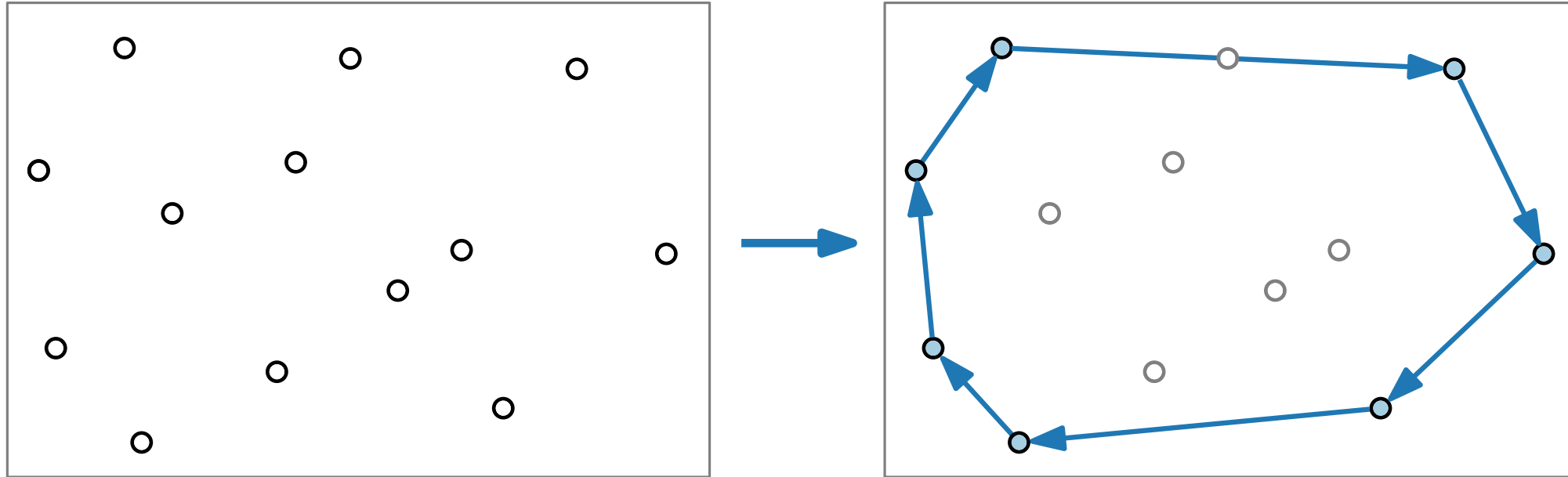
Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



Algorithmischer Ansatz

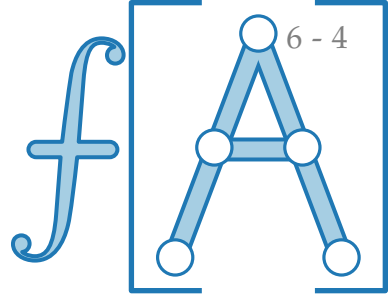
Geg.

Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



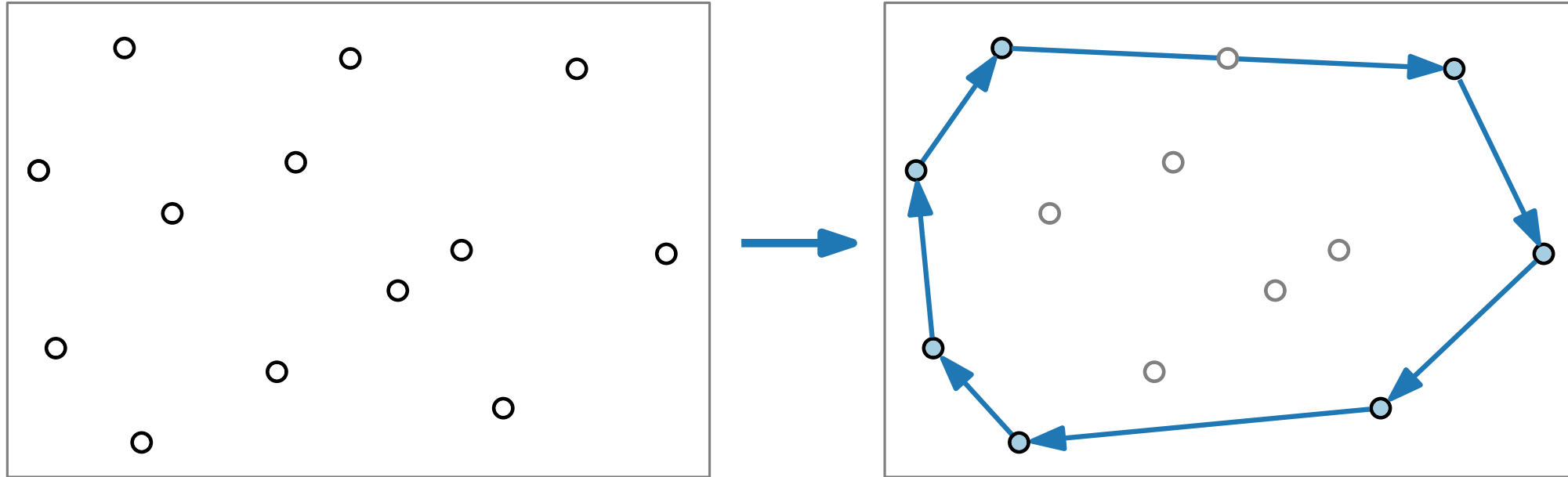
Ges.

Liste der Knoten auf Rand von $CH(S)$ im Uhrzeigersinn



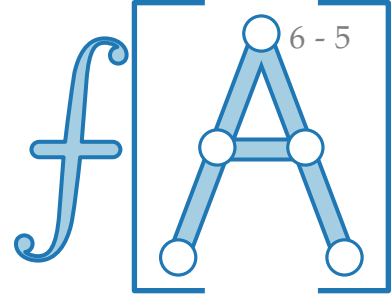
Algorithmischer Ansatz

Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$

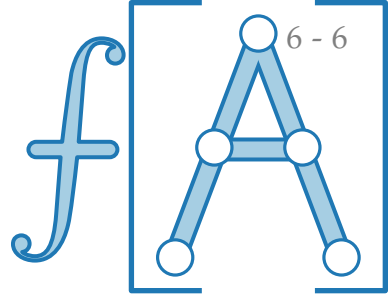


Ges. Liste der Knoten auf Rand von $\text{CH}(S)$ im Uhrzeigersinn

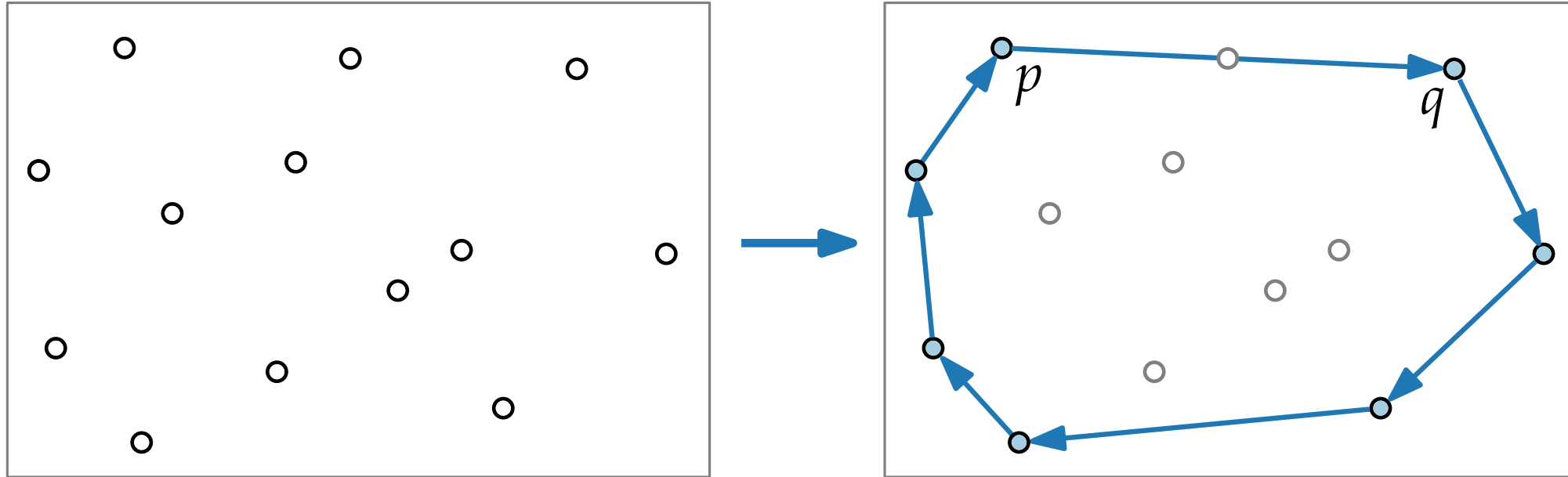
Beobachtung.



Algorithmischer Ansatz



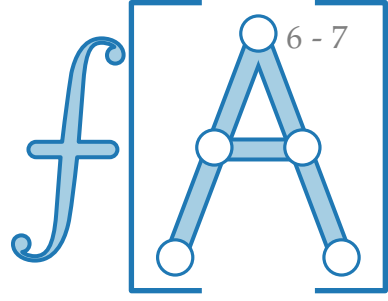
Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



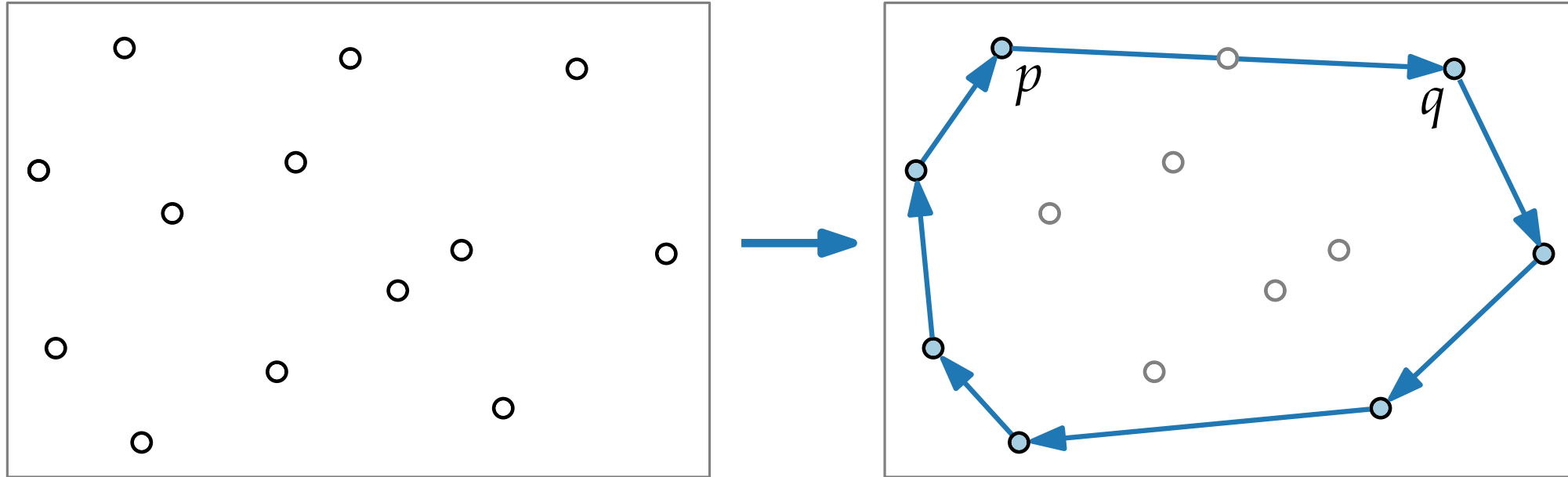
Ges. Liste der Knoten auf Rand von $\text{CH}(S)$ im Uhrzeigersinn

Beobachtung. (p, q) ist eine Kante auf $\text{CH}(S) \Leftrightarrow$

Algorithmischer Ansatz



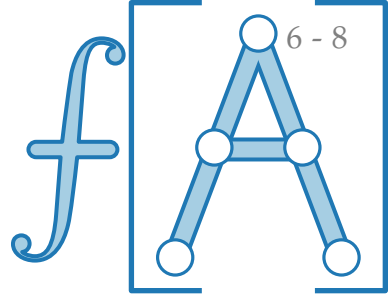
Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



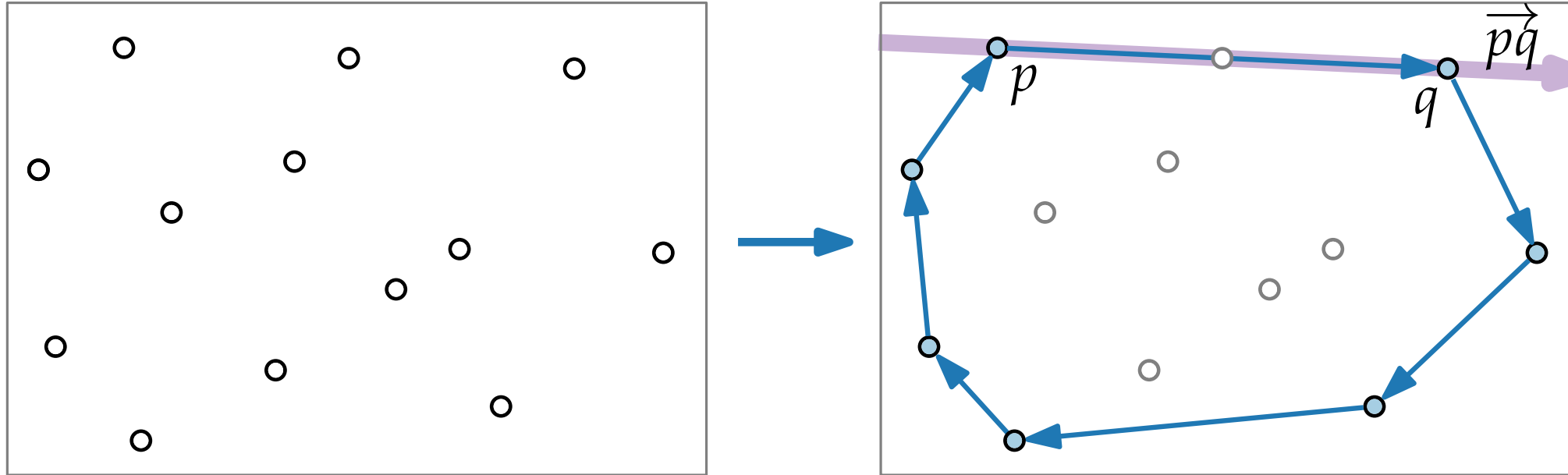
Ges. Liste der Knoten auf Rand von $\text{CH}(S)$ im Uhrzeigersinn

Beobachtung. (p, q) ist eine Kante auf $\text{CH}(S) \Leftrightarrow$
jeder Punkt in S liegt

Algorithmischer Ansatz



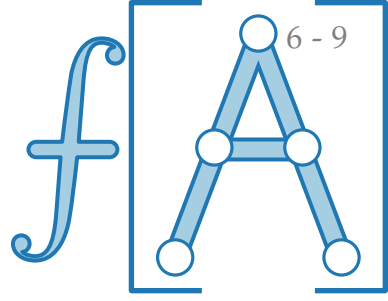
Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



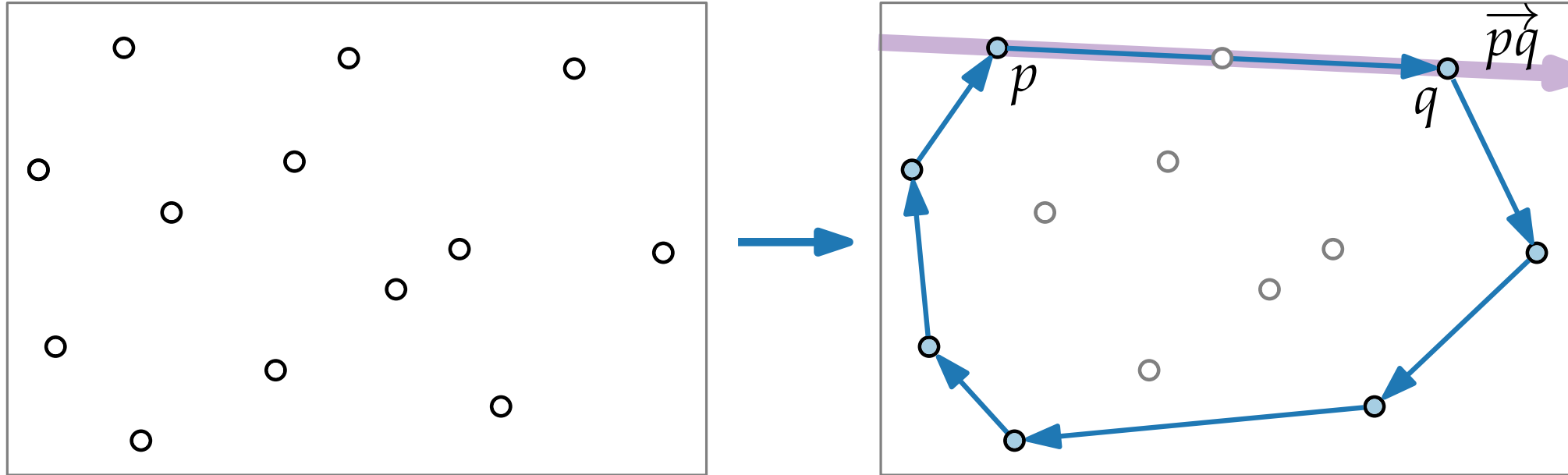
Ges. Liste der Knoten auf Rand von $\text{CH}(S)$ im Uhrzeigersinn

Beobachtung. (p, q) ist eine Kante auf $\text{CH}(S) \Leftrightarrow$
jeder Punkt in S liegt

Algorithmischer Ansatz



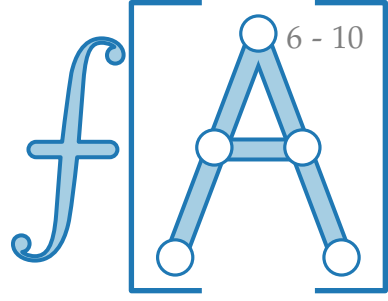
Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



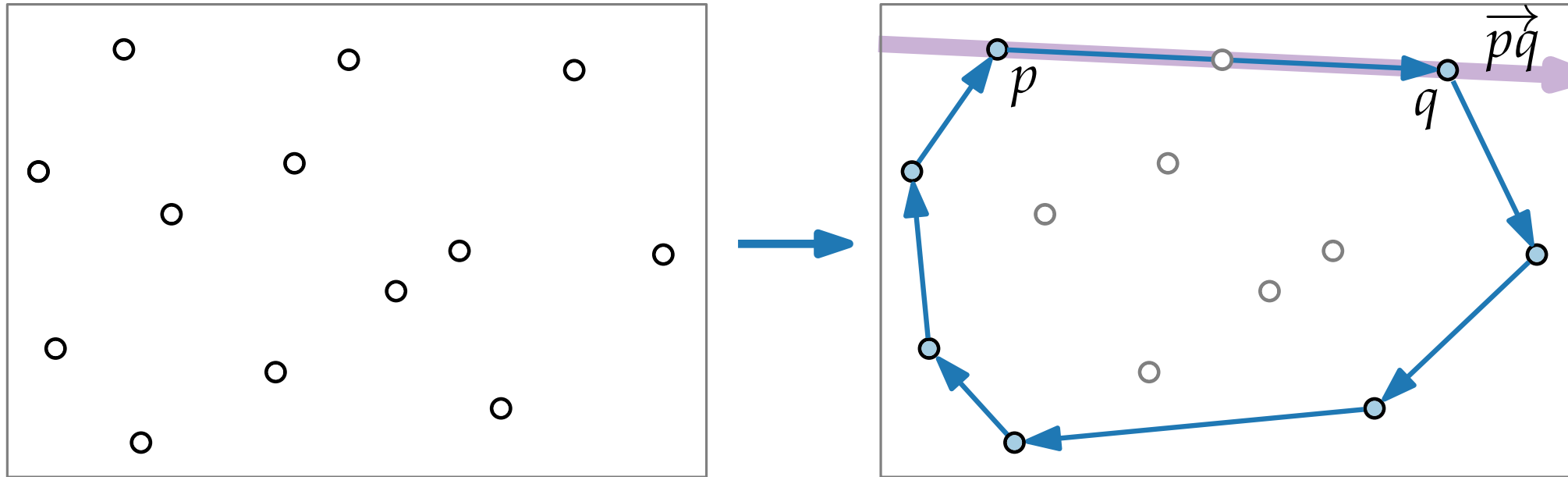
Ges. Liste der Knoten auf Rand von $\text{CH}(S)$ im Uhrzeigersinn

Beobachtung. (p, q) ist eine Kante auf $\text{CH}(S) \Leftrightarrow$
jeder Punkt in S liegt
■ streng rechts der gerichteten Geraden $\vec{pq}$ oder

Algorithmischer Ansatz



Geg. Menge S von n Punkten in der Ebene, d.h., $S \subset \mathbb{R}^2$



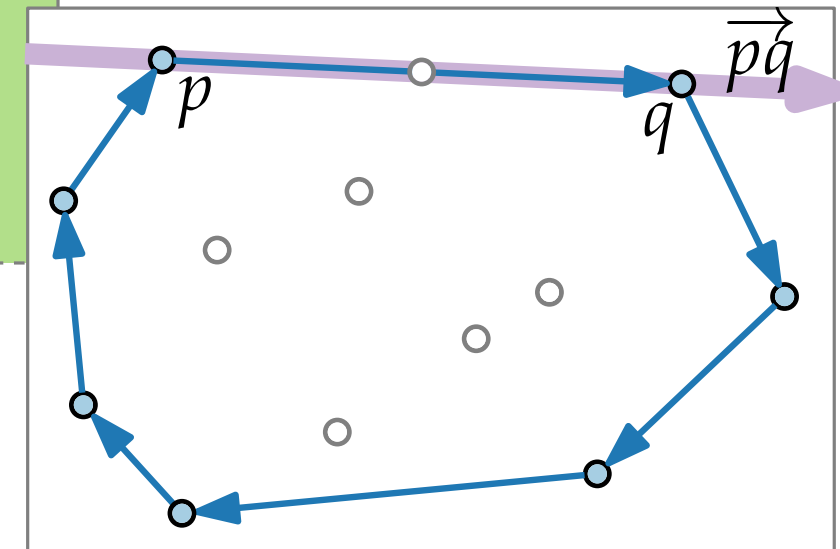
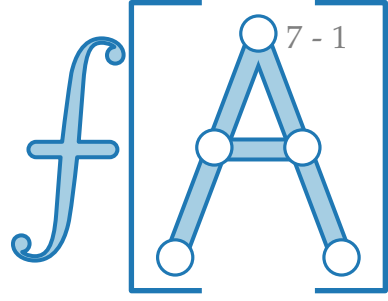
Ges. Liste der Knoten auf Rand von $\text{CH}(S)$ im Uhrzeigersinn

Beobachtung. (p, q) ist eine Kante auf $\text{CH}(S) \Leftrightarrow$
jeder Punkt in S liegt

- streng rechts der gerichteten Geraden $\overrightarrow{pq}$ oder
- auf dem Liniensegment $\overline{pq}$

Ein erster Algorithmus

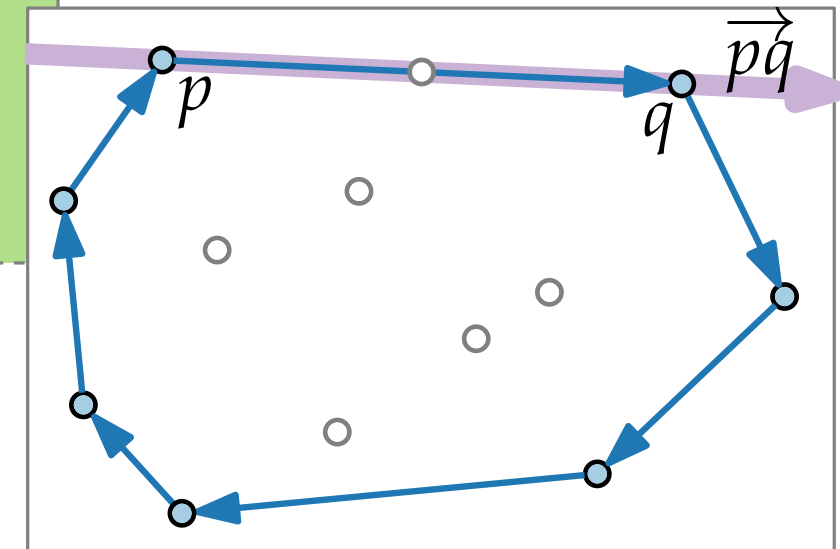
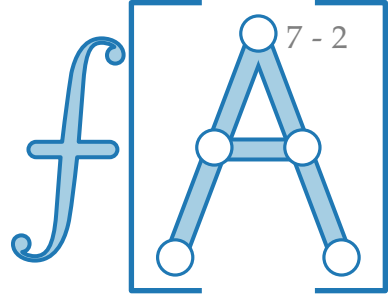
ERSTEKONVEXEHÜLLE(S)



Ein erster Algorithmus

ERSTEKONVEXEHÜLLE(S)

$$E = \emptyset$$

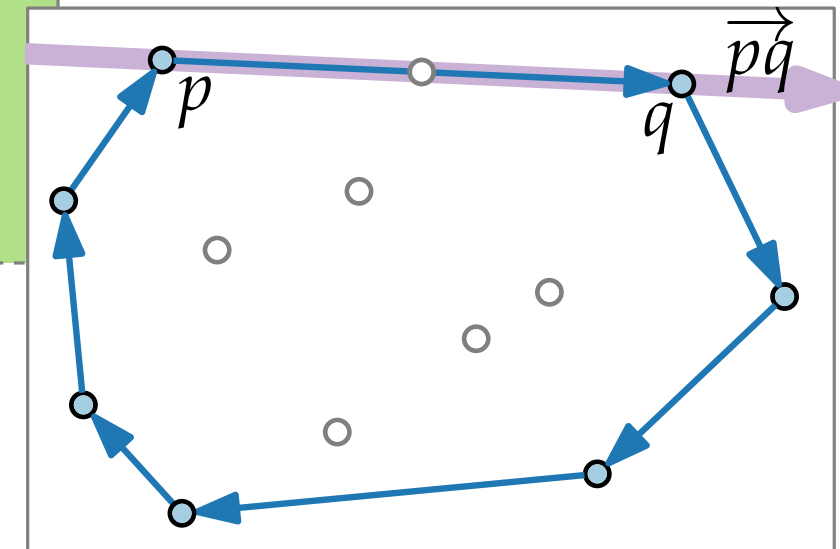
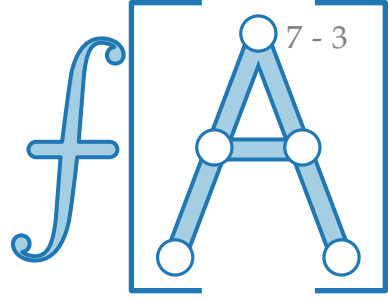


Ein erster Algorithmus

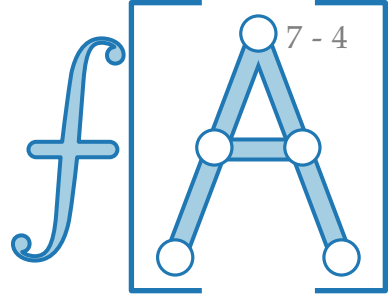
ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E
return L



Ein erster Algorithmus

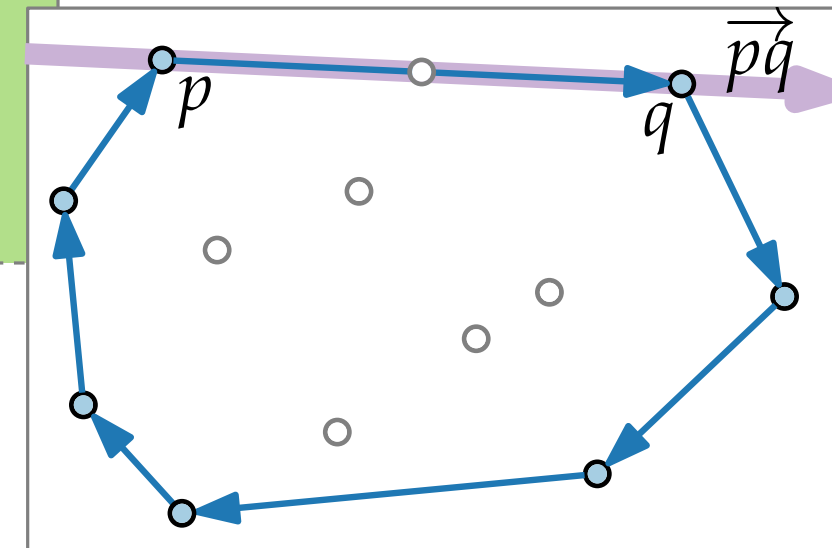


ERSTEKONVEXEHÜLLE(S)

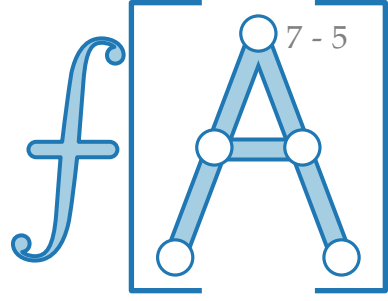
$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E
return L



Ein erster Algorithmus

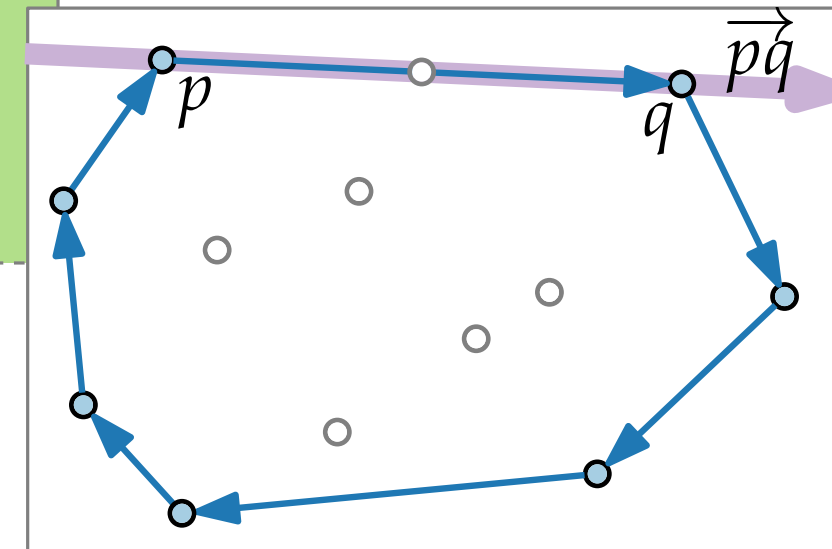


ERSTEKONVEXEHÜLLE(S)

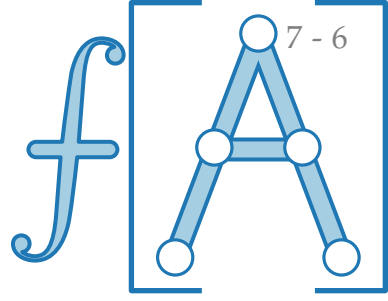
$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do** // teste: (p, q) Kante von $CH(S)$?

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E
return L



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

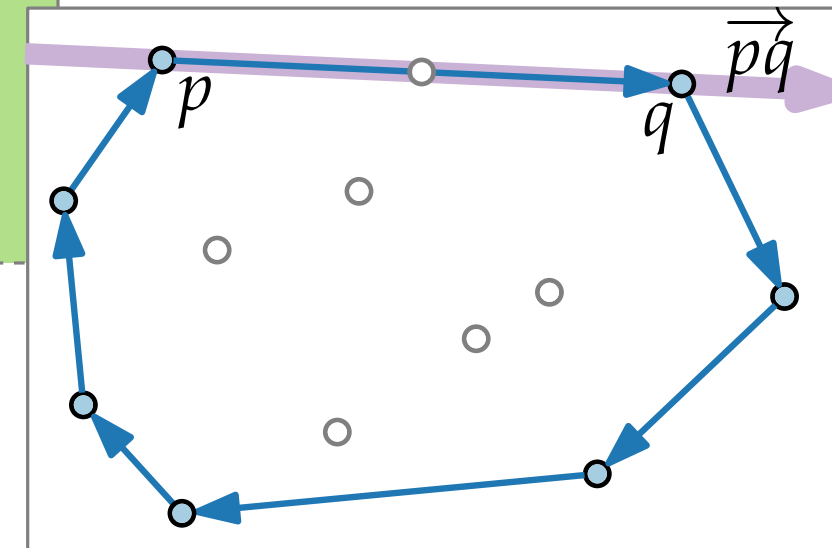
foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $CH(S)$?

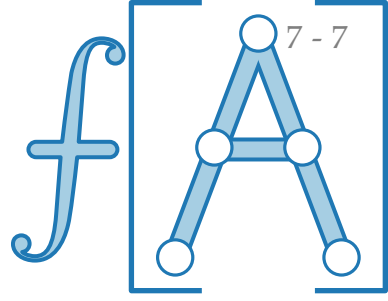
$valid = \text{true}$

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

return L



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do** // teste: (p, q) Kante von $CH(S)$?

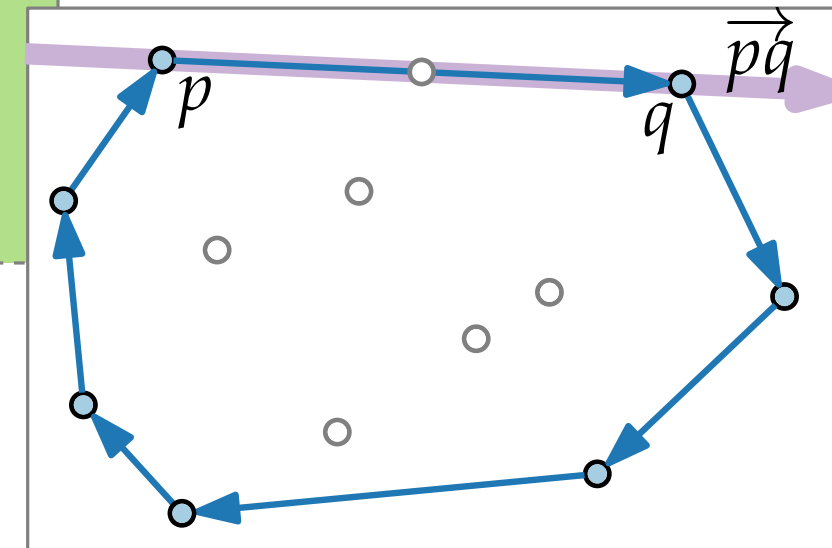
$valid = \text{true}$

if $valid$ **then**

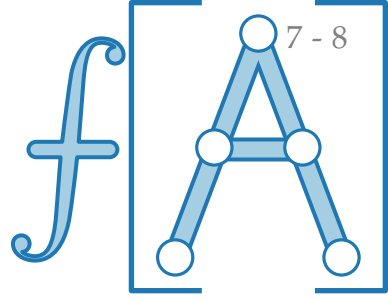
$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

return L



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $CH(S)$?

$valid = \text{true}$

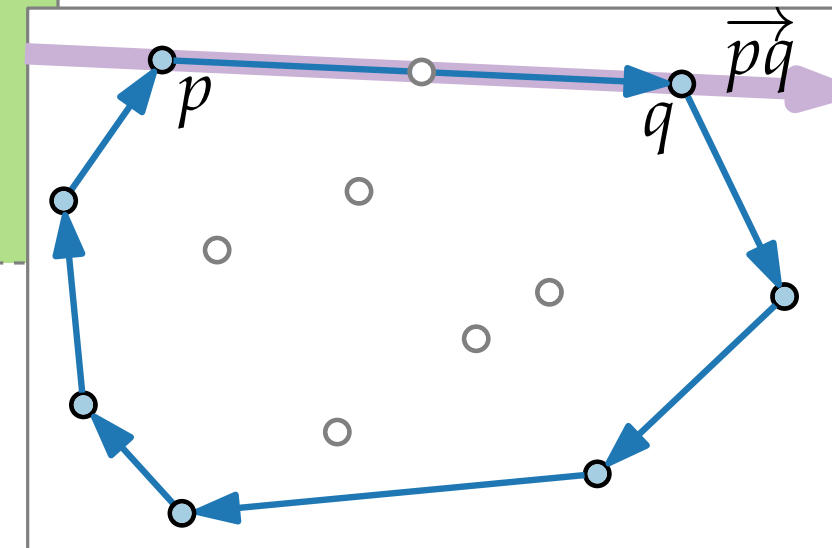
foreach $r \in S$ **do**

if $valid$ **then**

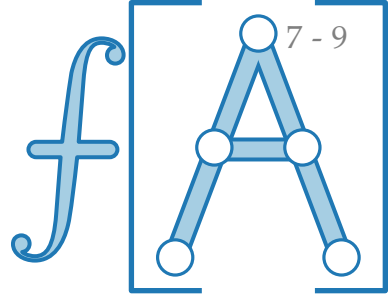
$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

return L



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do** // teste: (p, q) Kante von $CH(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

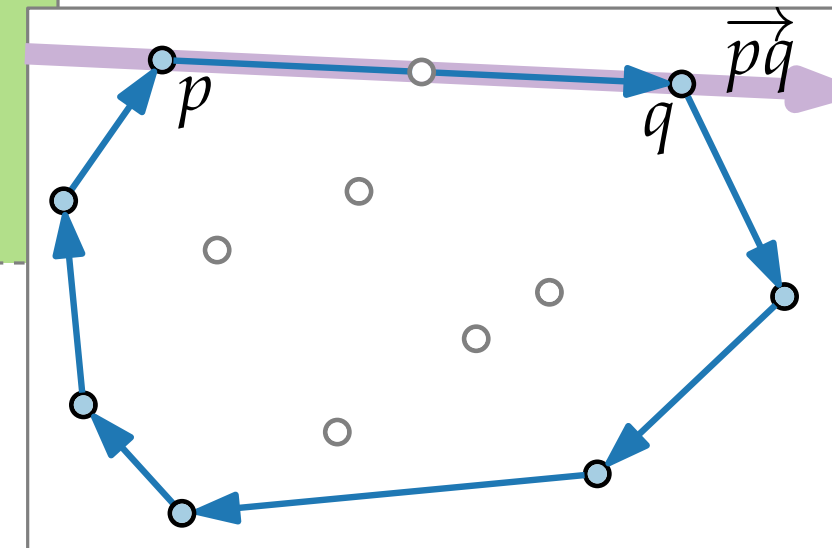
if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

if $valid$ **then**

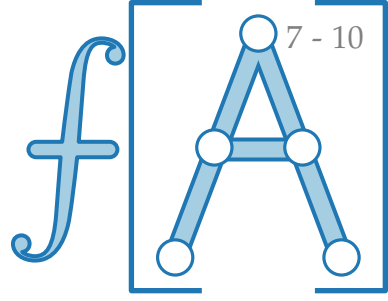
$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

return L



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do** // teste: (p, q) Kante von $\text{CH}(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

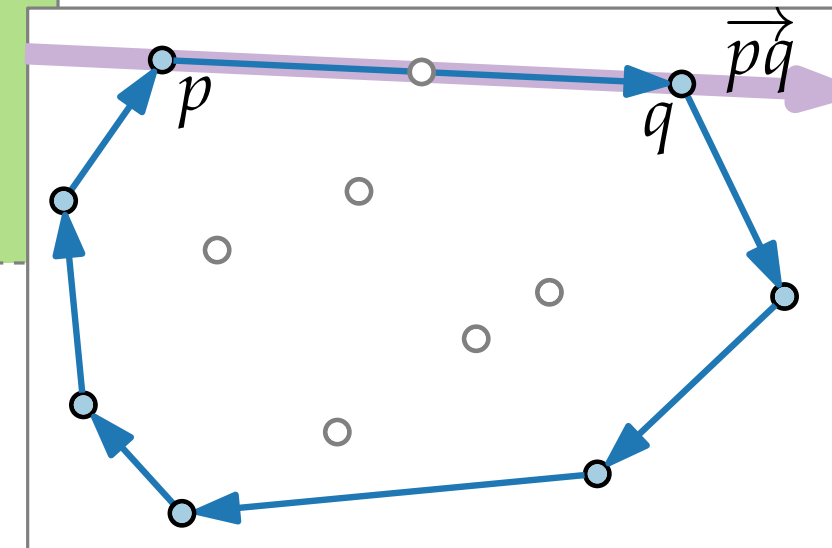
$valid = \text{false}$

if $valid$ **then**

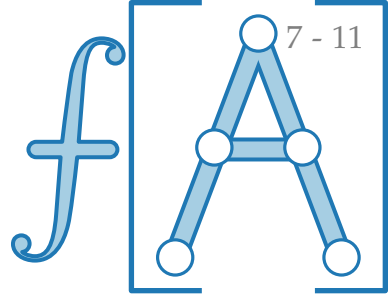
$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do** // teste: (p, q) Kante von $CH(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

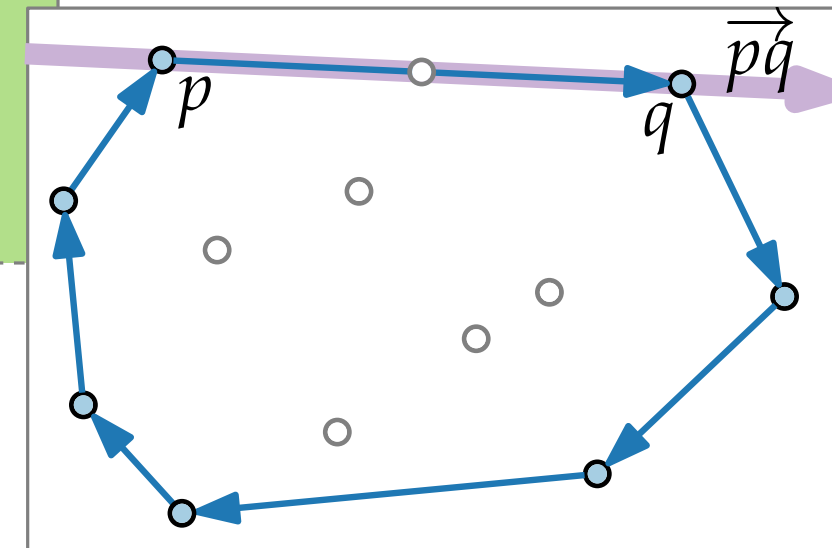
$valid = \text{false}$

if $valid$ **then**

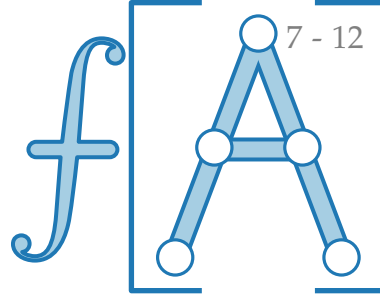
$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

return L



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $CH(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

if $valid$ **then**

$E = E \cup \{(p, q)\}$

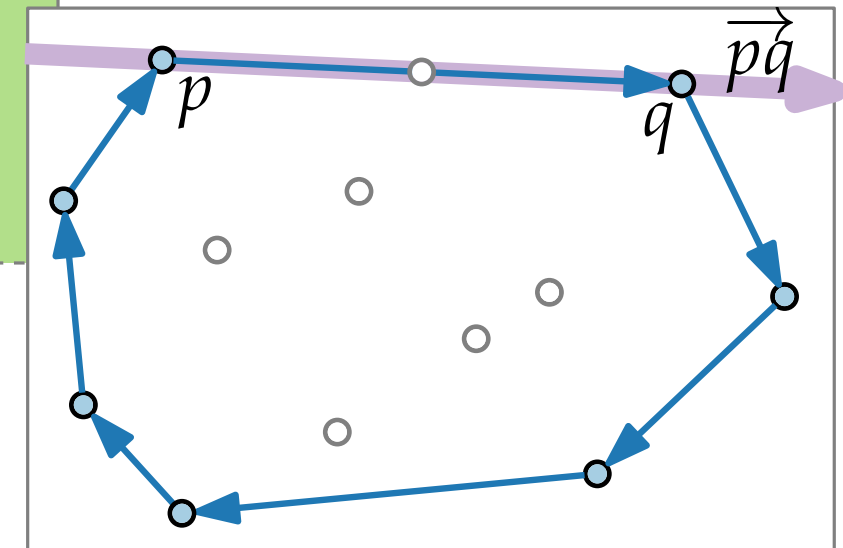
Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

return L

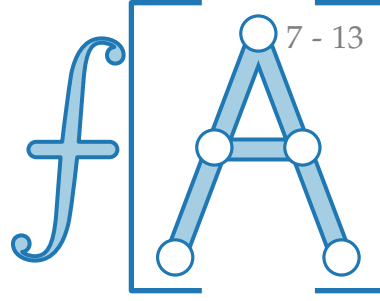
r streng rechts von $\overrightarrow{pq}$



$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $CH(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

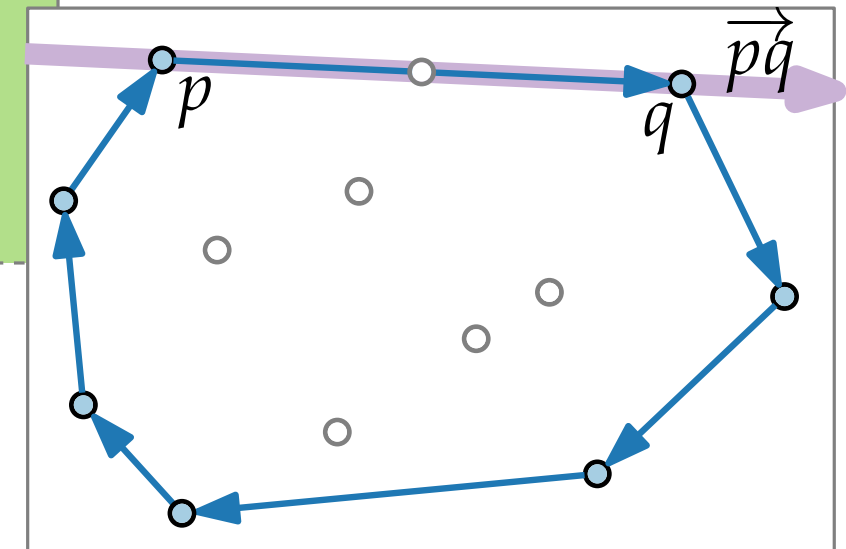
return L

r streng rechts von $\overrightarrow{pq}$

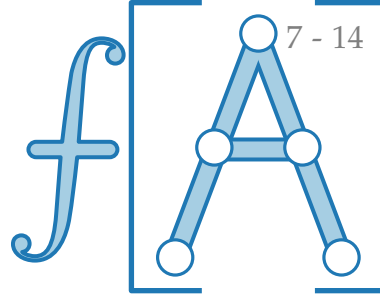


$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$

$\Rightarrow$ testen dauert $\mathcal{O}(1)$ Zeit



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $\text{CH}(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\vec{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

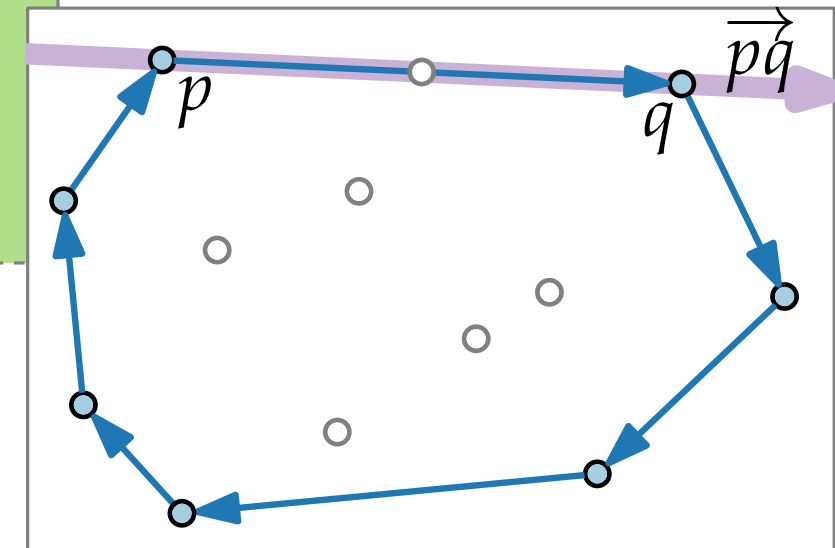
return L

r streng rechts von $\vec{pq}$

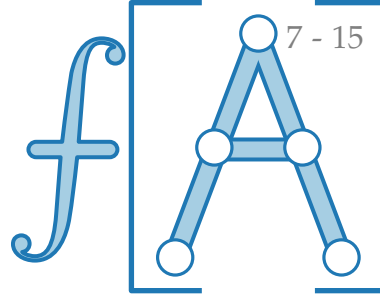


$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$

$\Rightarrow$ testen dauert $\mathcal{O}(1)$ Zeit



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $\text{CH}(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\vec{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

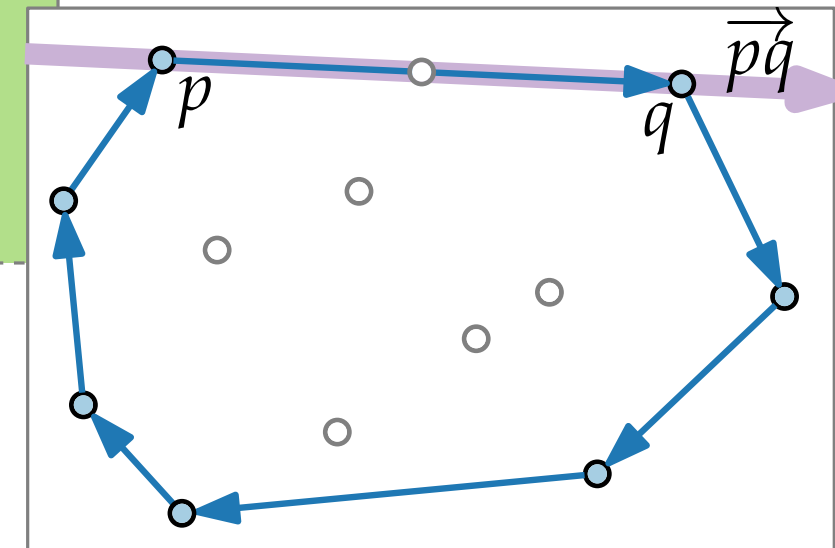
return L

r streng rechts von $\vec{pq}$

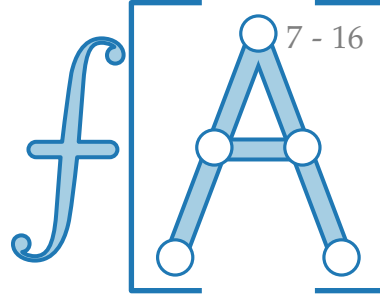


$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$

$\Rightarrow$ testen dauert $\mathcal{O}(1)$ Zeit



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $\text{CH}(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\vec{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

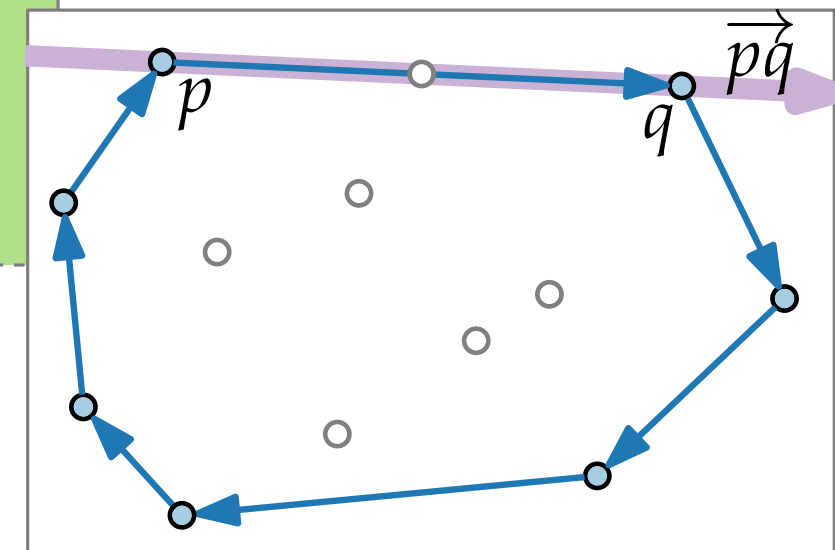
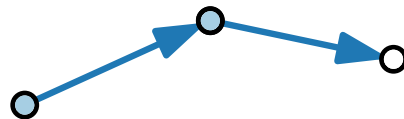
return L

r streng rechts von $\vec{pq}$

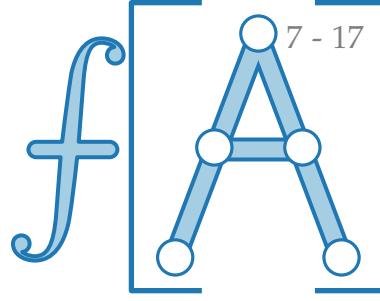


$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$

$\Rightarrow$ testen dauert $\mathcal{O}(1)$ Zeit



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $\text{CH}(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\vec{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

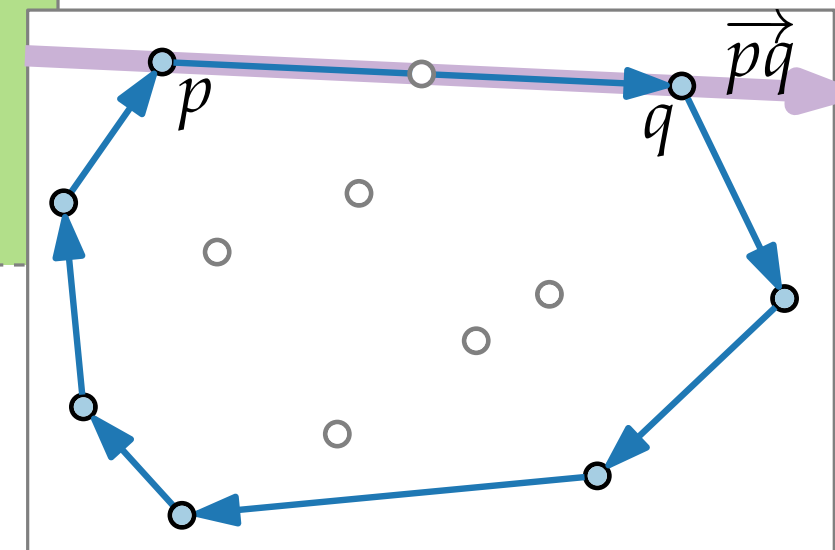
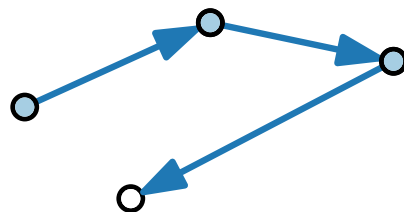
return L

r streng rechts von $\vec{pq}$

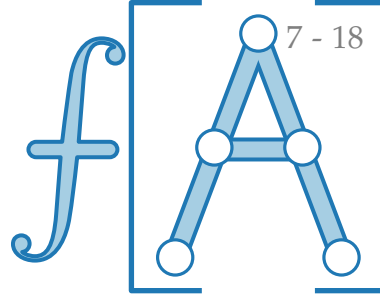


$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$

$\Rightarrow$ testen dauert $\mathcal{O}(1)$ Zeit



Ein erster Algorithmus



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

// teste: (p, q) Kante von $CH(S)$?

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\vec{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E

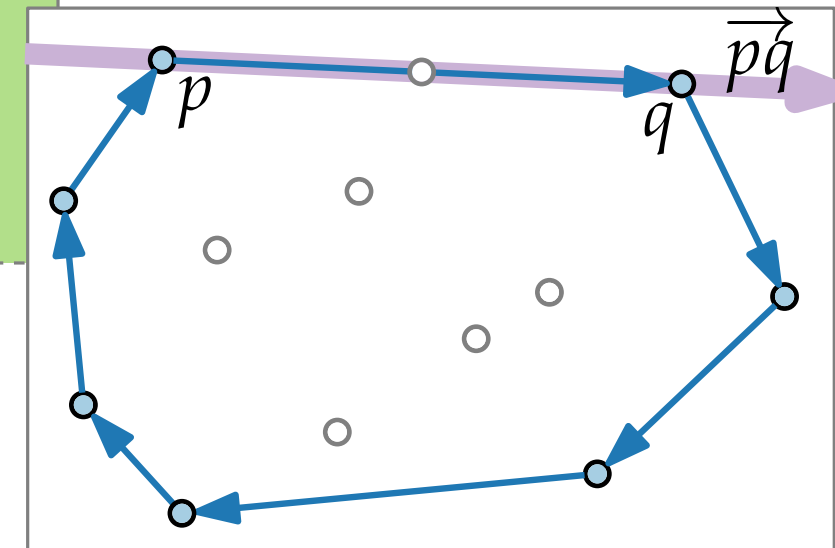
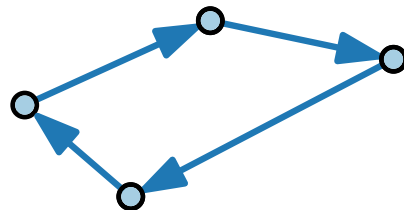
return L

r streng rechts von $\vec{pq}$

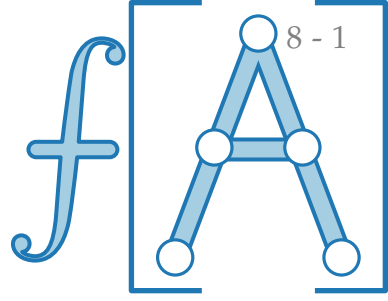


$$\begin{vmatrix} x_r & y_r & 1 \\ x_p & y_p & 1 \\ x_q & y_q & 1 \end{vmatrix} < 0$$

$\Rightarrow$ testen dauert $\mathcal{O}(1)$ Zeit



Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

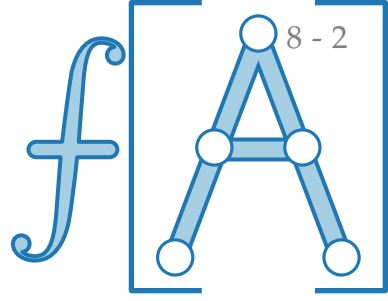
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

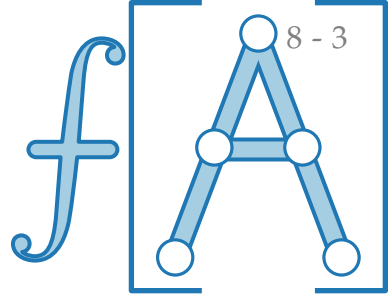
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

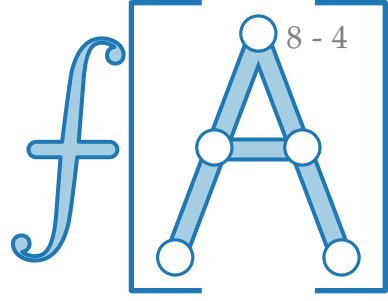
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

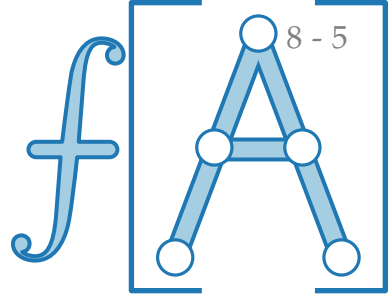
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

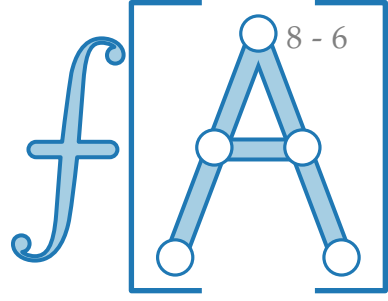
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then** $\Theta(1)$
 $valid = \text{false}$

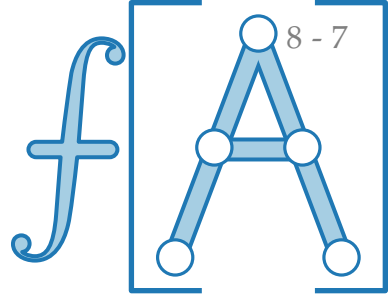
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

$n \cdot$

$\Theta(1)$

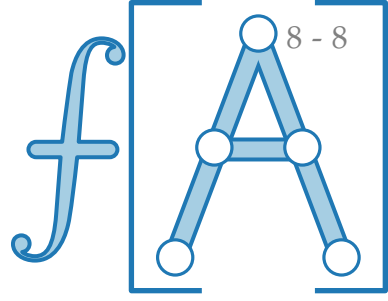
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do**

$valid = \text{true}$

foreach $r \in S$ **do**

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then**

$valid = \text{false}$

$n \cdot$

$\Theta(1)$

$\Theta(n)$

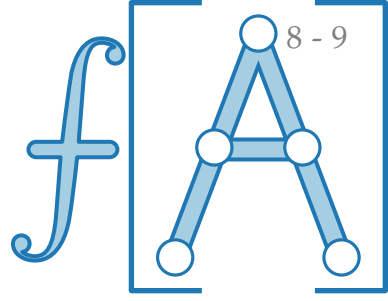
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ **do** $(n^2 - n) \cdot$

$valid = \text{true}$

foreach $r \in S$ **do** $n \cdot$

if not (r streng rechts von $\overrightarrow{pq}$ **or** $r \in \overline{pq}$) **then** $\Theta(1)$

$valid = \text{false}$ $\Theta(n)$

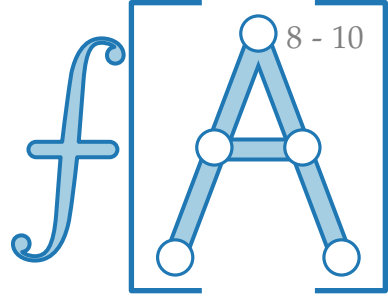
if $valid$ **then**

$E = E \cup \{(p, q)\}$

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit



ERSTEKONVEXEHÜLLE(S)

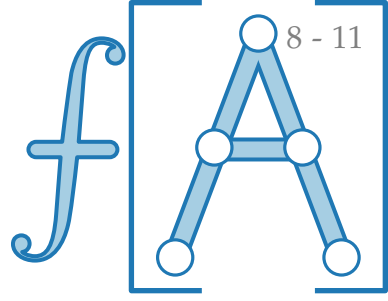
$E = \emptyset$

```
foreach  $(p, q) \in S \times S$  mit  $p \neq q$  do  $(n^2 - n) \cdot$   
     $valid = \text{true}$   
    foreach  $r \in S$  do  $n \cdot$   
        if not ( $r$  streng rechts von  $\overrightarrow{pq}$  or  $r \in \overline{pq}$ ) then  $\Theta(1)$   
             $valid = \text{false}$   $\Theta(n)$   
    if  $valid$  then  $\Theta(n^3)$   
         $E = E \cup \{(p, q)\}$ 
```

Konstruiere sortierte Liste L der Knoten von $\text{CH}(S)$ aus E

return L

Laufzeit

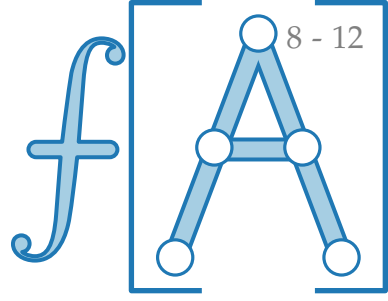


ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

```
foreach  $(p, q) \in S \times S$  mit  $p \neq q$  do  $(n^2 - n) \cdot$   
     $valid = \text{true}$   
    foreach  $r \in S$  do  $n \cdot$   
        if not ( $r$  streng rechts von  $\overrightarrow{pq}$  or  $r \in \overline{pq}$ ) then  $\Theta(1)$   
             $valid = \text{false}$   $\Theta(n)$   
        if  $valid$  then  $\Theta(n^3)$   
             $E = E \cup \{(p, q)\}$   
Konstruiere sortierte Liste  $L$  der Knoten von  $\text{CH}(S)$  aus  $E$   
return  $L$ 
```

Laufzeit

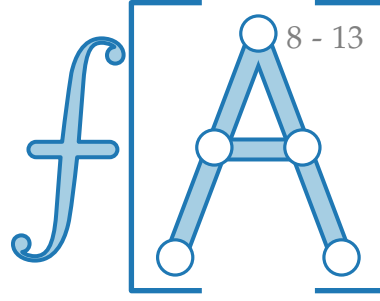


ERSTEKONVEXEHÜLLE(S)

$E = \emptyset$

```
foreach  $(p, q) \in S \times S$  mit  $p \neq q$  do  $(n^2 - n) \cdot$   
     $valid = \text{true}$   
    foreach  $r \in S$  do  $n \cdot$   
        if not ( $r$  streng rechts von  $\overrightarrow{pq}$  or  $r \in \overline{pq}$ ) then  $\Theta(1)$   
             $valid = \text{false}$   $\Theta(n)$   
    if  $valid$  then  $\Theta(n^3)$   
         $E = E \cup \{(p, q)\}$   
Konstruiere sortierte Liste  $L$  der Knoten von  $\text{CH}(S)$  aus  $E$   $\Theta(n)$   
return  $L$   $\mathcal{O}(n)$ 
```

Laufzeit



ERSTEKONVEXEHÜLLE(S)

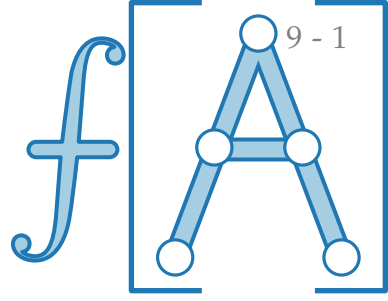
$E = \emptyset$

foreach $(p, q) \in S \times S$ mit $p \neq q$ do	$(n^2 - n) \cdot$	
$valid = \text{true}$		
foreach $r \in S$ do	$n \cdot$	
if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then	$\Theta(1)$	$\Theta(n)$
$valid = \text{false}$		$\Theta(n^3)$
if $valid$ then		
$E = E \cup \{(p, q)\}$		
Konstruiere sortierte Liste L der Knoten von $CH(S)$ aus E		$\Theta(n)$
return L		

Lemma. Wir können die konvexe Hülle von n Punkten in der Ebene in $\Theta(n^3)$ Zeit berechnen.

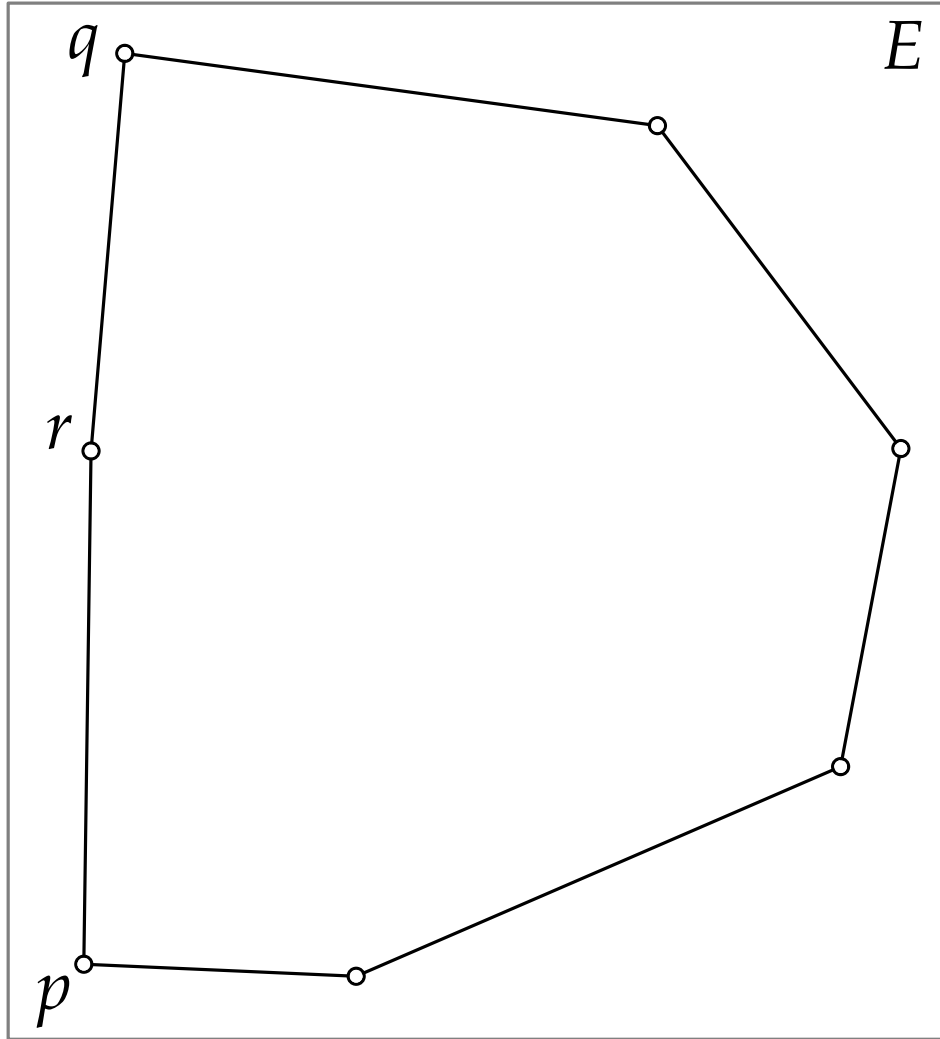
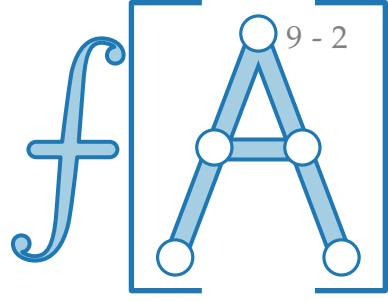
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
└ $valid = false$



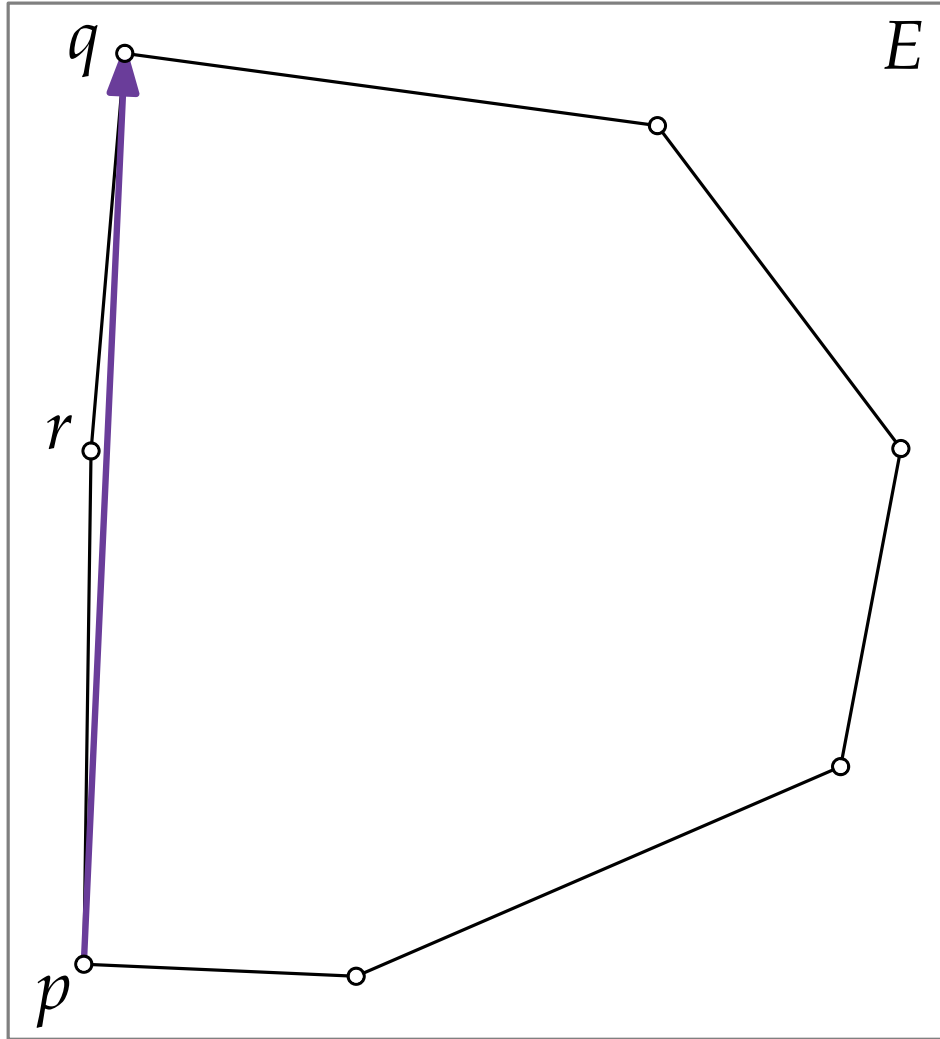
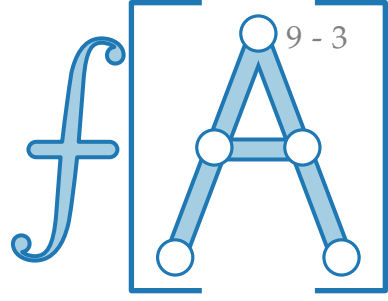
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
└ $valid = false$



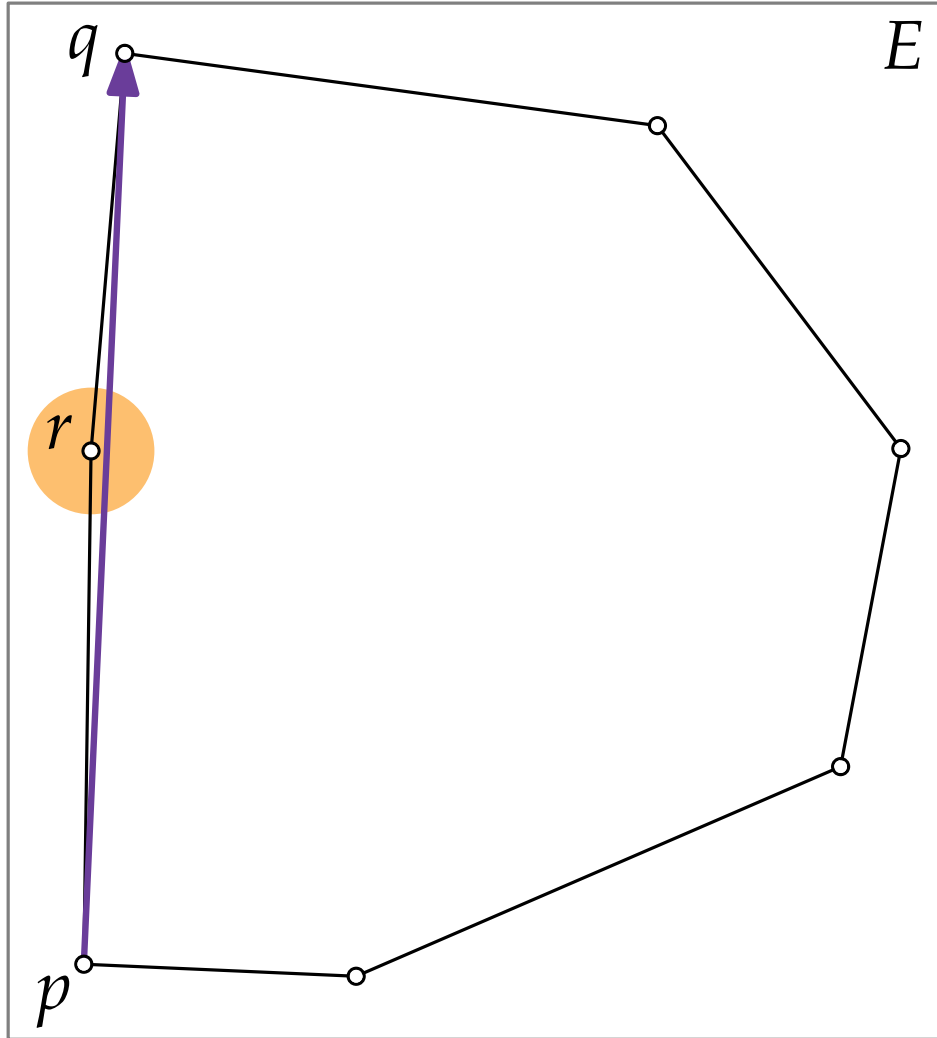
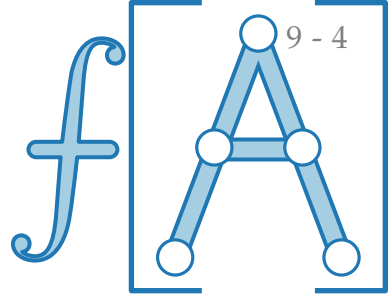
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
└ $valid = false$



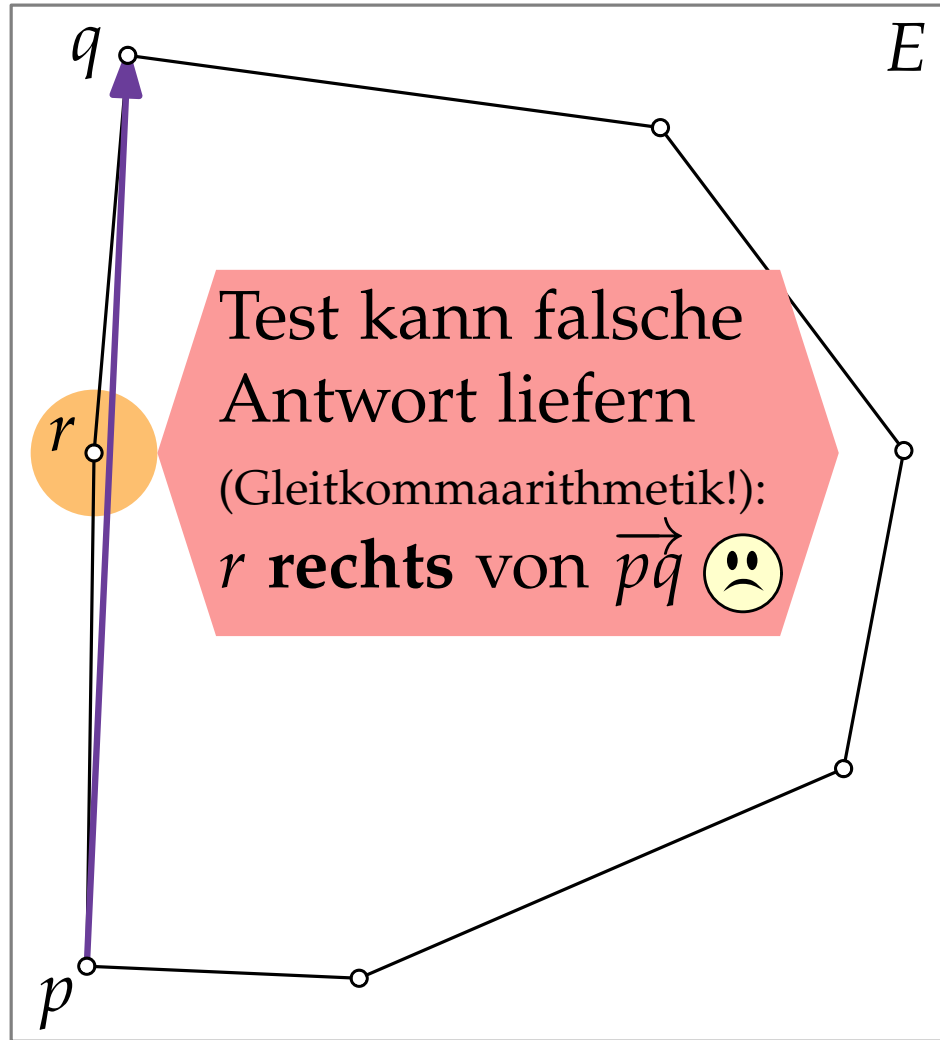
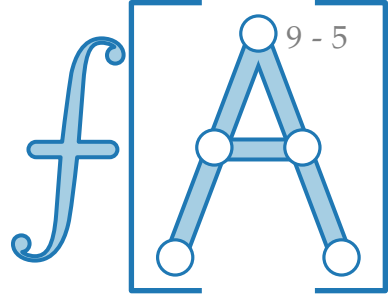
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
└ $valid = false$



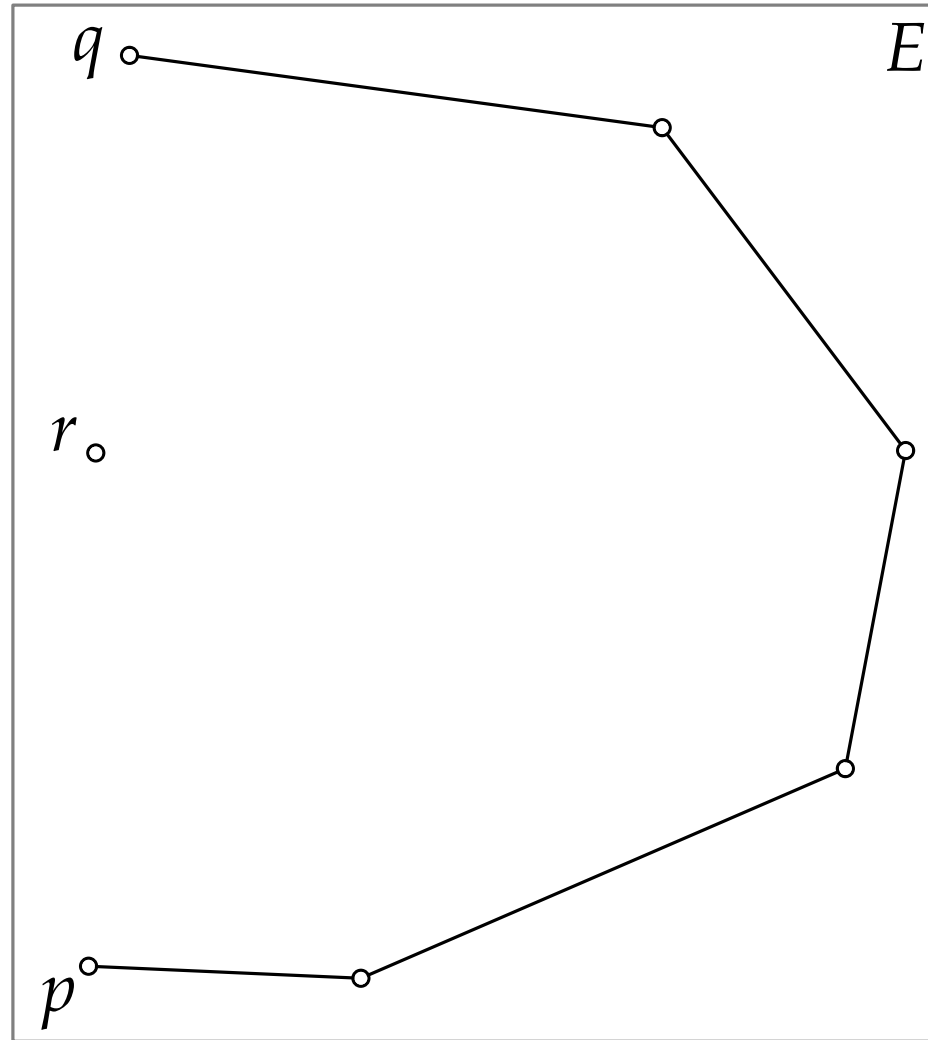
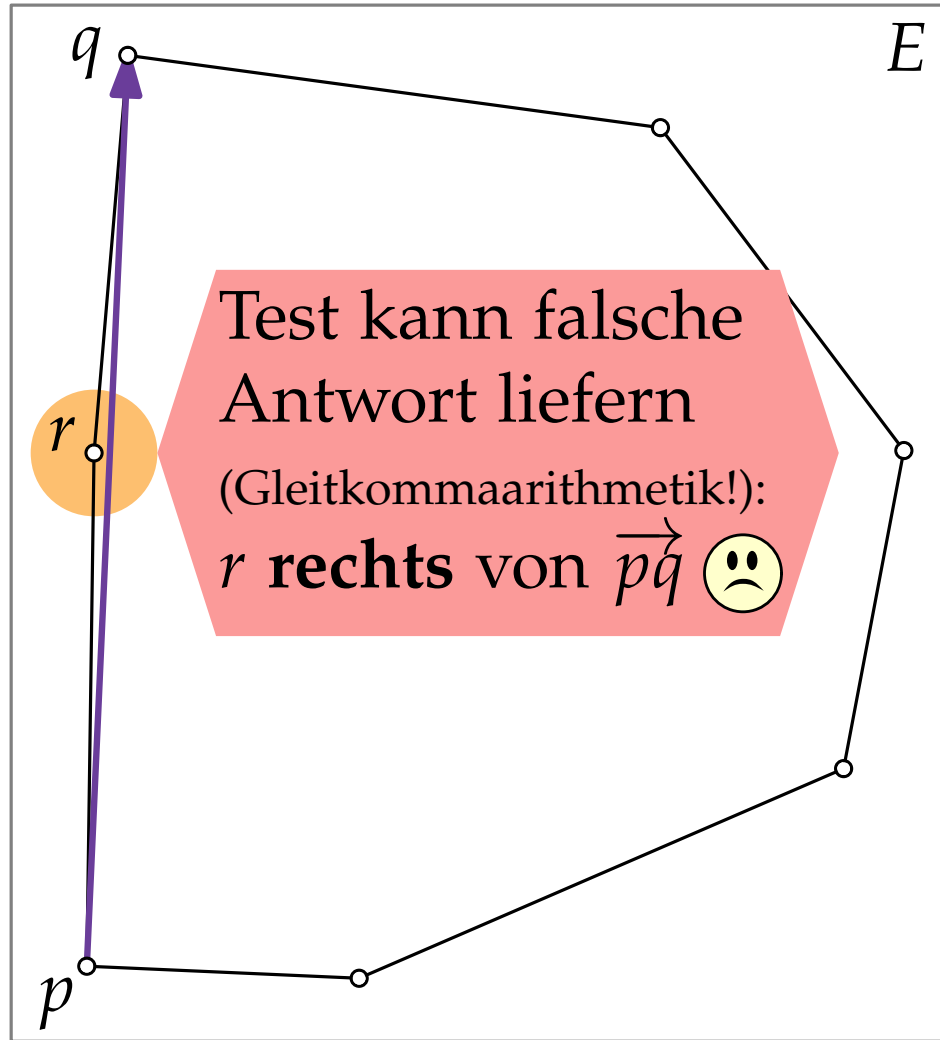
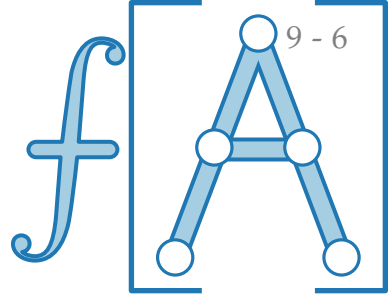
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
 $valid = false$



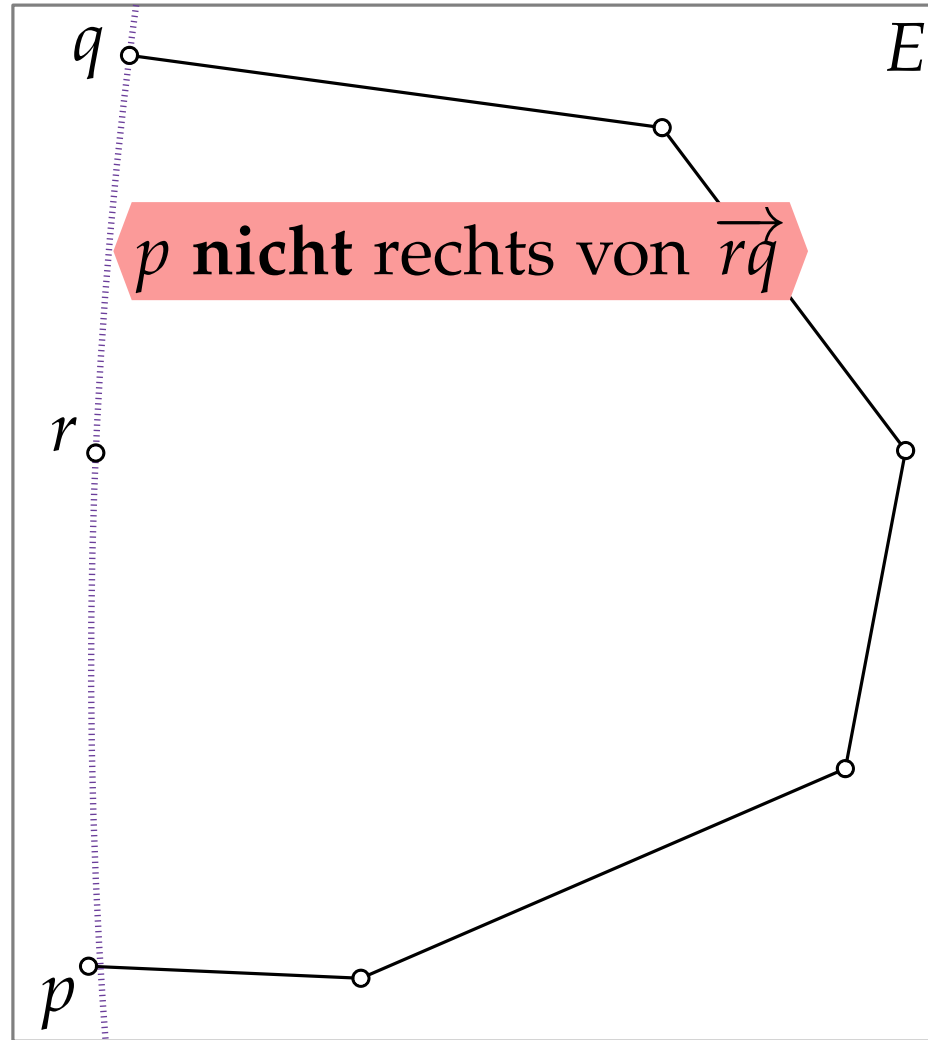
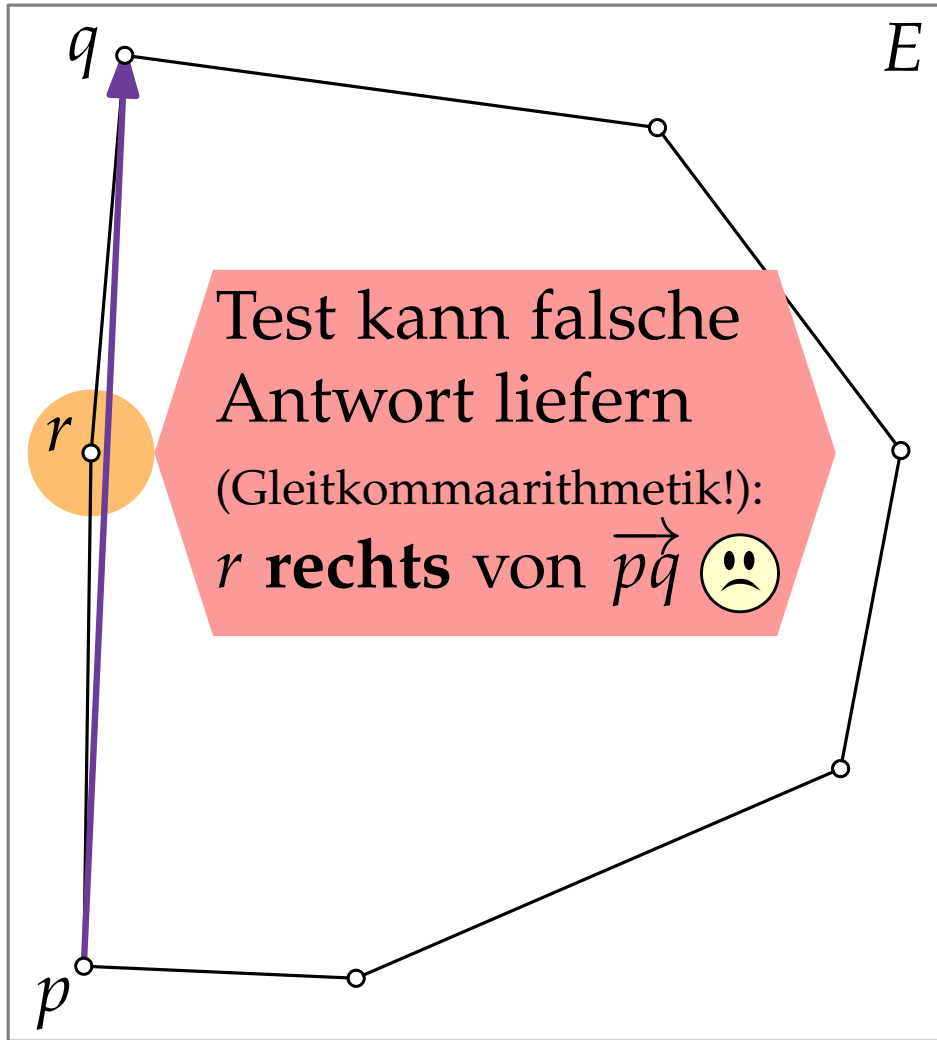
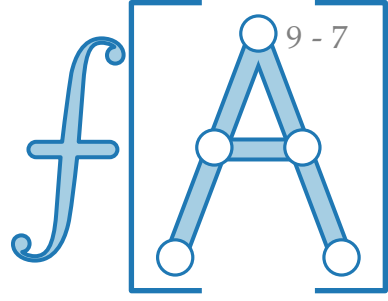
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
└ $valid = false$



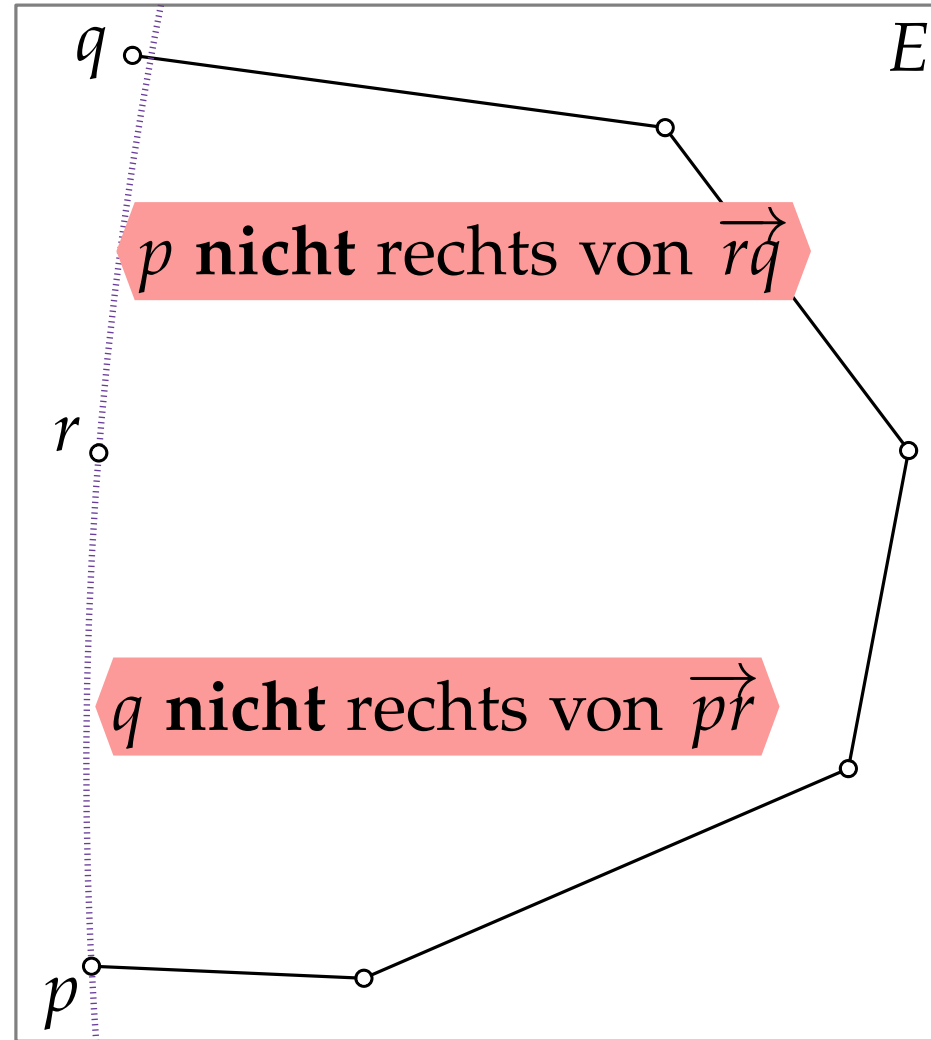
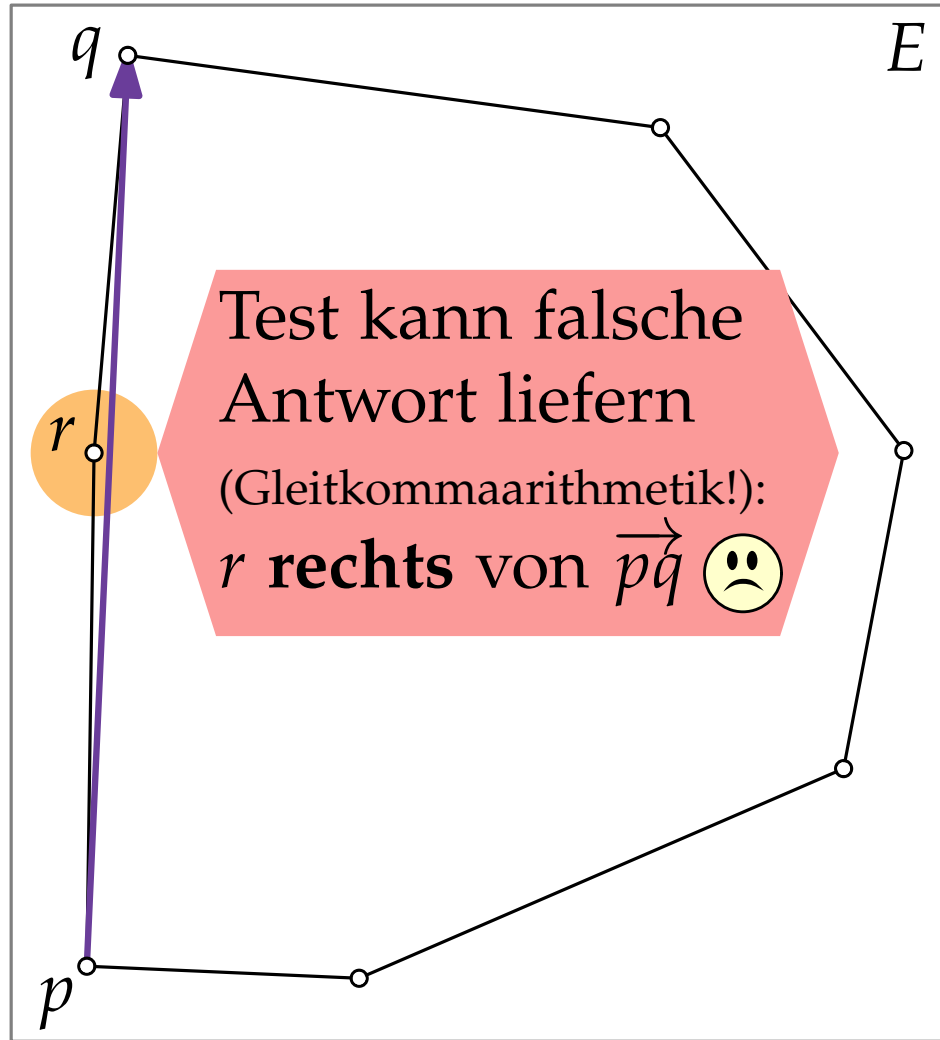
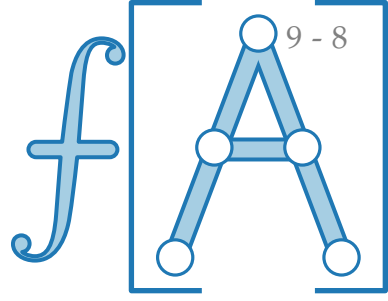
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
 $valid = false$



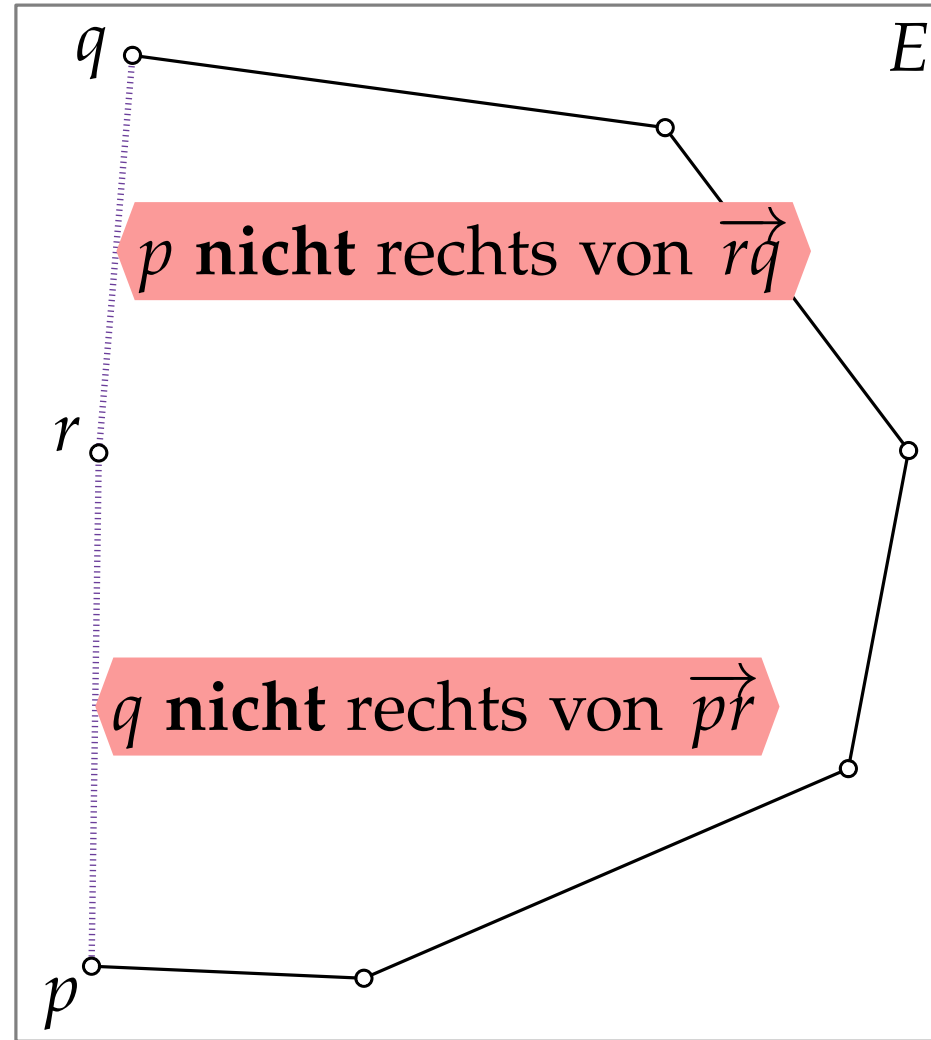
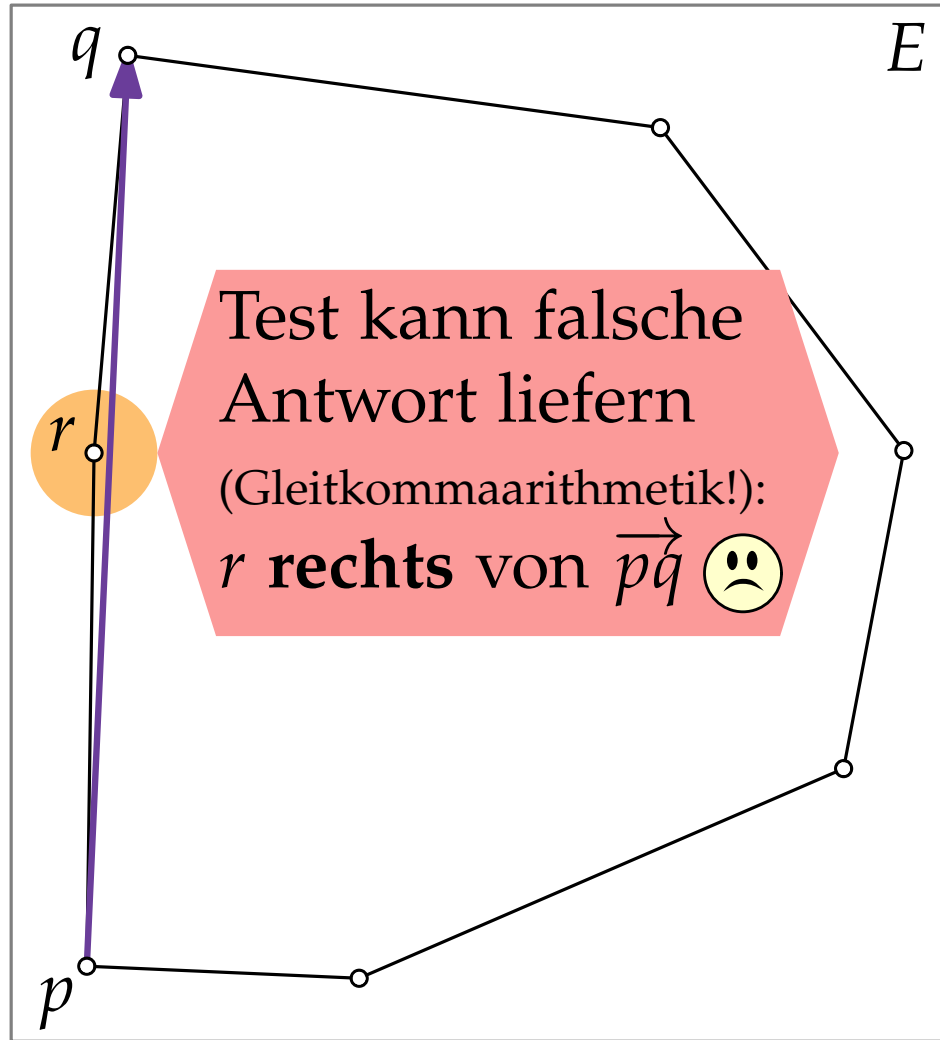
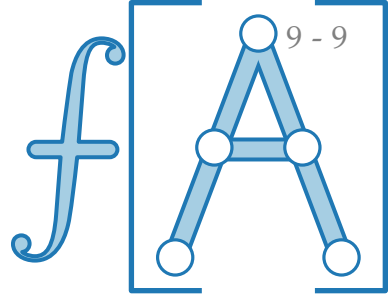
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
└ $valid = false$



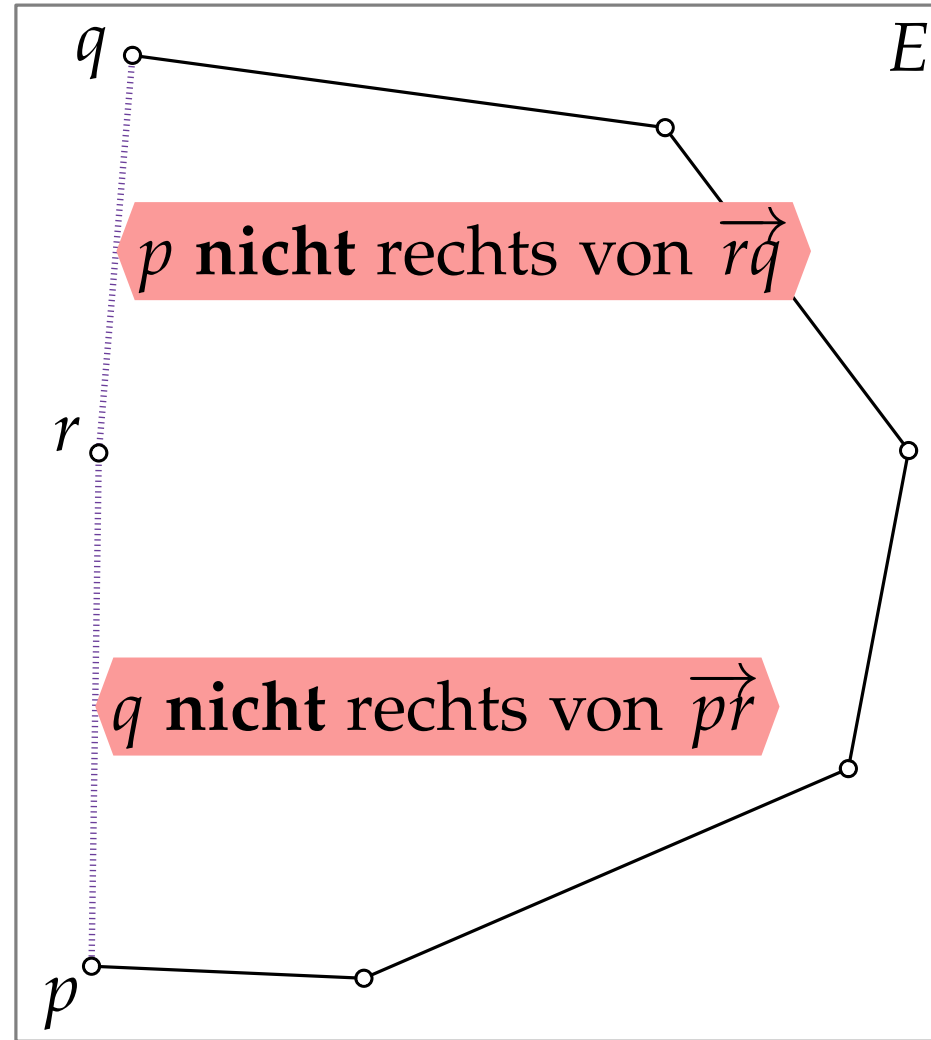
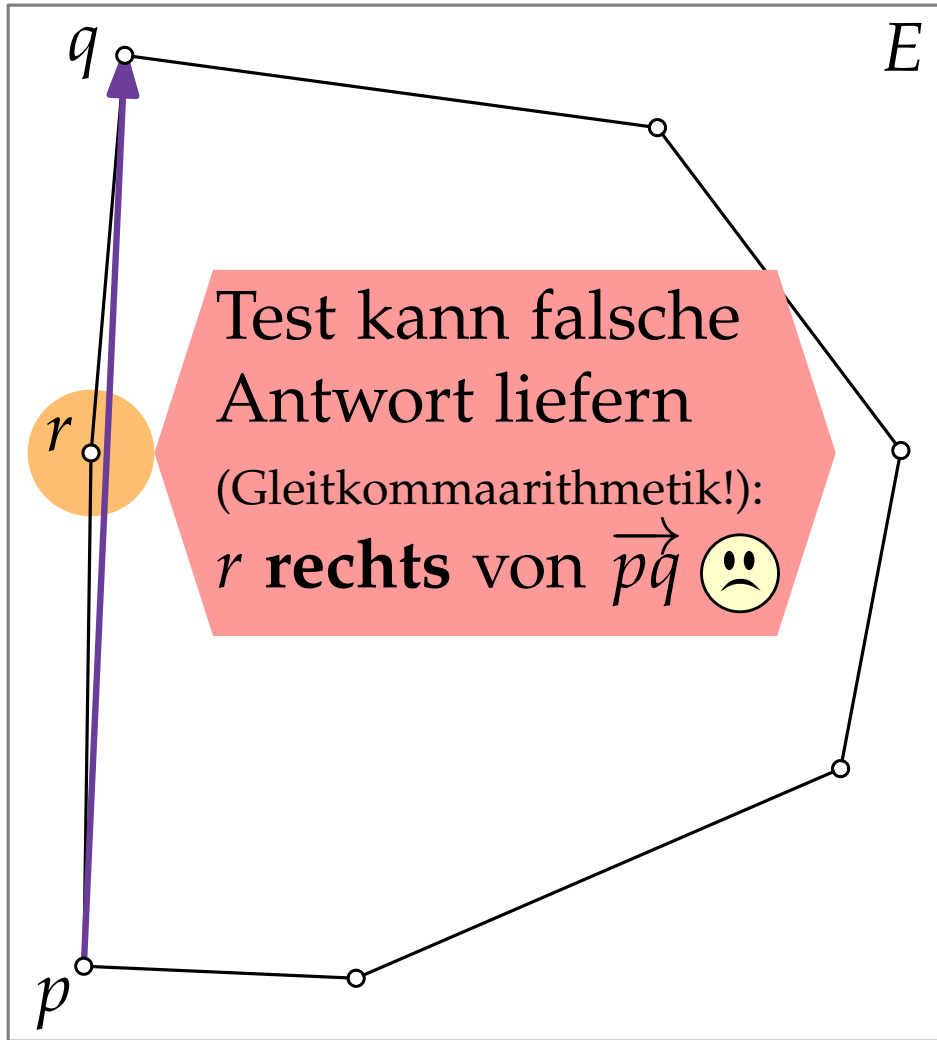
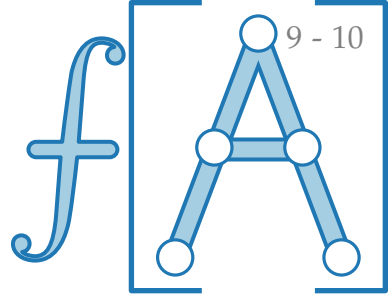
Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
 $valid = false$



Diskussion

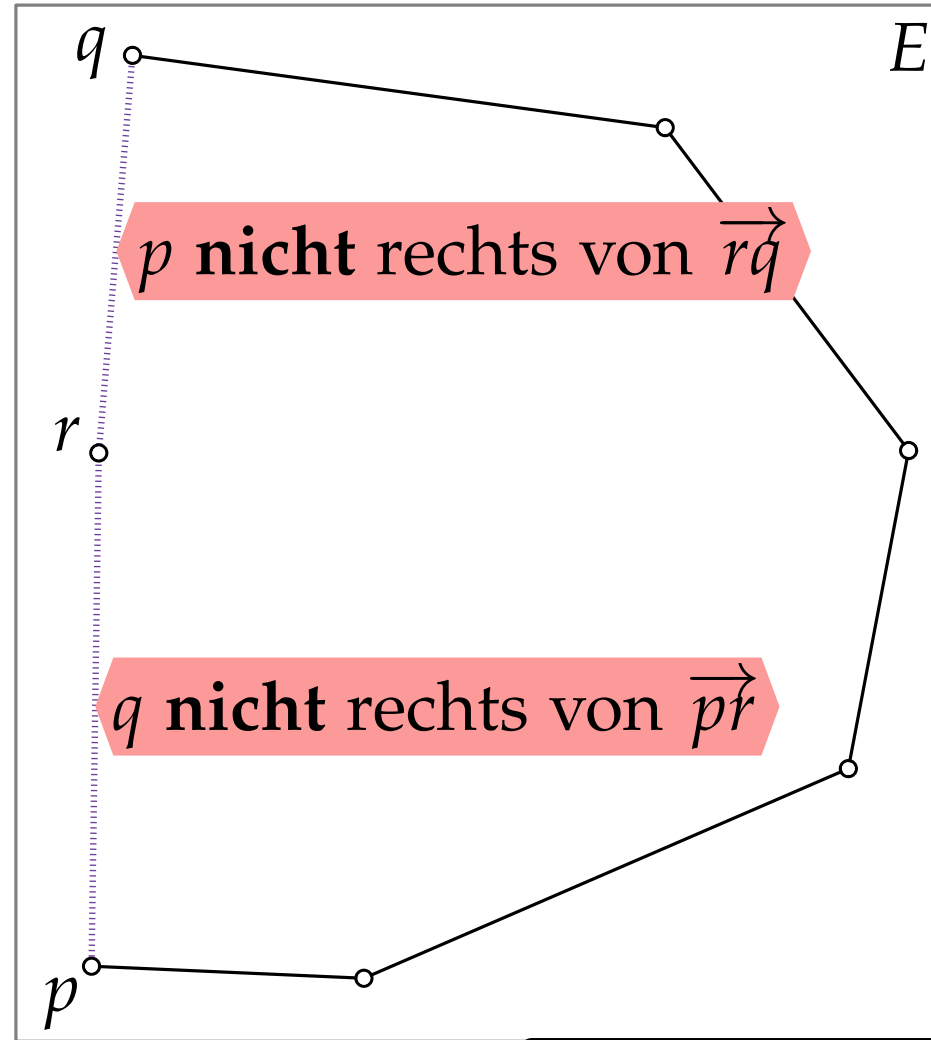
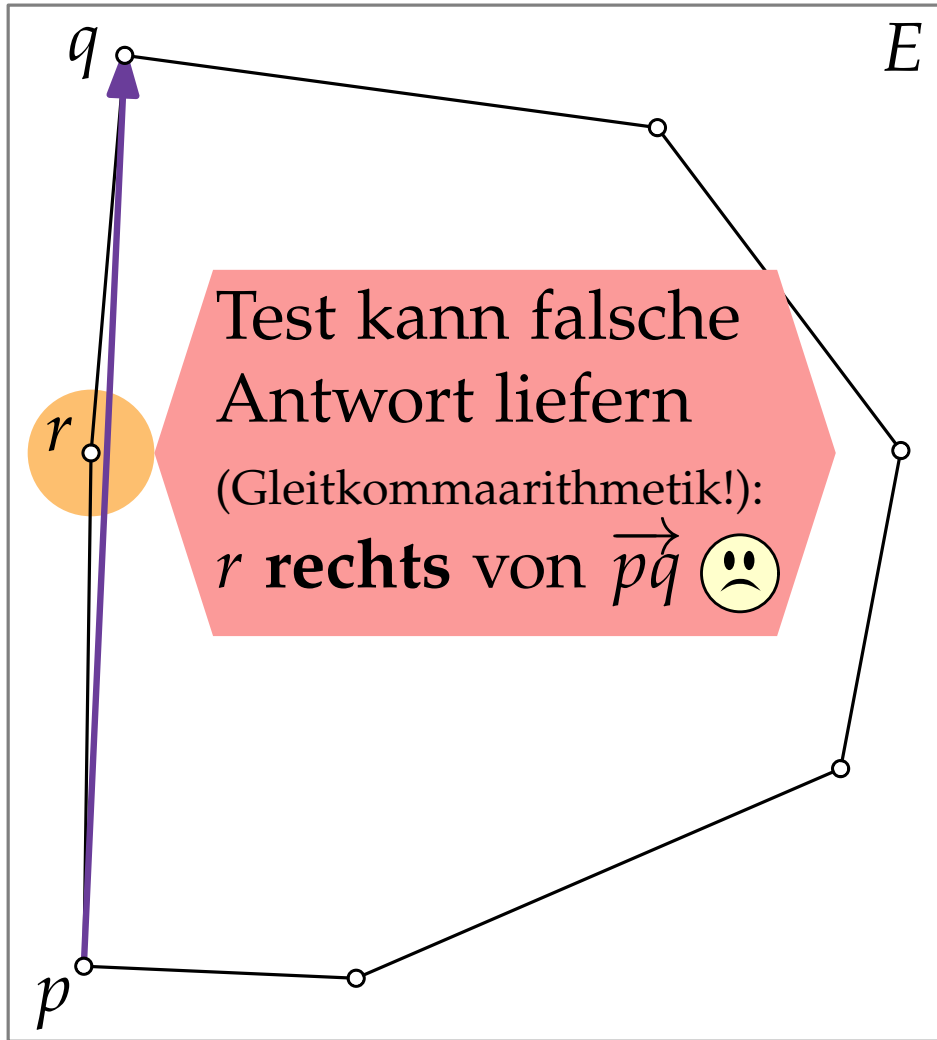
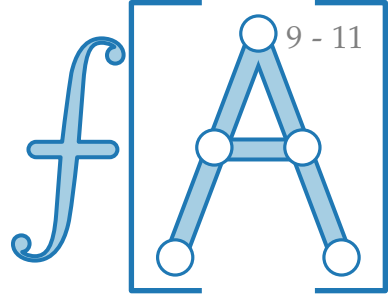
if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
 $valid = false$



Beobachtung. Algorithmus ERSTEKONVEXEHÜLLE ist nicht **robust**.

Diskussion

if not (r streng rechts von $\overrightarrow{pq}$ or $r \in \overline{pq}$) then
 $valid = false$



das Ergebnis ist evtl. nicht mal eine geschlossene Punktmenge!

Beobachtung. Algorithmus ERSTEKONVEXEHÜLLE ist nicht **robust**.

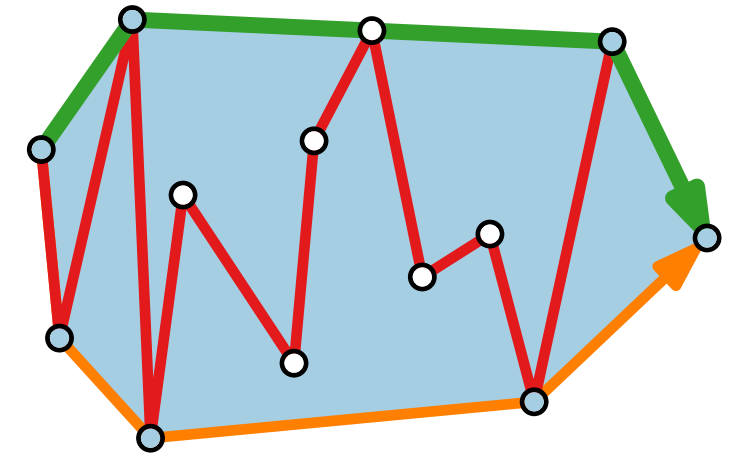
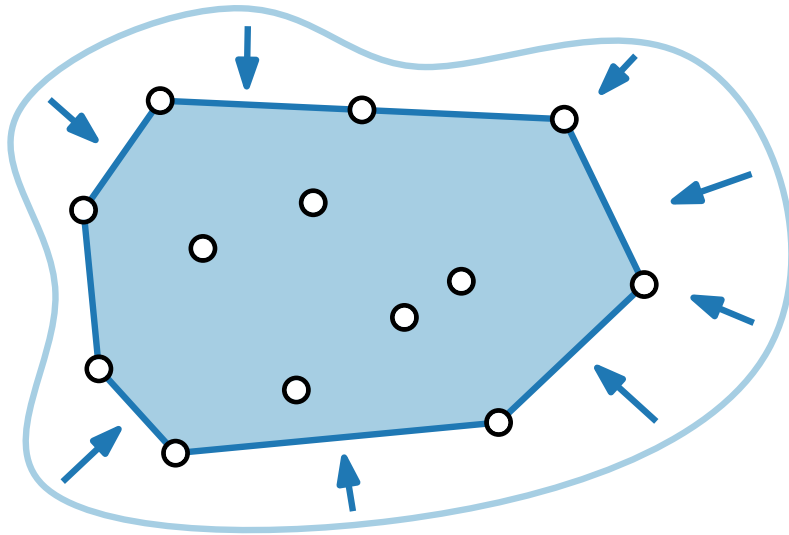
Fortgeschrittene Algorithmen

12. Vorlesung

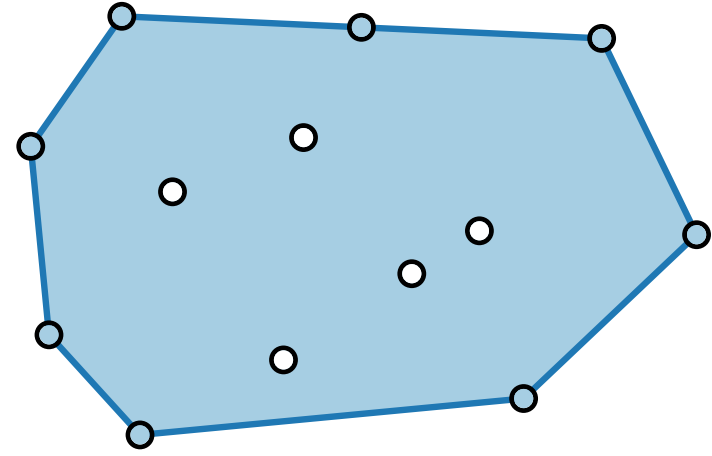
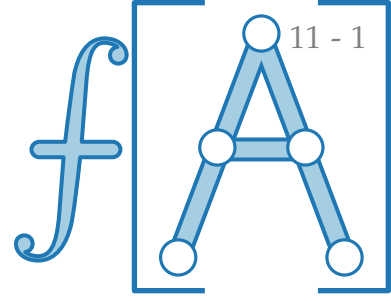
Algorithmische Geometrie I:

Konvexe Hülle

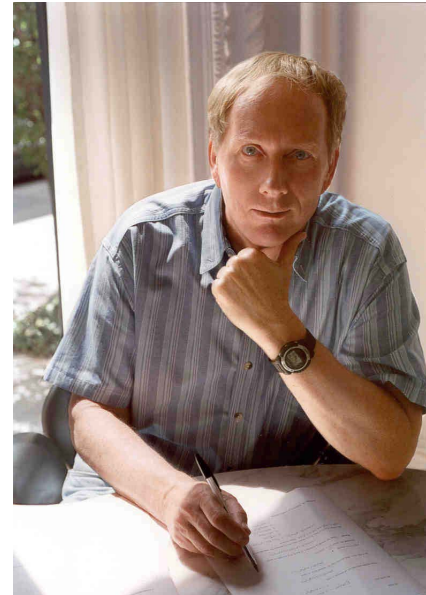
Teil III: ANDREW-GRAHAM-SCAN



ANDREW-GRAHAM-SCAN

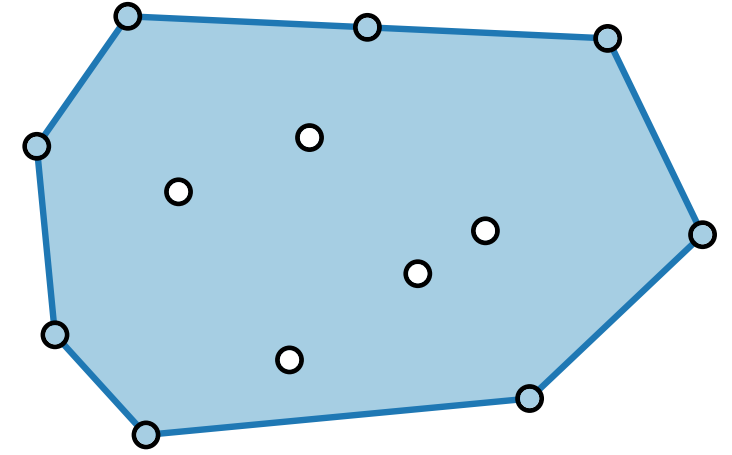
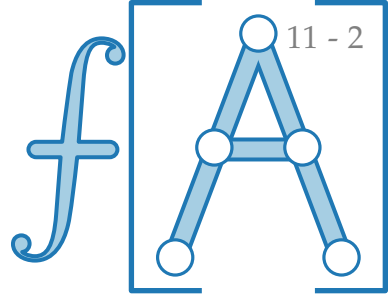


Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

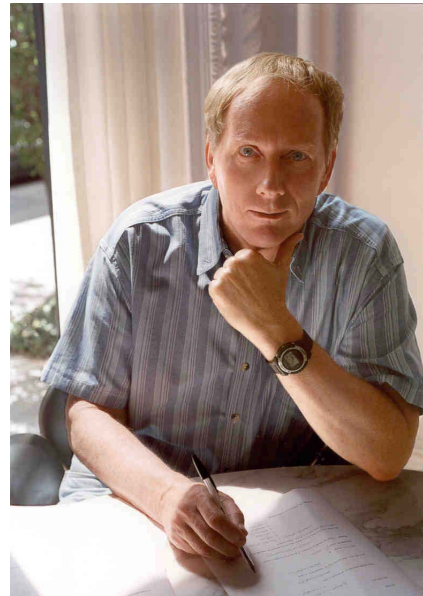


ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte

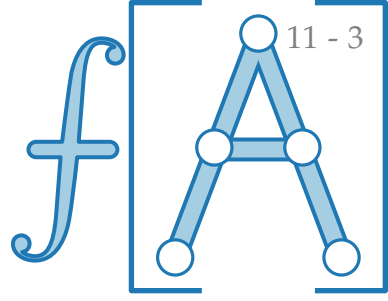


Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

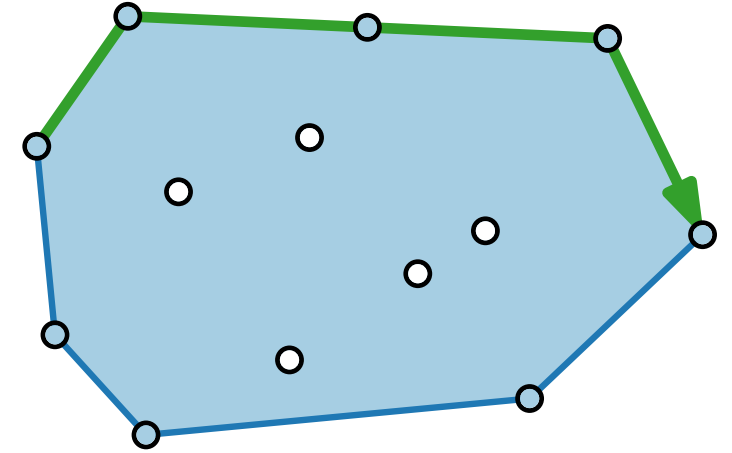


ANDREW-GRAHAM-SCAN

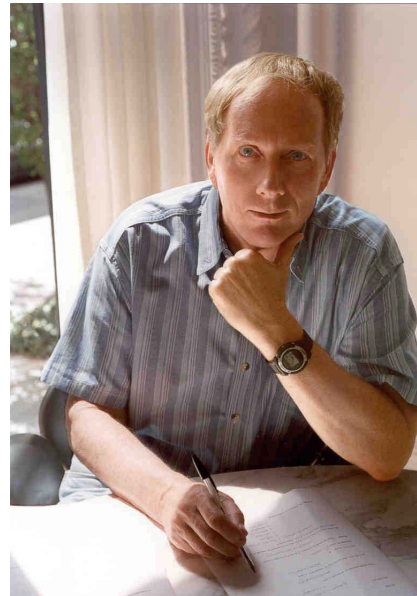
- Teile Berechnung in zwei Abschnitte



obere konvexe Hülle

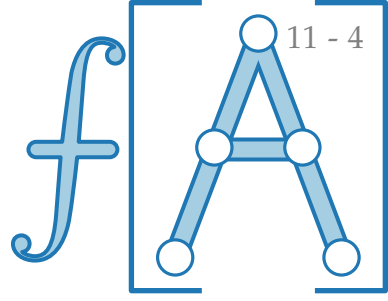


Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

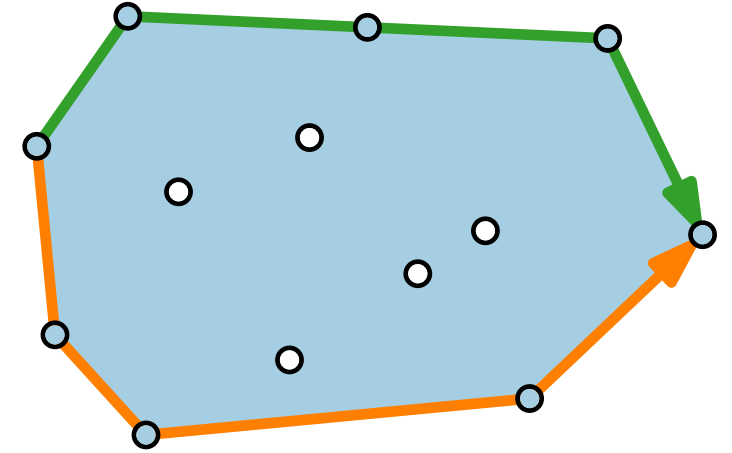


ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte

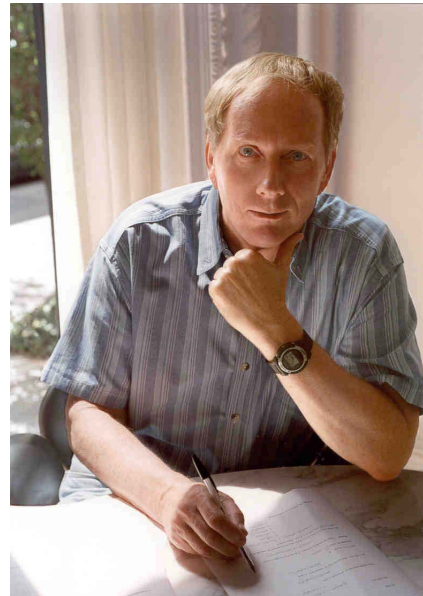


obere konvexe Hülle



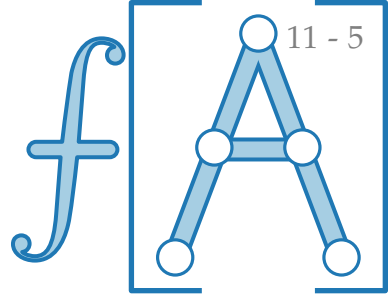
untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

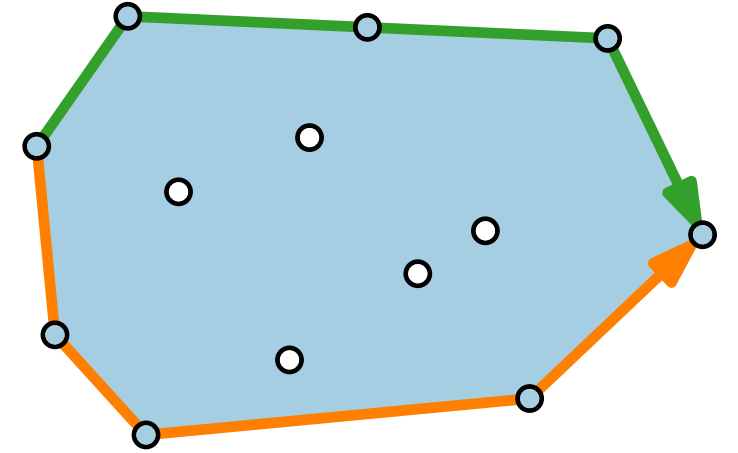


ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge

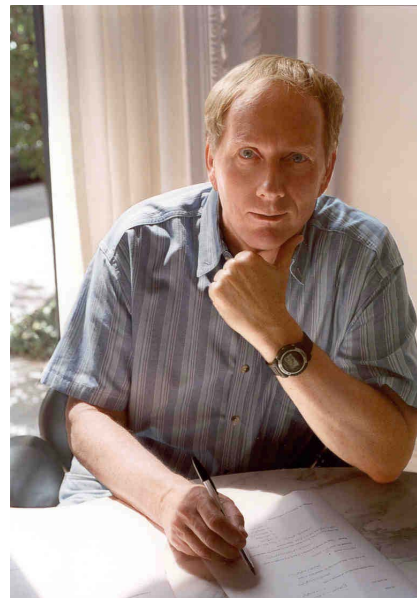


obere konvexe Hülle



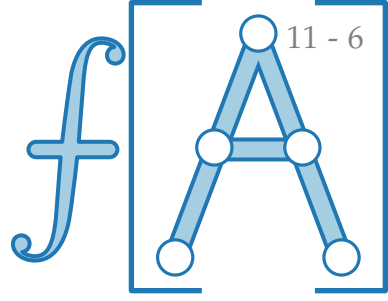
untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

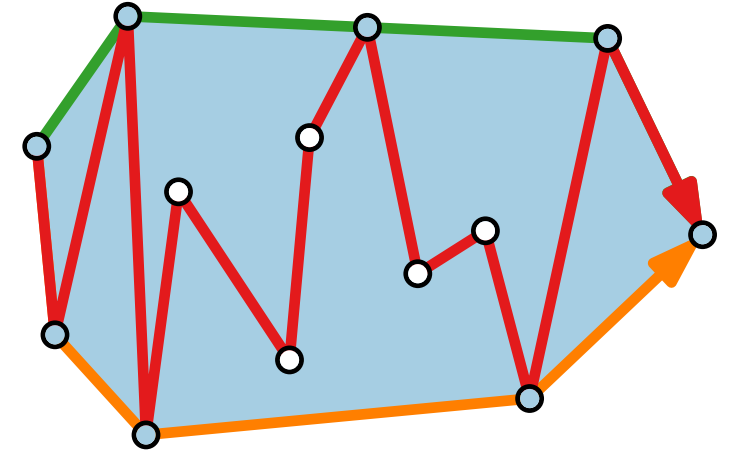


ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge

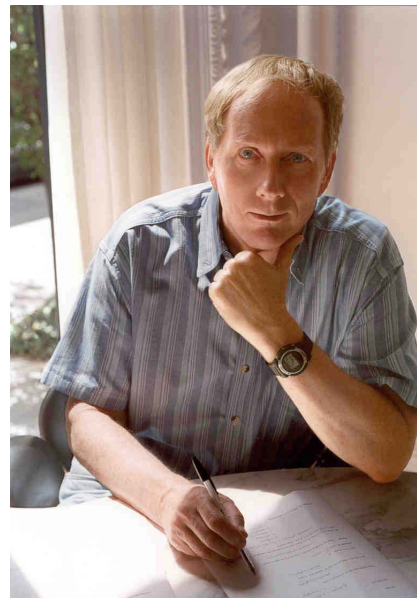


obere konvexe Hülle



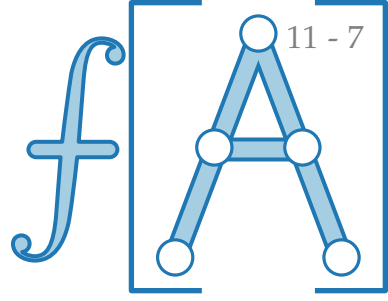
untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

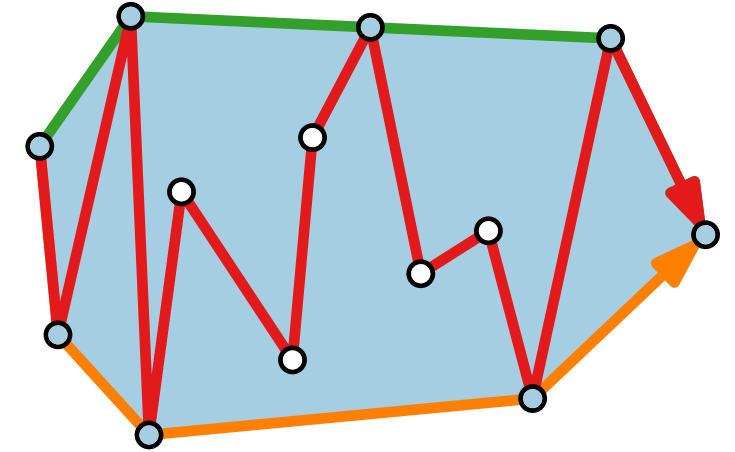


ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

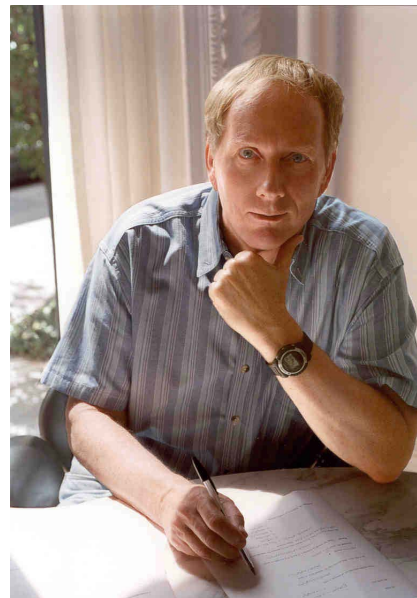


obere konvexe Hülle



untere konvexe Hülle

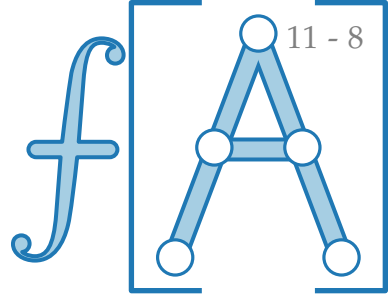
Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



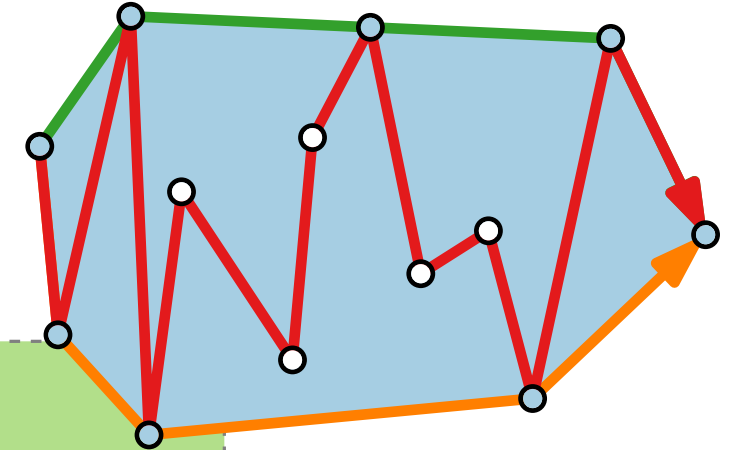
ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

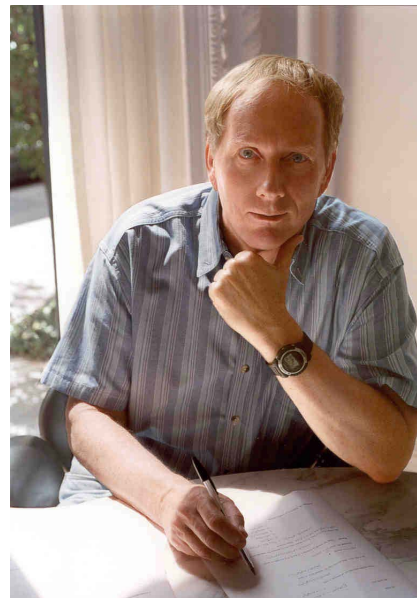


obere konvexe Hülle



untere konvexe Hülle

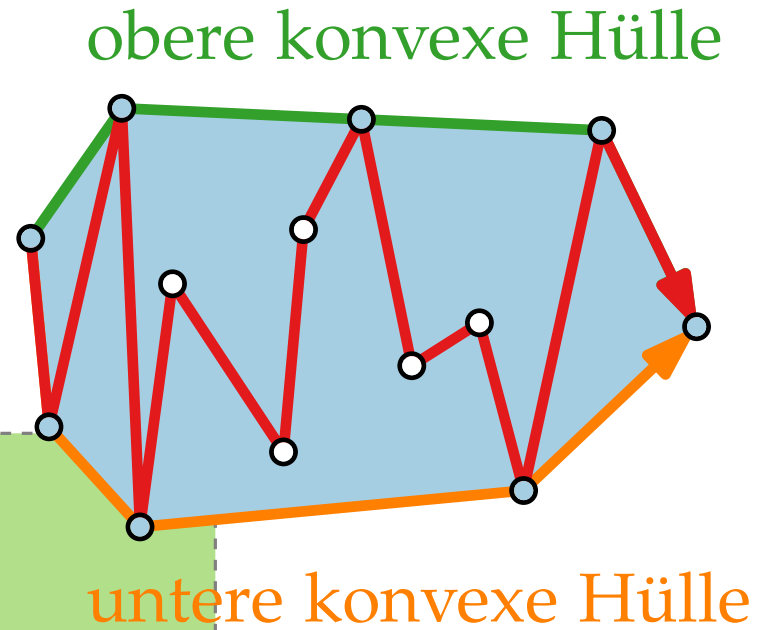
Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



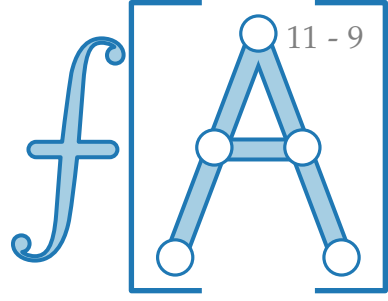
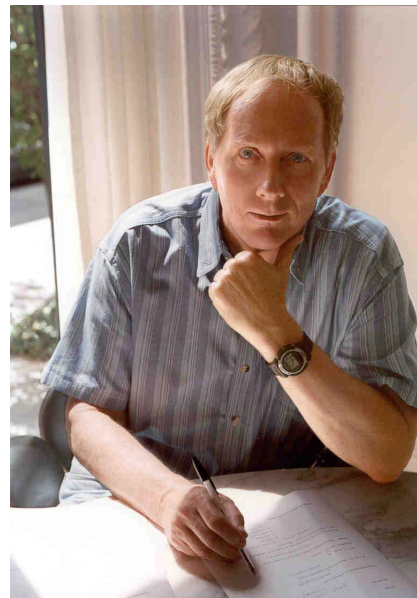
ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBERE KONVEXE HÜLLE(S)



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

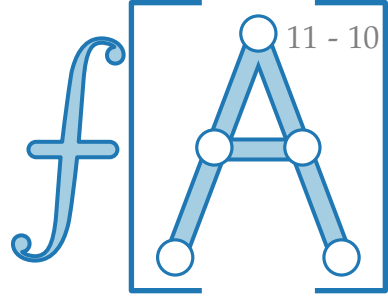


ANDREW-GRAHAM-SCAN

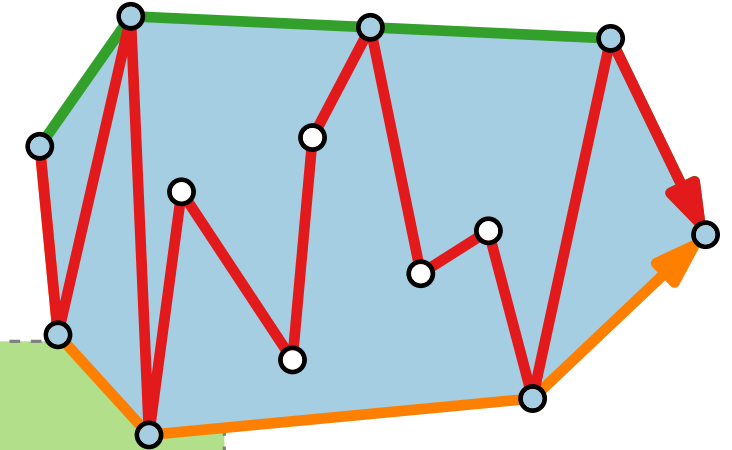
- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBERE KONVEXE HÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

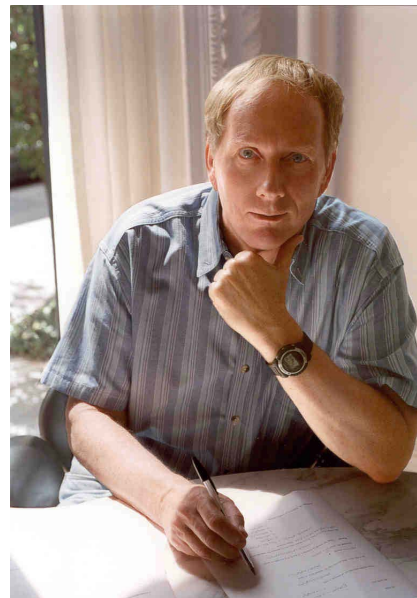


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

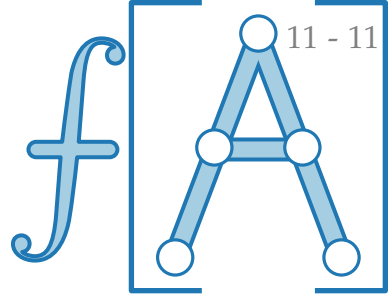


ANDREW-GRAHAM-SCAN

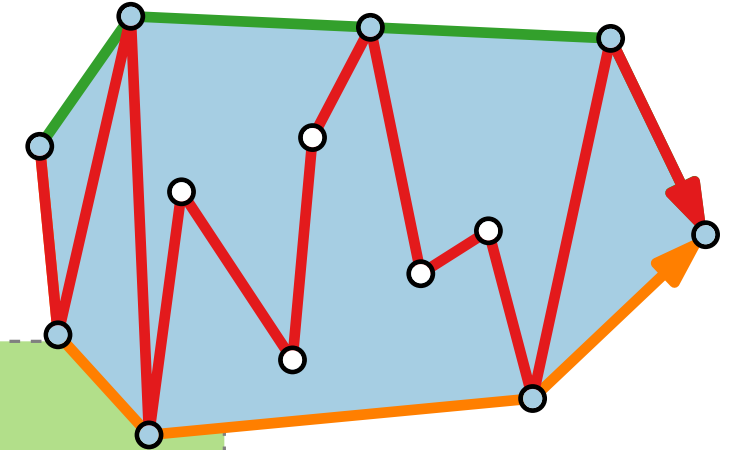
- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

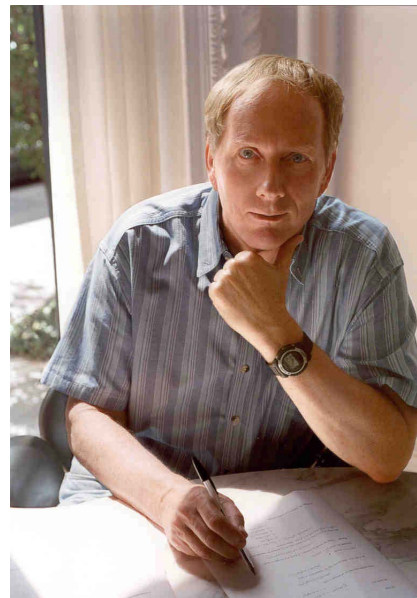


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA

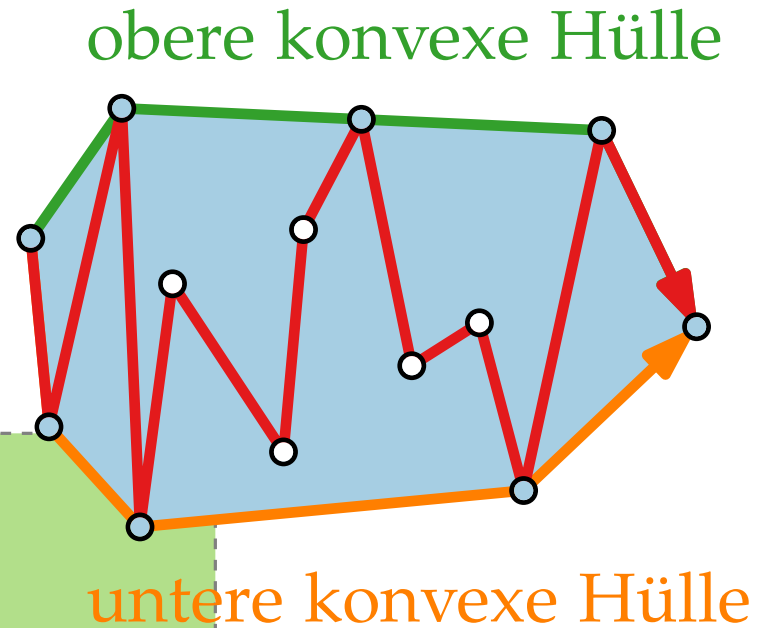


ANDREW-GRAHAM-SCAN

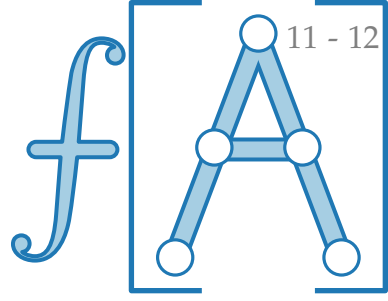
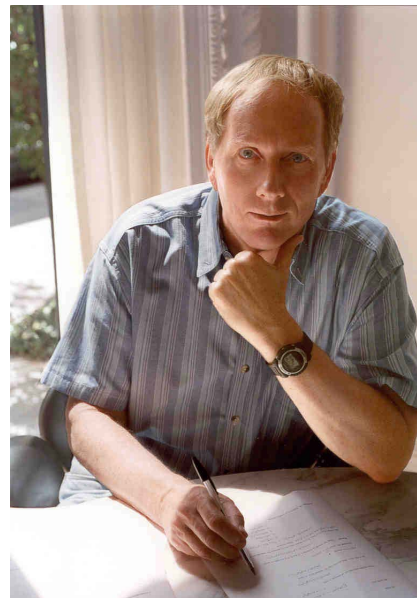
- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBERE KONVEXE HÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$
 $L = \langle p_1, p_2 \rangle$



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



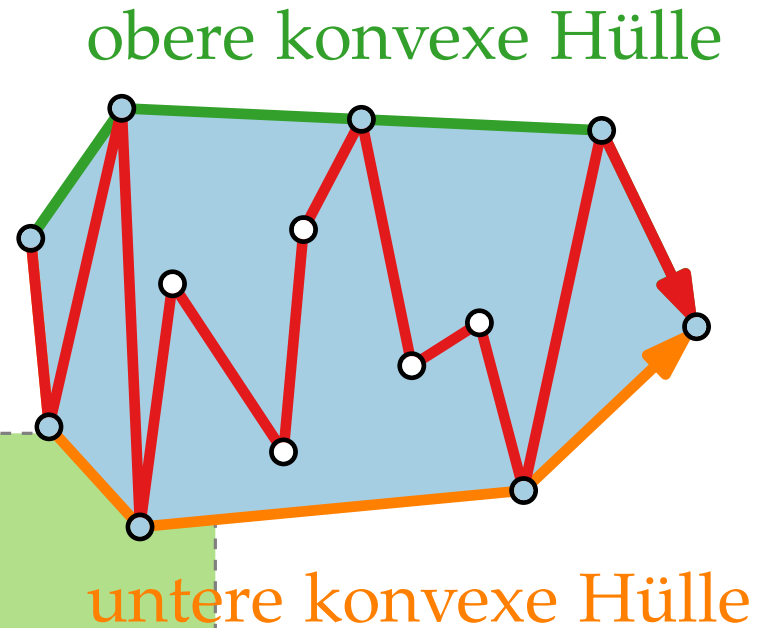
ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

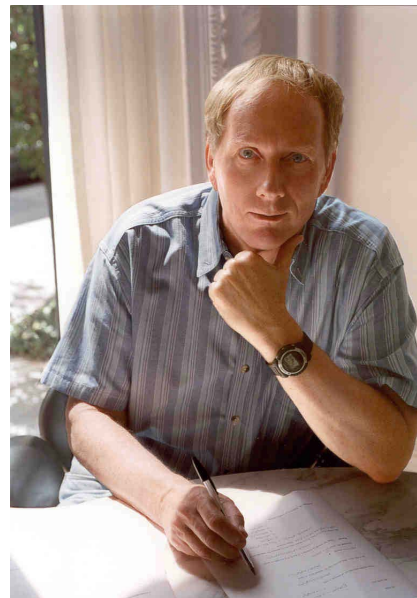
OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$
 $L = \langle p_1, p_2 \rangle$

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

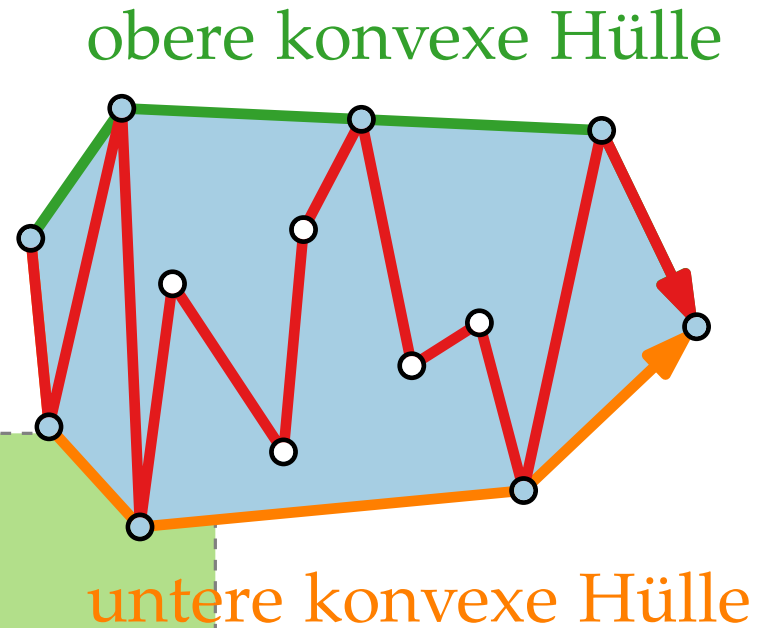
OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

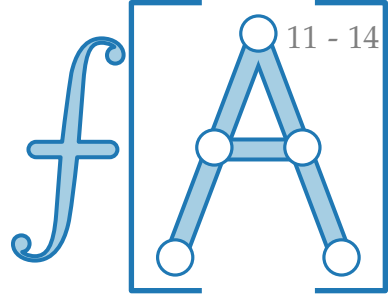
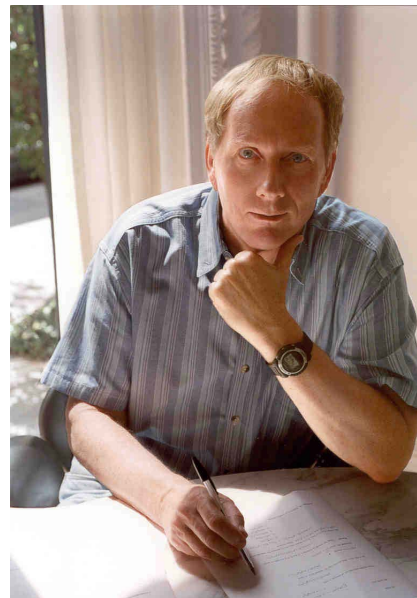
$L = \langle p_1, p_2 \rangle$

for $i = 3$ to n do

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBERE KONVEXE HÜLLE(S)

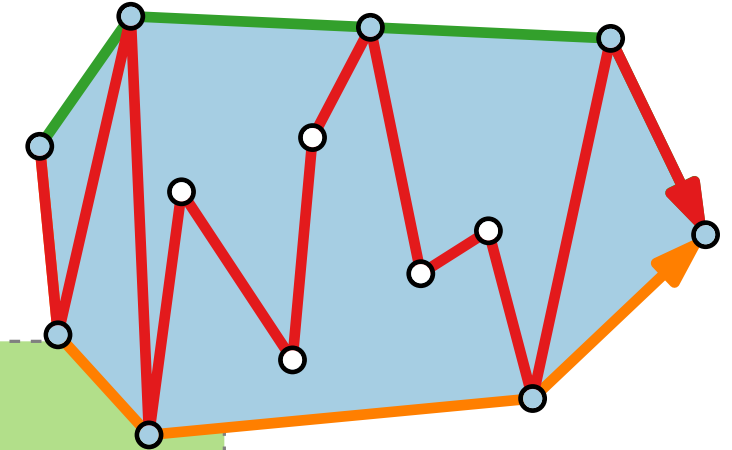
$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

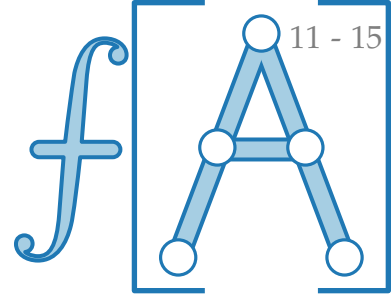
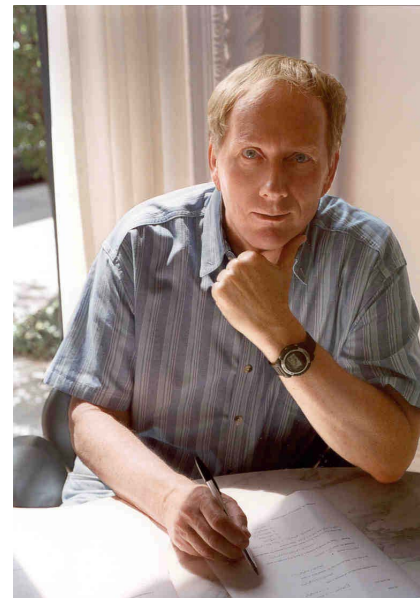
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

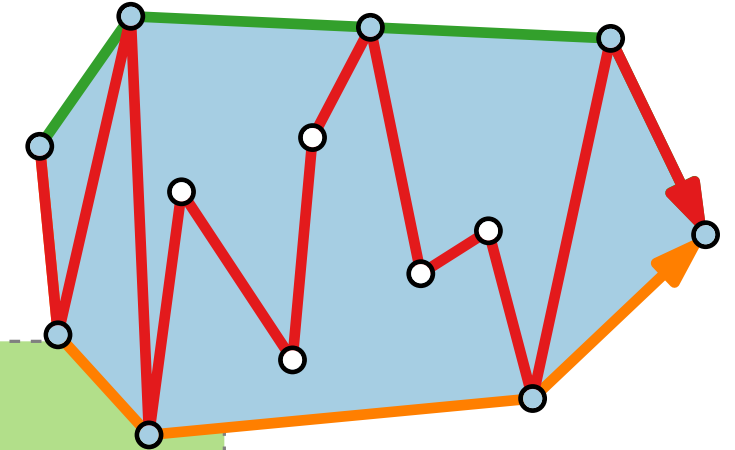
$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

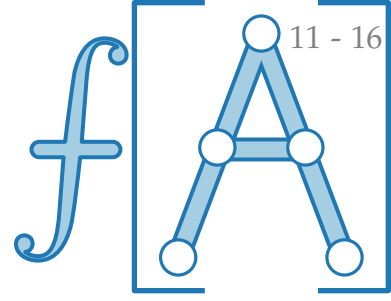
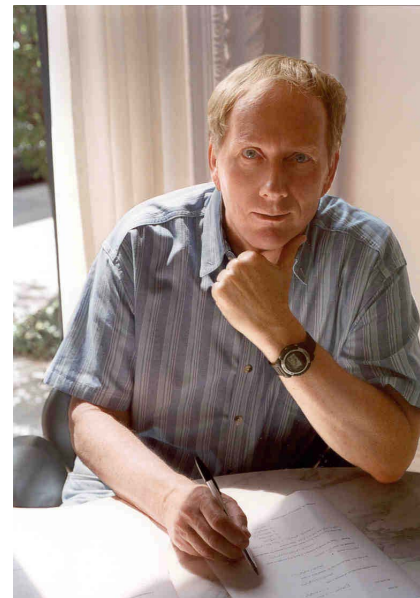
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

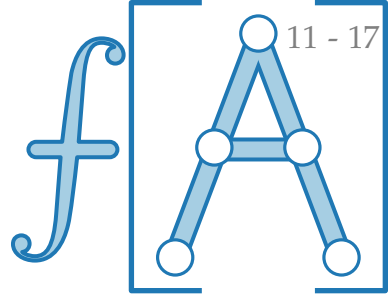
$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

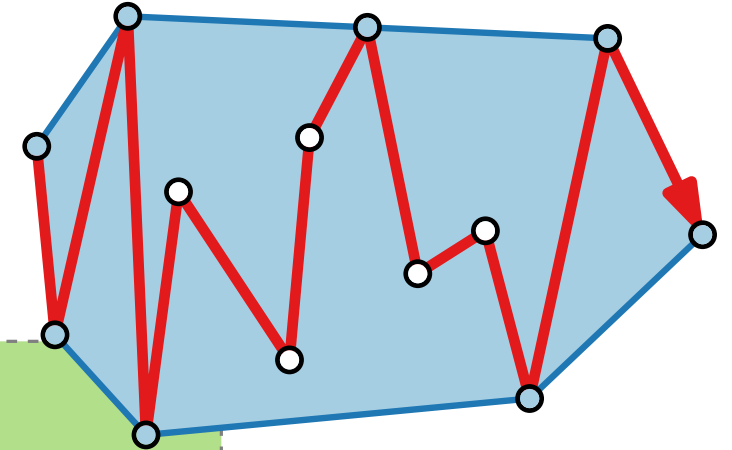
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

return L

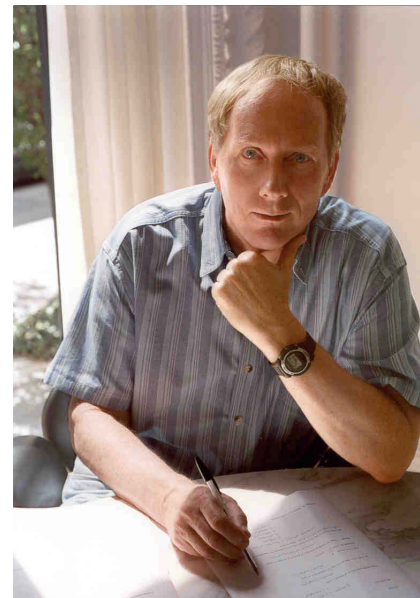


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

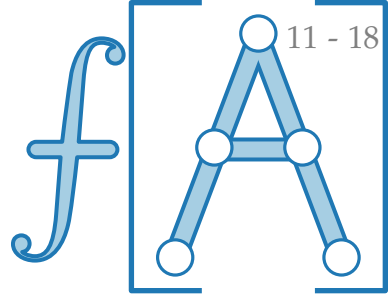
$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

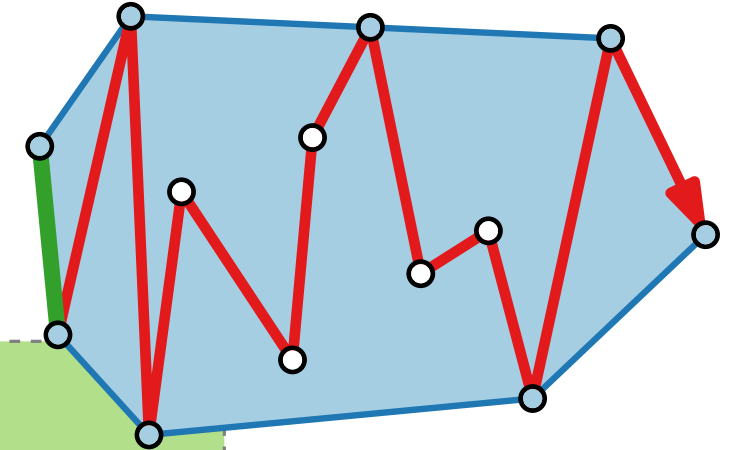
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

return L

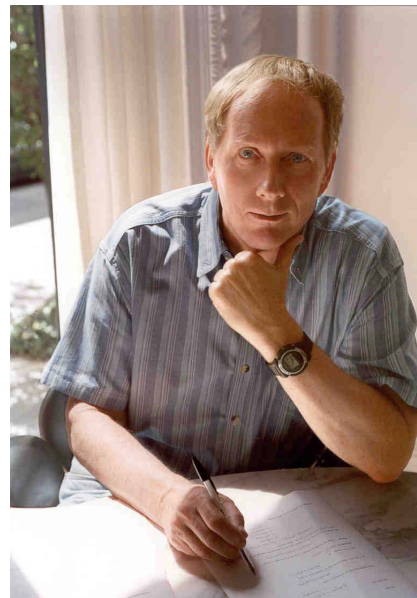


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

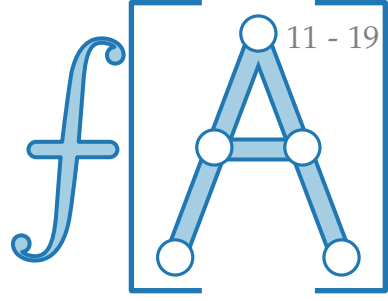
$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

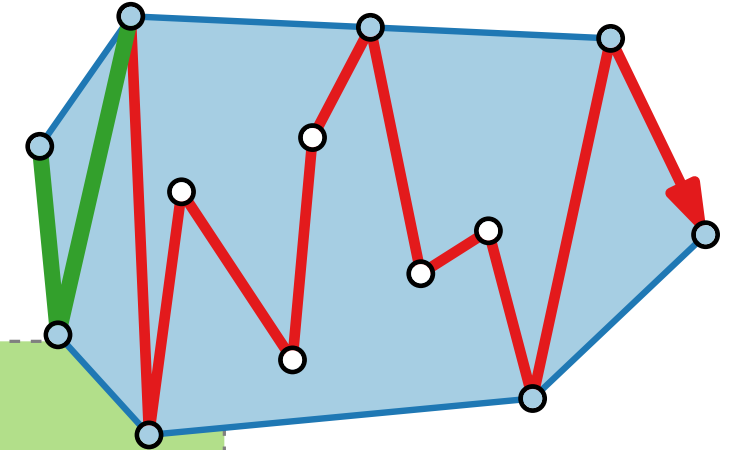
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

return L

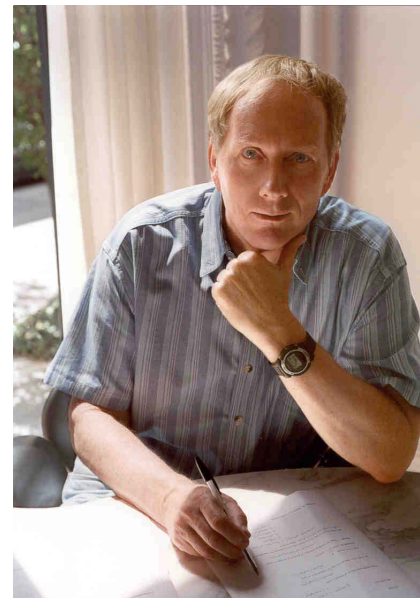


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

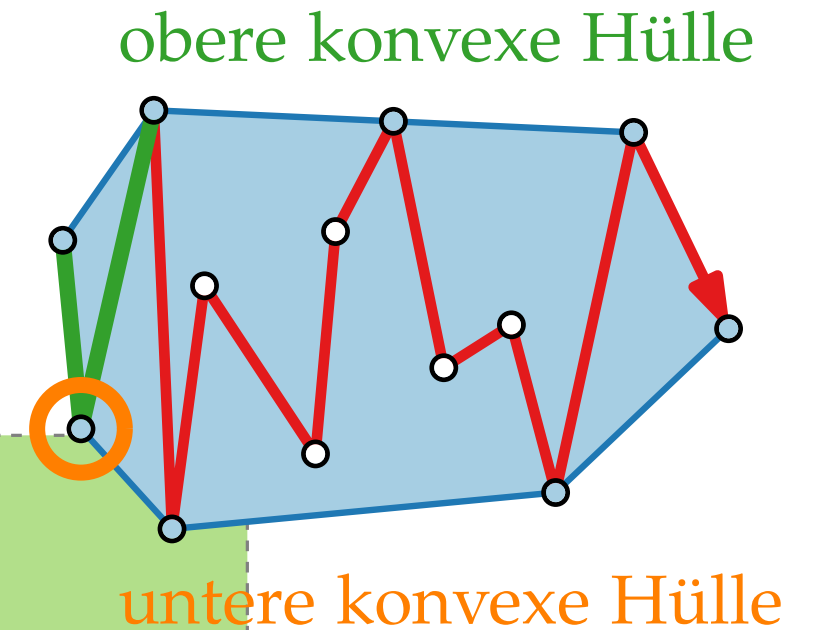
$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

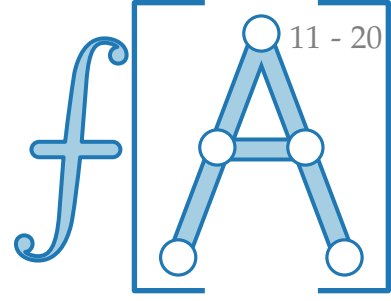
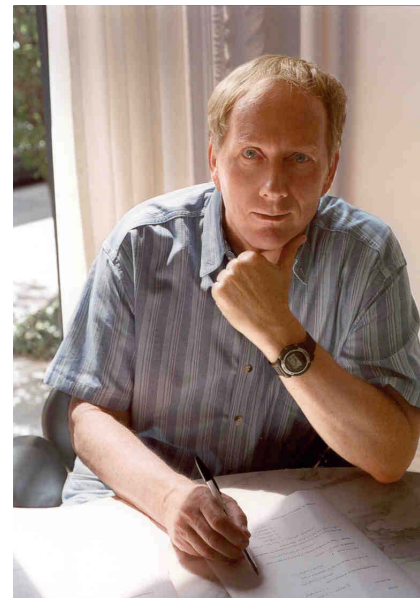
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

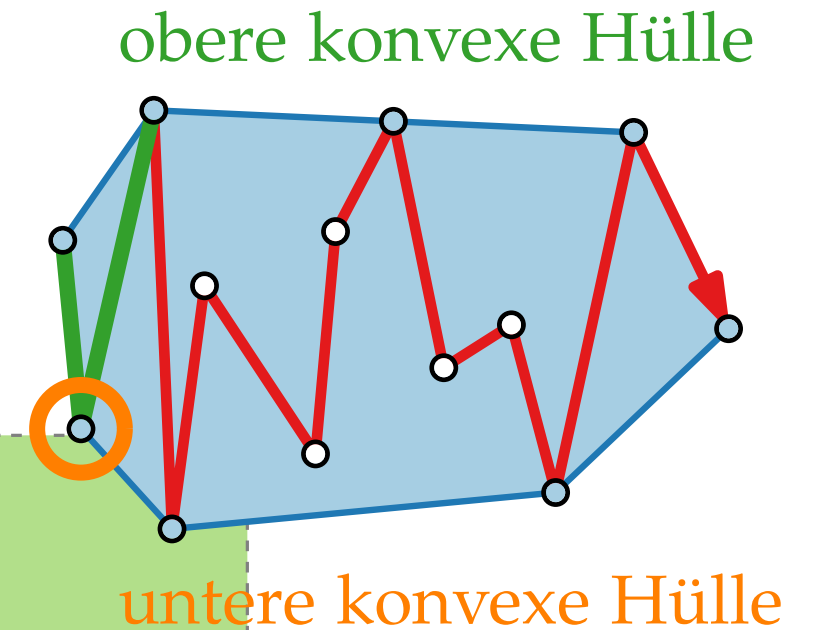
$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

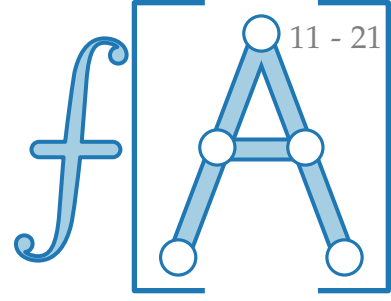
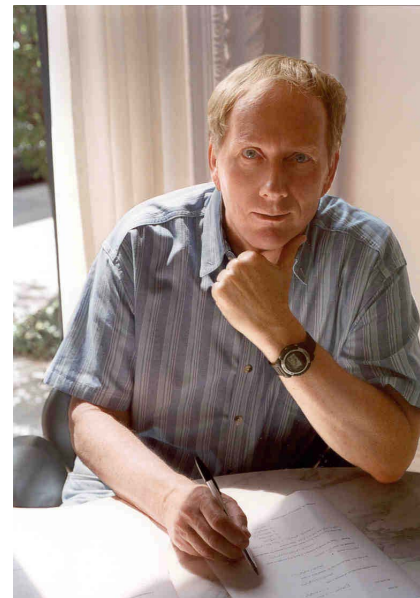
$L.\text{append}(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

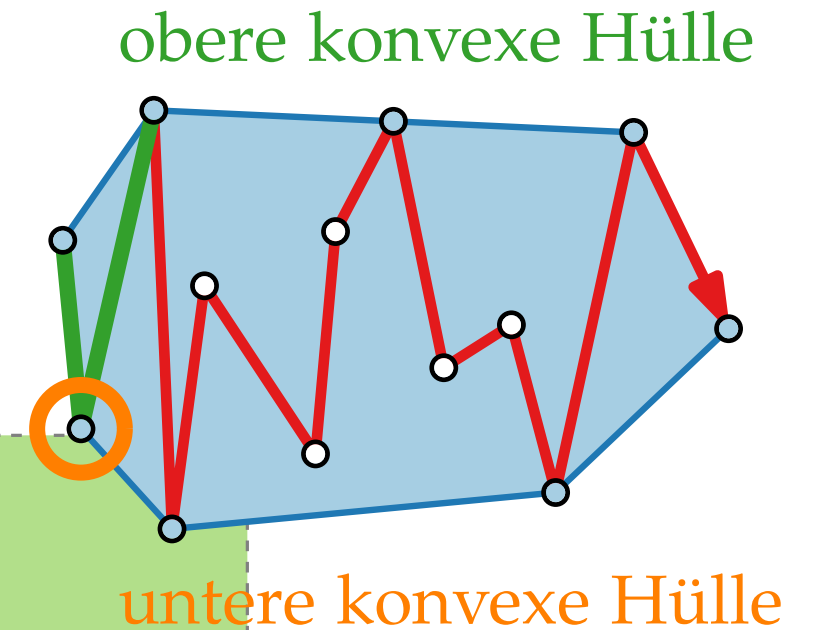
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

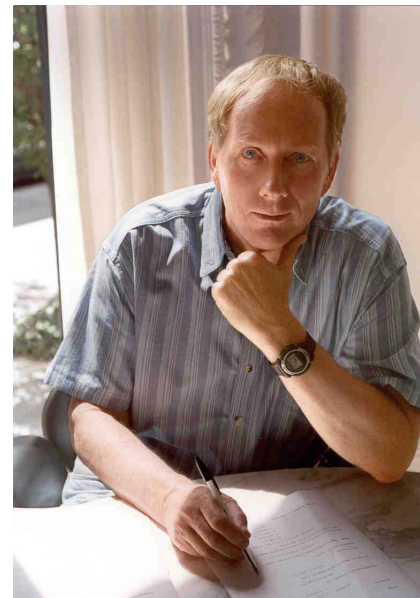
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

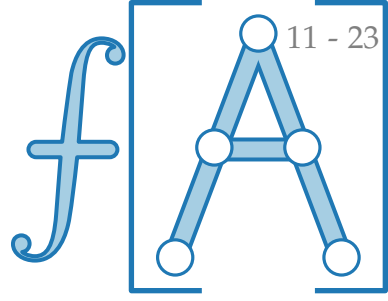
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

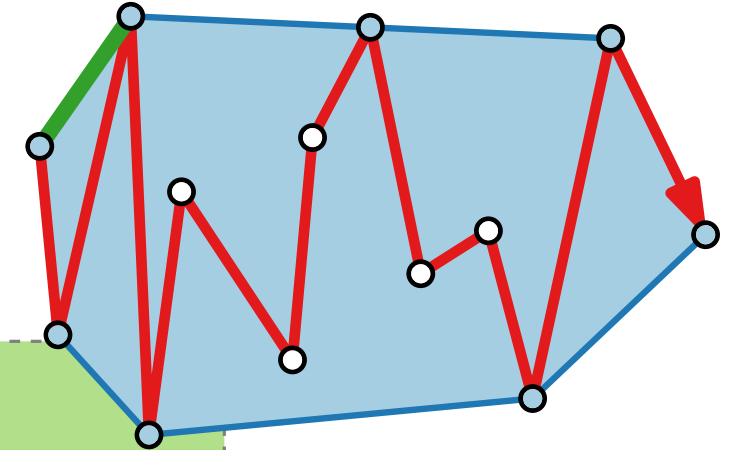
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

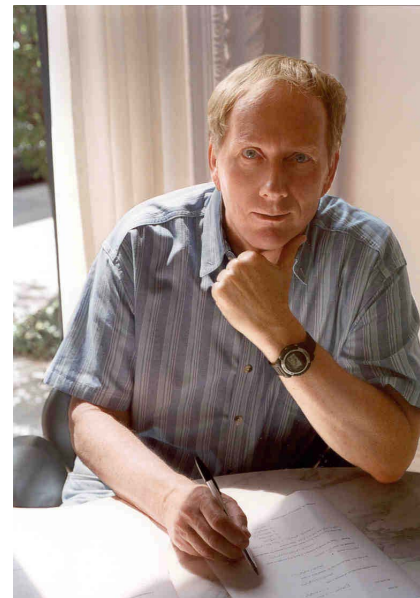


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

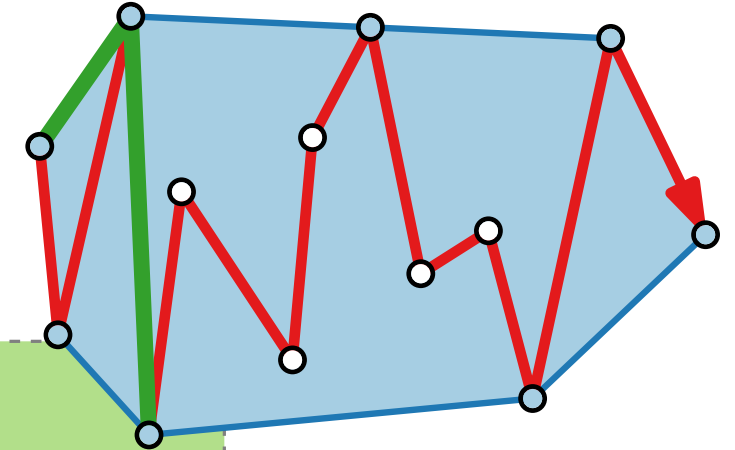
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

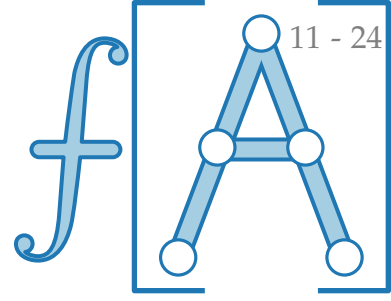
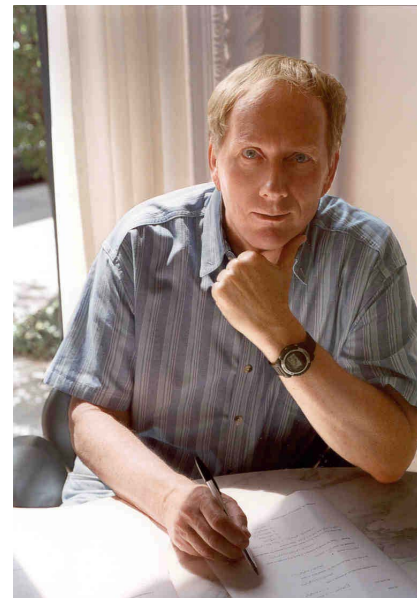
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

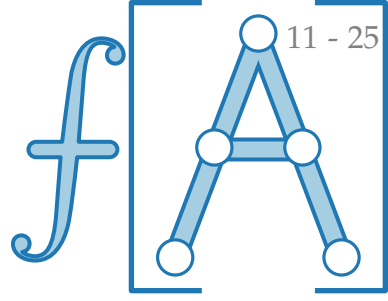
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

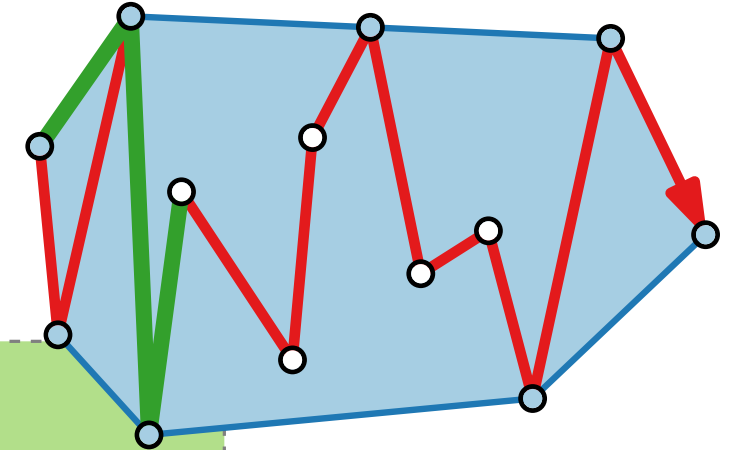
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

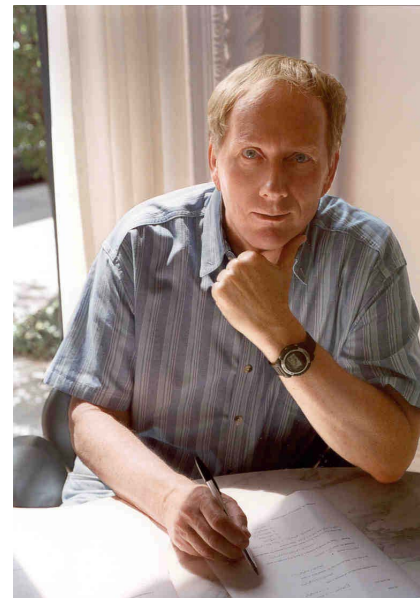


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

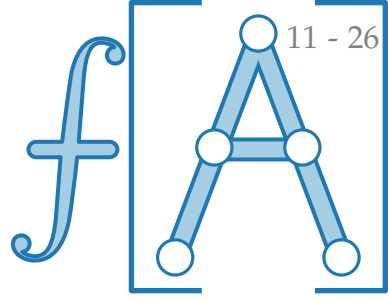
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

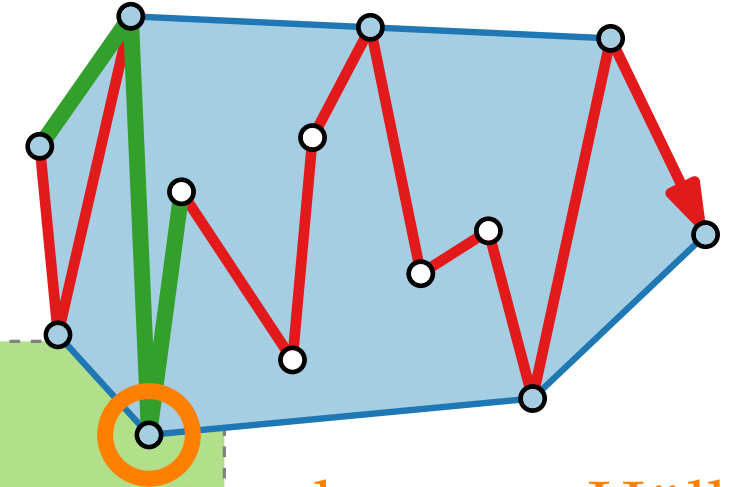
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

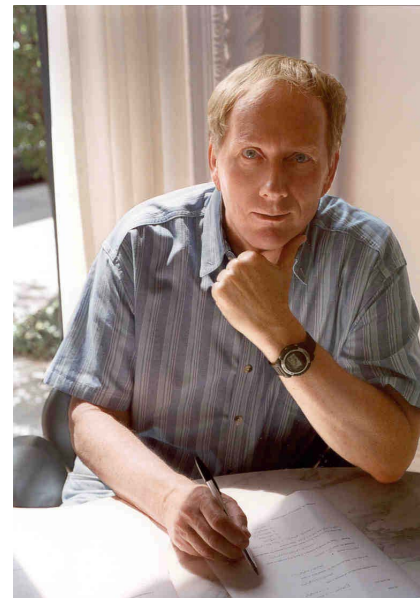


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

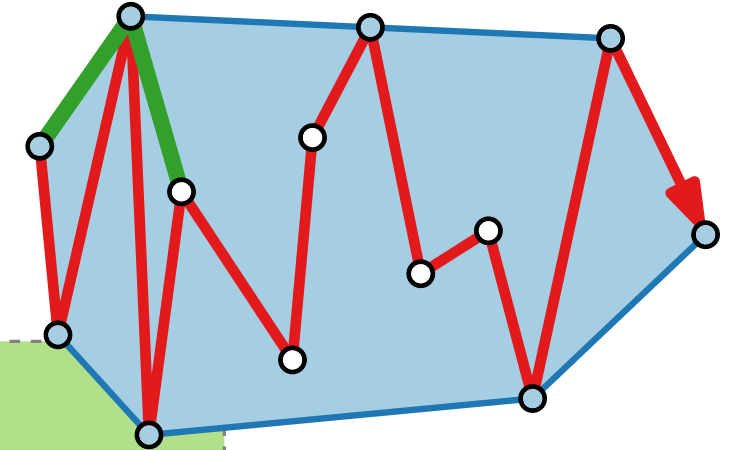
$L.\text{append}(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

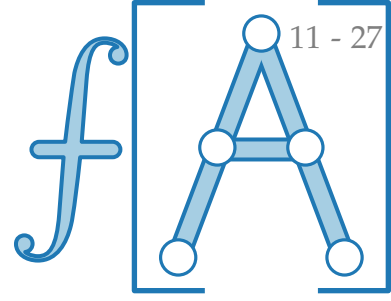
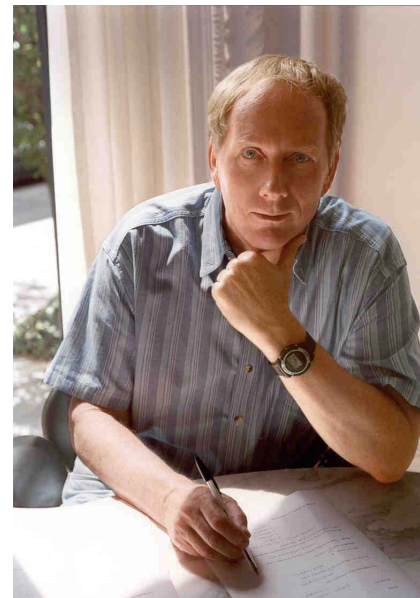
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

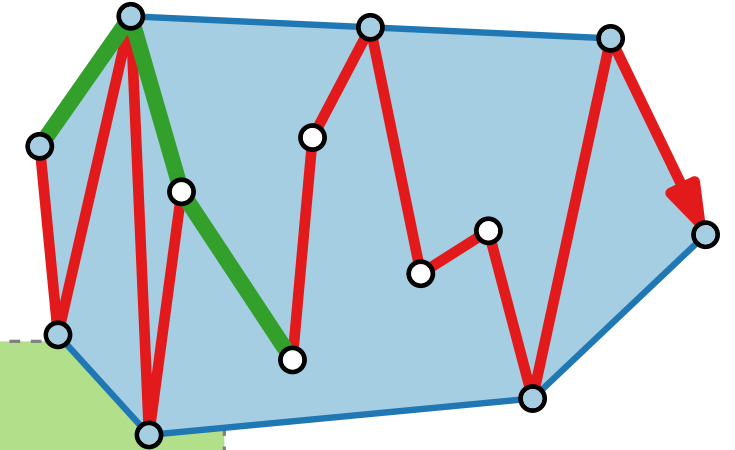
$L.\text{append}(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

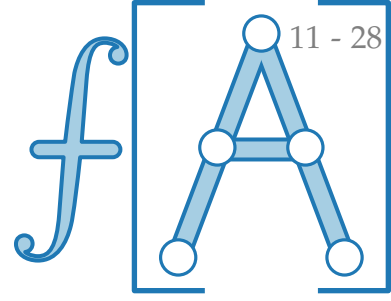
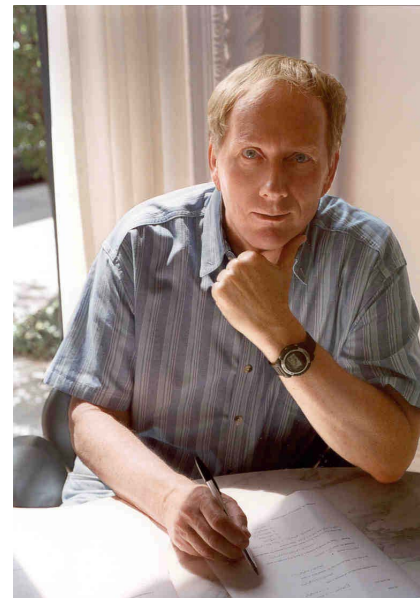
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

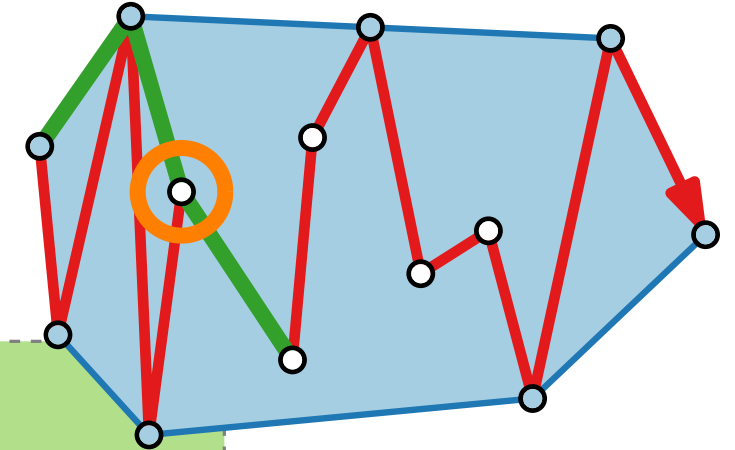
$L.\text{append}(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

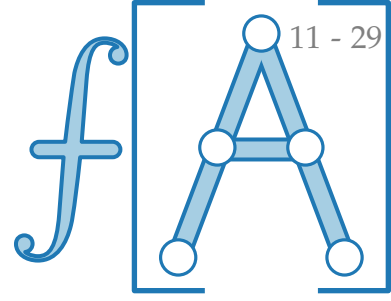
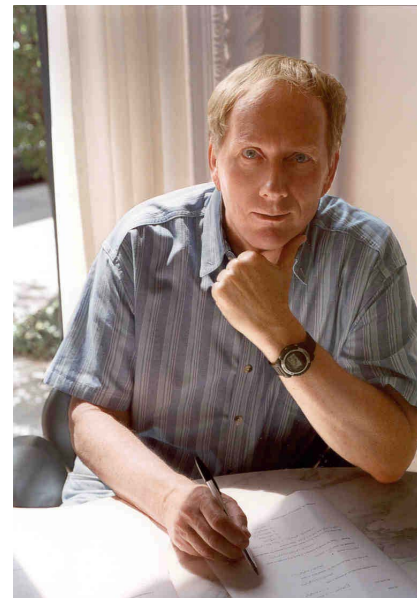
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

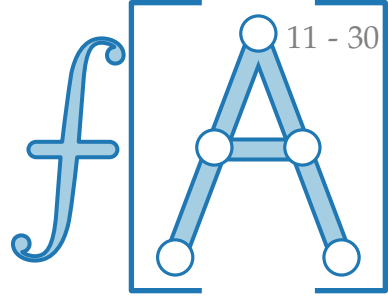
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

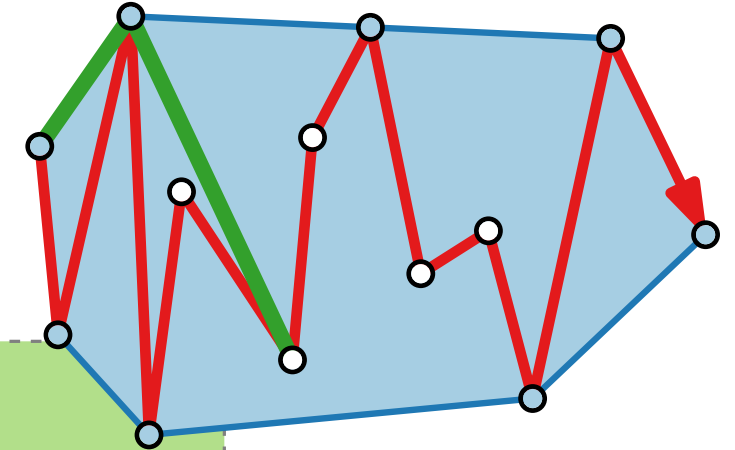
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

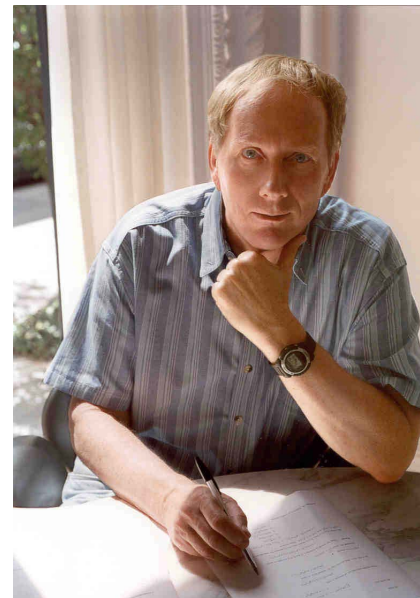


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

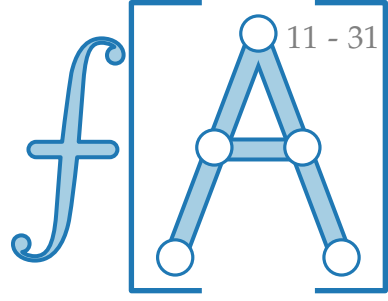
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

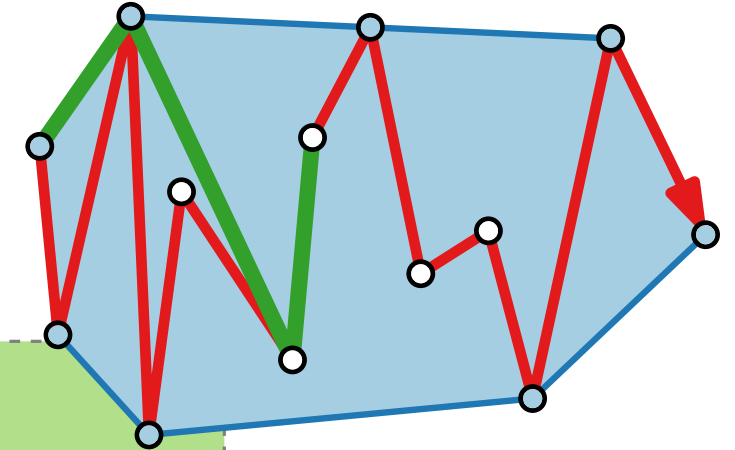
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

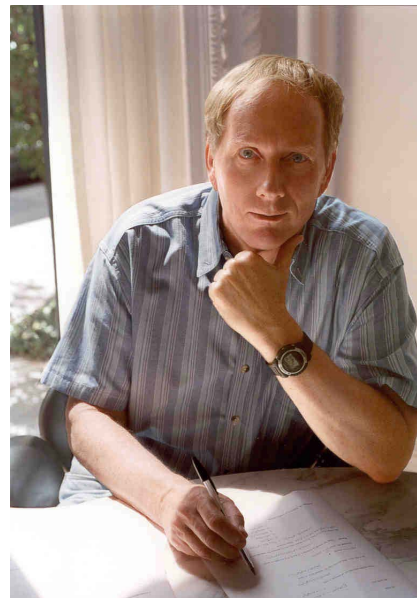


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

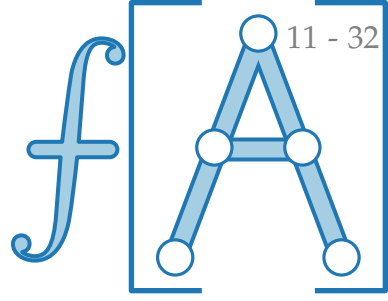
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

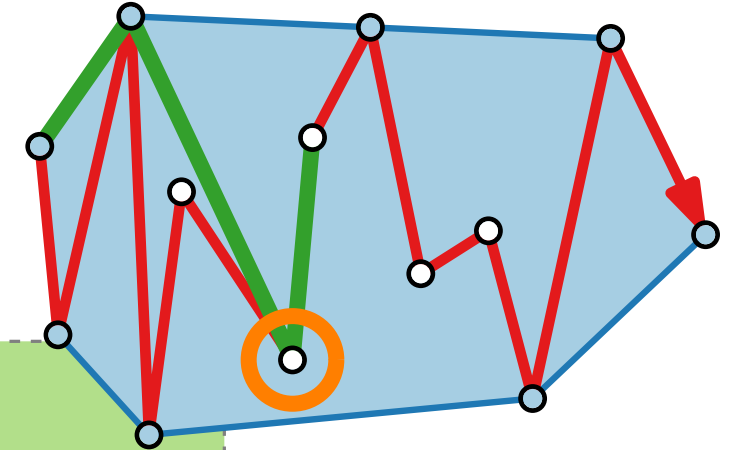
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

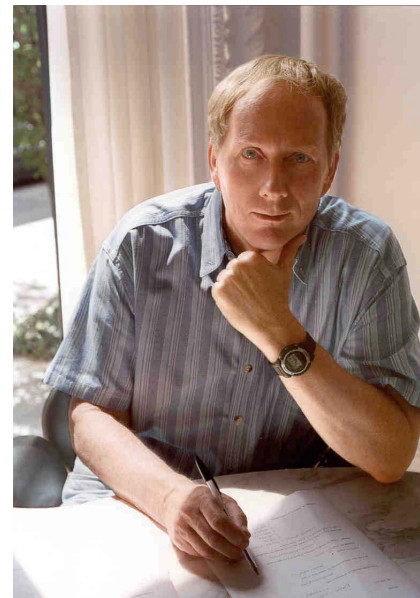


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

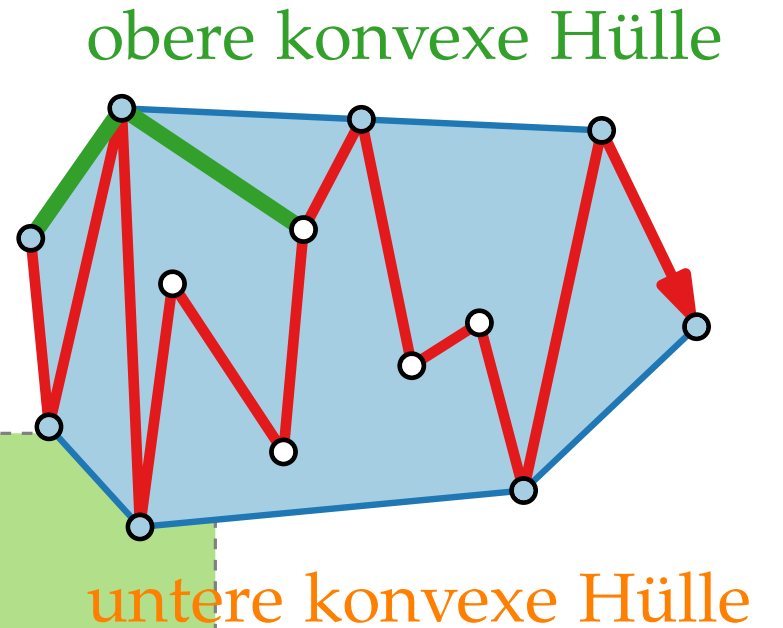
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

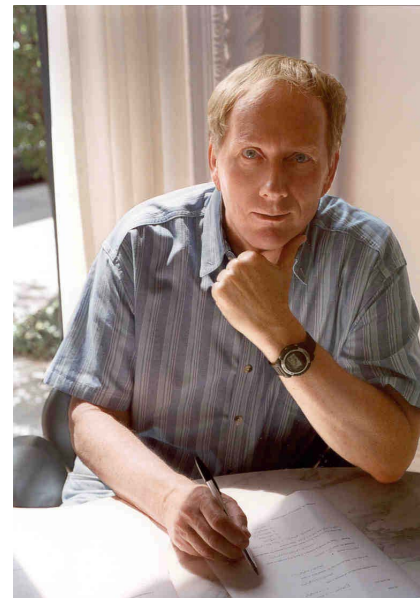
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

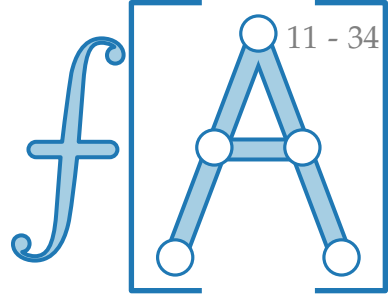
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

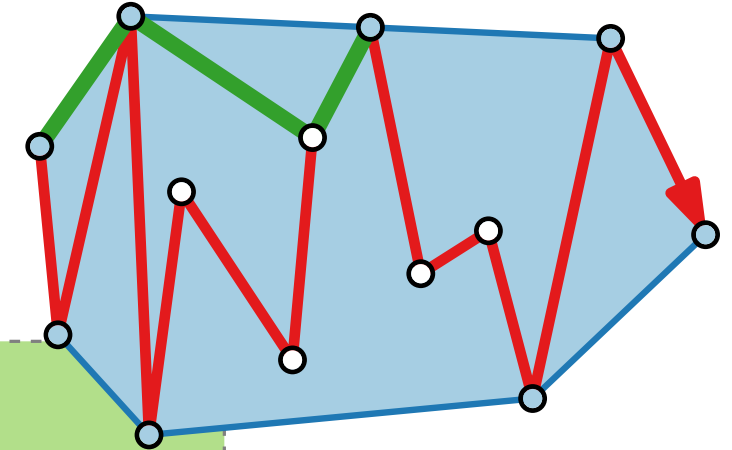
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

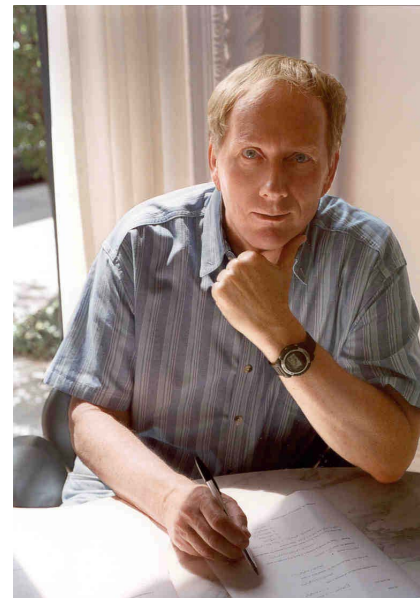


obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

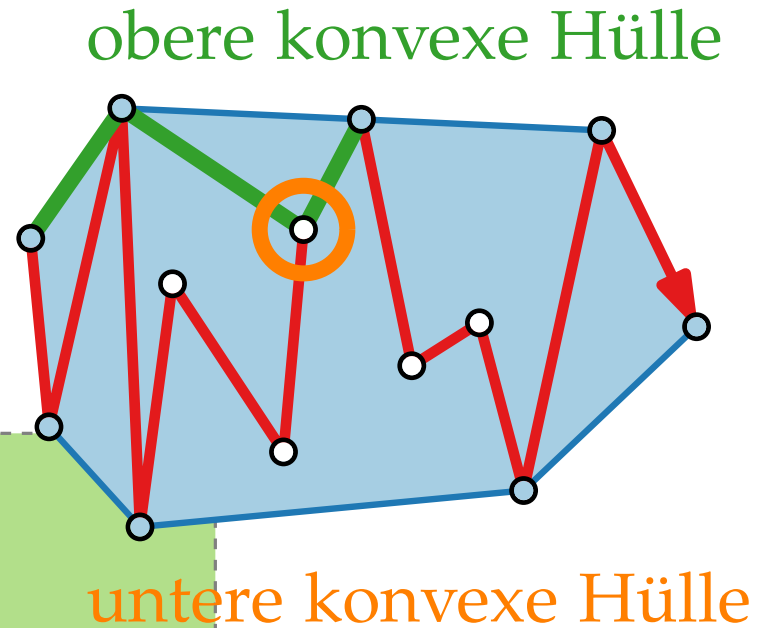
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

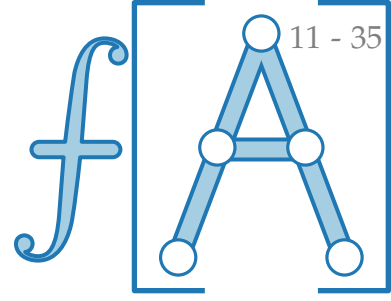
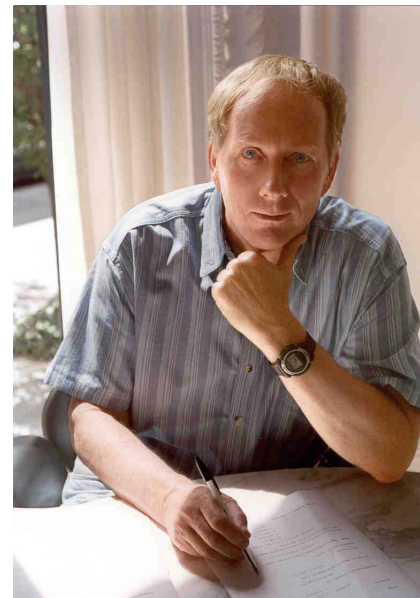
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

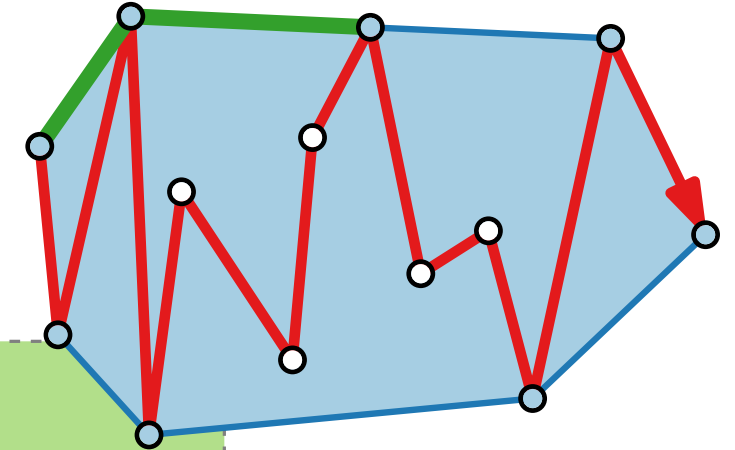
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

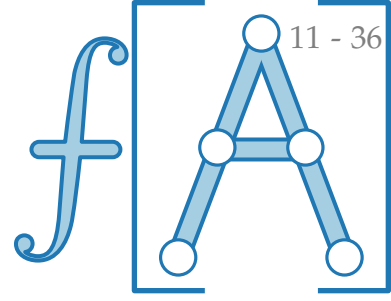
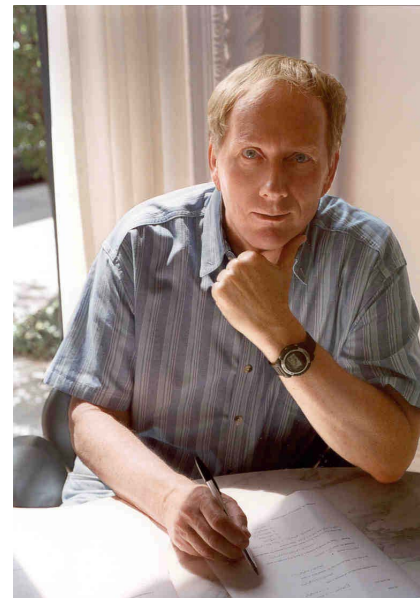
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

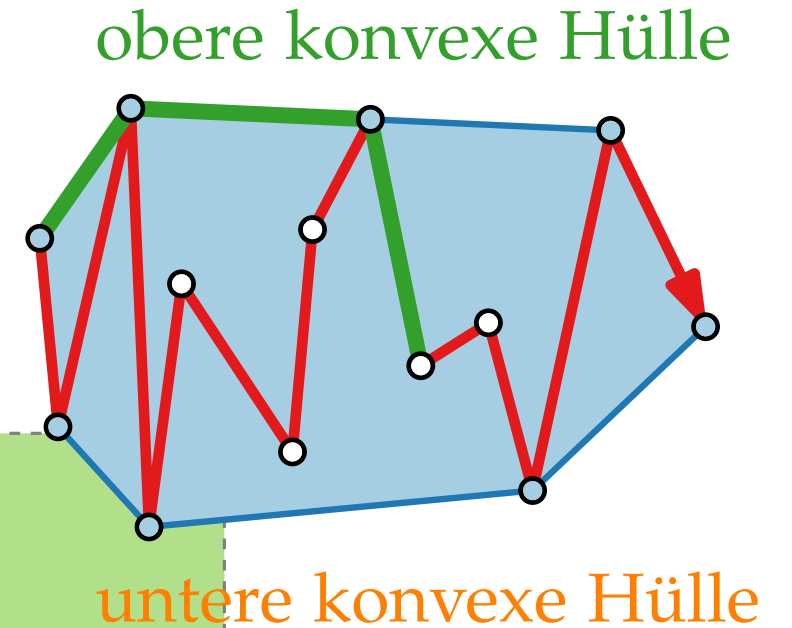
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

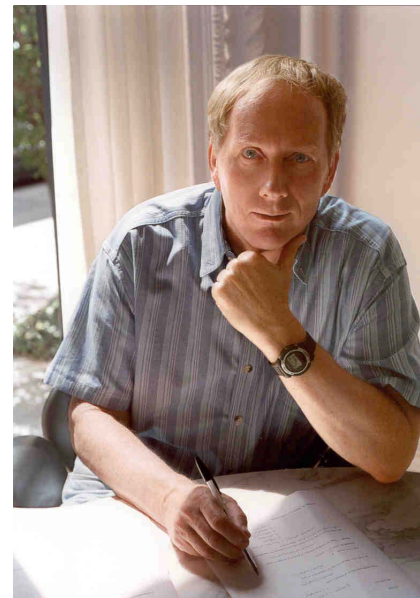
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

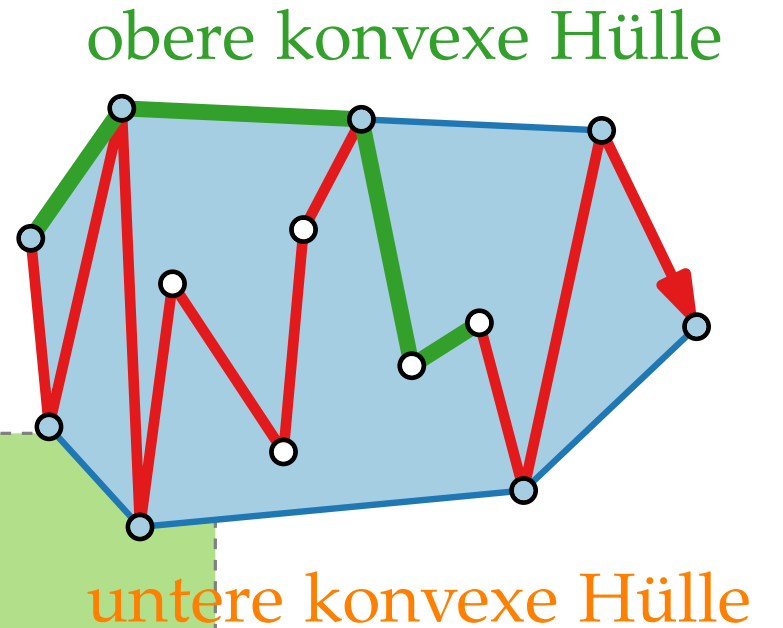
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

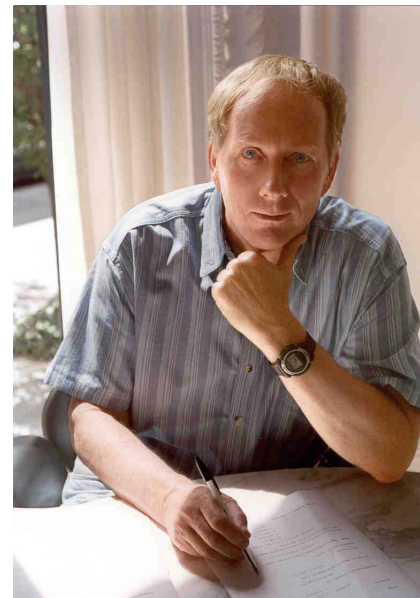
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

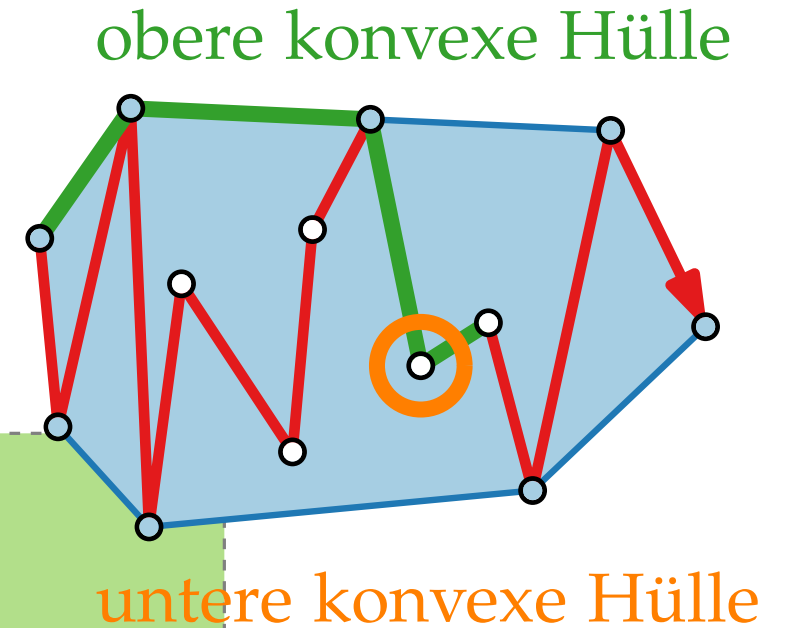
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

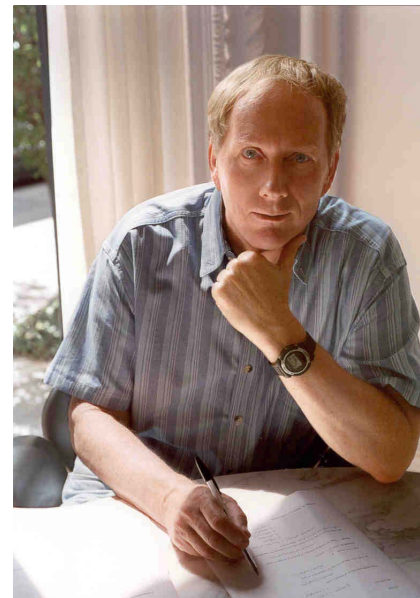
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

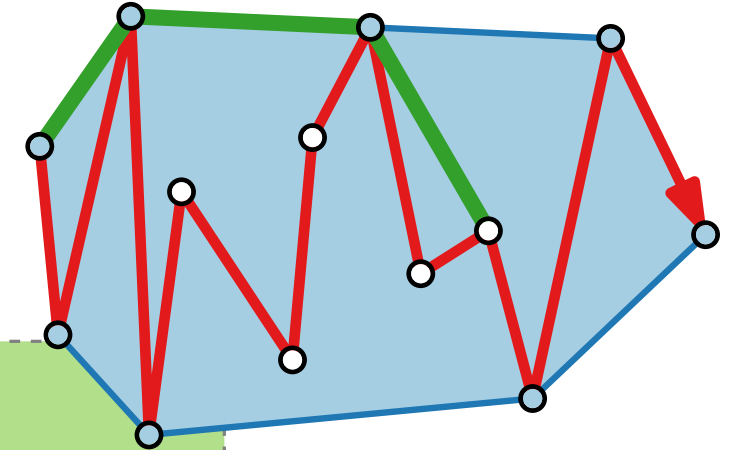
$L.\text{append}(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

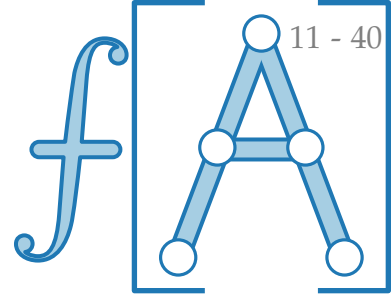
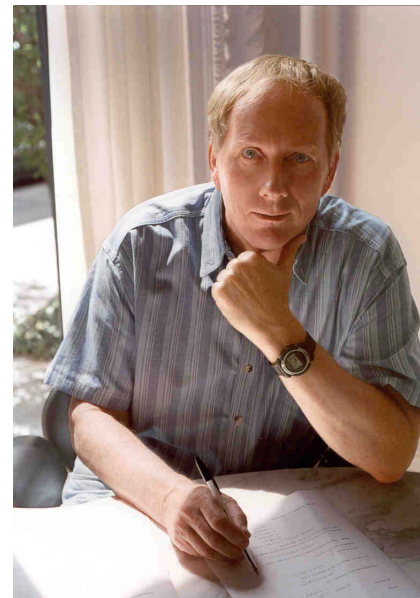
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

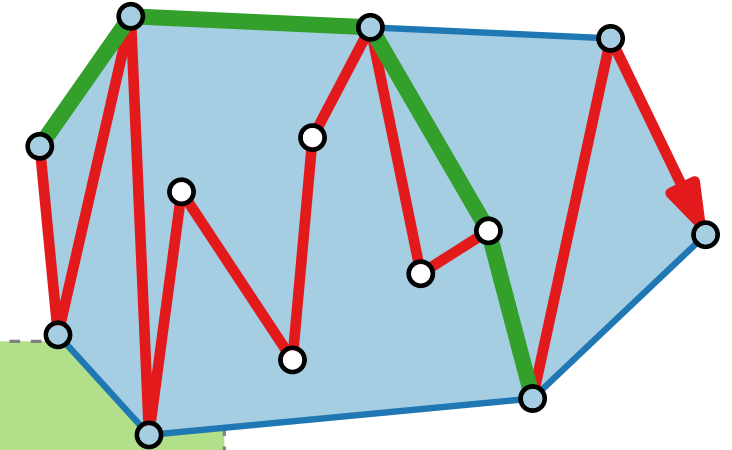
$L.\text{append}(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

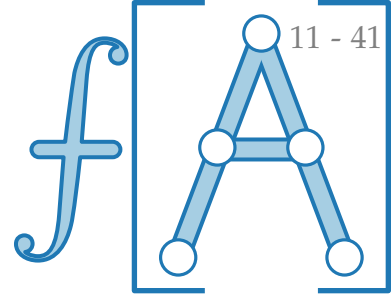
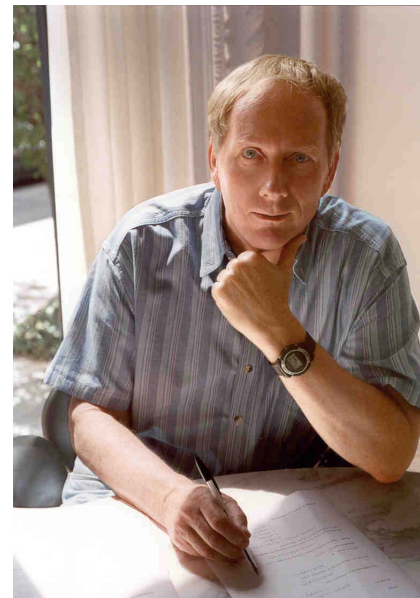
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

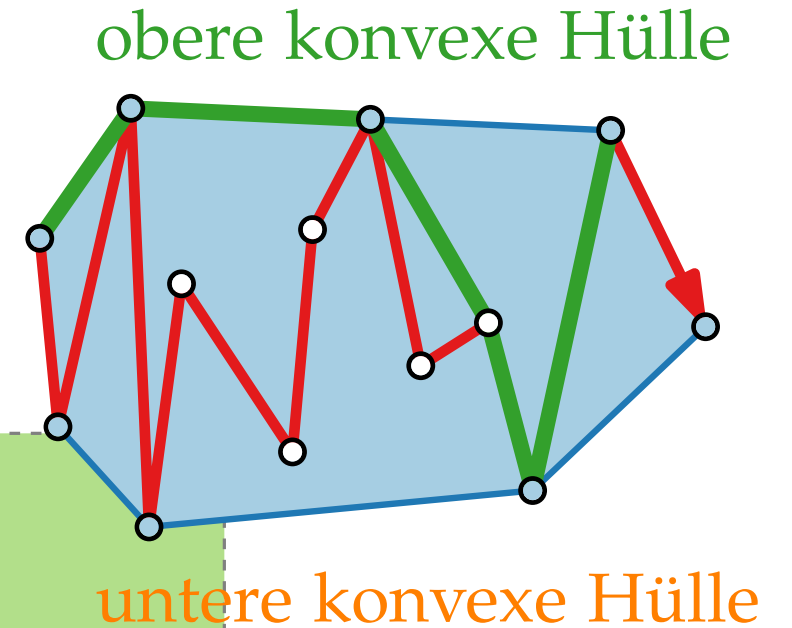
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.\text{append}(p_i)$

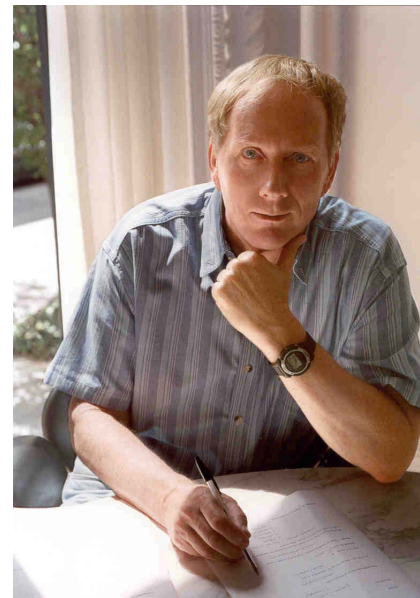
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

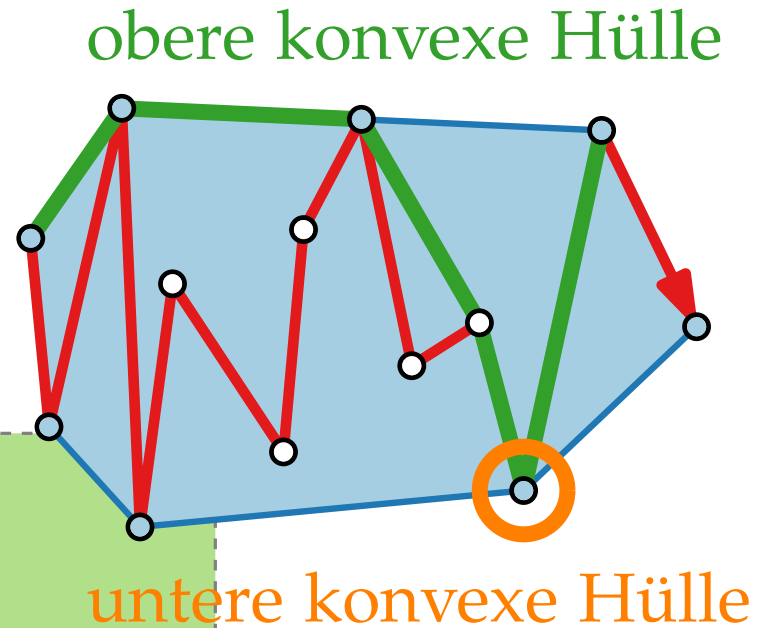
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

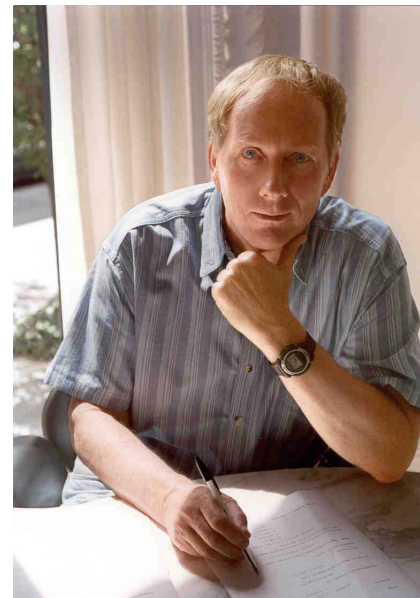
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

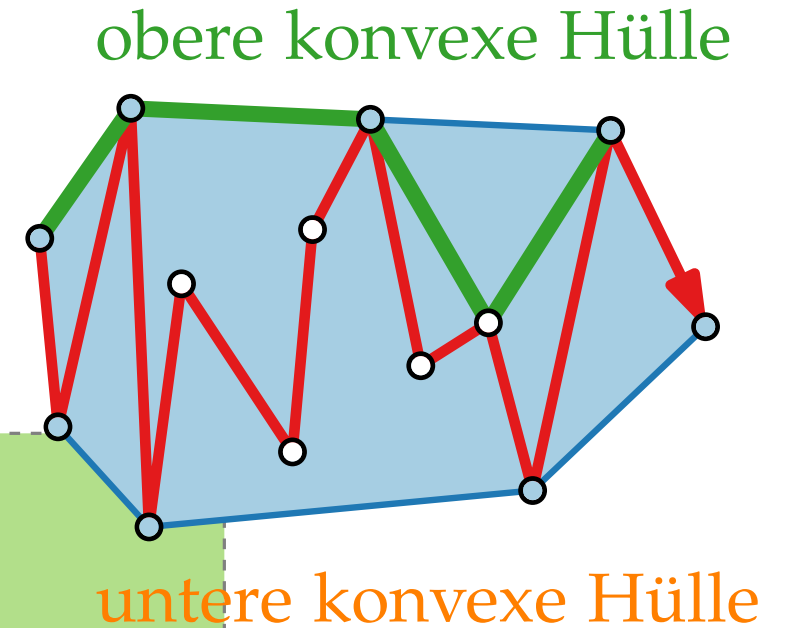
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

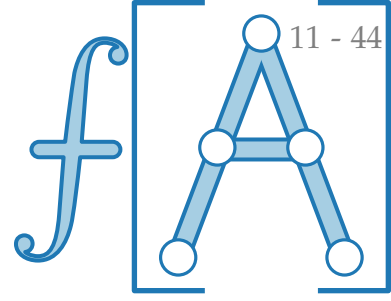
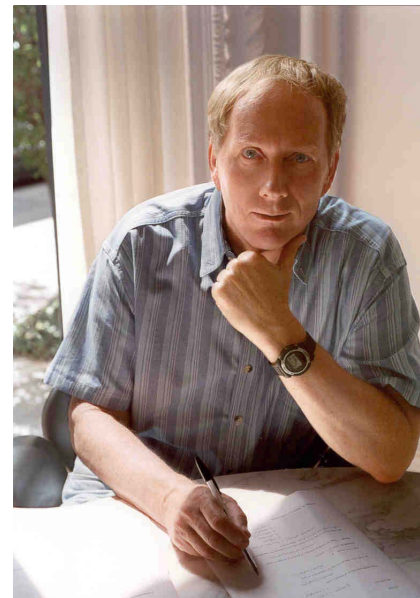
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

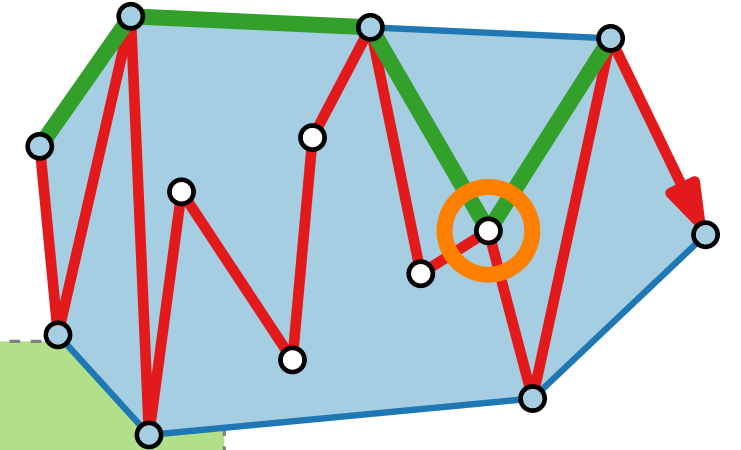
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

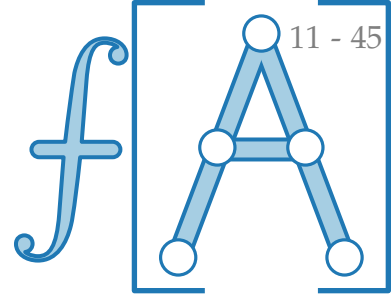
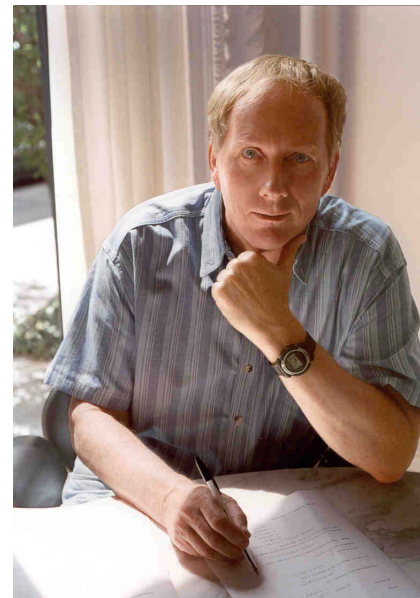
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle = \text{sortiere } S \text{ lexikografisch}$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

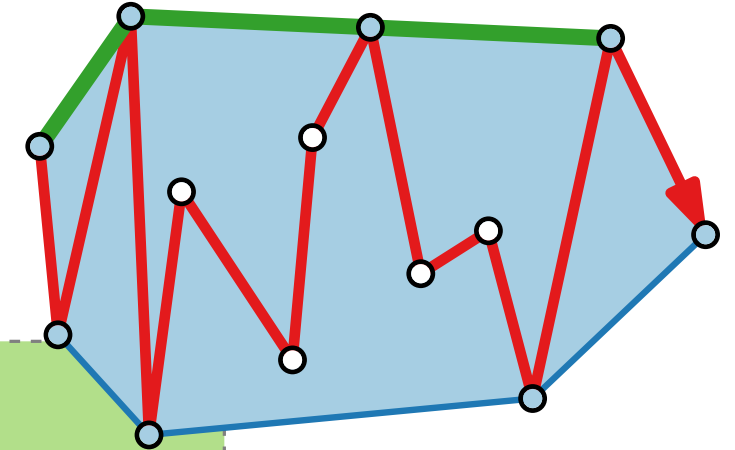
$L.\text{append}(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

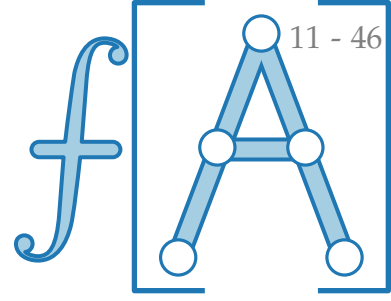
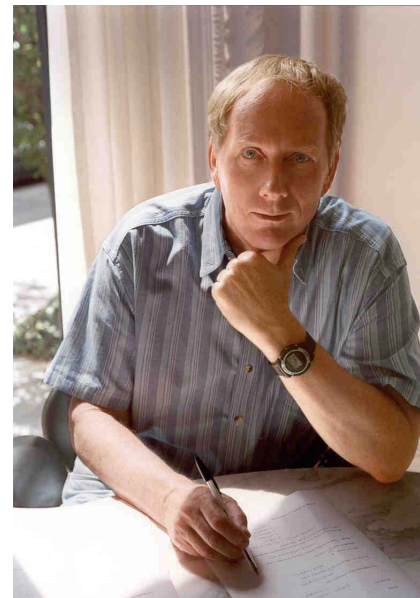
return L

obere konvexe Hülle



untere konvexe Hülle

Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

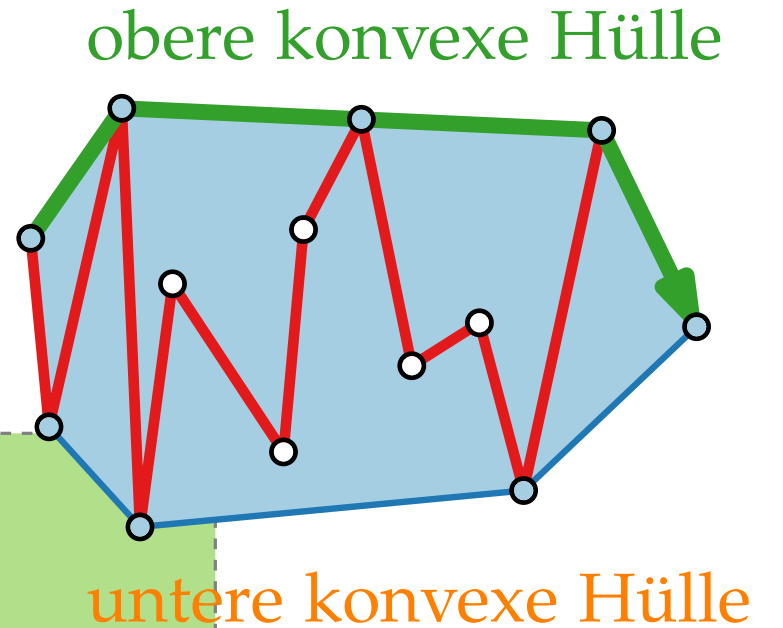
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

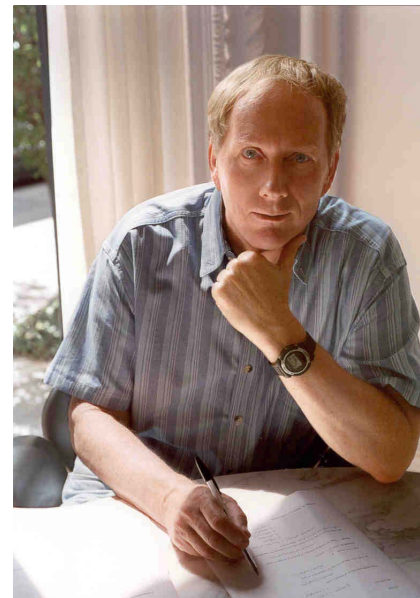
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



ANDREW-GRAHAM-SCAN

- Teile Berechnung in zwei Abschnitte
- Bringe Punkte in lexikografische Reihenfolge
- gehe inkrementell vor

OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

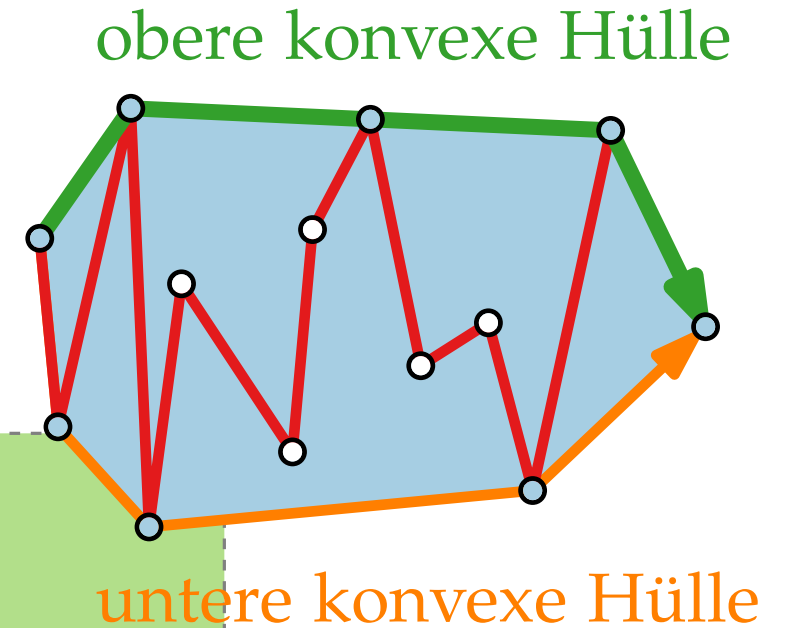
for $i = 3$ **to** n **do** // berechne obere konvexe Hülle von $\{p_1, p_2, \dots, p_i\}$

$L.append(p_i)$

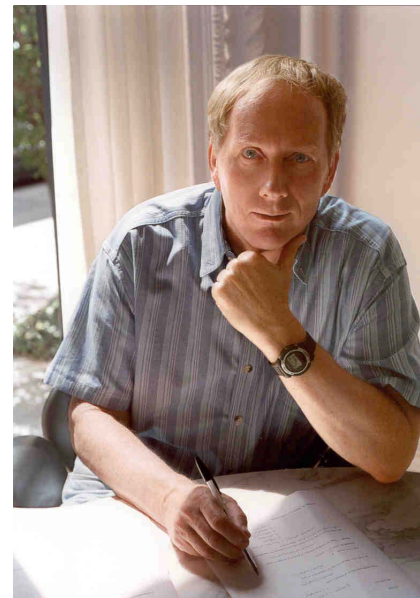
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

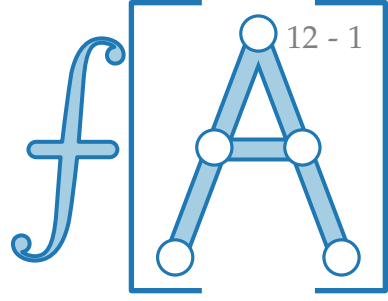
return L



Ronald Graham
*1935, Taft, CA
†2020, La Jolla, CA



Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

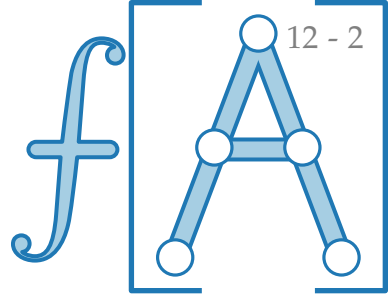
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle$ = sortiere S lexikografisch

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

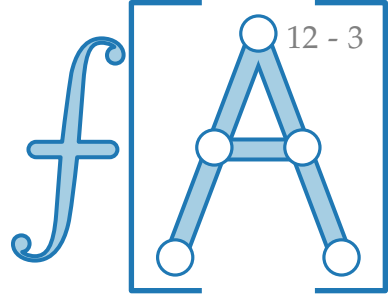
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

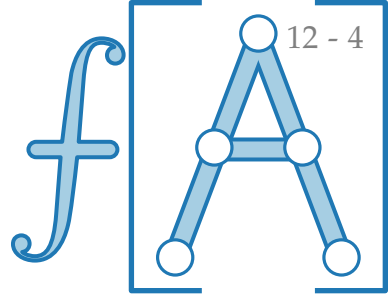
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

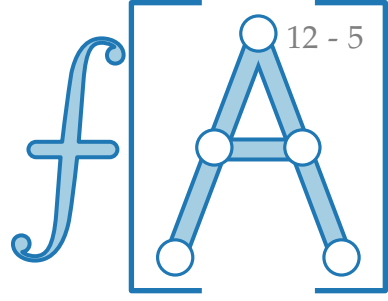
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

$(n - 2) \cdot$

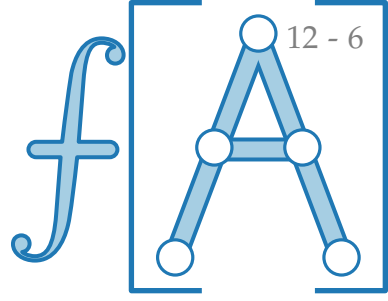
$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

$(n - 2) \cdot$

$L.append(p_i)$

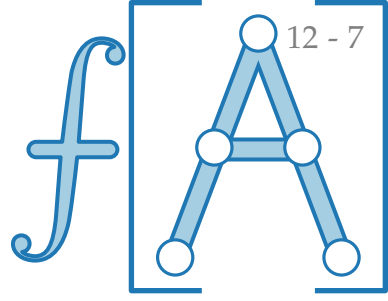
while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

?

return L

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

$(n - 2) \cdot$

$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

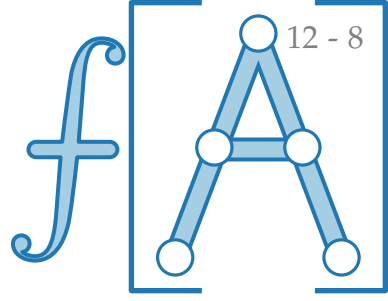
 entferne vorletzten Punkt aus L

?

return L

Jeder Punkt kann max. 1 mal entfernt werden

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

$(n - 2) \cdot$

$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

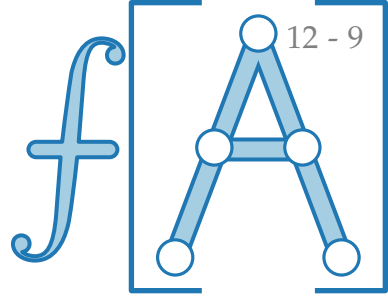
?

return L

Jeder Punkt kann max. 1 mal entfernt werden

➔ Überprüfung wird max. $2n$ mal ausgeführt

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

return L

~~$(n - 2) \cdot$~~

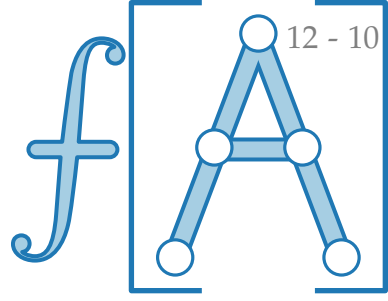
$\mathcal{O}(n)$



Jeder Punkt kann max. 1 mal entfernt werden

➔ Überprüfung wird max. $2n$ mal ausgeführt

Laufzeit



OBEREKONVEXEHÜLLE(S)

$\langle p_1, p_2, \dots, p_n \rangle =$ sortiere S lexikografisch $\mathcal{O}(n \log n)$

$L = \langle p_1, p_2 \rangle$

for $i = 3$ **to** n **do**

$L.append(p_i)$

while $|L| > 2$ **and** letzte 3 Punkte in L formen Linksknick **do**

 entferne vorletzten Punkt aus L

$\mathcal{O}(n)$

return L

~~$(n - 2) \cdot$~~



Jeder Punkt kann max. 1 mal entfernt werden

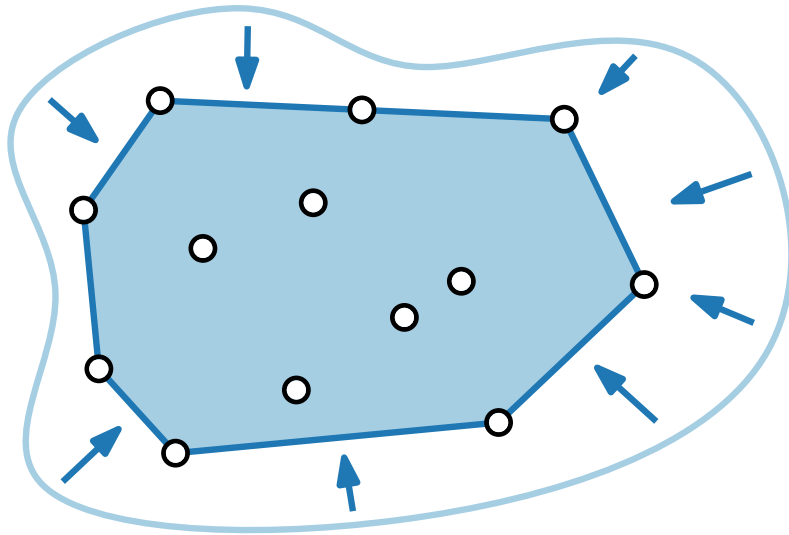
➔ Überprüfung wird max. $2n$ mal ausgeführt

Satz. ANDREW-GRAHAM-SCAN berechnet die konvexe Hülle von n Punkten in der Ebene robust in $\mathcal{O}(n \log n)$ Zeit.

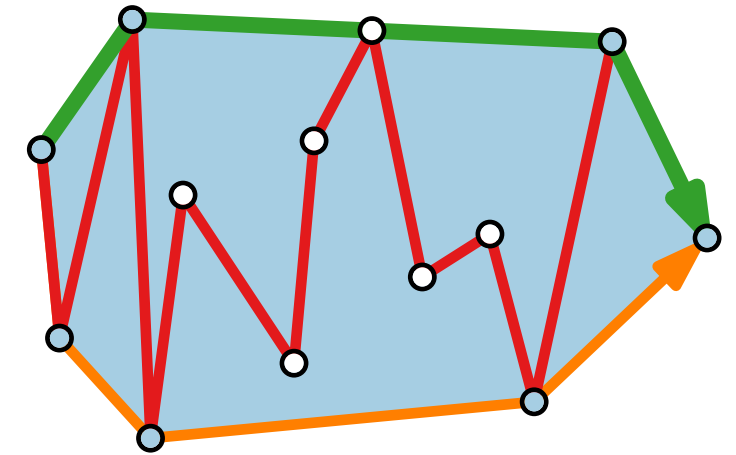
Fortgeschrittene Algorithmen

12. Vorlesung Algorithmische Geometrie I: Konvexe Hülle

Teil IV: Ausgabesensitive Algorithmen



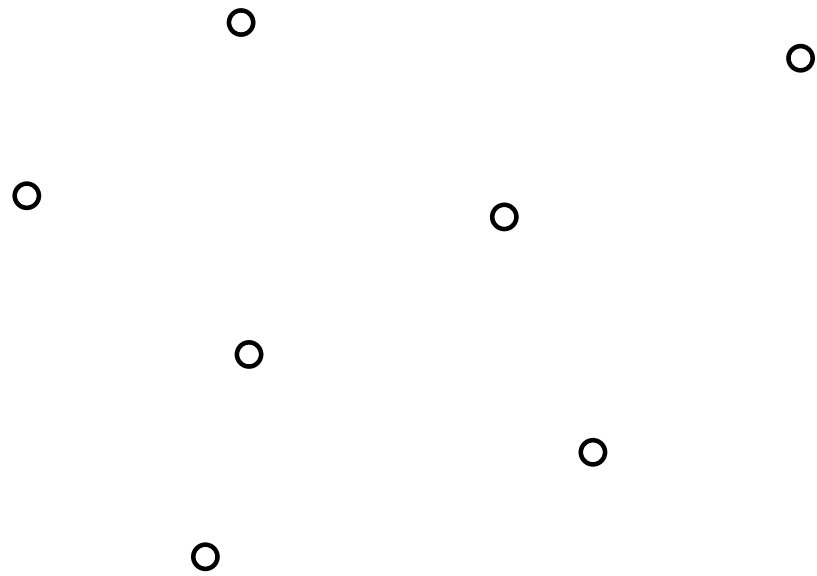
Philipp Kindermann



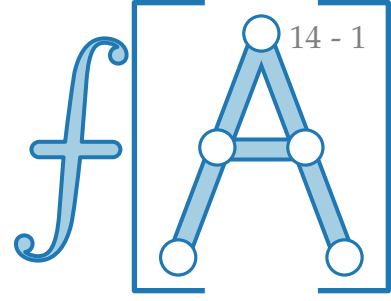
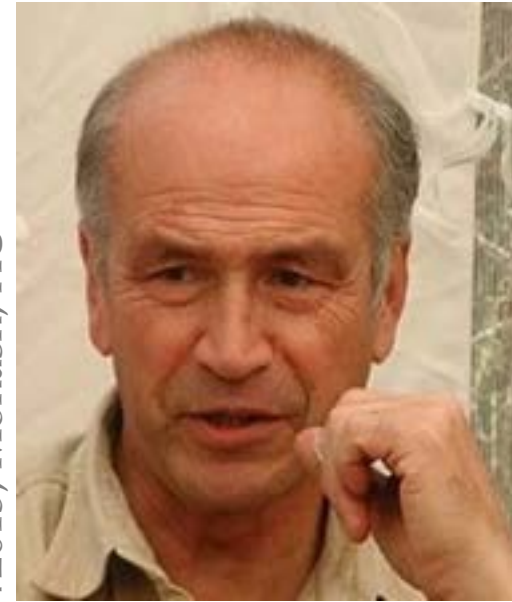
Wintersemester 2025/26

Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

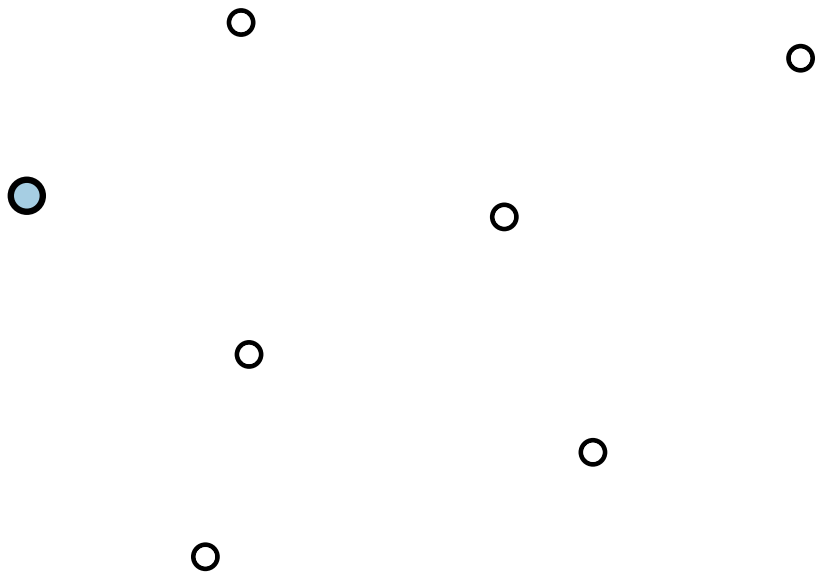


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

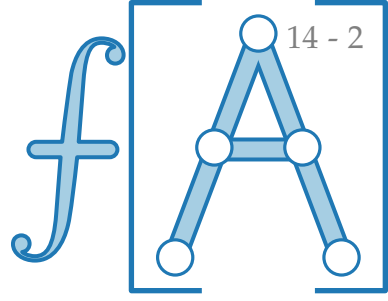
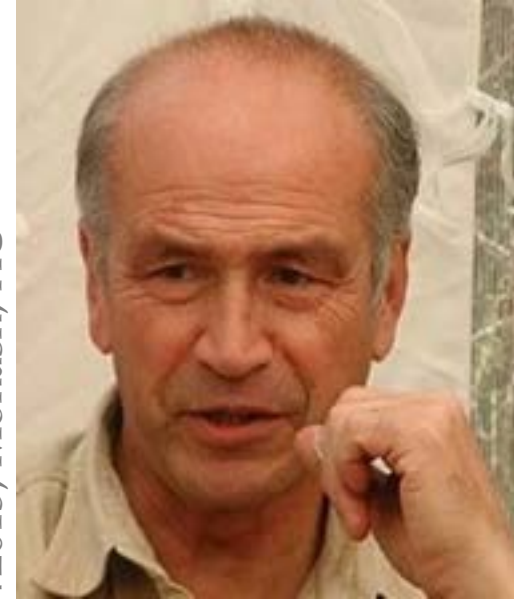


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

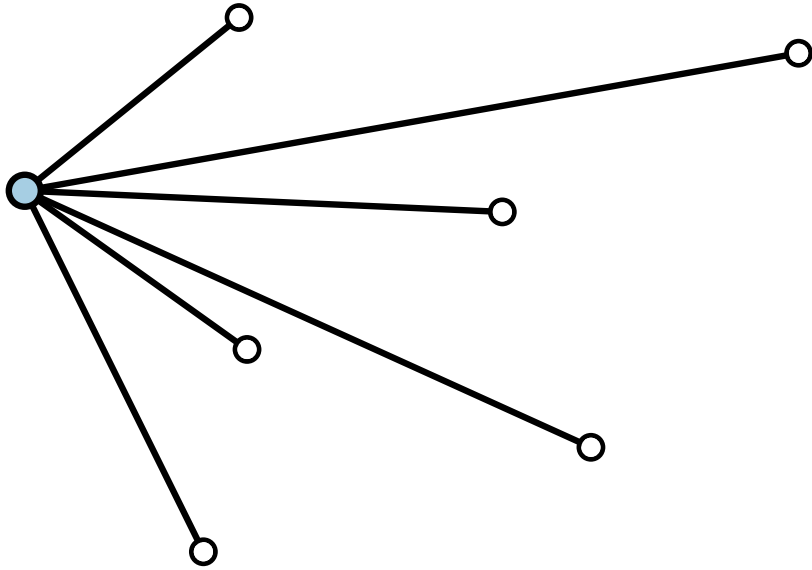


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

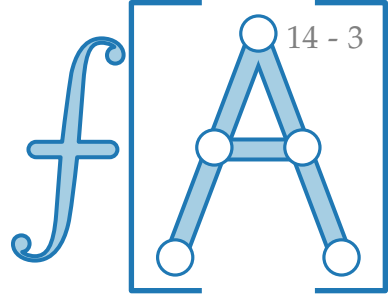
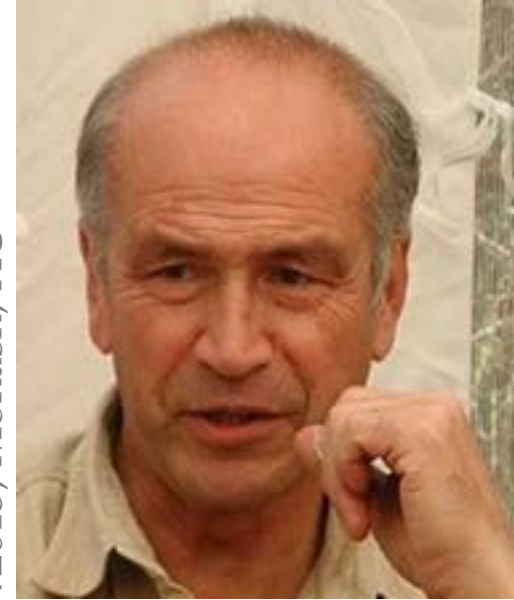


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

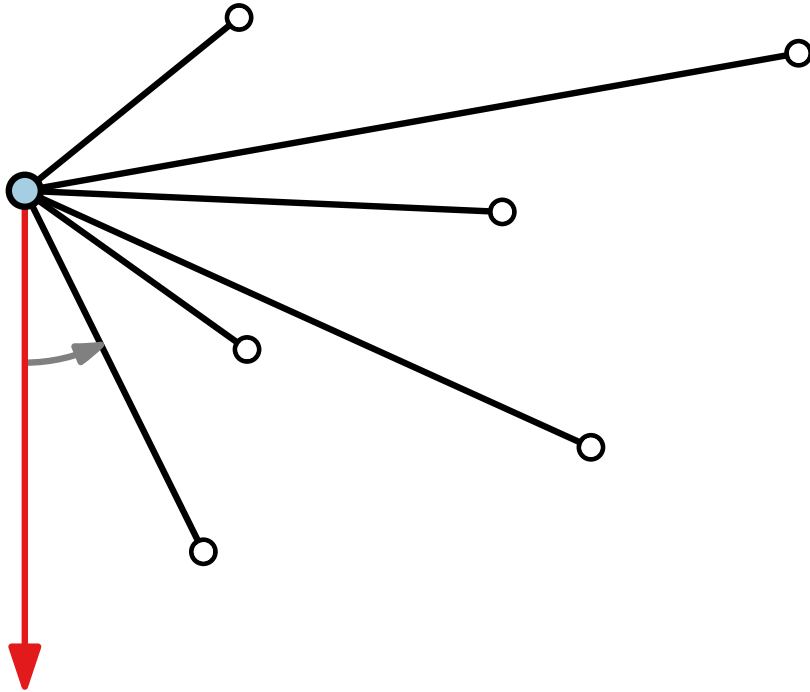


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

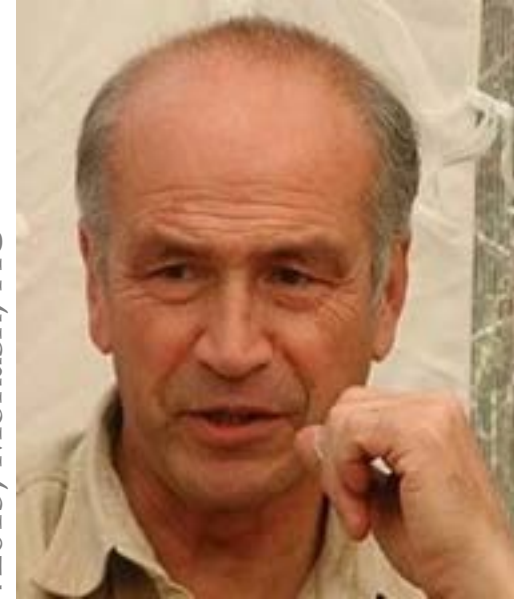


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

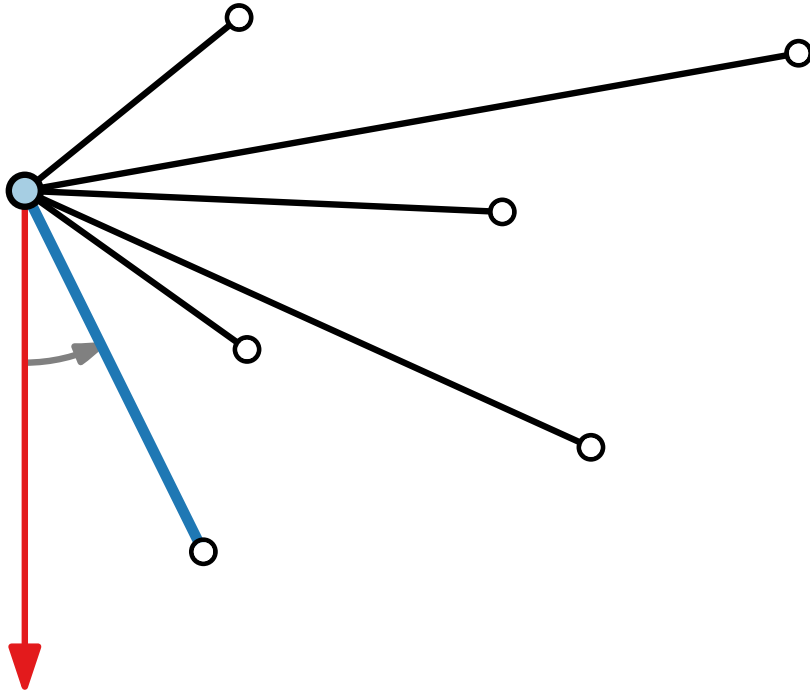


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

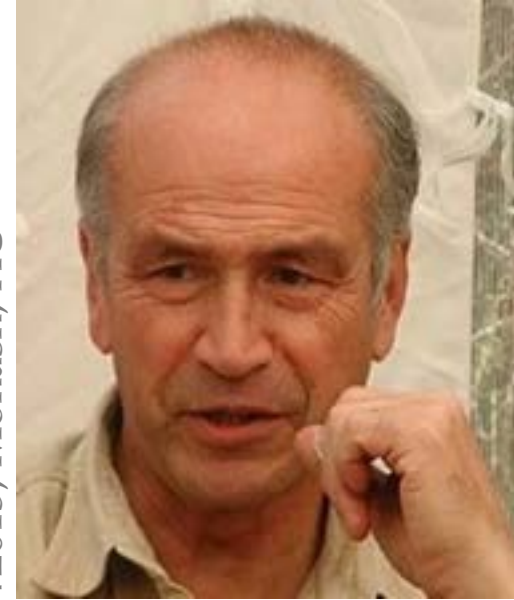


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

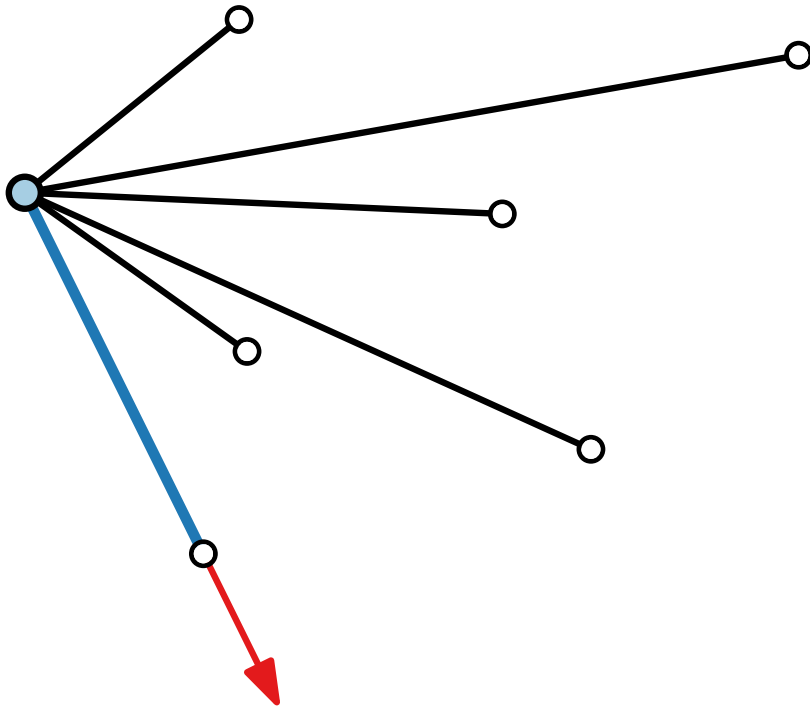


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

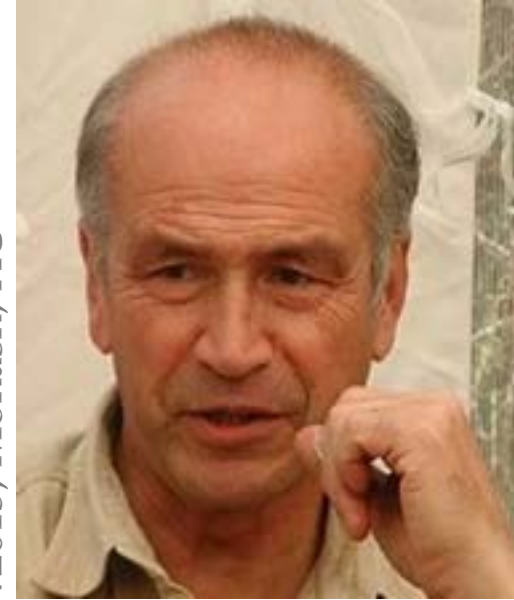


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

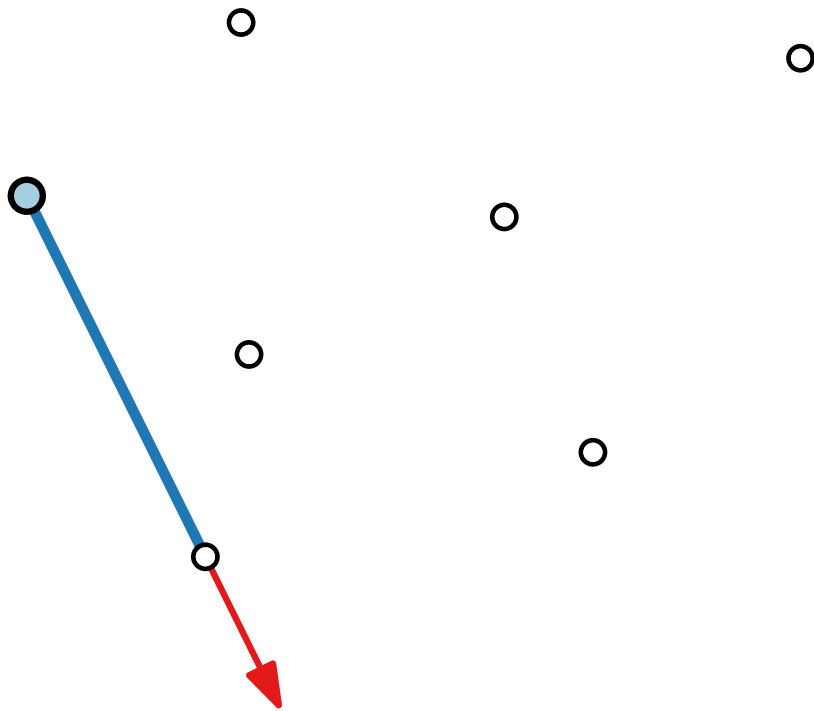


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

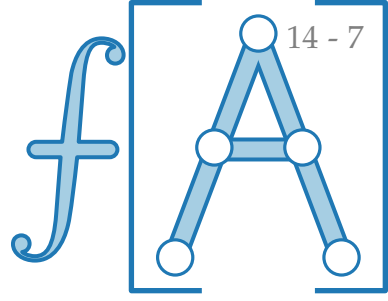
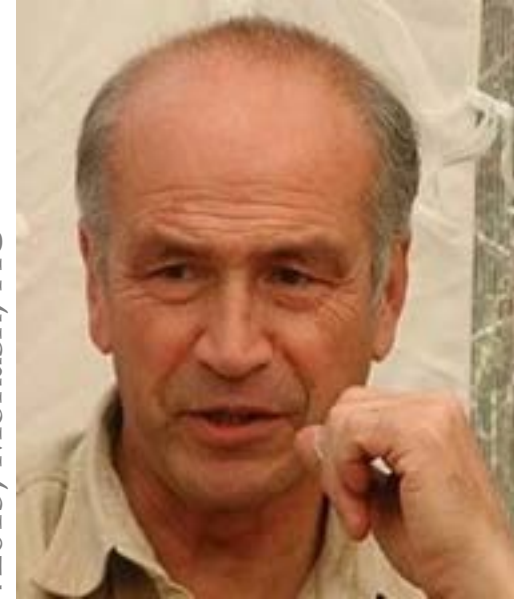


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

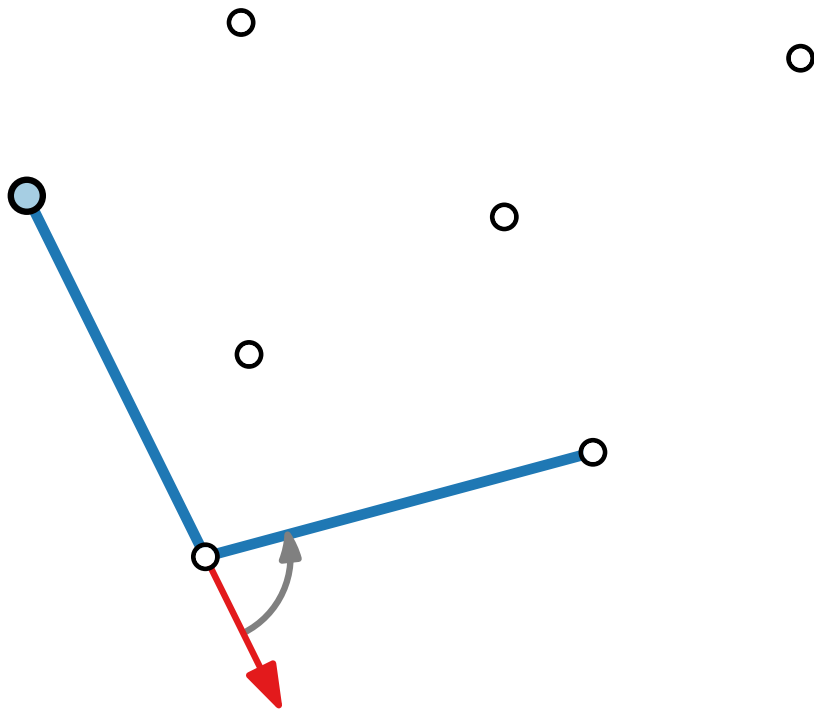


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

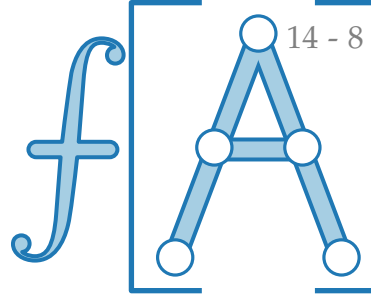
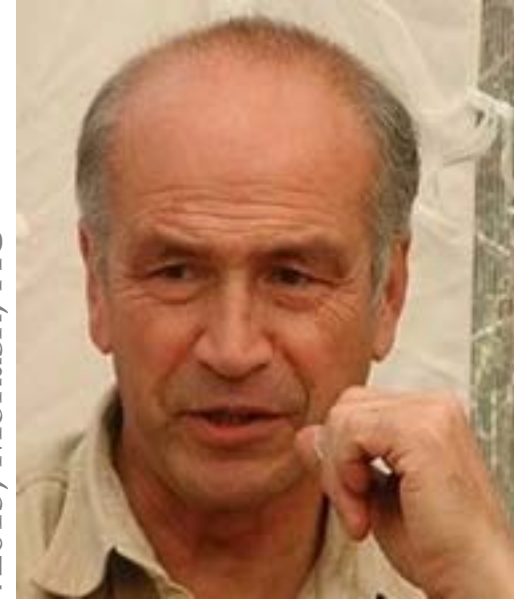


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

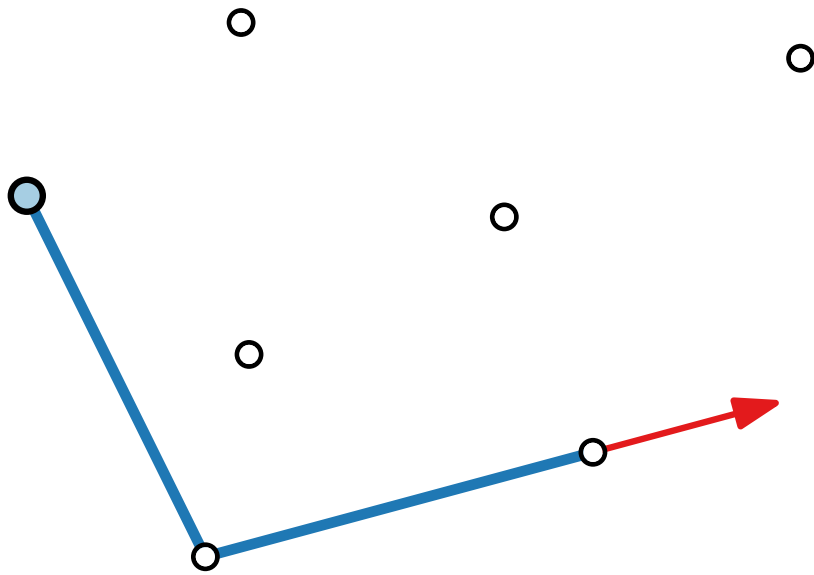


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

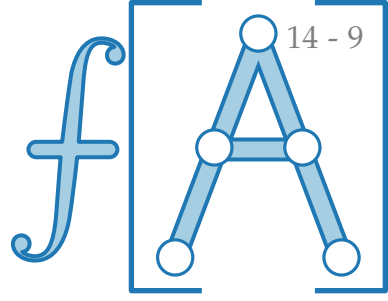
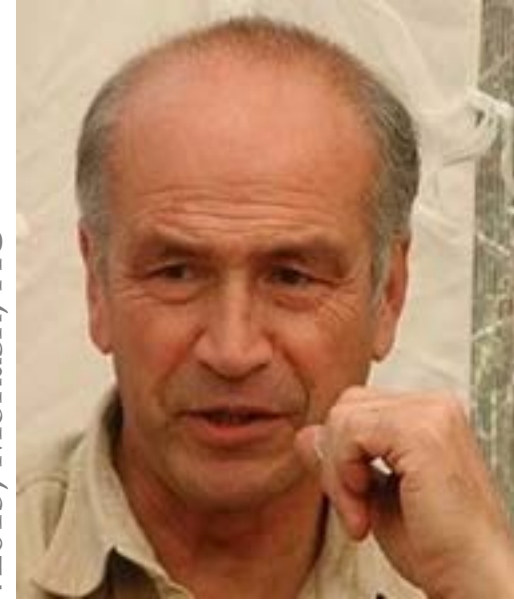


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

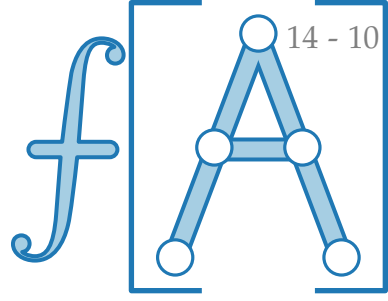
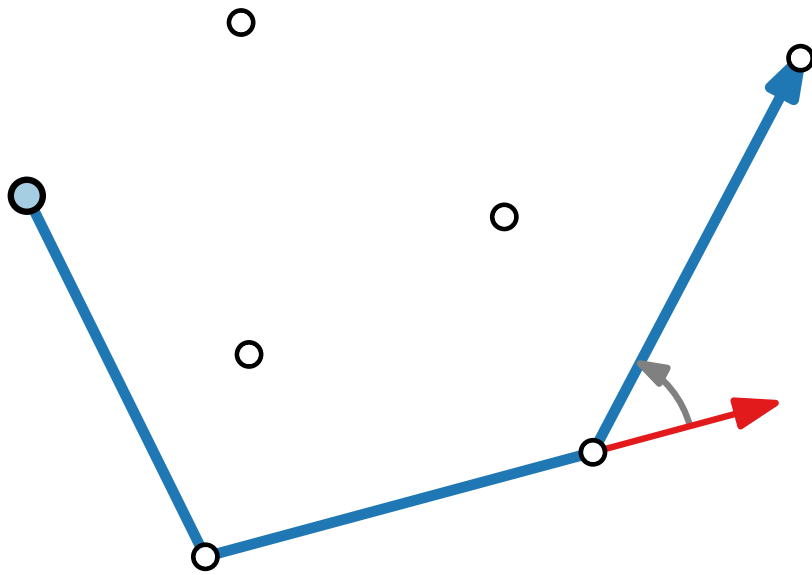


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

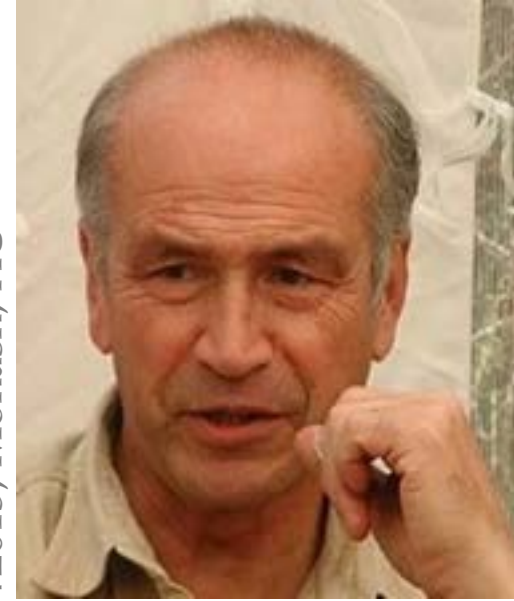


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

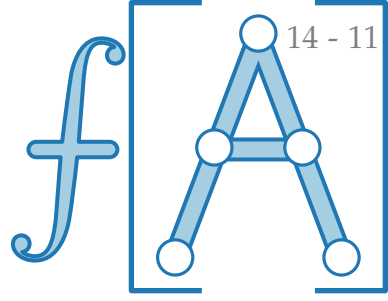
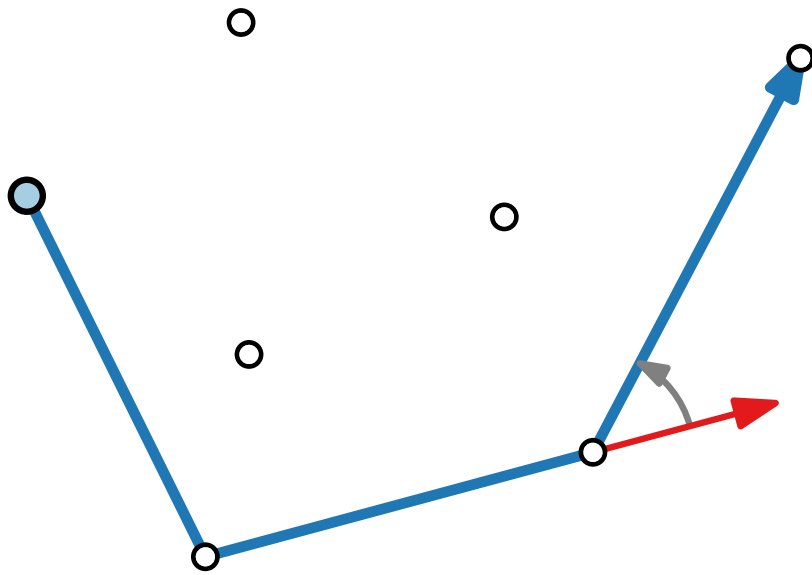


Ray A. Jarvis
*1941, Burma
†2013, Monash, AU

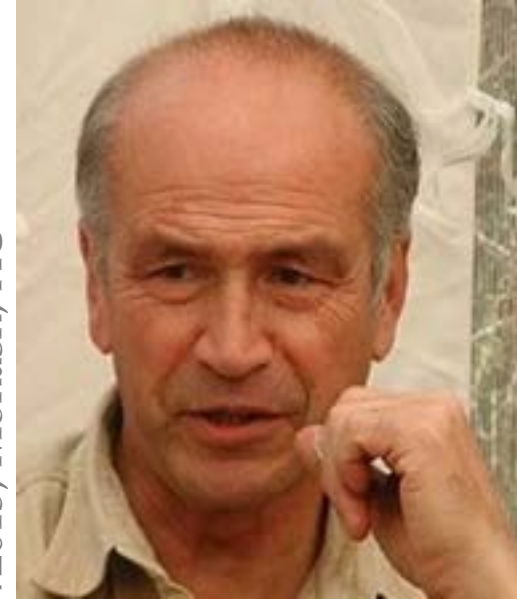


Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)
Laufzeit?



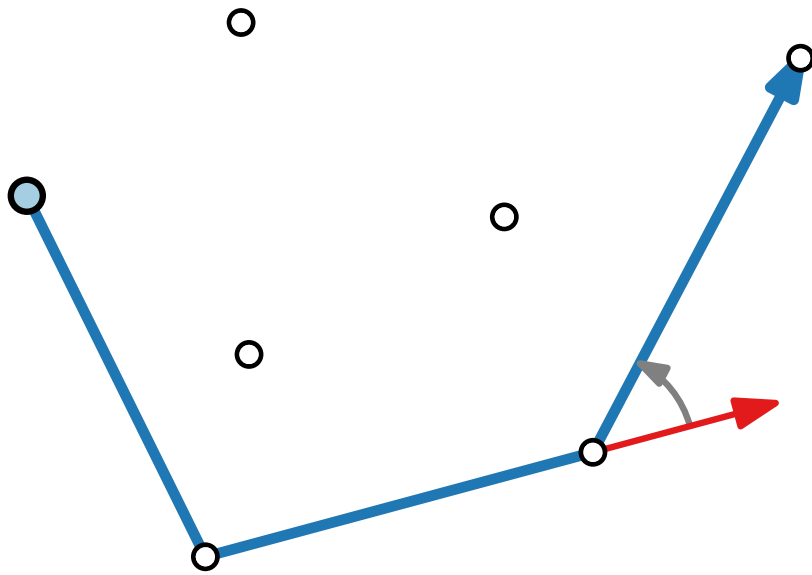
Ray A. Jarvis
*1941, Burma
†2013, Monash, AU



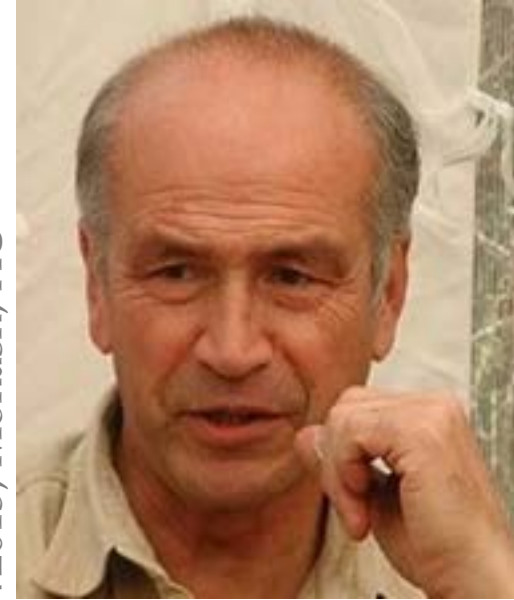
Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

Laufzeit? $\mathcal{O}(n \cdot h)$



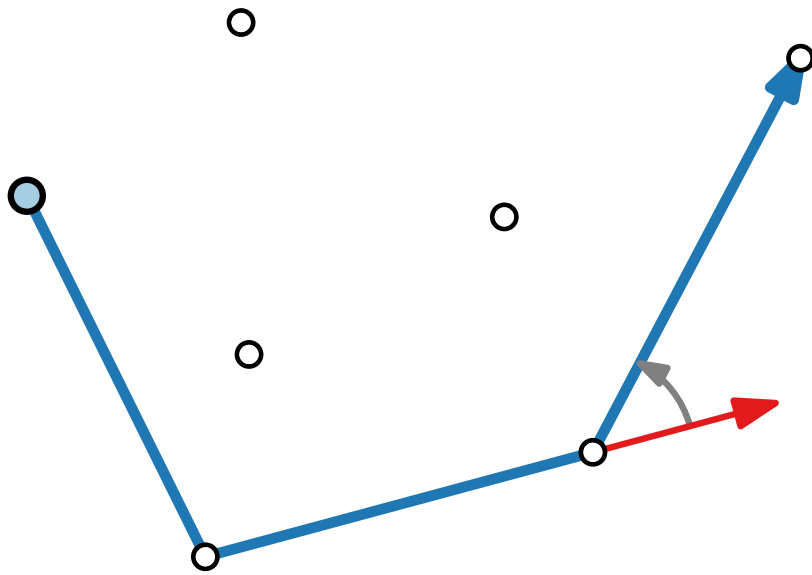
Ray A. Jarvis
*1941, Burma
†2013, Monash, AU



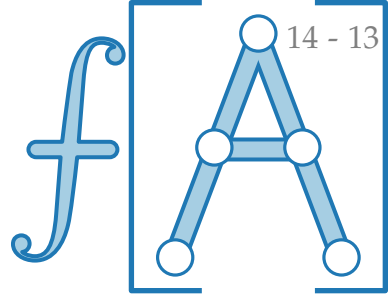
Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

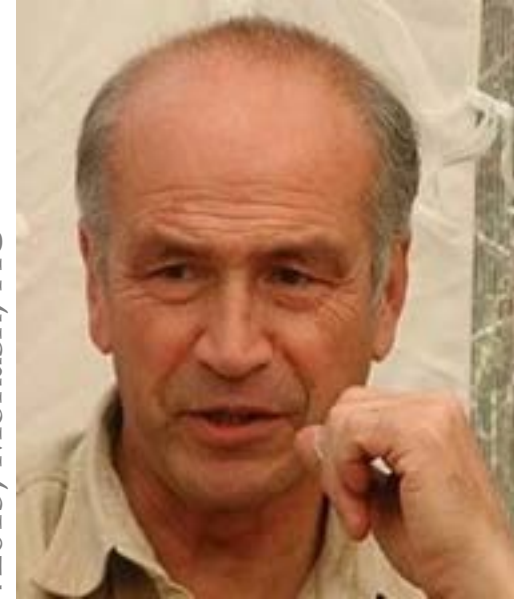
Laufzeit? $\mathcal{O}(n \cdot h)$



... wobei $h = |\text{CH}(S)|$ = Größe der Ausgabe



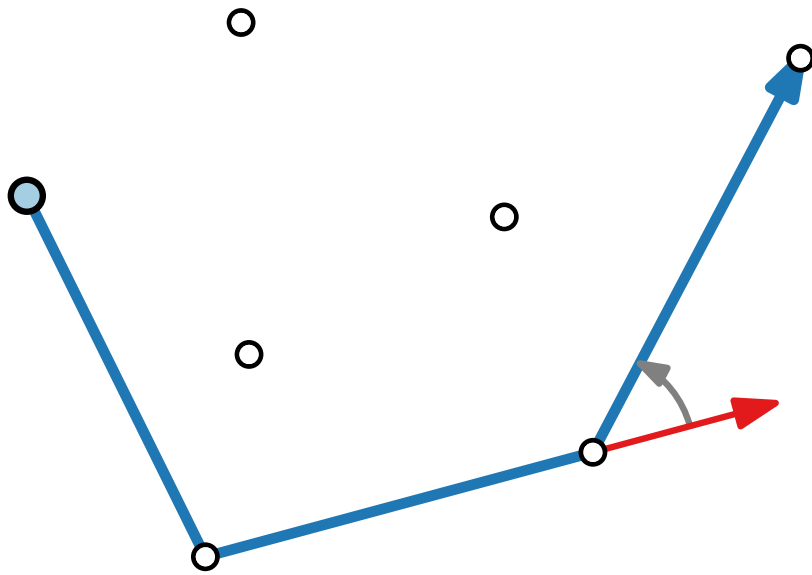
Ray A. Jarvis
*1941, Burma
†2013, Monash, AU



Ausgabesensitive Algorithmen

- Jarvis Gift-Wrapping-Algorithmus (aka JARVISMARCH)

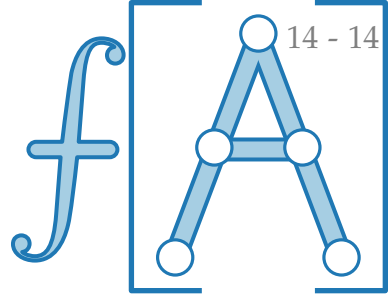
Laufzeit? $\mathcal{O}(n \cdot h)$



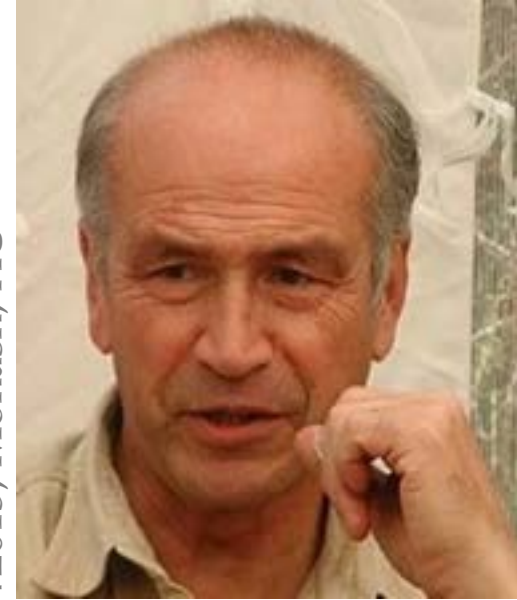
- Chan's exponentielle Suche Algorithmus

$\mathcal{O}(n \log h)$

... wobei $h = |\text{CH}(S)|$ = Größe der Ausgabe



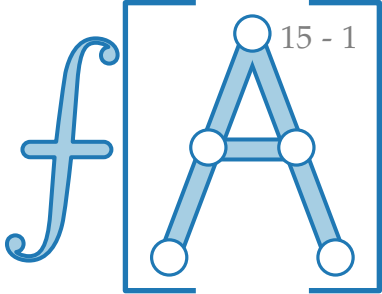
Ray A. Jarvis
*1941, Burma
†2013, Monash, AU



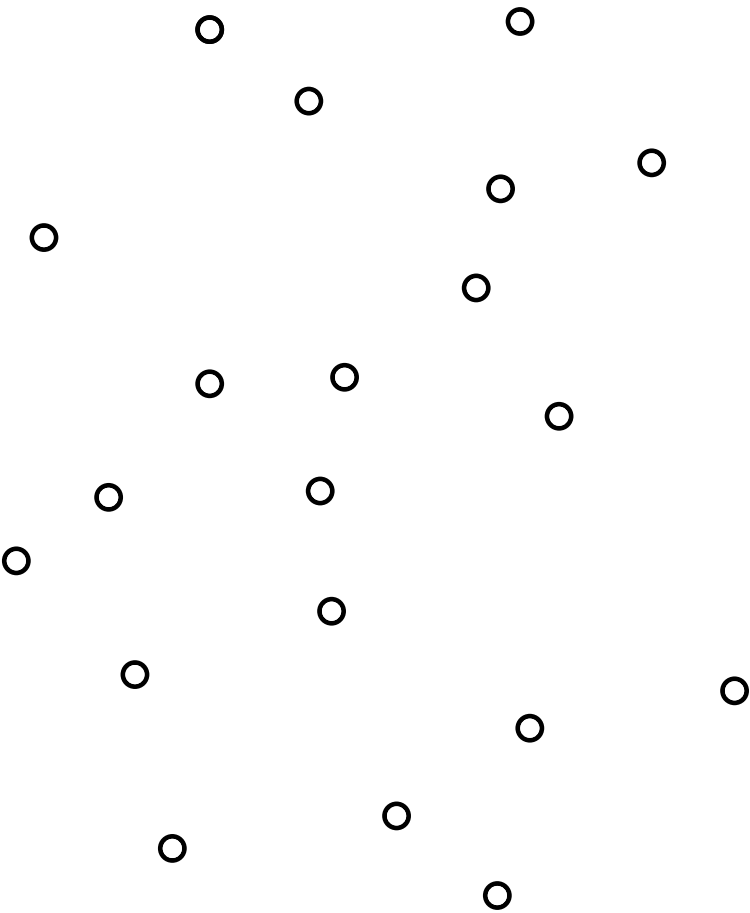
Timothy M. Chan
*1976, USA



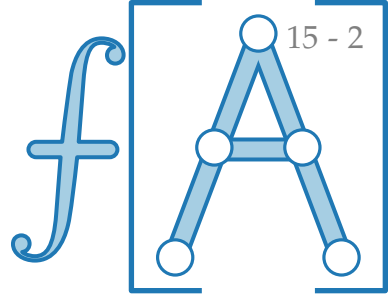
Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

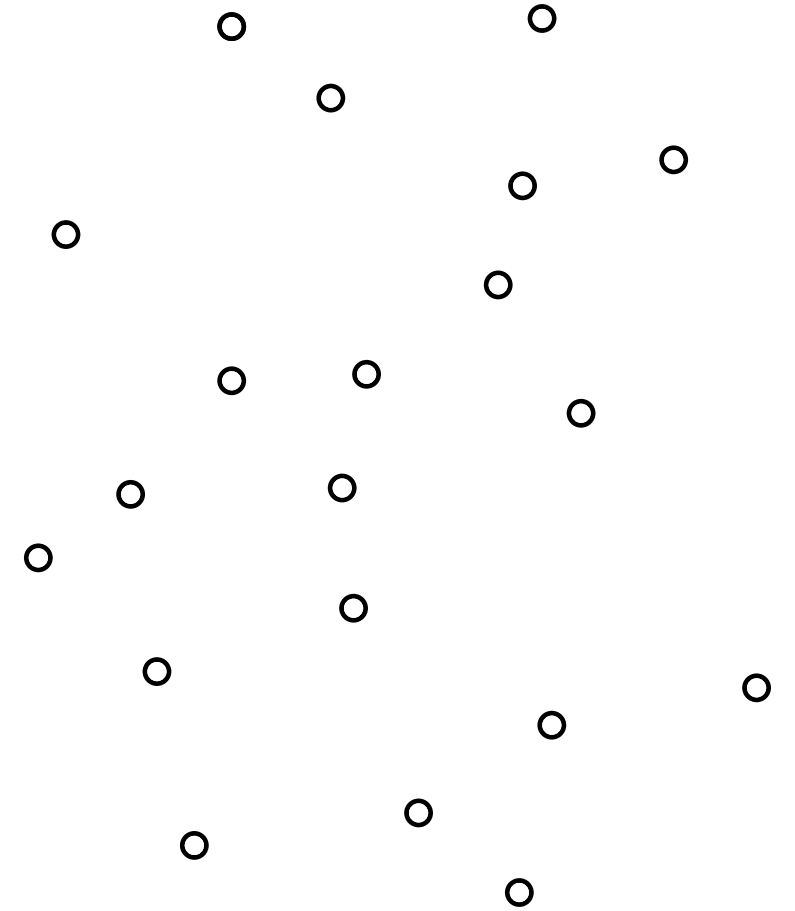


Chans Algorithmus

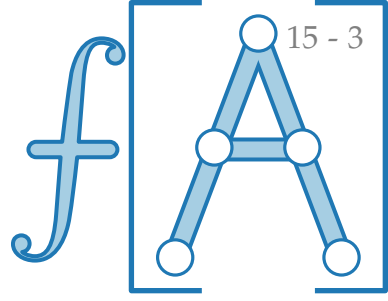


CHANSKONVEXEHÜLLE(S, m)

z.B. 5



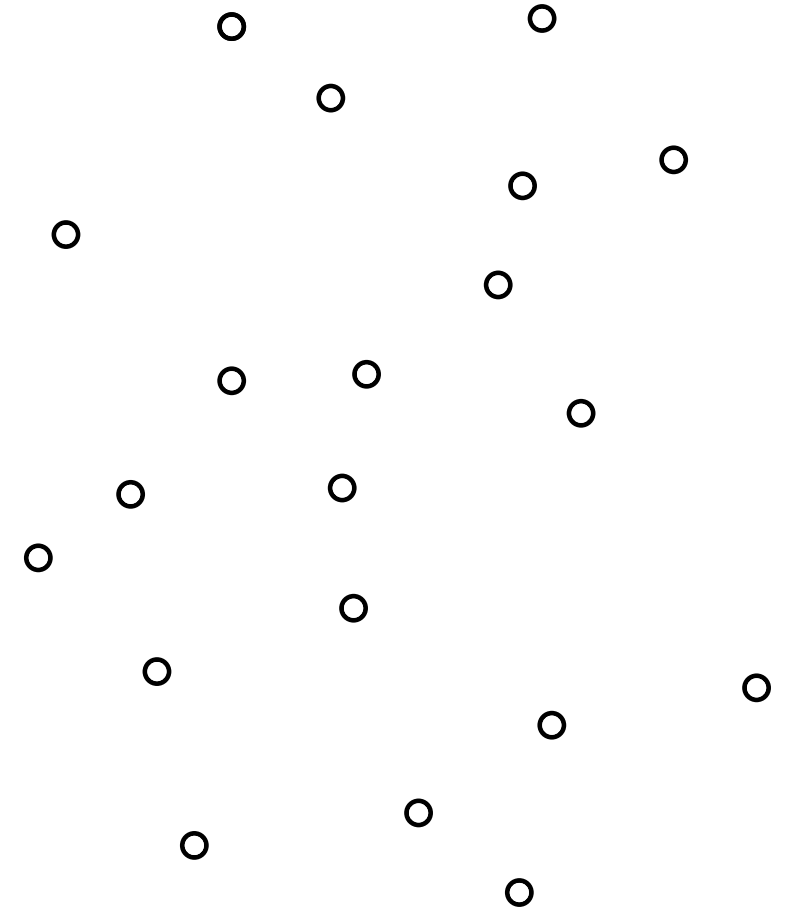
Chans Algorithmus



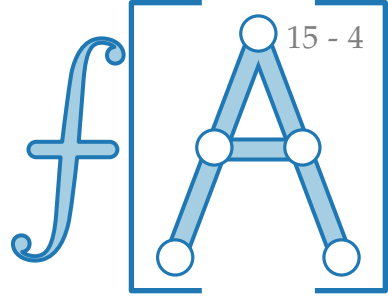
z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m



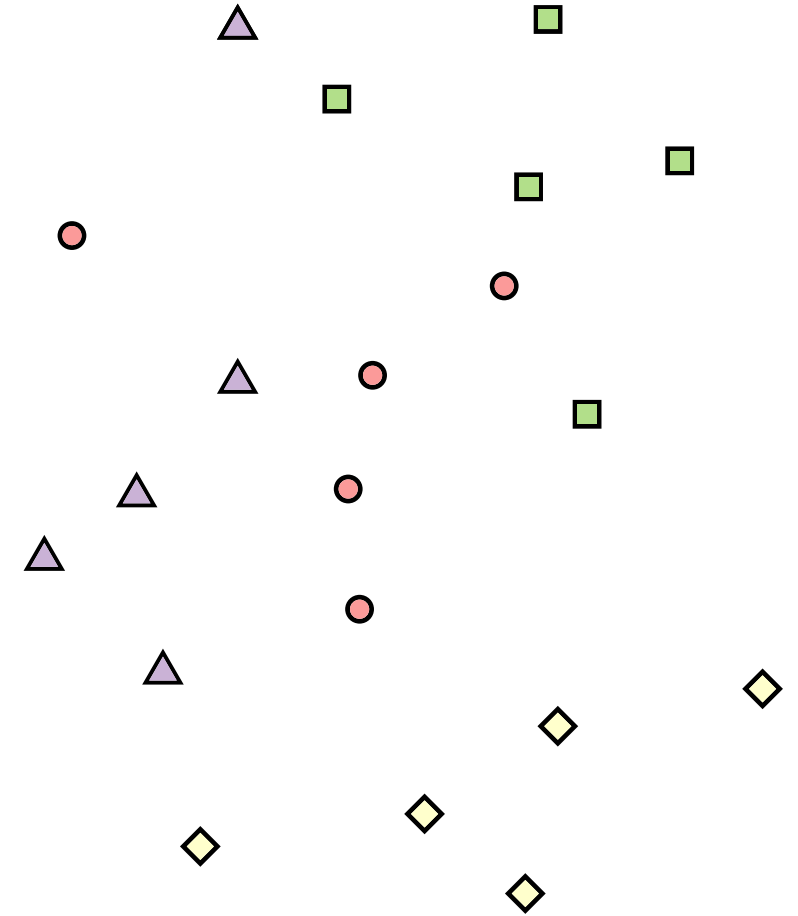
Chans Algorithmus



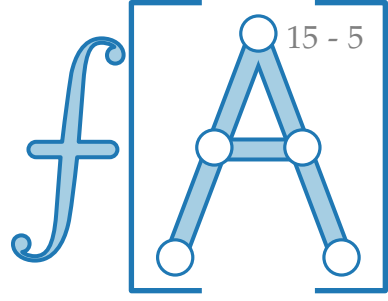
z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m



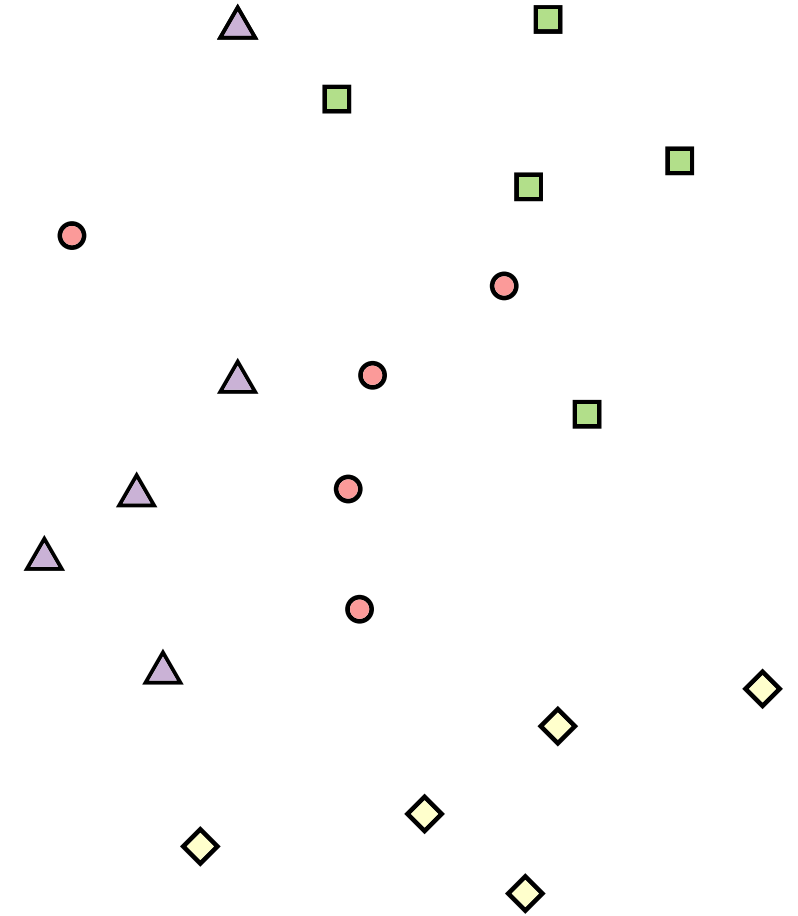
Chans Algorithmus



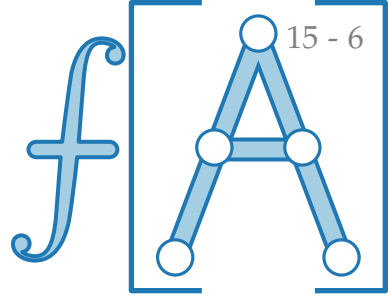
z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$



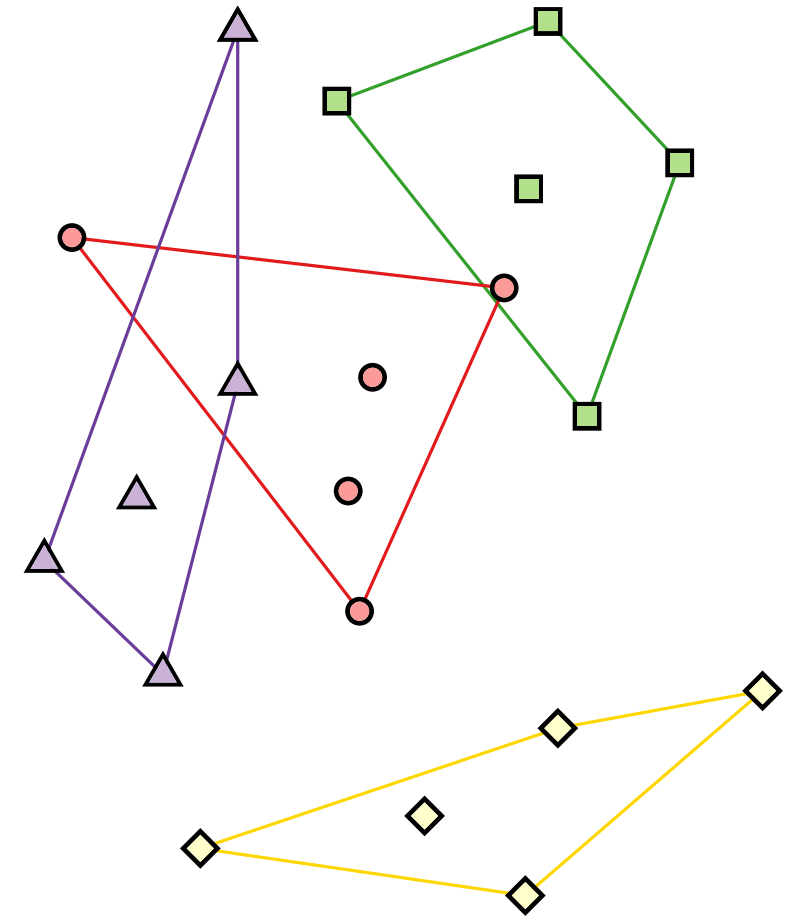
Chans Algorithmus



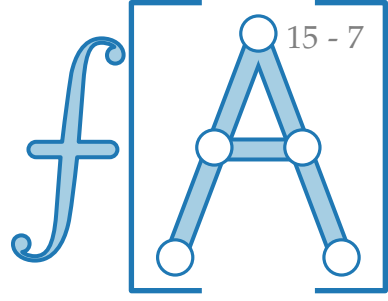
z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$



Chans Algorithmus



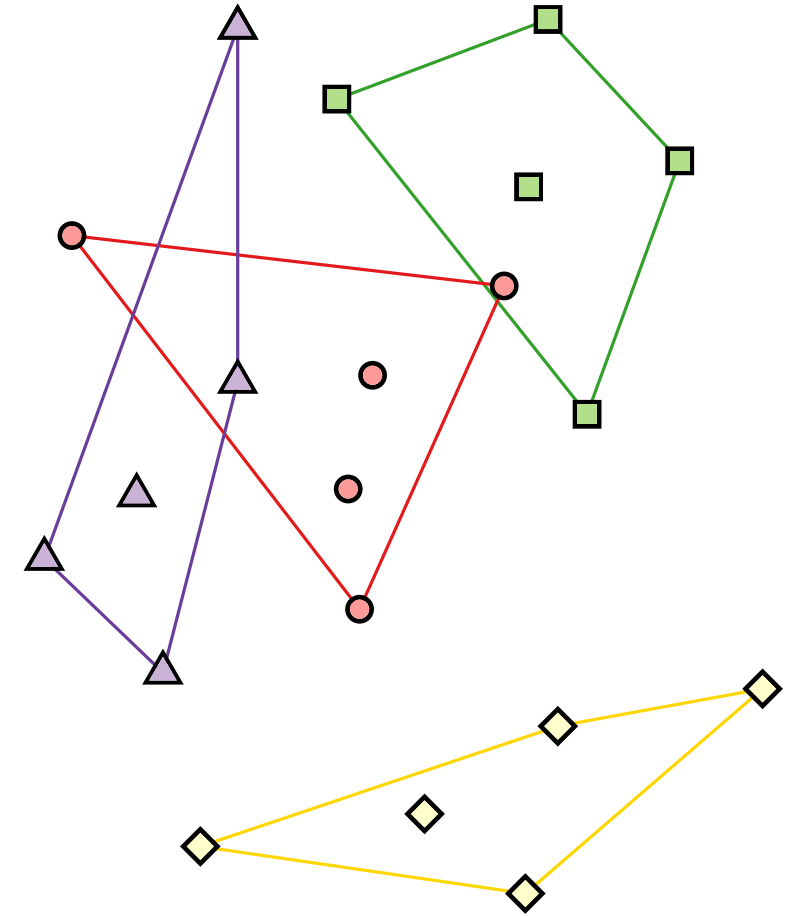
z.B. 5

CHANSKONVEXEHÜLLE(S, m)

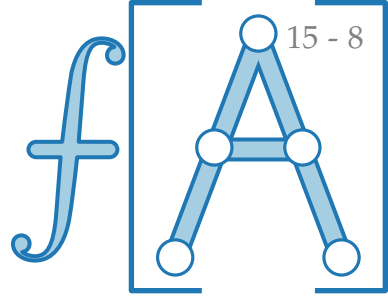
Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P



Chans Algorithmus



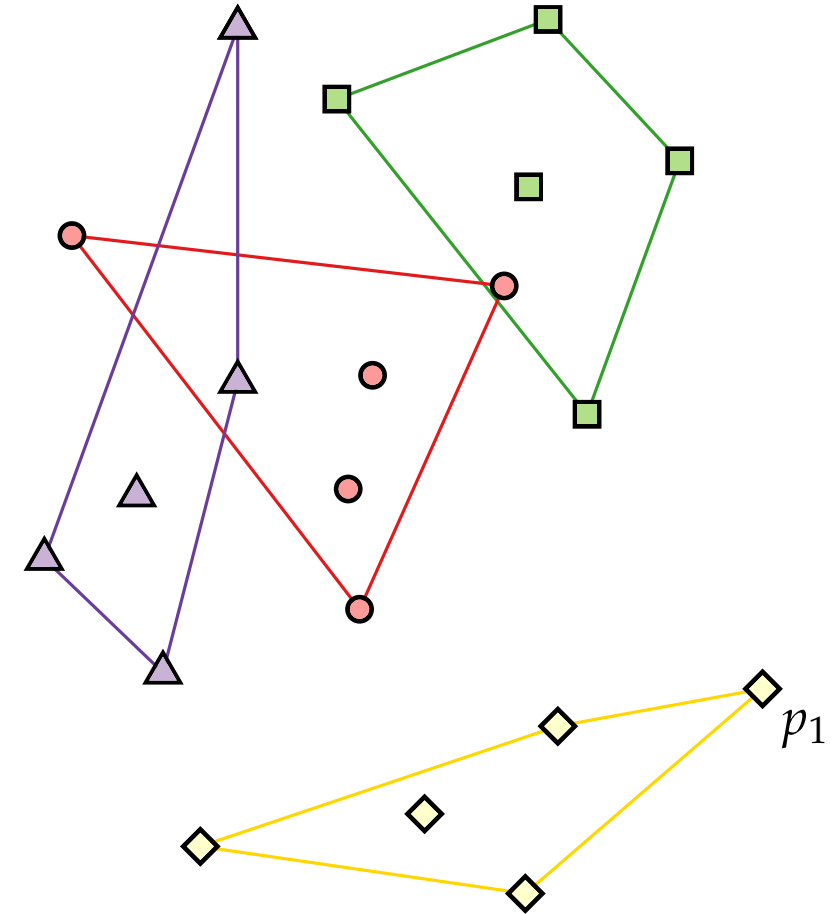
z.B. 5

CHANSKONVEXEHÜLLE(S, m)

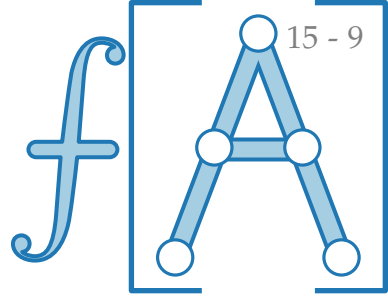
Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P



Chans Algorithmus



z.B. 5

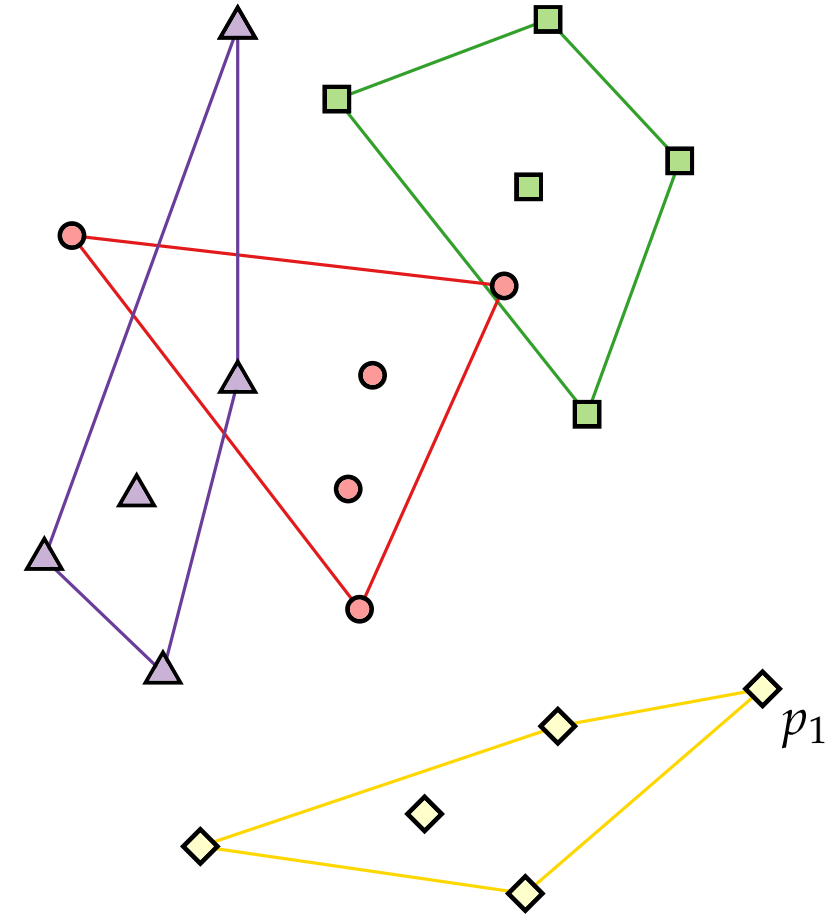
CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

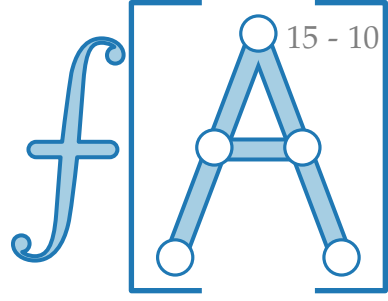
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

z.B. 5

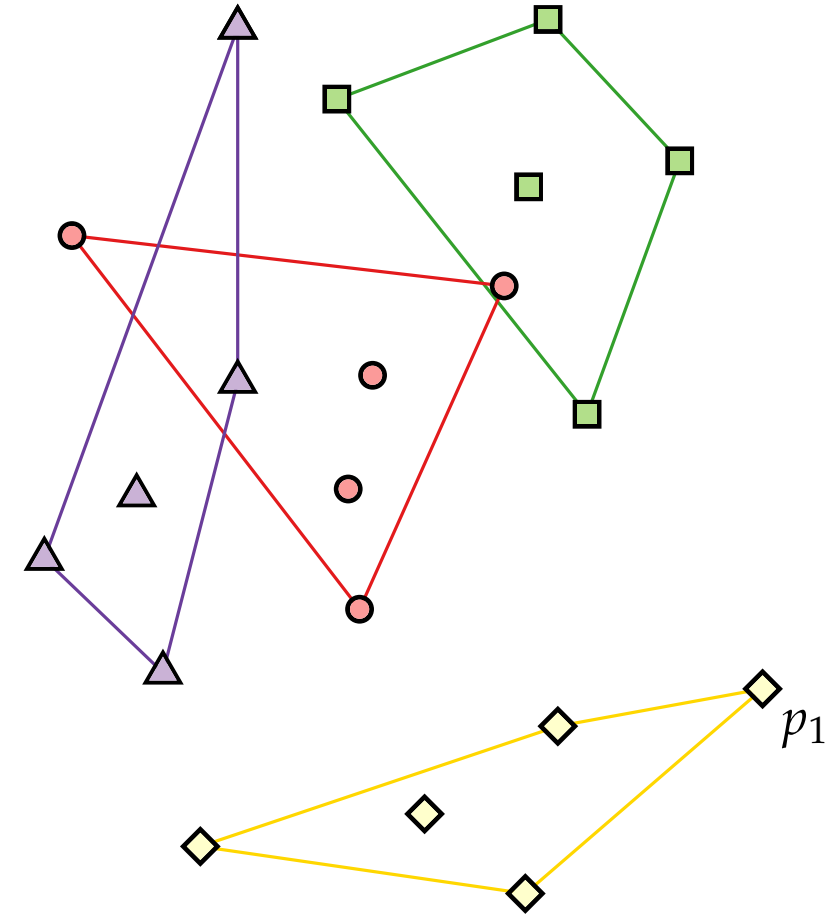
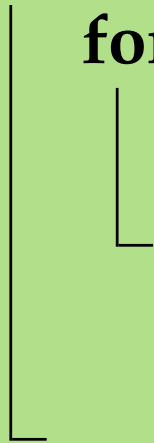
Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

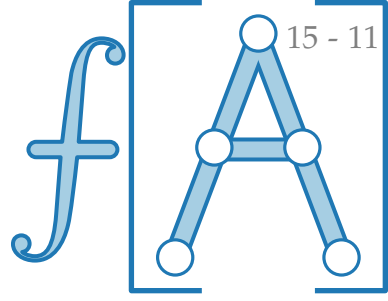
p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

z.B. 5

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

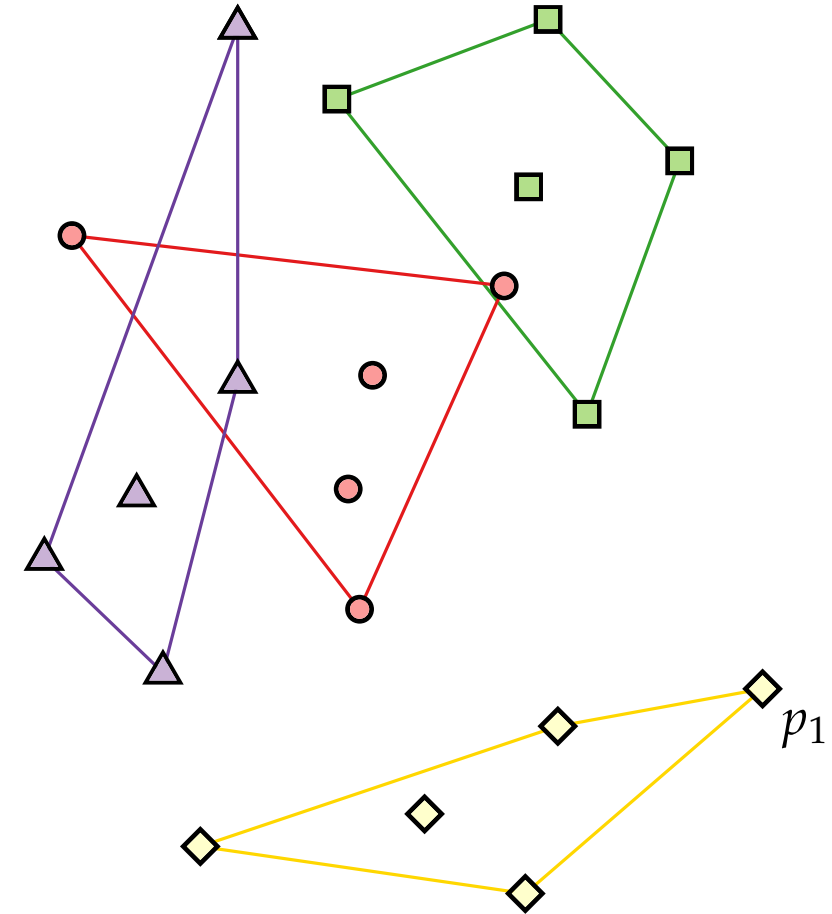
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

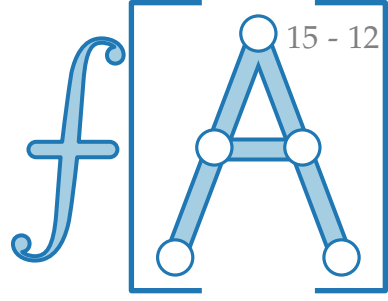
for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

z.B. 5

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

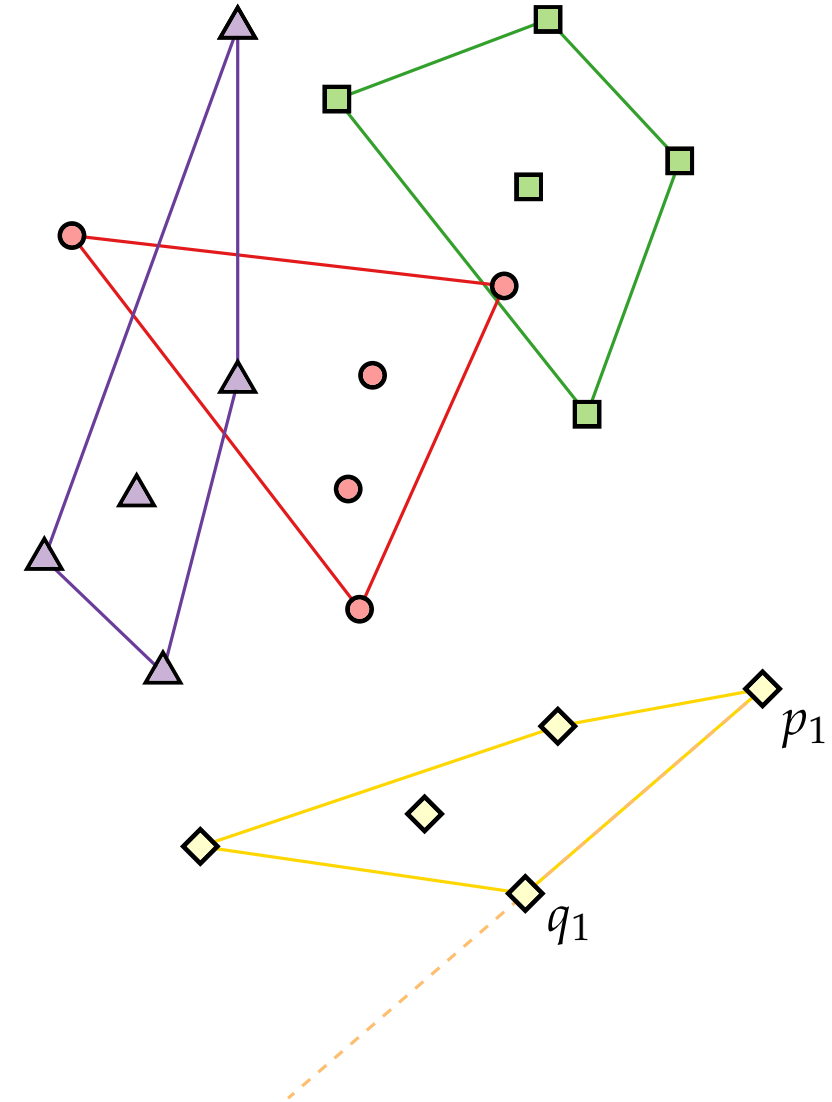
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

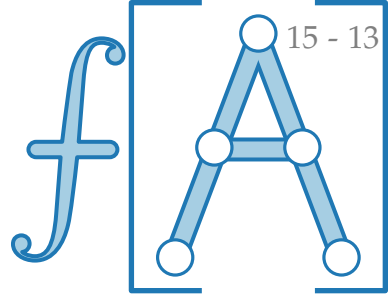
for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

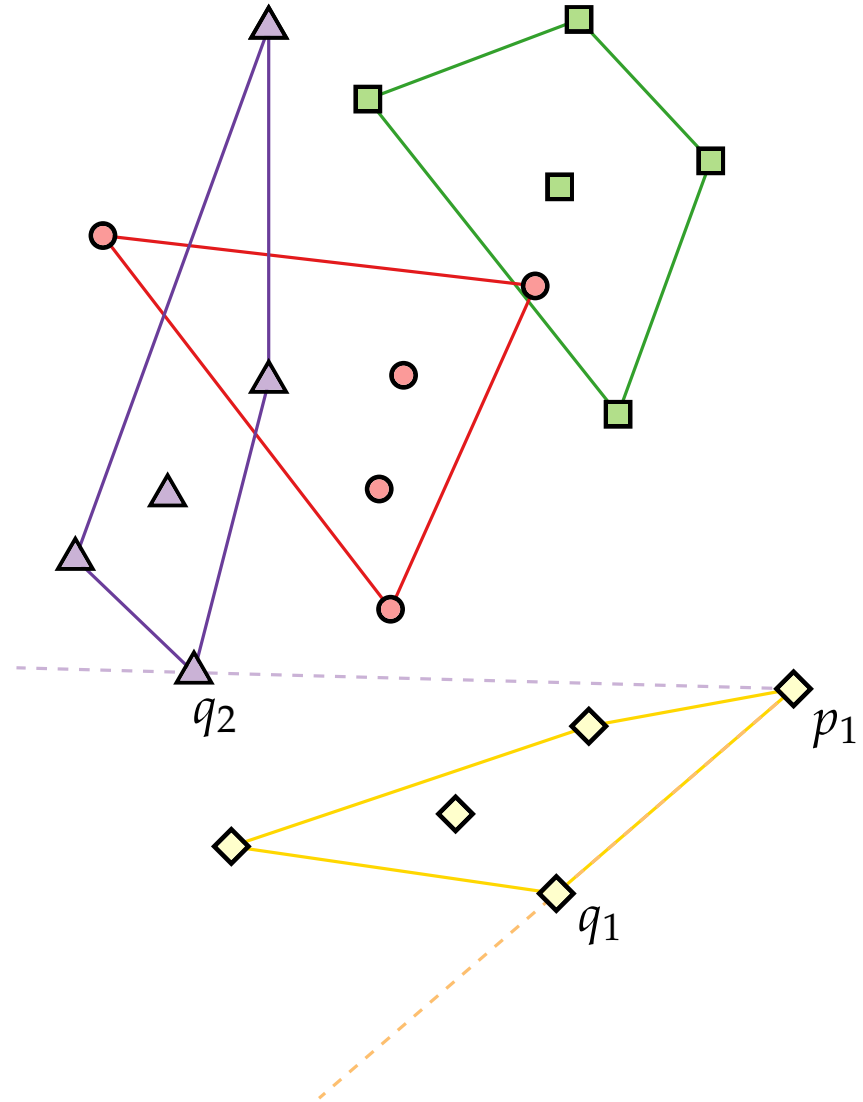
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

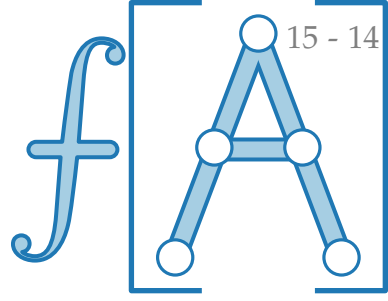
for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

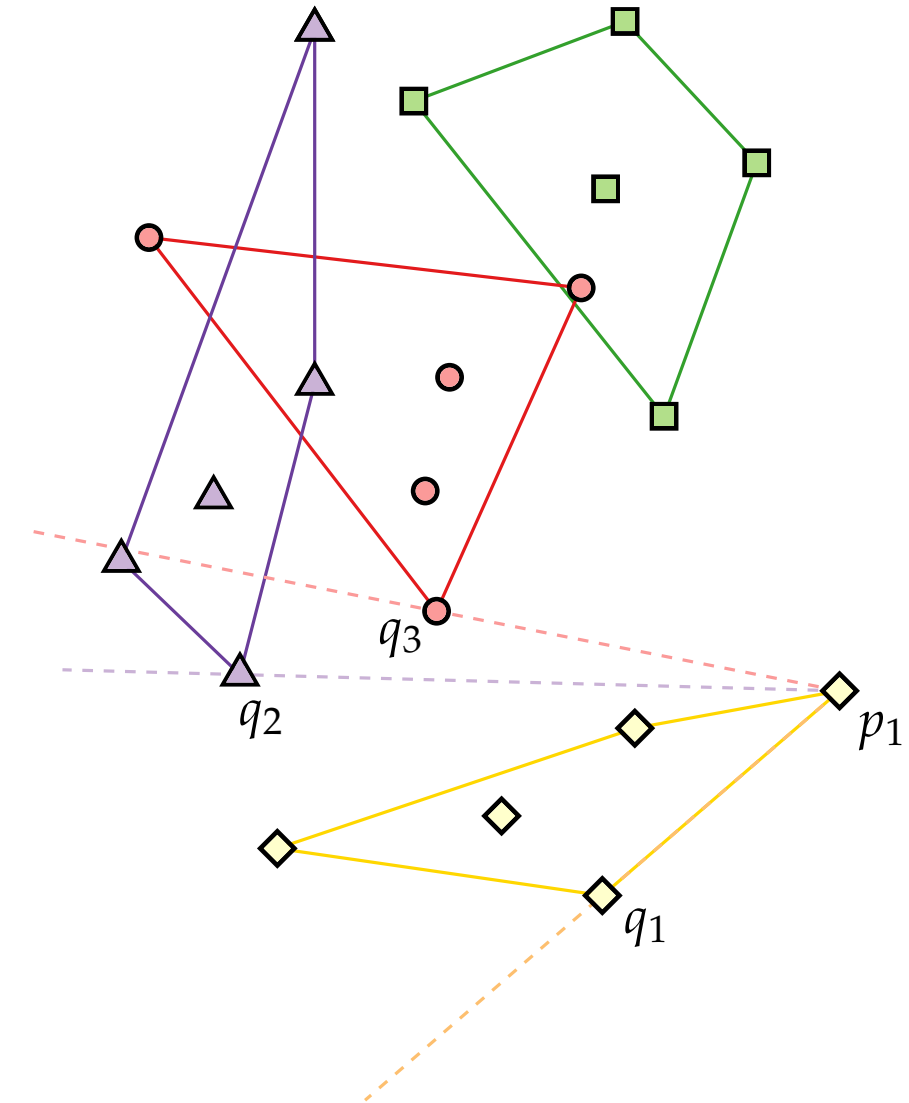
p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

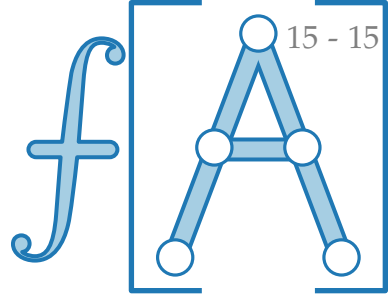
foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

z.B. 5



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

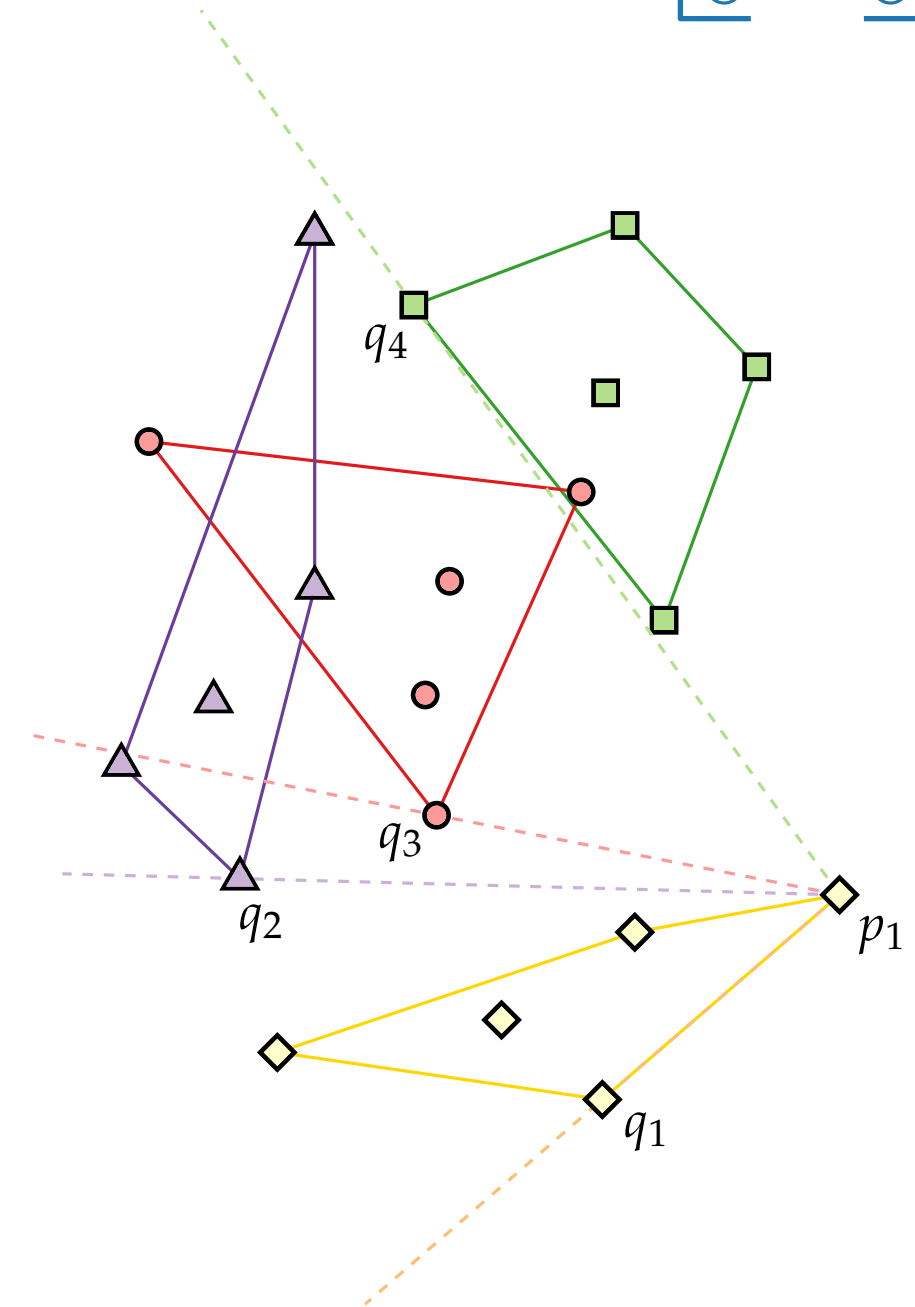
p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

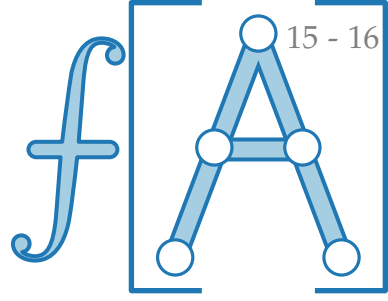
foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

z.B. 5



Chans Algorithmus



z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

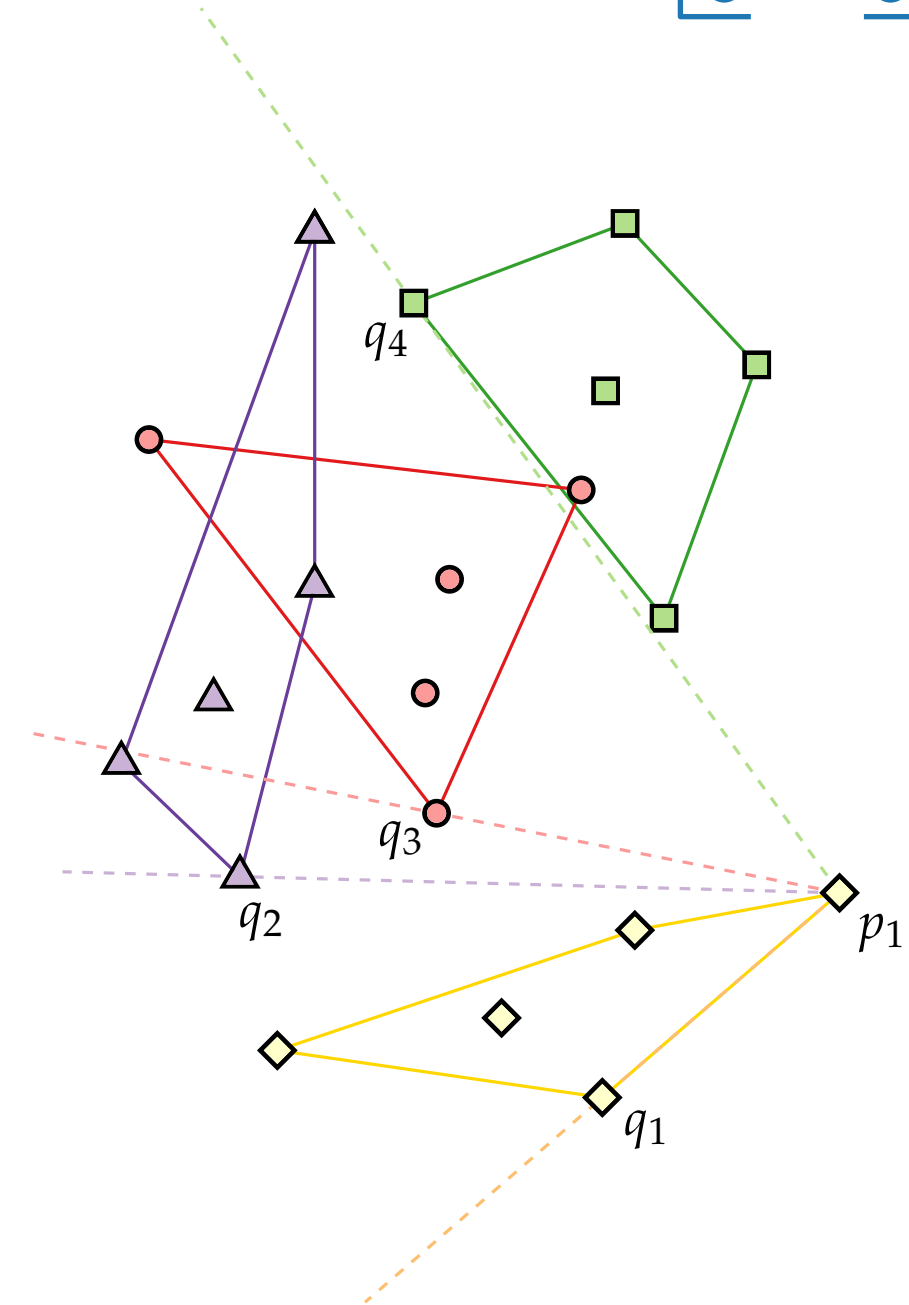
p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

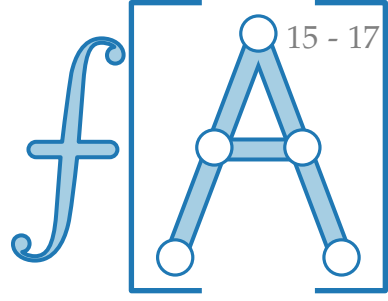
foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$



Chans Algorithmus



z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

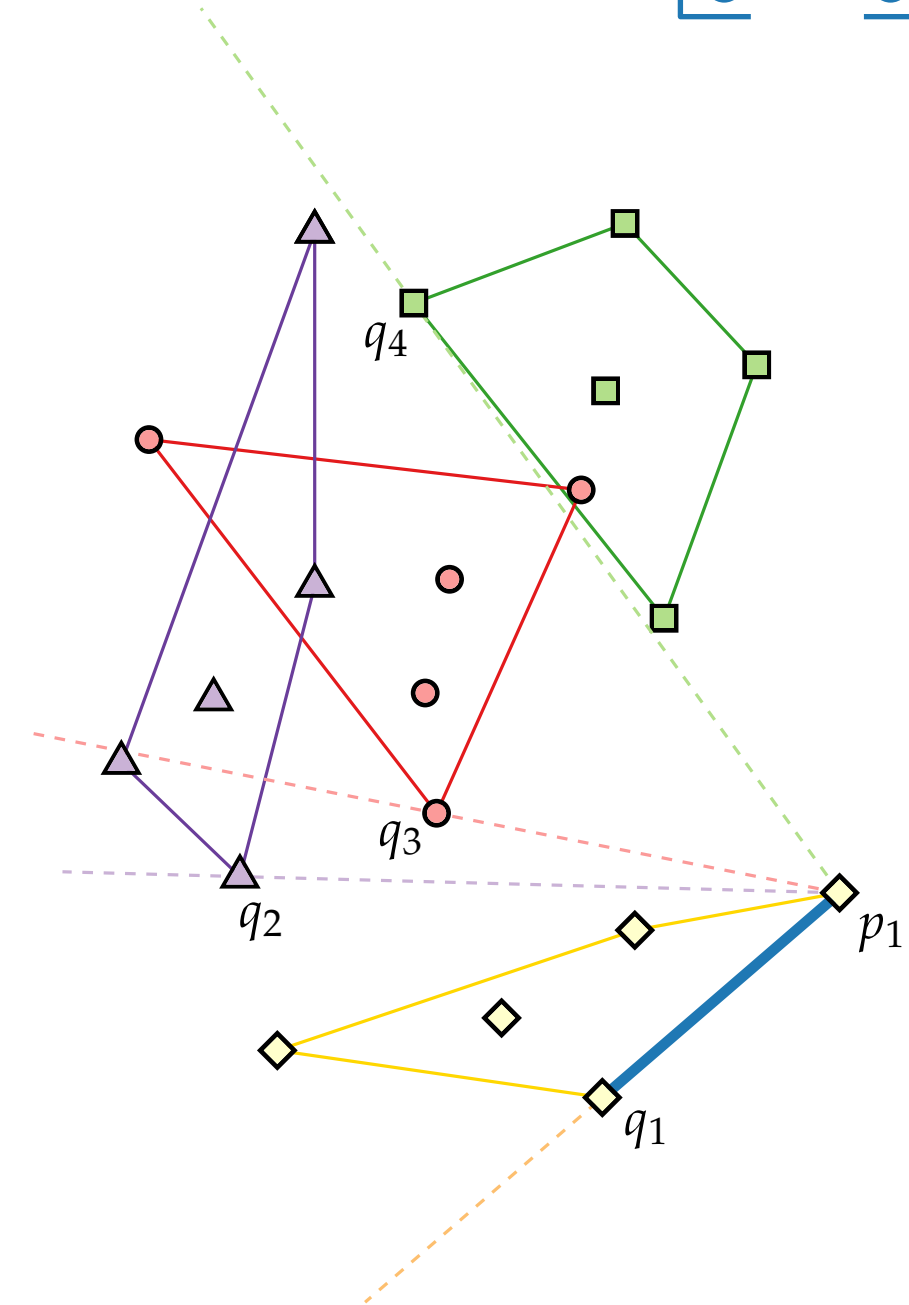
p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

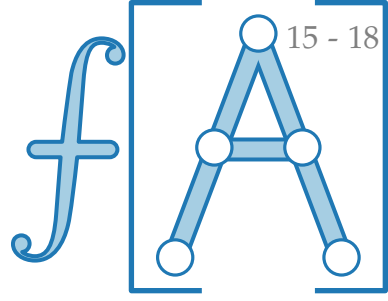
foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

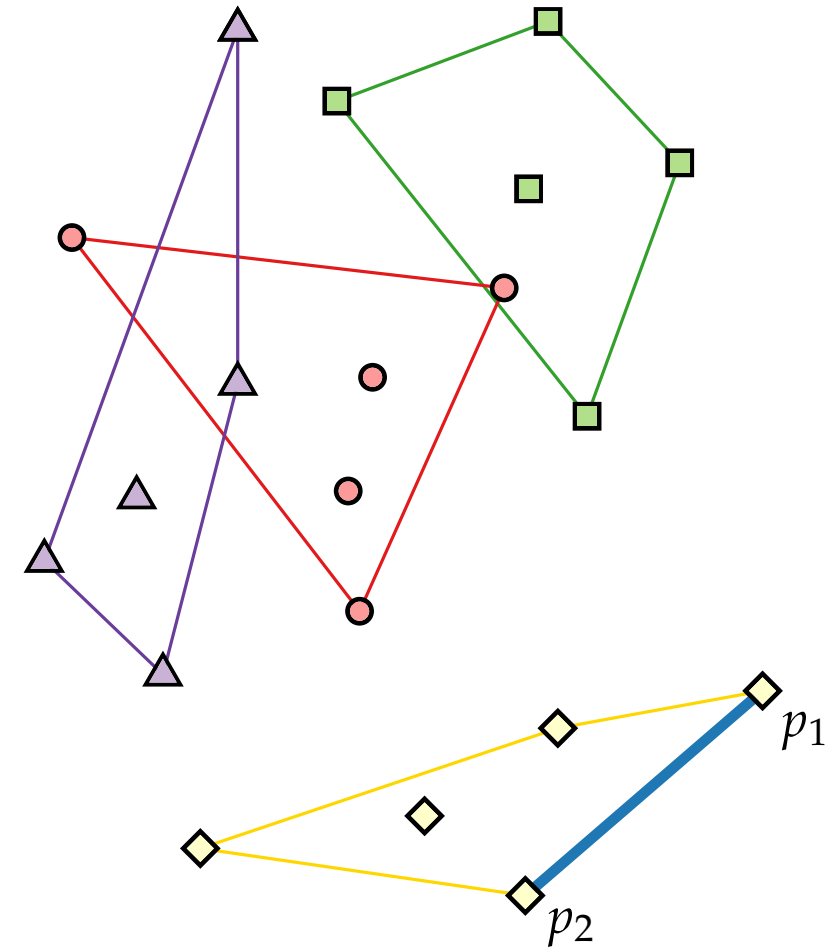
for $k = 2$ **to** m **do**

foreach S_i **do**

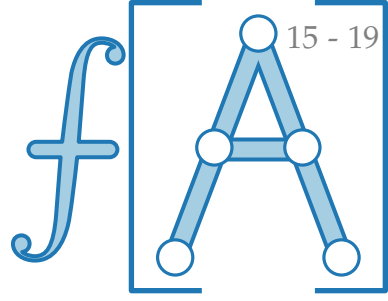
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

z.B. 5



Chans Algorithmus



z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

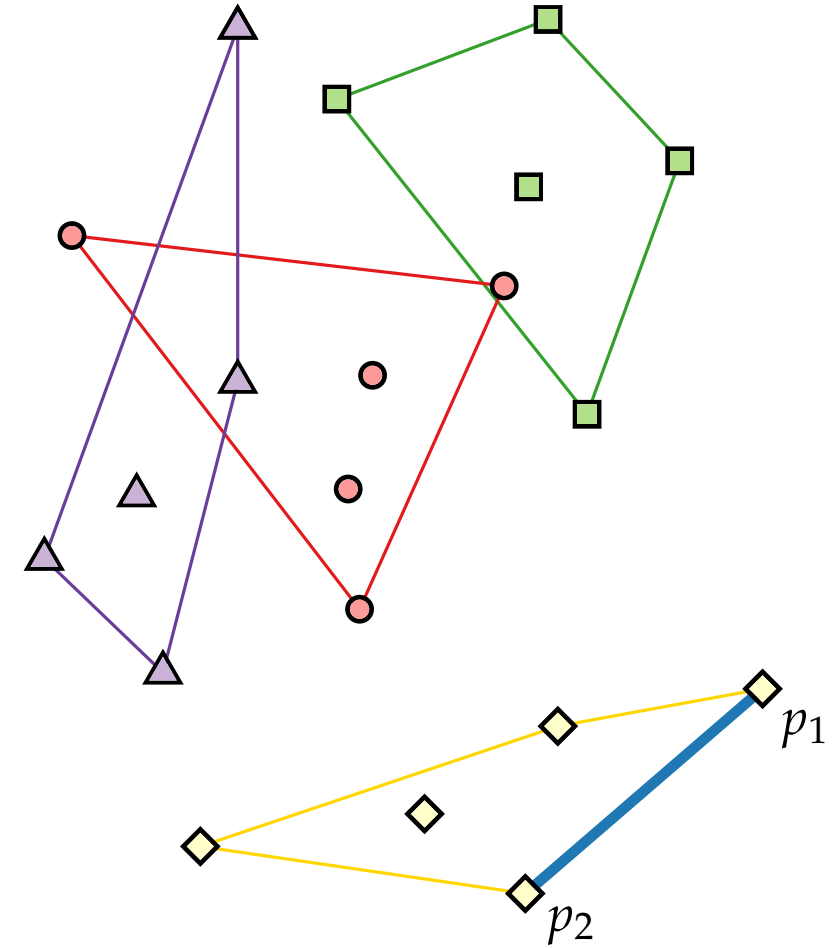
for $k = 2$ **to** m **do**

foreach S_i **do**

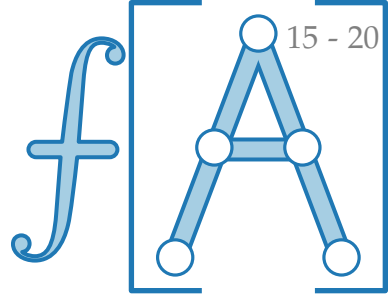
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

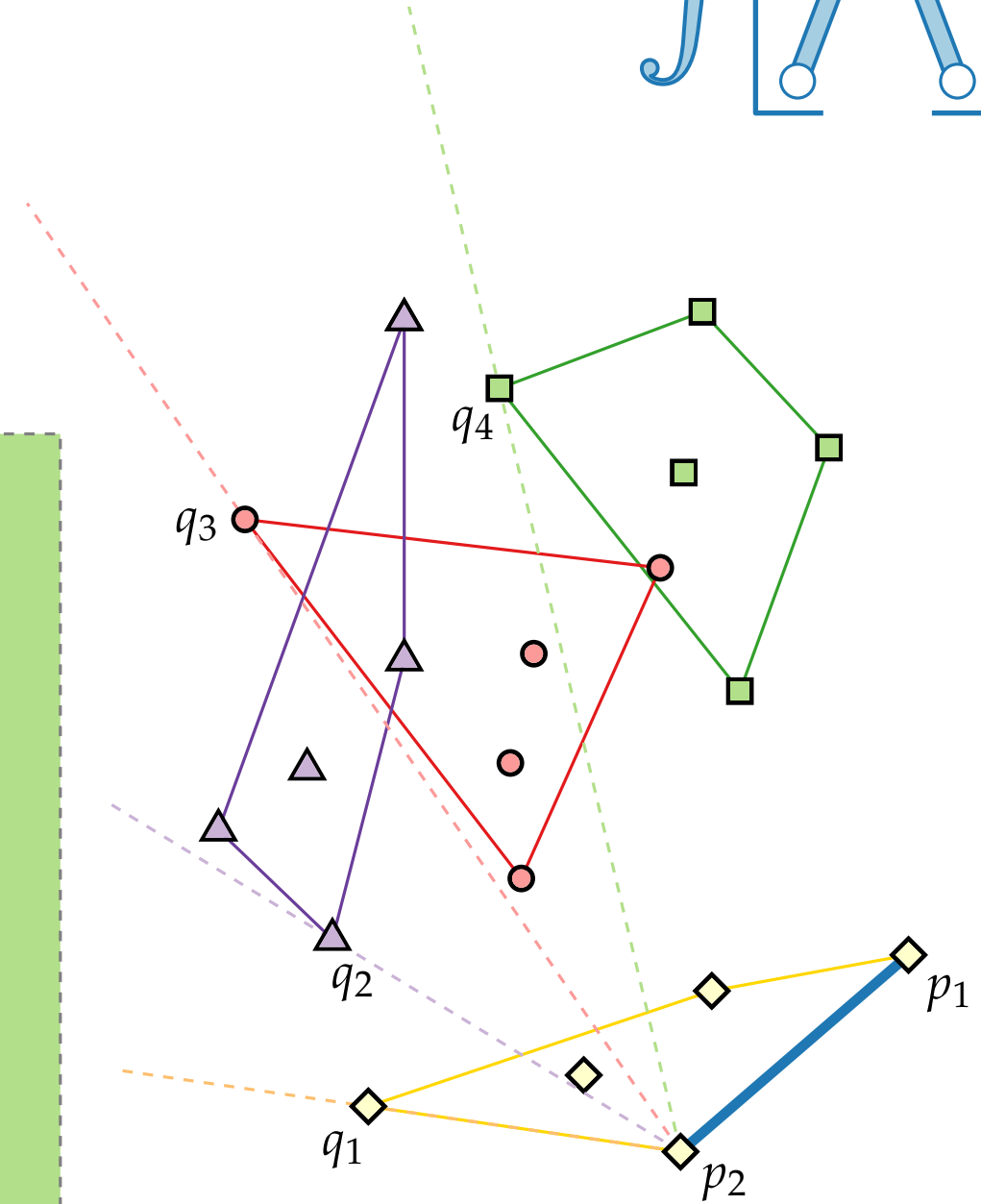
for $k = 2$ **to** m **do**

foreach S_i **do**

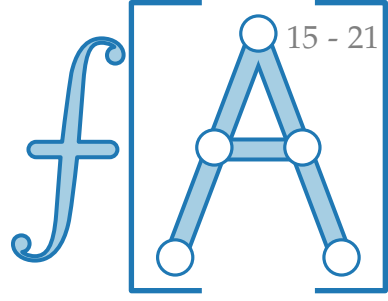
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$



Chans Algorithmus



z.B. 5

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

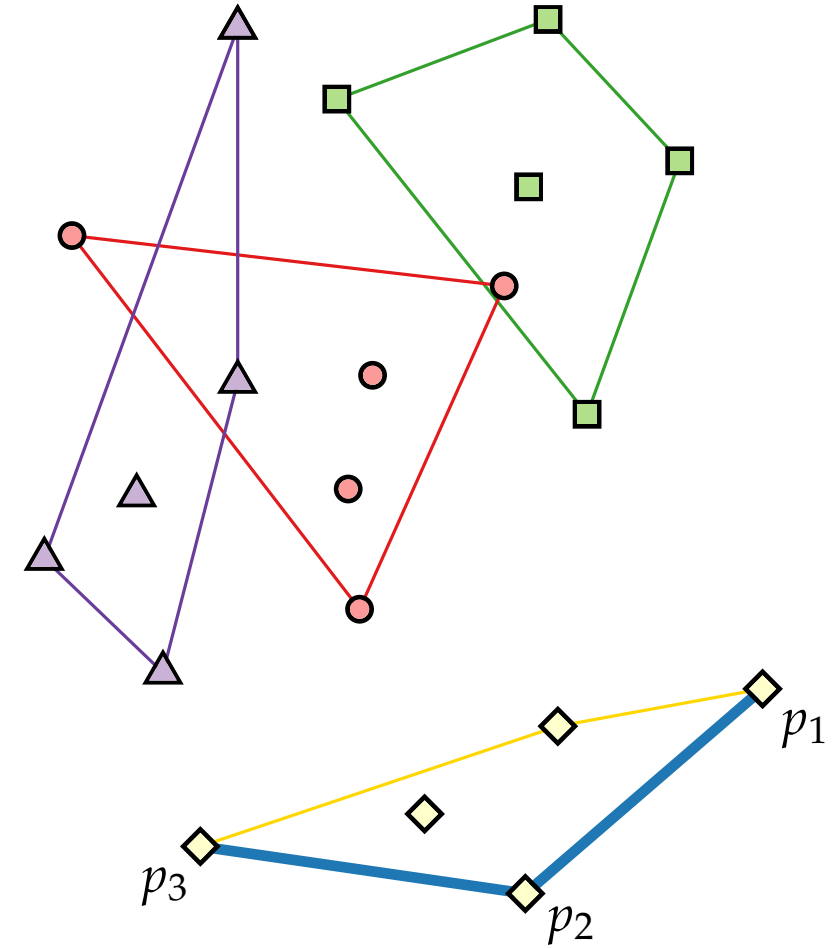
for $k = 2$ **to** m **do**

foreach S_i **do**

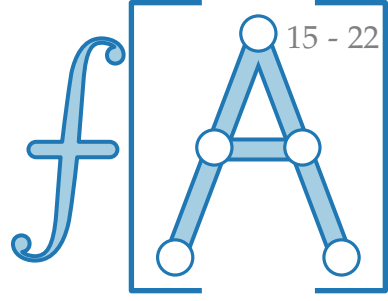
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

z.B. 5

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

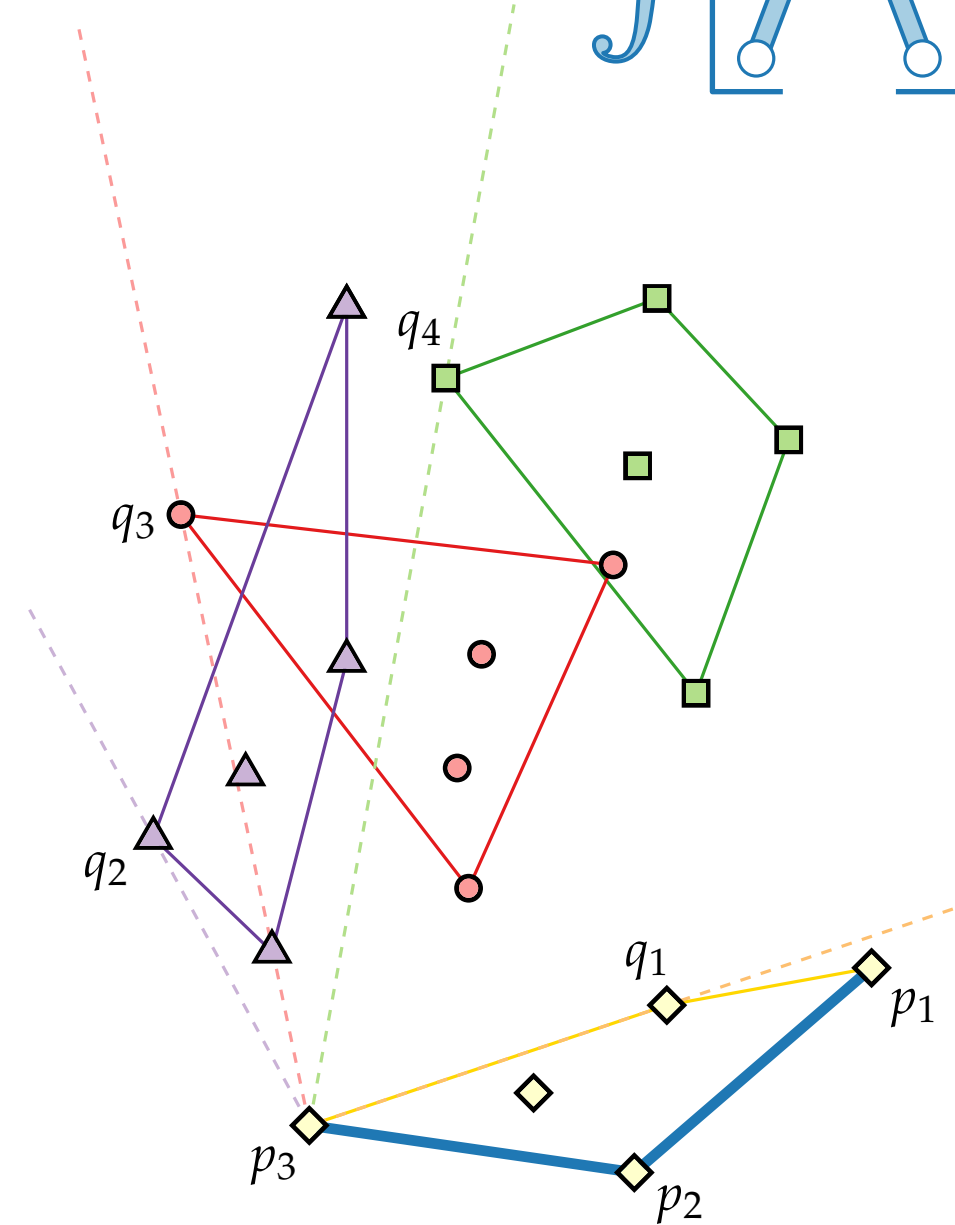
for $k = 2$ **to** m **do**

foreach S_i **do**

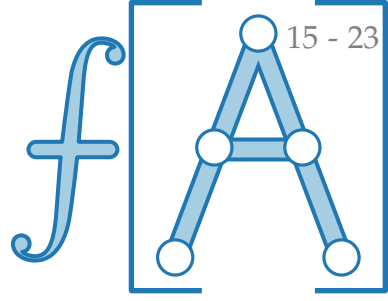
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

z.B. 5

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

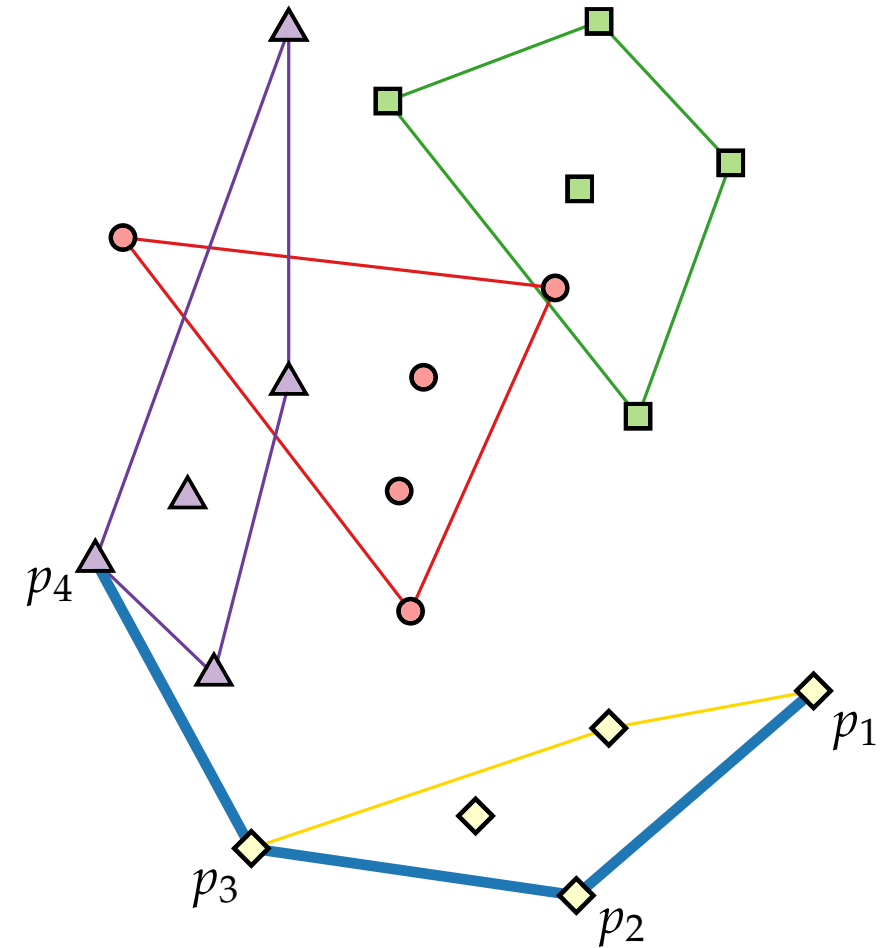
for $k = 2$ **to** m **do**

foreach S_i **do**

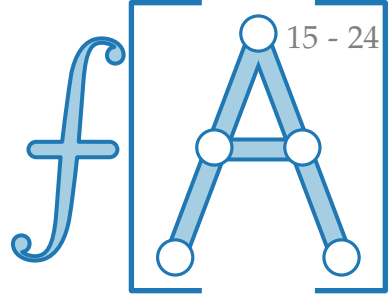
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

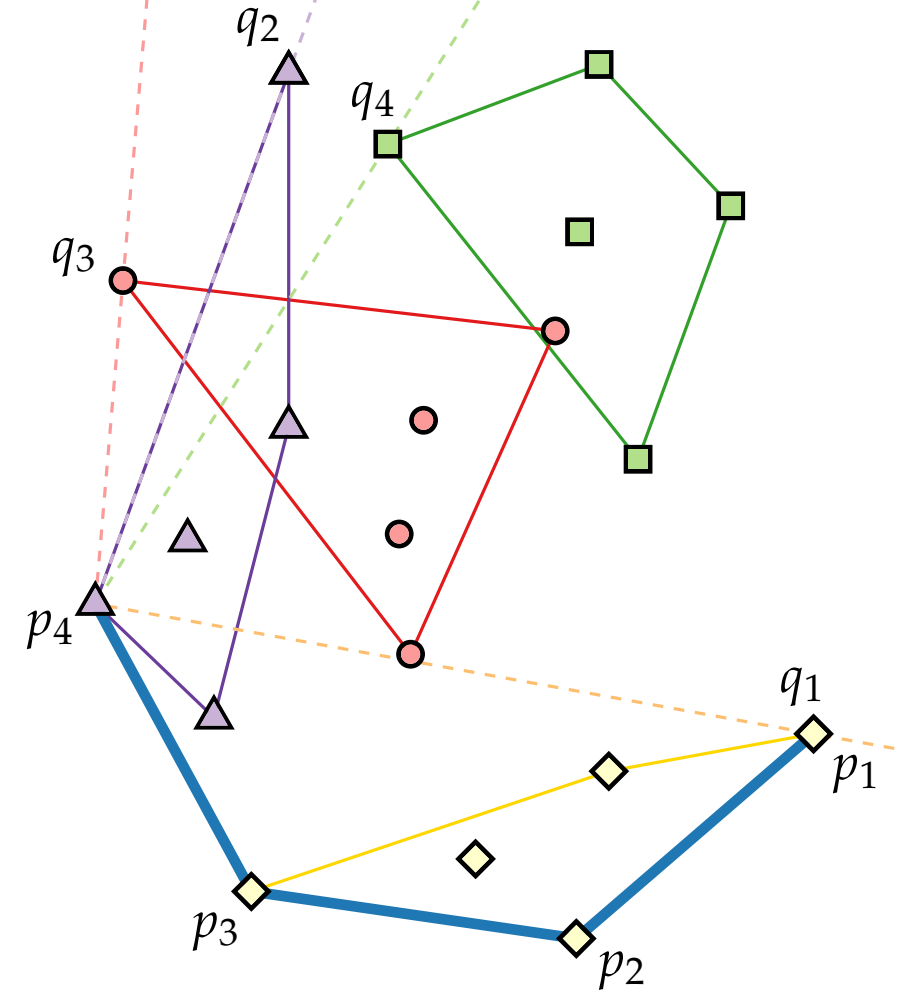
for $k = 2$ **to** m **do**

foreach S_i **do**

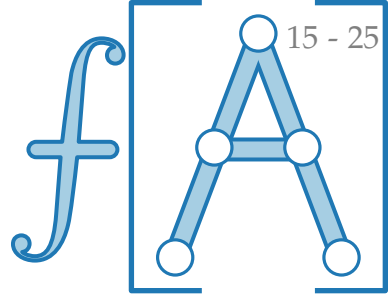
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

z.B. 5

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

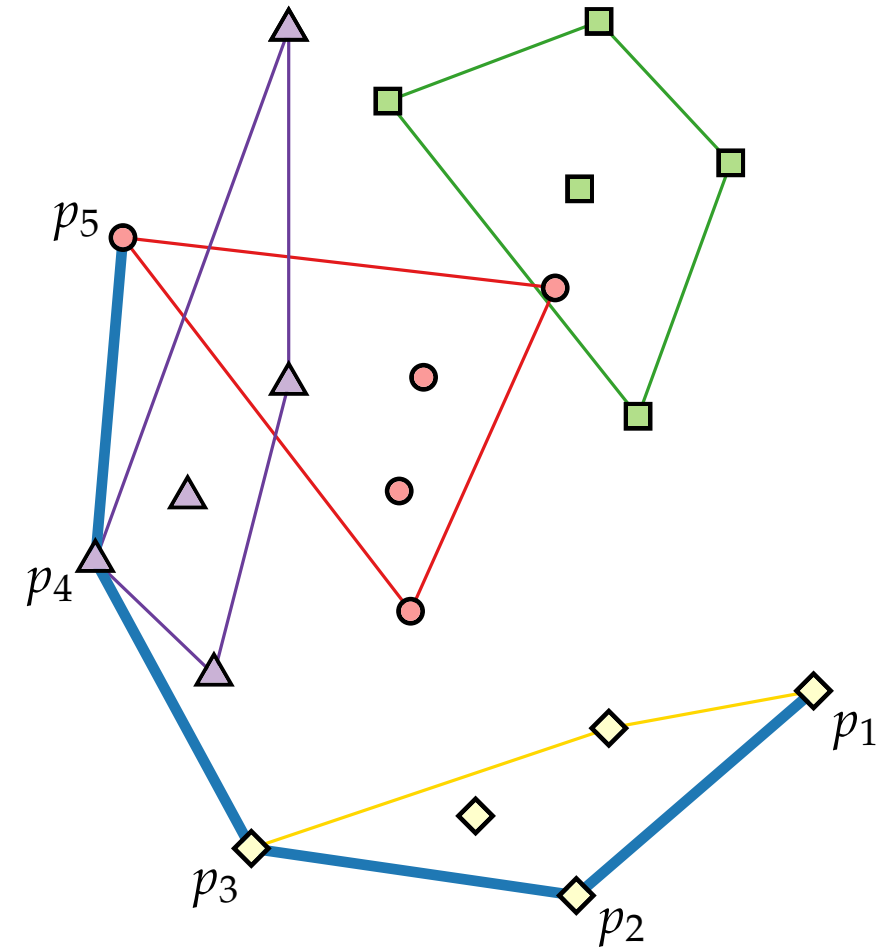
for $k = 2$ **to** m **do**

foreach S_i **do**

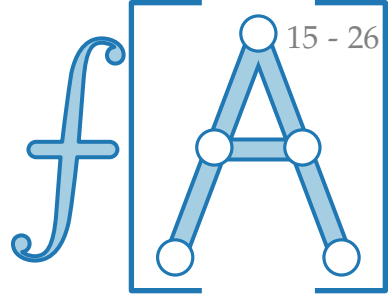
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

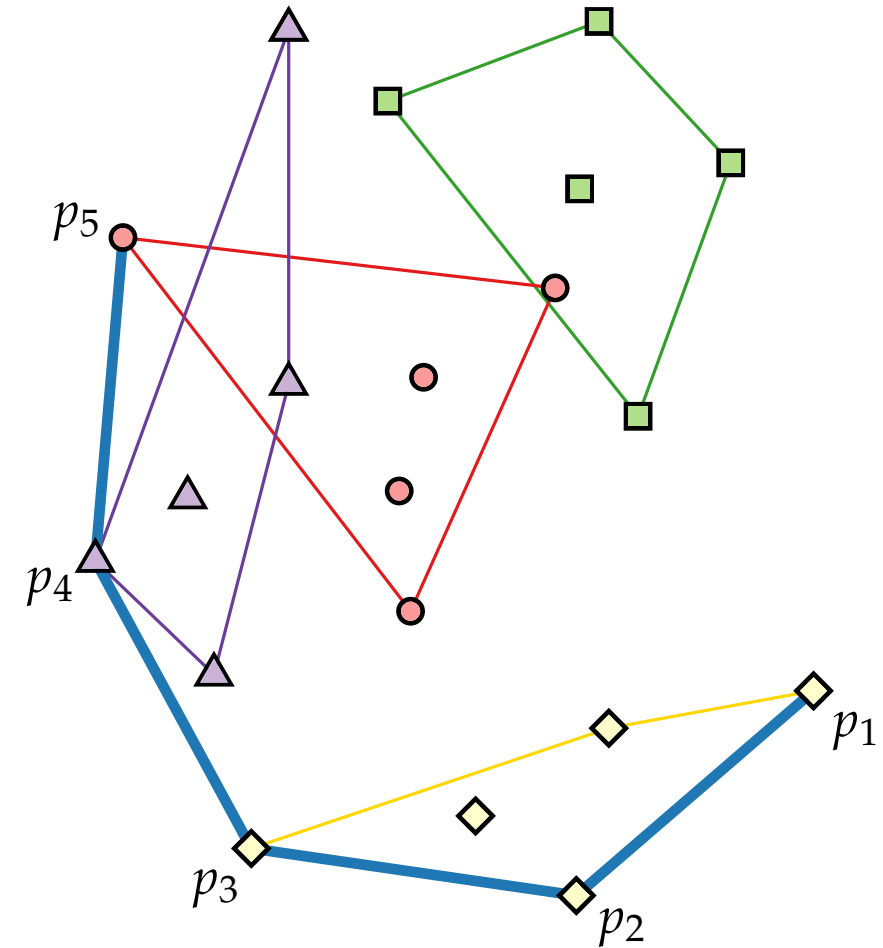
foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

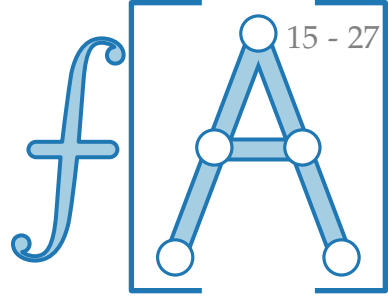
$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$



Chans Algorithmus



CHANSKONVEXEHÜLLE(S, m)

z.B. 5

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

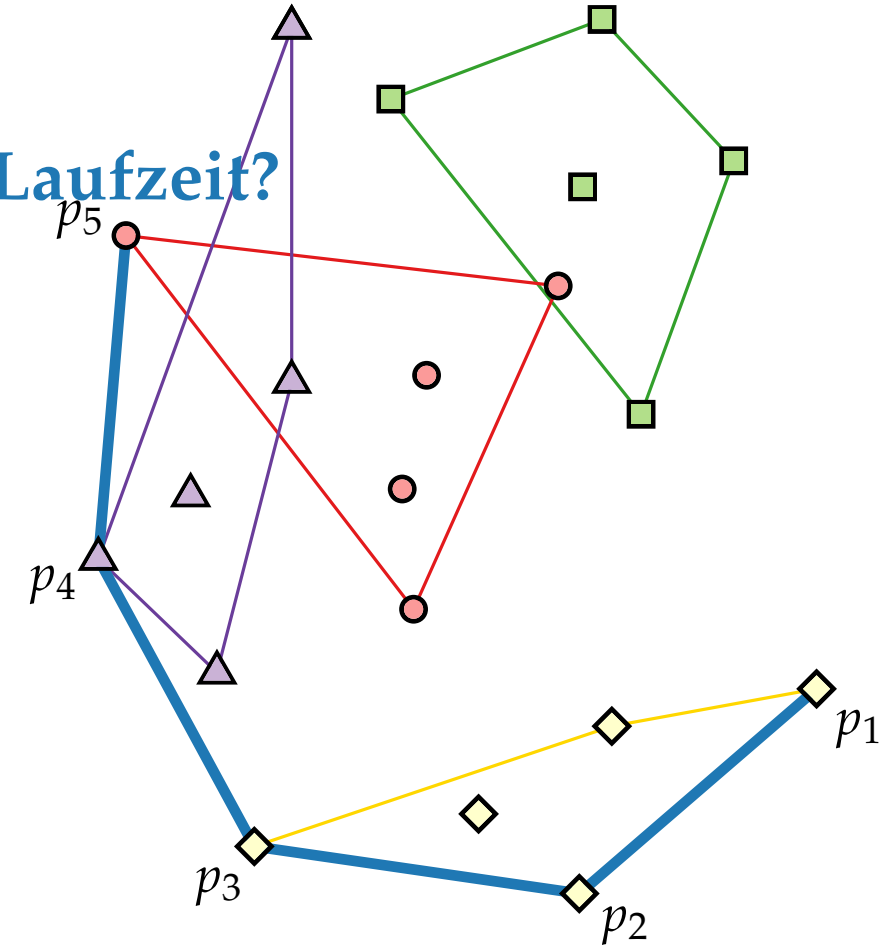
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

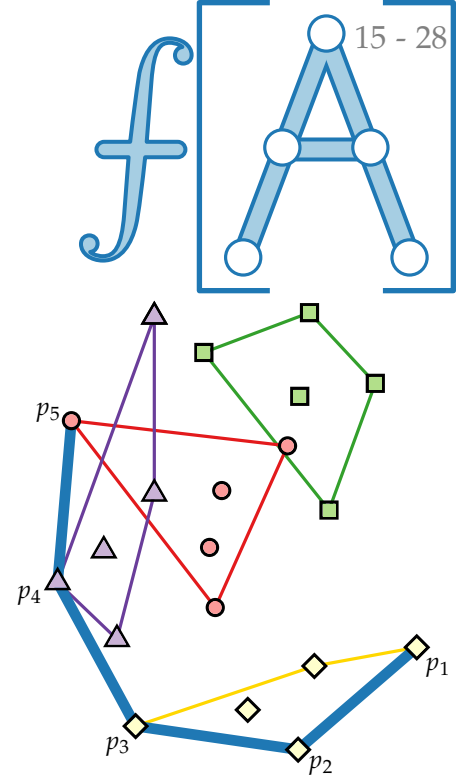
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

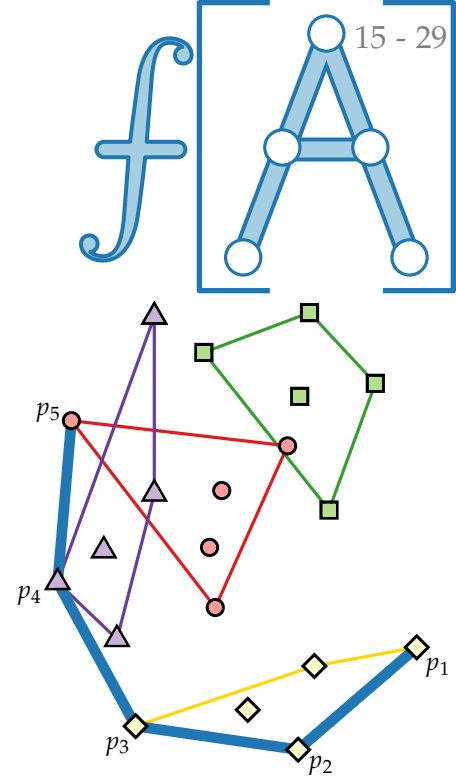
q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

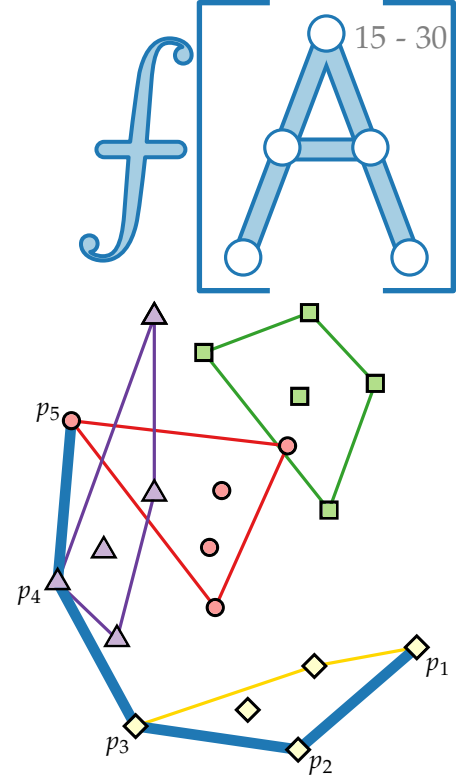
$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

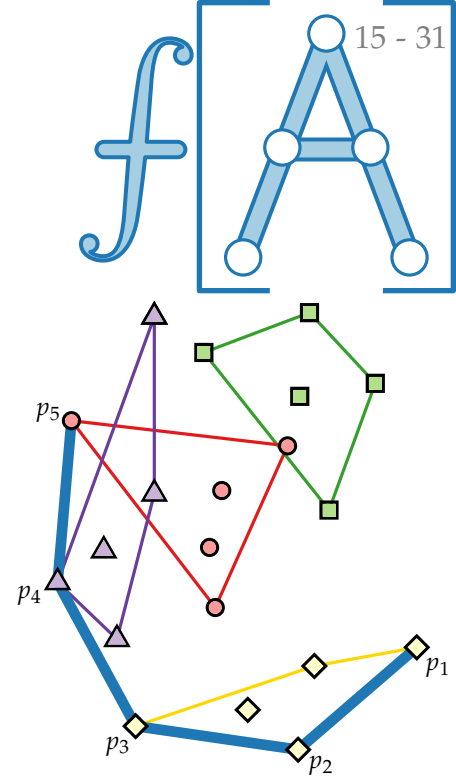
$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

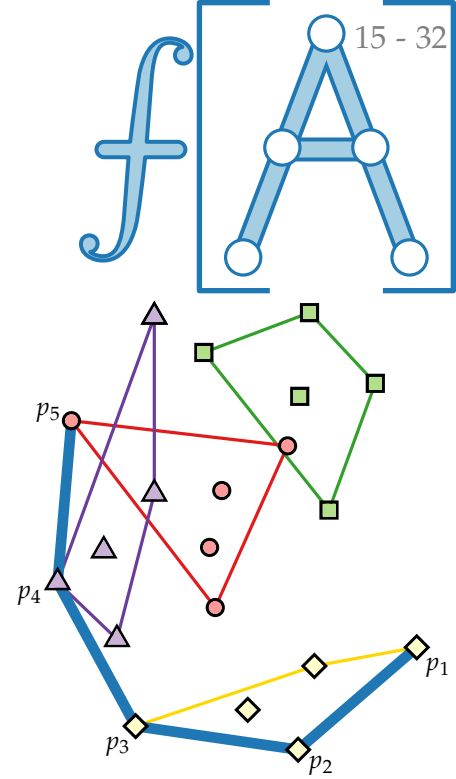
if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

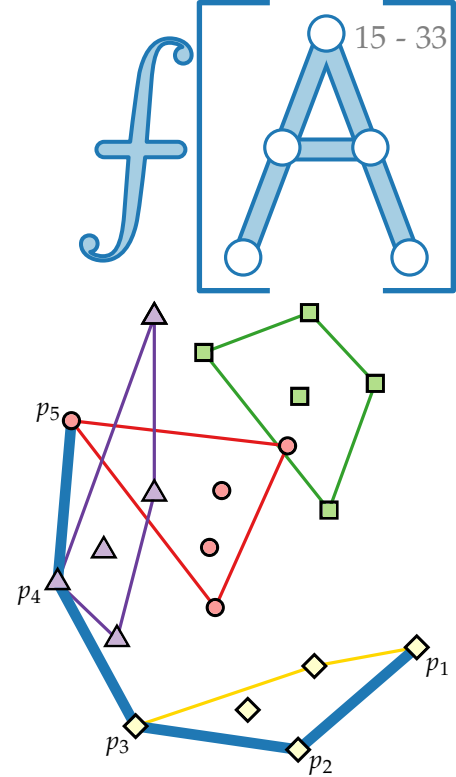
if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

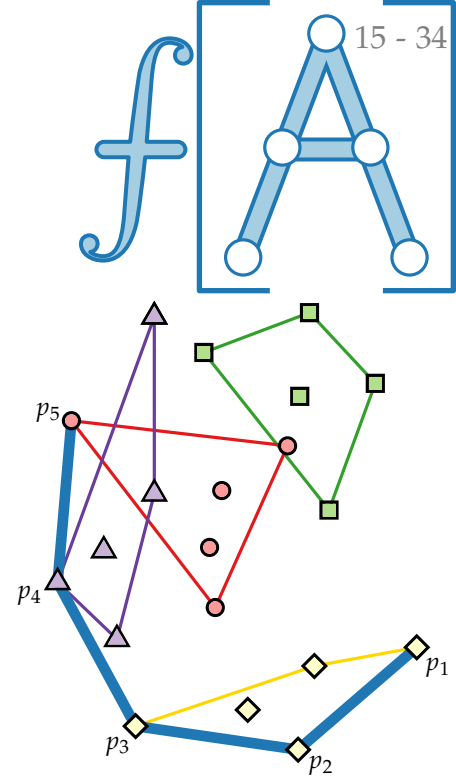
return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot$

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

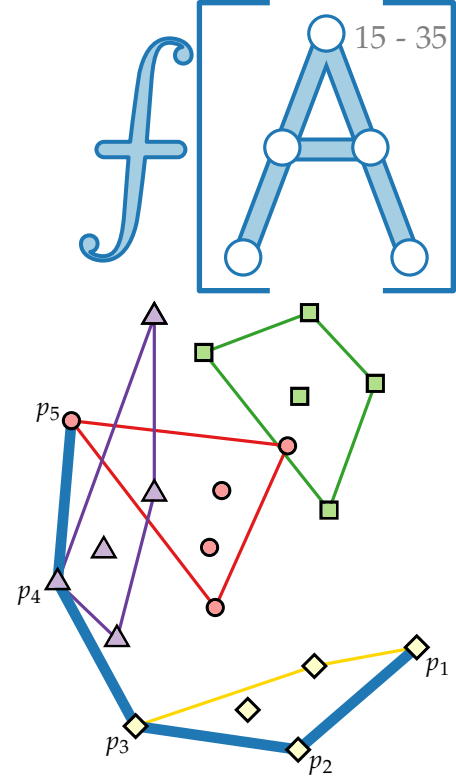
return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) =$

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

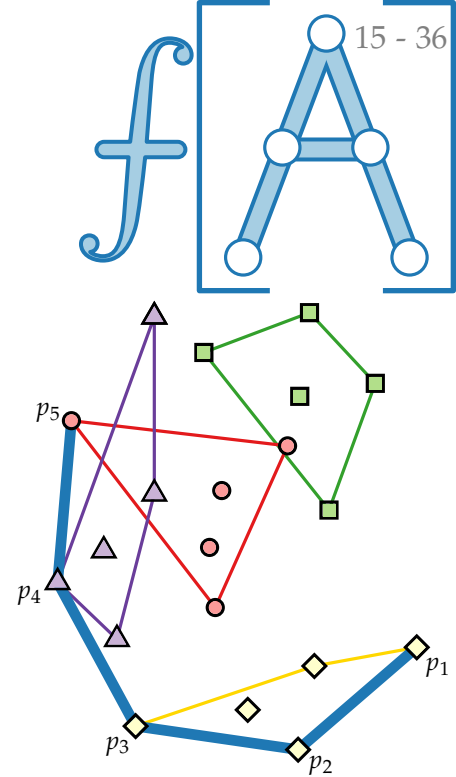
return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

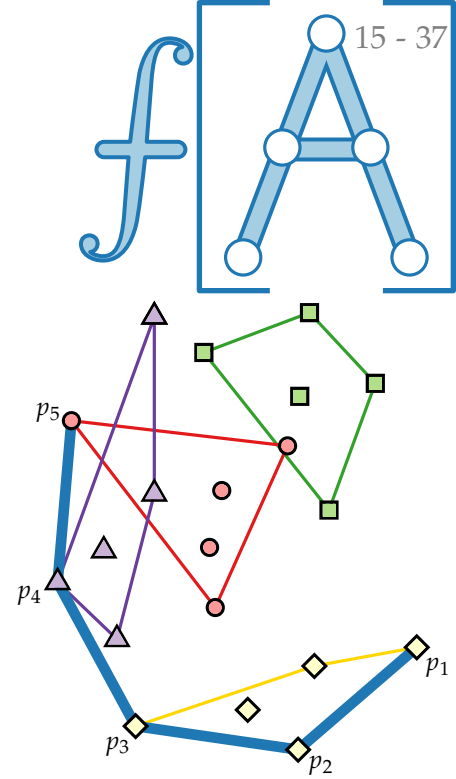
return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

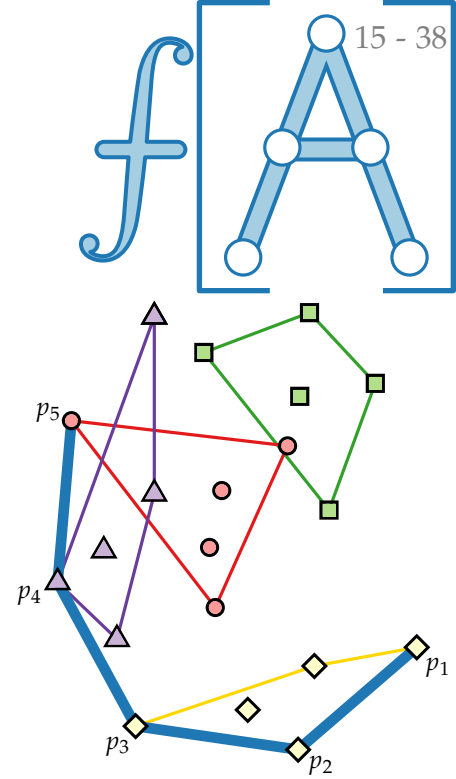
Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

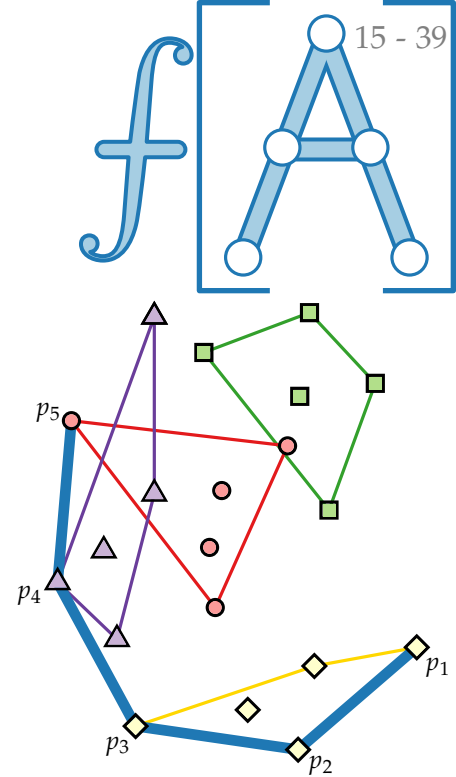
Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

```
foreach  $S_i$  do  $C(S_i) = \text{GRAHAMSCAN}(S_i)$ 
```

p_1 = rechterster Punkt in P

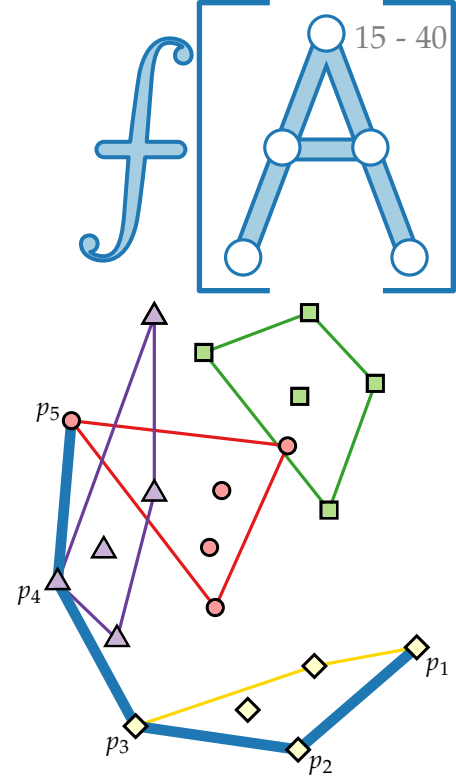
for $k = 2$ **to** m **do****foreach S_i do**

$q_i = \text{Punkt von } CH(S_i), \text{ so dass}$
 $CH(S_i) \text{ rechts von } \overrightarrow{p_{k-1}q_i}.$

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

```
return ⊙
```



Laufzeit?

$$\mathcal{O}(n)$$

$$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$$

$$\mathcal{O}(n)$$

$m.$

$$\frac{n}{m} \cdot$$

Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

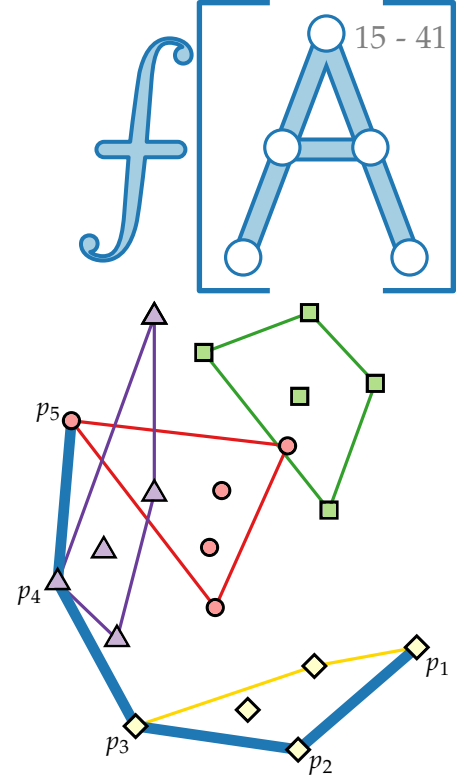
$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

$\frac{n}{m} \cdot$



The figure shows a complex graph with several components. A primary path is highlighted with thick blue edges, connecting vertices labeled p_1 , p_2 , p_3 , p_4 , and p_5 . This path starts at p_1 , goes through p_2 and p_3 , then branches into two parallel paths between p_4 and p_5 . One branch uses purple triangle-shaped vertices and edges, while the other uses red circle-shaped vertices and edges. Additionally, there are green square-shaped vertices connected by thin green edges, forming a separate cycle-like structure. Yellow diamond-shaped vertices are also present, some connected to the main path and others isolated.

Laufzeit?

$$\mathcal{O}(n)$$
$$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$$
$$\mathcal{O}(n)$$
 $m \cdot$

binäre
Suche

return $\emptyset$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

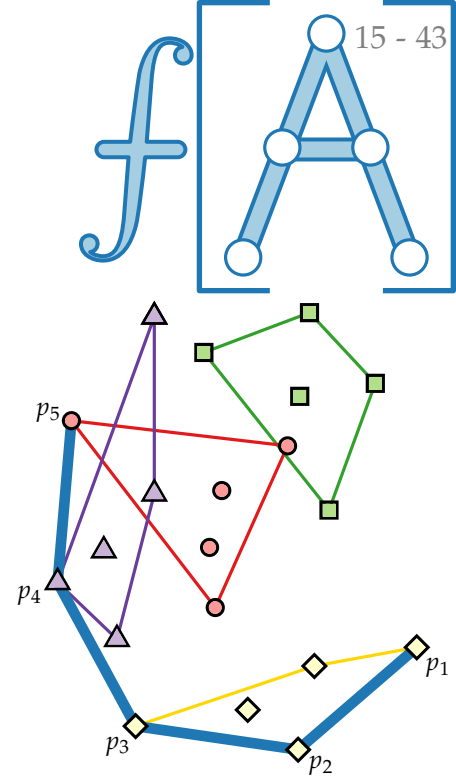
$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

$\frac{n}{m} \cdot$

$\mathcal{O}(\log m)$



Laufzeit?

$$\mathcal{O}(n)$$
$$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$$
$$\mathcal{O}(n)$$
 $m \cdot$

binäre
Suche

$$\frac{n}{m}.$$
$$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$$

```
return ⊙
```


Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

q_i = Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

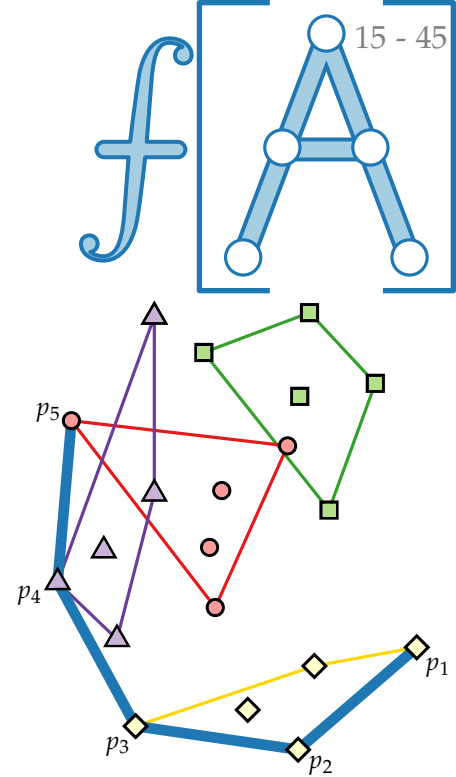
$\mathcal{O}(n)$

$m \cdot$

$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

```
foreach  $S_i$  do  $C(S_i) = \text{GRAHAMSCAN}(S_i)$ 
```

p_1 = rechterster Punkt in P

for $k = 2$ **to** m **do****foreach S_i do**

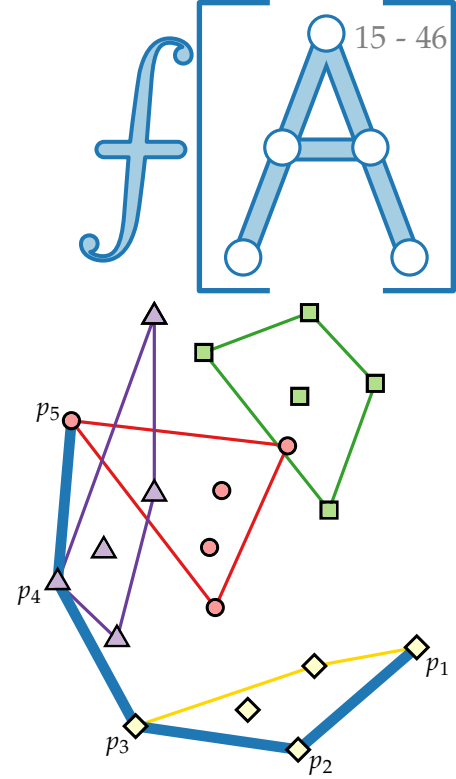
$q_i = \text{Punkt von } CH(S_i), \text{ so dass}$
 $CH(S_i) \text{ rechts von } \overrightarrow{p_{k-1}q_i}.$

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1} p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$



Laufzeit?

$$\mathcal{O}(n)$$
$$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$$
$$\mathcal{O}(n)$$
 $m \cdot$
$$\frac{n}{m} \cdot$$
$$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$$
$$\frac{n}{m} = \mathcal{O}(n)$$
 $\mathcal{O}(1)$

Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

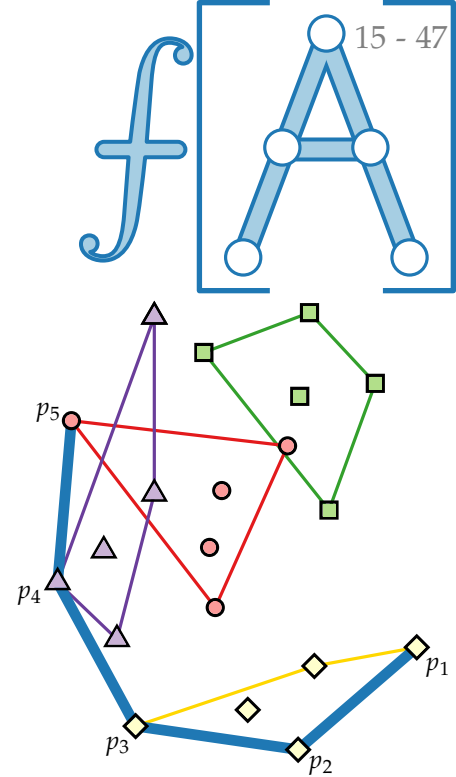
$m \cdot$

$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$

$\mathcal{O}(1)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

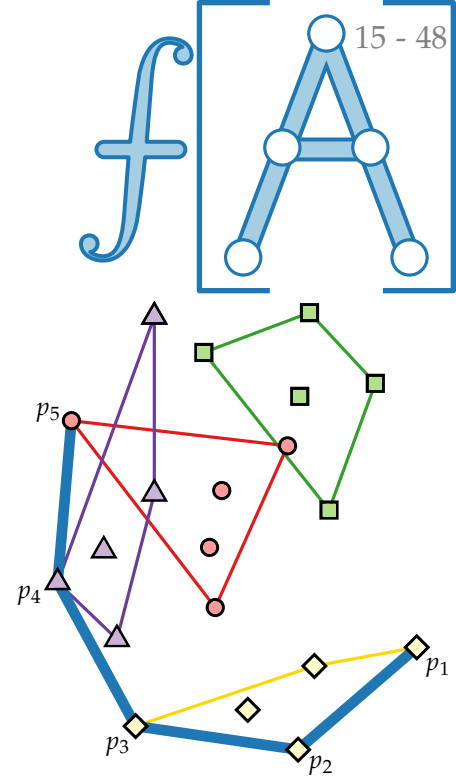
$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$

$\mathcal{O}(1)$

$\mathcal{O}(n \log m)$



Chans Algorithmus

Satz. Wenn wir $h = |\text{CH}(S)|$ kennen, dann berechnet CHANSKONVEXEHÜLLE die konvexe Hülle von S in $\mathcal{O}(n \log h)$ Zeit.

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ **to** m **do**

foreach S_i **do**

$q_i =$ Punkt von $\text{CH}(S_i)$, so dass
 $\text{CH}(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

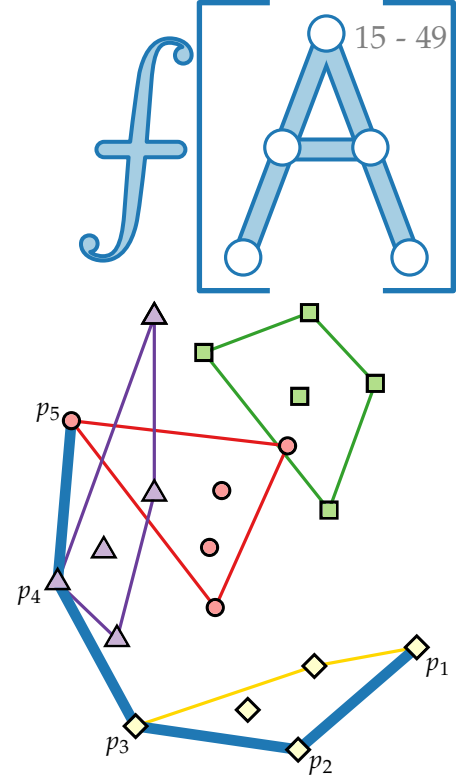
$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$

$\mathcal{O}(1)$

$\mathcal{O}(n \log m)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S)

for $i = 1, \dots, \log \log n$ do

|

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i do $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ to m do

foreach S_i do

| $q_i =$ Punkt von $CH(S_i)$, so dass
| $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

| $p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

| if $p_k = p_1$ then return $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

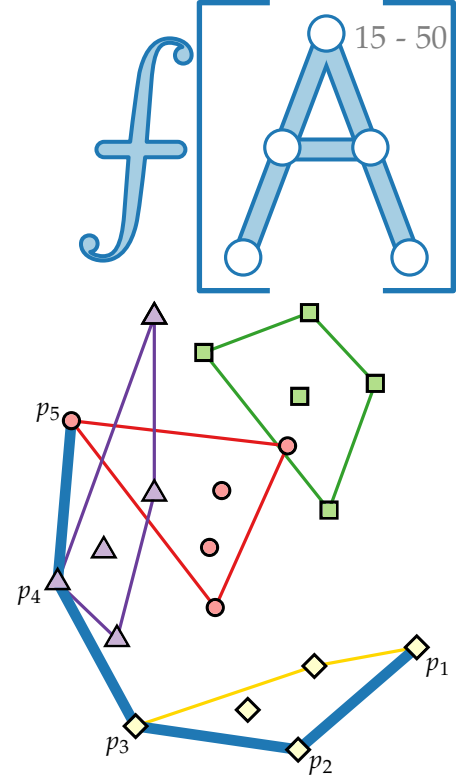
$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$

$\mathcal{O}(1)$

$\mathcal{O}(n \log m)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S)

for $i = 1, \dots, \log \log n$ **do**

$L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ to m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

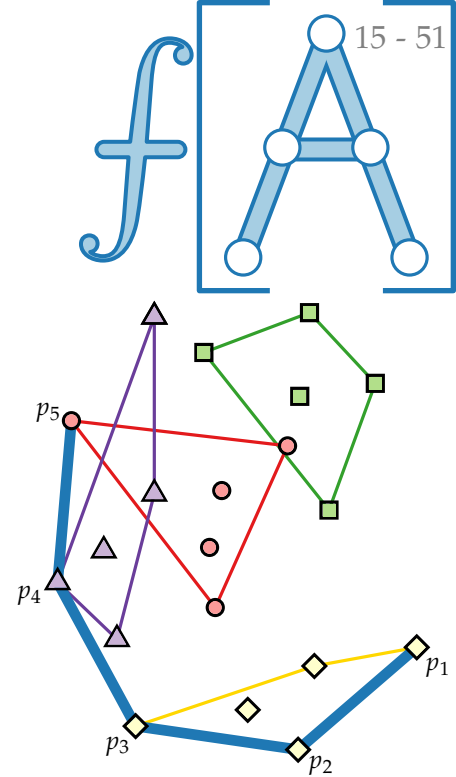
$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$

$\mathcal{O}(1)$

$\mathcal{O}(n \log m)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S)

for $i = 1, \dots, \log \log n$ **do**

$L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$

$m = 2, 4, 16, 256, \dots$

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ to m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

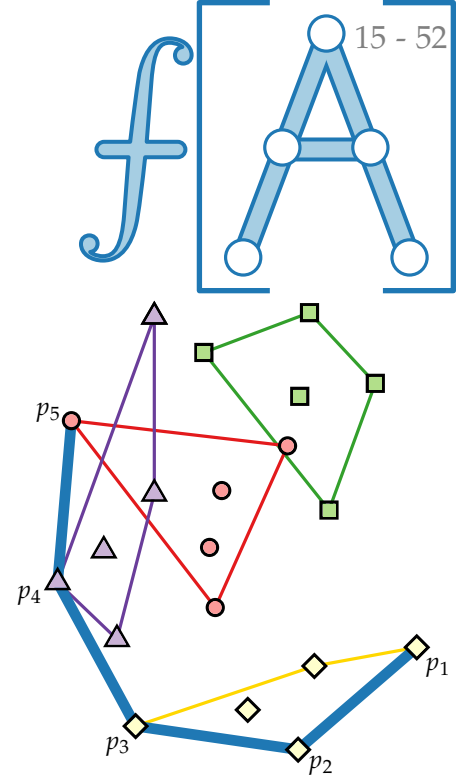
$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$

$\mathcal{O}(1)$

$\mathcal{O}(n \log m)$



Chans Algorithmus

CHANSKONVEXEHÜLLE(S)

for $i = 1, \dots, \log \log n$ **do**

$L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$

if $L \neq \emptyset$ **then return** L

$m = 2, 4, 16, 256, \dots$

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$

$p_1 =$ rechtester Punkt in P

for $k = 2$ to m **do**

foreach S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

Laufzeit?

$\mathcal{O}(n)$

$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$

$\mathcal{O}(n)$

$m \cdot$

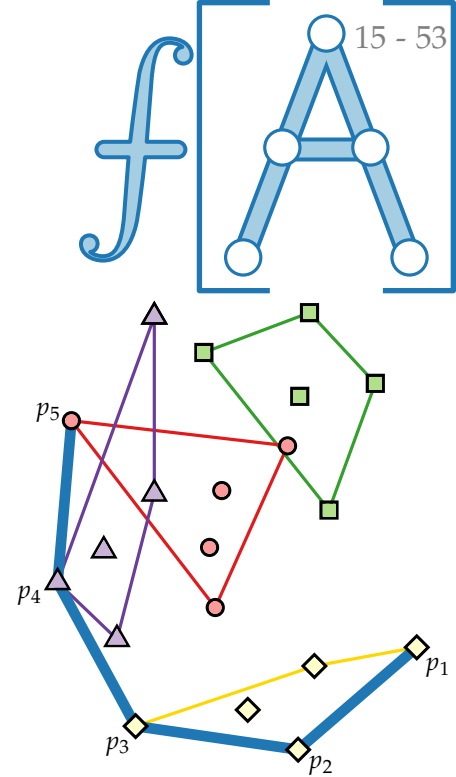
$\frac{n}{m} \cdot$

$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$

$\frac{n}{m} = \mathcal{O}(n)$

$\mathcal{O}(1)$

$\mathcal{O}(n \log m)$



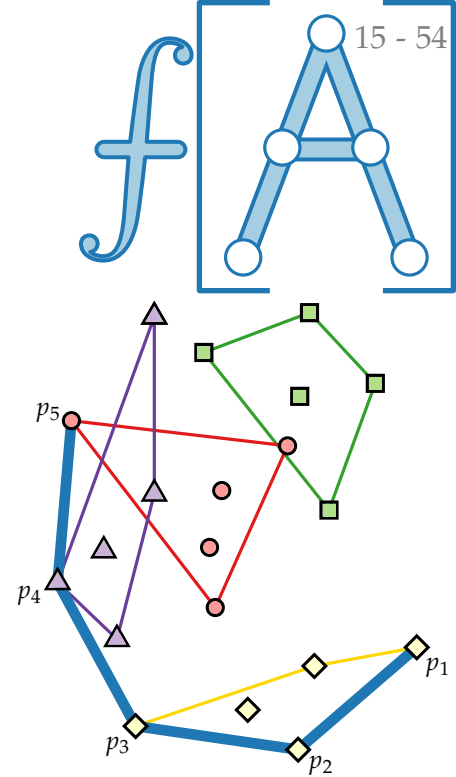
Chans Algorithmus

CHANSKONVEXEHÜLLE(S) Spätestens fertig wenn $i = \log \log h$
for $i = 1, \dots, \log \log n$ **do** $m = 2, 4, 16, 256, \dots$
 $L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$
 if $L \neq \emptyset$ **then return** L

CHANSKONVEXEHÜLLE(S, m)
 Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$
 $p_1 = \text{rechtster Punkt in } P$
for $k = 2$ **to** m **do**
 foreach S_i **do**
 $q_i = \text{Punkt von } CH(S_i), \text{ so dass}$
 $CH(S_i) \text{ rechts von } \overrightarrow{p_{k-1}q_i}.$ binäre Suche
 $p_k = q_i, \text{ so dass } q_1, \dots, q_{n/m} \text{ rechts von } \overrightarrow{p_{k-1}p_k}$
 if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$
return $\emptyset$

Laufzeit?

$$\begin{aligned} & \mathcal{O}(n) \\ & \frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m) \\ & \mathcal{O}(n) \\ & m \cdot \\ & \frac{n}{m} \cdot \\ & \mathcal{O}(\log m) = \mathcal{O}(n \log m) \\ & \frac{n}{m} = \mathcal{O}(n) \\ & \mathcal{O}(1) \\ & \underbrace{\hspace{15em}}_{\mathcal{O}(n \log m)} \end{aligned}$$



Chans Algorithmus

```

CHANSKONVEXEHÜLLE( $S$ )
  for  $i = 1, \dots, \log \log n$  do
     $L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$ 
    if  $L \neq \emptyset$  then return  $L$ 

```

CHANSKONVEXEHÜLLE(S, m)

Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m

```
foreach  $S_i$  do  $C(S_i) = \text{GRAHAMSCAN}(S_i)$ 
```

p_1 = rechtester Punkt in P

for $k = 2$ **to** m **do****foreach** S_i **do**

$q_i =$ Punkt von $CH(S_i)$, so dass
 $CH(S_i)$ rechts von $\overrightarrow{p_{k-1}q_i}$.

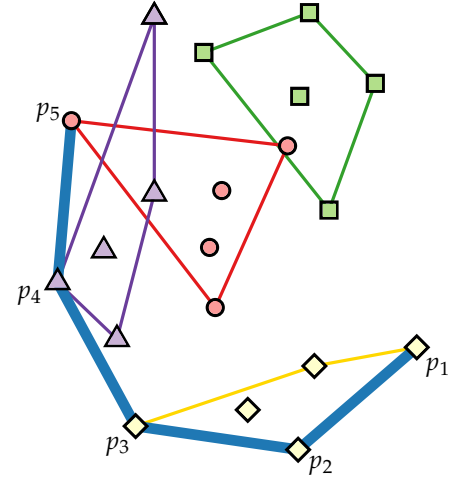
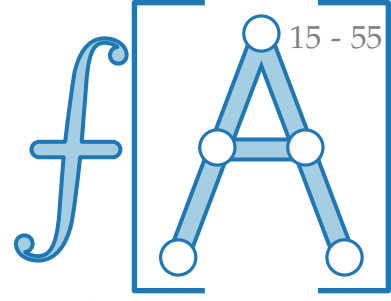
binäre
Suche

$p_k = q_i$, so dass $q_1, \dots, q_{n/m}$ rechts von $\overrightarrow{p_{k-1}p_k}$

if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$

return $\emptyset$

$$\sum_{i=1}^{\log \log h} \mathcal{O}(n \log 2^{2^i})$$



Laufzeit?

$$\mathcal{O}(n)$$
$$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$$
$$\mathcal{O}(n)$$
 $m \cdot$
$$\frac{n}{m}.$$
$$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$$
$$\frac{n}{m} = \mathcal{O}(n)$$
$$\mathcal{O}(1)$$
$$\mathcal{O}(n \log m)$$

Chans Algorithmus

CHANSKONVEXEHÜLLE(S) Spätestens fertig wenn $i = \log \log h$
for $i = 1, \dots, \log \log n$ **do** $m = 2, 4, 16, 256, \dots$
 $L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$
 if $L \neq \emptyset$ **then return** L

CHANSKONVEXEHÜLLE(S, m)
 Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$
 $p_1 = \text{rechtster Punkt in } P$
for $k = 2$ **to** m **do**
 foreach S_i **do**
 $q_i = \text{Punkt von } CH(S_i), \text{ so dass}$
 $CH(S_i) \text{ rechts von } \overrightarrow{p_{k-1}q_i}.$ binäre Suche
 $p_k = q_i, \text{ so dass } q_1, \dots, q_{n/m} \text{ rechts von } \overrightarrow{p_{k-1}p_k}$
 if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$
return $\emptyset$

$$\sum_{i=1}^{\log \log h} \mathcal{O}(n \log 2^{2^i})$$

$$= \mathcal{O}(n \cdot \sum_{i=1}^{\log \log h} 2^i)$$

Laufzeit?

$$\mathcal{O}(n)$$

$$\frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$$

$$\mathcal{O}(n)$$

$$m \cdot$$

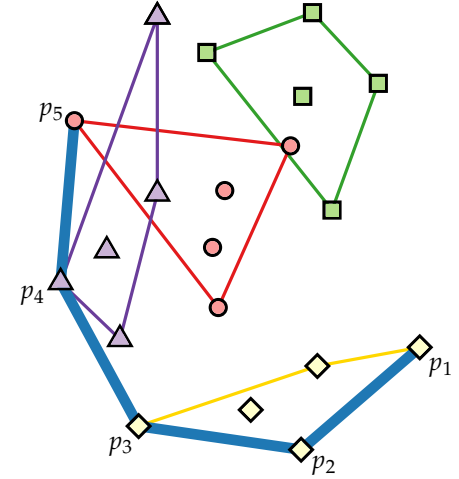
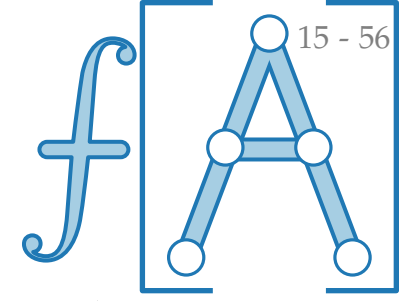
$$\frac{n}{m} \cdot$$

$$\mathcal{O}(\log m) = \mathcal{O}(n \log m)$$

$$\frac{n}{m} = \mathcal{O}(n)$$

$$\mathcal{O}(1)$$

$$\mathcal{O}(n \log m)$$



Chans Algorithmus

```

CHANSKONVEXEHÜLLE(S) Spätestens fertig wenn  $i = \log \log h$ 
  for  $i = 1, \dots, \log \log n$  do  $m = 2, 4, 16, 256, \dots$ 
  |    $L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$ 
  |   if  $L \neq \emptyset$  then return  $L$ 
  
```

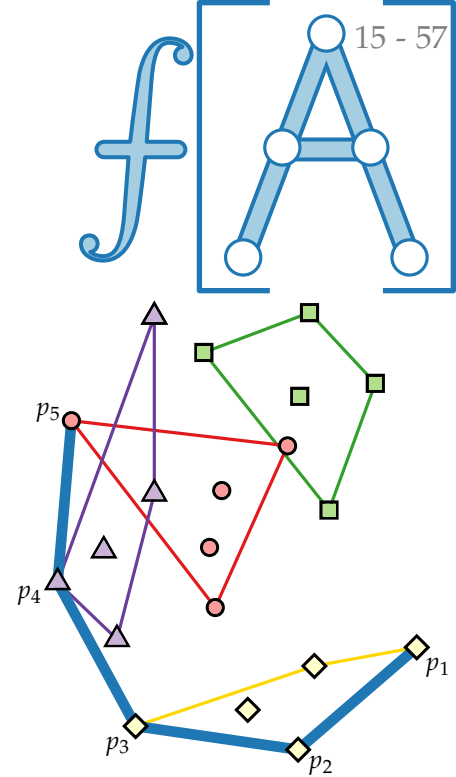
```

CHANSKONVEXEHÜLLE( $S, m$ )
  Zerlege  $S$  in  $n/m$  Mengen  $S_1, \dots, S_{n/m}$  der Größe  $m$ 
  foreach  $S_i$  do  $C(S_i) = \text{GRAHAMSCAN}(S_i)$ 
   $p_1 =$  rechtester Punkt in  $P$ 
  for  $k = 2$  to  $m$  do
  |   foreach  $S_i$  do
  |   |    $q_i =$  Punkt von  $CH(S_i)$ , so dass binäre Suche
  |   |    $CH(S_i)$  rechts von  $\overrightarrow{p_{k-1}q_i}$ .
  |   |    $p_k = q_i$ , so dass  $q_1, \dots, q_{n/m}$  rechts von  $\overrightarrow{p_{k-1}p_k}$ 
  |   |   if  $p_k = p_1$  then return  $\langle p_1, \dots, p_k \rangle$ 
  return  $\emptyset$ 
  
```

$$\begin{aligned}
 & \sum_{i=1}^{\log \log h} \mathcal{O}(n \log 2^{2^i}) \\
 &= \mathcal{O}(n \cdot \sum_{i=1}^{\log \log h} 2^i) \\
 &= \mathcal{O}(n \cdot 2^{1+\log \log h})
 \end{aligned}$$

Laufzeit?

$$\begin{aligned}
 & \mathcal{O}(n) \\
 & \frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m) \\
 & \mathcal{O}(n) \\
 & m \cdot \\
 & \frac{n}{m} \cdot \\
 & \mathcal{O}(\log m) = \mathcal{O}(n \log m) \\
 & \frac{n}{m} = \mathcal{O}(n) \\
 & \mathcal{O}(1) \\
 & \underbrace{\hspace{10em}}_{\mathcal{O}(n \log m)}
 \end{aligned}$$



Chans Algorithmus

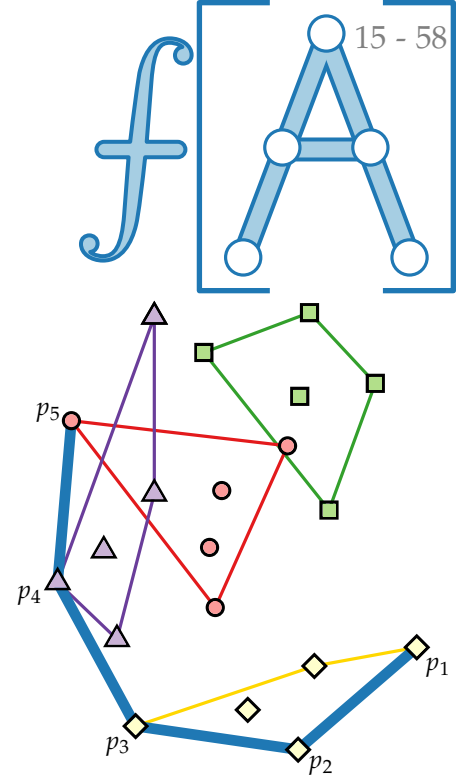
CHANSKONVEXEHÜLLE(S) Spätestens fertig wenn $i = \log \log h$
for $i = 1, \dots, \log \log n$ **do** $m = 2, 4, 16, 256, \dots$
 $L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$
 if $L \neq \emptyset$ **then return** L

CHANSKONVEXEHÜLLE(S, m)
 Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$
 $p_1 = \text{rechtester Punkt in } P$
for $k = 2$ **to** m **do**
 foreach S_i **do**
 $q_i = \text{Punkt von } CH(S_i), \text{ so dass}$
 $CH(S_i) \text{ rechts von } \overrightarrow{p_{k-1}q_i}.$ binäre Suche
 $p_k = q_i, \text{ so dass } q_1, \dots, q_{n/m} \text{ rechts von } \overrightarrow{p_{k-1}p_k}$
 if $p_k = p_1$ **then return** $\langle p_1, \dots, p_k \rangle$
return $\emptyset$

$$\begin{aligned} & \sum_{i=1}^{\log \log h} \mathcal{O}(n \log 2^{2^i}) \\ &= \mathcal{O}(n \cdot \sum_{i=1}^{\log \log h} 2^i) \\ &= \mathcal{O}(n \cdot 2^{1+\log \log h}) \\ &= \mathcal{O}(n \cdot \log h) \end{aligned}$$

Laufzeit?

$$\begin{aligned} & \mathcal{O}(n) \\ & \frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m) \\ & \mathcal{O}(n) \\ & m \cdot \\ & \frac{n}{m} \cdot \\ & \mathcal{O}(\log m) = \mathcal{O}(n \log m) \\ & \frac{n}{m} = \mathcal{O}(n) \\ & \mathcal{O}(1) \\ & \underbrace{\hspace{10em}}_{\mathcal{O}(n \log m)} \end{aligned}$$



Chans Algorithmus

$\text{CHANSKONVEXEHÜLLE}(S)$ Spätestens fertig wenn $i = \log \log h$
for $i = 1, \dots, \log \log n$ **do** $m = 2, 4, 16, 256, \dots$
 $L = \text{CHANSKONVEXEHÜLLE}(S, 2^{2^i})$
 if $L \neq \emptyset$ **then return** L

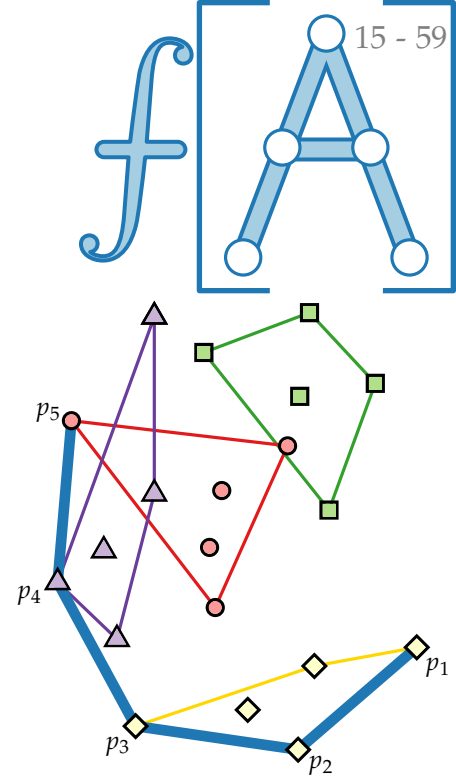
$\text{CHANSKONVEXEHÜLLE}(S, m)$
 Zerlege S in n/m Mengen $S_1, \dots, S_{n/m}$ der Größe m
foreach S_i **do** $C(S_i) = \text{GRAHAMSCAN}(S_i)$
 $p_1 = \text{rechtster Punkt in } P$
for $k = 2$ **to** m **do**
 foreach S_i **do**
 $q_i = \text{Punkt von } CH(S_i), \text{ so dass}$
 $CH(S_i) \text{ rechts von } \overrightarrow{p_{k-1}q_i}.$ binäre Suche
 $p_k = q_i, \text{ so dass } q_1, \dots, q_{n/m} \text{ rechts von } \overrightarrow{p_{k-1}p_k}$

Satz. CHANSKONVEXEHÜLLE berechnet die konvexe Hülle von S in $\mathcal{O}(n \log h)$ Zeit, wobei $h = |CH(S)|$.

$$\begin{aligned}
 & \sum_{i=1}^{\log \log h} \mathcal{O}(n \log 2^{2^i}) \\
 &= \mathcal{O}(n \cdot \sum_{i=1}^{\log \log h} 2^i) \\
 &= \mathcal{O}(n \cdot 2^{1+\log \log h}) \\
 &= \mathcal{O}(n \cdot \log h)
 \end{aligned}$$

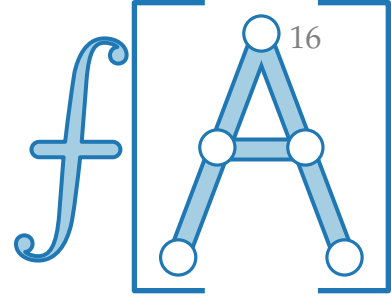
Laufzeit?

$$\begin{aligned}
 & \mathcal{O}(n) \\
 & \frac{n}{m} \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m) \\
 & \mathcal{O}(n) \\
 & m \cdot \\
 & \frac{n}{m} \cdot \\
 & \mathcal{O}(\log m) = \mathcal{O}(n \log m) \\
 & \frac{n}{m} = \mathcal{O}(n) \\
 & \mathcal{O}(1) \\
 & \underbrace{\hspace{10em}}_{\mathcal{O}(n \log m)}
 \end{aligned}$$



Chans Algorithmus

[Text kopiert am 17. Oktober 2017 von:
https://en.wikipedia.org/wiki/Chan's_algorithm]



Initially, we assume that the value of h is known and make a parameter $m = h$. This assumption is not realistic, but we remove it later. The algorithm starts by arbitrarily partitioning P into at most $1 + \frac{n}{m}$ subsets Q with at most m points each. Then, it computes the convex hull of each subset Q using an $\mathcal{O}(n \log n)$ algorithm – **Graham's scan**. Note that, as there are $\mathcal{O}(n/m)$ subsets of $\mathcal{O}(m)$ points each, this phase takes $\mathcal{O}(n/m) \cdot \mathcal{O}(m \log m) = \mathcal{O}(n \log m)$ time.

The second phase consists of executing the **Jarvis' march** algorithm and using the precomputed convex hulls to speed up the execution. At each step in Jarvis's march, we have a point p_i in the convex hull, and need to find a point $p_{i+1} = f(p_i, P)$ such that all other points of P are to the right of the line $p_i p_{i+1}$. If we know the convex hull of a set Q of m points, then we can compute $f(p_i, Q)$ in $\mathcal{O}(\log m)$ time, by using binary search. We can compute $f(p_i, Q)$ for all the $\mathcal{O}(n/m)$ subsets Q in $\mathcal{O}(n/m \log m)$ time. Then, we can determine $f(p_i, P)$ using the same technique as normally used in Jarvis's march, but only considering the points that are $f(p_i, Q)$ for some subset Q . As Jarvis's march repeats this process $\mathcal{O}(h)$ times, the second phase also takes $\mathcal{O}(n \log m)$ time, and therefore $\mathcal{O}(n \log h)$ time if $m = h$.

By running the two phases described above, we can compute the convex hull of n points in $\mathcal{O}(n \log h)$ time, assuming that we know the value of h . If we make $m < h$, we can abort the execution after $m + 1$ steps, therefore spending only $\mathcal{O}(n \log m)$ time (but not computing the convex hull). We can initially set m as a small constant (we use 2 for our analysis, but in practice numbers around 5 may work better), and increase the value of m until $m > h$, in which case we obtain the convex hull as a result.

If we increase the value of m too slowly, we may need to repeat the steps mentioned before too many times, and the execution time will be large. On the other hand, if we increase the value of m too quickly, we risk making m much larger than h , also increasing the execution time. Similar to strategy used by Chazelle and Matoušek's algorithm, Chan's algorithm squares the value of m at each iteration, and makes sure that m is never larger than n . In other words, at iteration t (starting at 1), we have $m = \min(n, 2^{2^t})$. The total running time of the algorithm is

$$\sum_{t=1}^{\lceil \log \log h \rceil} \mathcal{O}(n \log(2^{2^t})) = \mathcal{O}(n) \sum_{t=1}^{\lceil \log \log h \rceil} \mathcal{O}(2^t) = \mathcal{O}\left(n \cdot 2^{1 + \lceil \log \log h \rceil}\right) = \mathcal{O}(n \log h).$$

To generalize this construction for the 3-dimensional case, an $\mathcal{O}(n \log n)$ algorithm to compute the 3-dimensional convex hull should be used instead of Graham scan, and a 3-dimensional version of Jarvis's march needs to be used. The time complexity remains $\mathcal{O}(n \log h)$.